

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO MARANHÃO
CAMPUS SÃO LUÍS – MONTE CASTELO
CURSO DE ENGENHARIA ELÉTRICA INDUSTRIAL

THIAGO LIMA ALVES DE OLIVEIRA

**SISTEMA INTELIGENTE DE CONTROLE E GERENCIAMENTO REMOTO DE
PROCESSOS INDUSTRIAIS BASEADO EM REDES NEURAIS ARTIFICIAIS**

São Luís

2026

THIAGO LIMA ALVES DE OLIVEIRA

**SISTEMA INTELIGENTE DE CONTROLE E GERENCIAMENTO REMOTO DE
PROCESSOS INDUSTRIAIS BASEADO EM REDES NEURAIIS ARTIFICIAIS**

Monografia, apresentada ao curso de Engenharia Elétrica Industrial do Instituto Federal de Educação, Ciência e Tecnologia do Maranhão Campus Monte Castelo, como requisito para obtenção do título de Engenheiro Eletricista Industrial.

Orientador: Prof. Dr. Francisco Borges Carreiro

THIAGO LIMA ALVES DE OLIVEIRA

**SISTEMA INTELIGENTE DE CONTROLE E GERENCIAMENTO REMOTO DE
PROCESSOS INDUSTRIAIS BASEADO EM REDES NEURAIS ARTIFICIAIS**

Monografia, apresentada ao curso de Engenharia Elétrica Industrial do Instituto Federal de Educação, Ciência e Tecnologia do Maranhão Campus Monte Castelo, como requisito para obtenção do título de Engenheiro Eletricista Industrial.

Aprovada em: ____ de _____ de _____.

BANCA EXAMINADORA

Prof. Dr. Francisco Borges Carreiro (Orientador)

Doutor em Engenharia

Instituto Federal de Educação, Ciência e Tecnologia do Maranhão (IFMA)

Prof. Dr. Arnaldo Pinheiro de Azevedo Junior (Examinador 1)

Doutor em Engenharia

Instituto Federal de Educação, Ciência e Tecnologia do Maranhão (IFMA)

Prof. Dr. Almir Souza Silva Neto (Examinador 2)

Doutor em Engenharia

Instituto Federal de Educação, Ciência e Tecnologia do Maranhão (IFMA)

AGRADECIMENTOS

Primeiramente, agradeço a Deus, fonte de força, sabedoria e perseverança, por me sustentar ao longo de toda esta jornada acadêmica e permitir que eu chegasse até aqui.

À minha esposa, Adriana, pelo apoio incondicional, compreensão nos momentos de ausência e por ser meu alicerce nos dias mais desafiadores. Sua parceria foi essencial para que este trabalho se concretizasse.

Às minhas filhas, Liv e Gianna, razão maior de tudo, que mesmo tão pequenas, foram grande motivação para seguir firme. Que este esforço sirva também como exemplo de dedicação e persistência para suas vidas.

À minha família, pois a importância de cada um de vocês fez parte dos meus momentos de reflexão e foram fonte de motivação para seguir em frente nesta etapa tão importante da minha vida.

Ao meu orientador, Prof. Francisco, pela confiança depositada neste trabalho desde sua concepção, pela paciência nos momentos de dúvida e pelas orientações que conduziram a realização deste projeto.

Aos meus professores, pelos ensinamentos transmitidos, pela orientação e pelo comprometimento com a formação acadêmica e profissional, pois cada contribuição foi fundamental para o desenvolvimento deste trabalho.

Aos colegas e amigos que, direta ou indiretamente, contribuíram com apoio, incentivo e troca de conhecimento ao longo dessa trajetória.

Por fim, agradeço a todos que fizeram parte desta etapa tão importante da minha vida, contribuindo para meu crescimento pessoal e profissional.

“Porque o Senhor dá a sabedoria; da sua boca é
que vem o conhecimento e o entendimento.”

(Provérbios 2:6)

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema inteligente de controle e gerenciamento remoto de um processo industrial simulado, fundamentado em técnicas de Inteligência Artificial (IA) e integração com computação em nuvem. O cenário de aplicação consiste em uma planta de triagem automatizada, modelada no ambiente de simulação Factory I/O, cujo processo é controlado por cinco Redes Neurais Artificiais (RNAs) especializadas, treinadas por aprendizado supervisionado para atuar em diferentes etapas do fluxo produtivo e realizar o direcionamento dos materiais de acordo com as condições operacionais e os pedidos de produção. A comunicação entre a planta simulada e o sistema de controle é estabelecida por meio do protocolo industrial Modbus TCP/IP, sendo avaliada tanto em uma infraestrutura local executada em Python quanto em uma arquitetura embarcada baseada no microcontrolador ESP32. A camada de supervisão é composta por uma interface web que possibilita a inserção de pedidos, o acompanhamento remoto dos indicadores de produção, a visualização do estado da planta e o treinamento das redes neurais, além da integração com uma API de Inteligência Artificial Generativa para a elaboração de relatórios a partir dos dados operacionais. A abordagem desenvolvida possibilitou a integração dos diferentes recursos empregados em uma arquitetura única para controle e supervisão de processos industriais simulados.

Palavras-chave: Rede Neural Artificial; Automação industrial; Indústria 4.0; Computação em Nuvem; Inteligência Artificial; Modbus.

ABSTRACT

This work presents the development of an intelligent system for remote control and management of a simulated industrial process, based on Artificial Intelligence (AI) techniques and integration with cloud computing. The application scenario consists of an automated sorting plant, modeled in the Factory I/O simulation environment, whose process is controlled by five specialized Artificial Neural Networks (ANNs), trained through supervised learning to operate at different stages of the production flow and to direct materials according to operational conditions and production orders. Communication between the simulated plant and the control system is established through the Modbus TCP/IP industrial protocol, being evaluated both on a local infrastructure running in Python and on an embedded architecture based on the ESP32 microcontroller. The supervisory layer consists of a web interface that enables order entry, remote monitoring of production indicators, visualization of the plant's status, and neural network training, in addition to integration with a Generative Artificial Intelligence API for generating reports from operational data. The developed approach enabled the integration of the different resources employed into a single architecture for the control and supervision of simulated industrial processes.

Keywords: Artificial Neural Network; Industrial Automation; Industry 4.0; Cloud Computing; Artificial Intelligence; Modbus.

LISTA DE FIGURAS

Figura 1 – Evolução da indústria.....	25
Figura 2 – Sistema em malha aberta.....	26
Figura 3 – Sistema em malha fechada.....	27
Figura 4 – Fluxo de varredura do CLP	28
Figura 5 – Esquema de sistema distribuído	29
Figura 6 – Arquitetura sistema SCADA.....	31
Figura 7 – Sistema Ciber-Físico	32
Figura 8 – Pirâmide da integração vertical.....	34
Figura 9 – Ilustração integração horizontal	35
Figura 10 – Camadas TCP/IP	37
Figura 11 – Composição Modbus TCP/IP.....	39
Figura 12 – Fluxo de requisição HTTP	41
Figura 13 – Ilustração da composição da interface web.....	44
Figura 14 – Ilustração de um neurônio.....	49
Figura 15 – Representação de um perceptron	50
Figura 16 – Perceptron multicamada.....	52
Figura 17 – Curva característica sigmoide	55
Figura 18 – Curva característica tangente hiperbólica	56
Figura 19 – Fluxo do back propagation.....	61
Figura 20 – Camadas do sistema	67
Figura 21 – Tela inicial Factory I/O	68
Figura 22 – Visão geral da planta.....	69
Figura 23 – Esteira transportadora.....	69
Figura 24 – Fluxograma do processo industrial	70
Figura 25 – Itens tratados no sistema	71
Figura 26 – Módulo emissor.....	71
Figura 27 – Sensor óptico.....	71
Figura 28 – Pivô de desvio	73
Figura 29 – Sensor difuso.....	73
Figura 30 – Visão geral do encaminhamento dos itens.....	73
Figura 31 – Elevador e esteiras auxiliares	74
Figura 32 – Manipulador cartesiano.....	76

Figura 33 – Dispositivo de remoção.....	76
Figura 34 – Ponto de remoção dos itens do sistema.....	77
Figura 35 – Fluxo para Forward Pass.....	87
Figura 36 – Fluxograma de entrada de produtos	93
Figura 37 – Fluxograma controle de expedição	95
Figura 38 – Eixos do manipulador cartesiano	96
Figura 39 – Fluxograma Elevador	97
Figura 40 – Arquitetura de controle	101
Figura 41 – Ilustração da computação em nuvem	108
Figura 42 – Itens produzidos e itens enviados para o elevador	110
Figura 43 – Processados para a linha A e para linha B	110
Figura 44 – Tempo de atendimento.....	112
Figura 45 – Comando para ligar o sistema	113
Figura 46 – Representação sinótica da planta	113
Figura 47 – Gerenciamento de pedidos	114
Figura 48 – Comparação de acionamento das instâncias da rede Expedição.....	115
Figura 49 – Lista de histórico de valores das RNAs	115
Figura 50 – Seleção da versão de treinamento da rede.....	116
Figura 51 – Inserção de conjunto de dados para treinamento da RNA	117
Figura 52 – Definição da quantidade de épocas e da taxa de aprendizagem.....	117
Figura 53 – Evolução da acurácia no treinamento	118
Figura 54 – Desenvolvimento do erro no treinamento	118
Figura 55 – Pesos entre Oculta-saída	119
Figura 56 – Evolução Bias saída	119
Figura 57 – Comando de teste para um conjunto de entradas.....	119
Figura 58 – Exibição dos valores de um conjunto de pesos.....	120
Figura 59 – Botão para gerar relatório por IA	121
Figura 60 – Configuração do banco de dados	123
Figura 61 – Estrutura da tabela Pedidos no banco de dados	124
Figura 62 – Estrutura da tabela Historico no banco de dados	125
Figura 63 – Estrutura da tabela treinamento no banco de dados	126
Figura 64 – Estrutura da tabela configs no banco de dados	127
Figura 65 – Estrutura da tabela api no banco de dados	128
Figura 66 – Fluxo de inicialização dos parâmetros da rede	129

Figura 67 – Comunicação HTTP.....	135
Figura 68 – Comunicação com serviço externo	137
Figura 69 – Configuração servidor FactoryI/O	138
Figura 70 – Configuração entradas e saídas Factory I/O.....	138
Figura 71 – Endereçamentos Factory I/O.....	141
Figura 72 – Conjunto de dados para treinamento da rede prioridade.....	143
Figura 73 – Acurácia geral da rede prioridade	145
Figura 74 – Erro geral da rede prioridade.....	146
Figura 75 – Conjunto de dados para treinamento da rede elevador	147
Figura 76 – Acurácia das saídas da rede elevador.....	148
Figura 77 – Erro geral da rede elevador	149
Figura 78 – Conjunto de dados para treinamento da rede esteira.....	150
Figura 79 – Acurácia das saídas da rede esteira	151
Figura 80 – Erros por saídas da rede esteira	152
Figura 81 – Conjunto de dados para treinamento da rede expedição	154
Figura 82 – Acurácia geral da rede expedição	155
Figura 83 – Erros por saídas da rede expedição	156
Figura 84 – Conjunto de dados de treinamento da rede despacho	157
Figura 85 – Acurácia geral da rede despacho.....	158
Figura 86 – Erro geral da rede despacho	159
Figura 87 – Inserção de pedidos Pendentes.....	161
Figura 88 – Pedidos em fabricação.....	161
Figura 89 – Pedidos finalizados.....	162
Figura 90 – Saídas da rede prioridade	163
Figura 91 – Saídas da rede esteira	163
Figura 92 – Saídas da rede Expedição A.....	164
Figura 93 – Saídas da rede Expedição B	164
Figura 94 – Saídas da rede despacho.....	165
Figura 95 – Saídas da rede elevador.....	166
Figura 96 – Distribuição dos itens produzidos	167
Figura 97 – Itens enviados para armazenamento.....	167
Figura 98 – Itens enviados para esteira A.....	168
Figura 99 – Itens enviados para esteira B.....	168
Figura 100 – Tempo médio entre itens no sistema.....	174

Figura 101 – Tempo médio de encaminhamento	175
Figura 102 – Armazenamento vertical no momento chegada do segundo item X.....	178
Figura 103 – Relatório gerado por IA generativa.....	180
Figura 104 – Sinótico da planta em produção	182
Figura 105 – Print do log ESP32	184
Figura 106 – Rede Expedição B no teste comparativo.....	184
Figura 107 – Print do log do controlador Python	185
Figura 108 – Rede Elevador no teste comparativo.....	185

LISTA DE TABELAS

Tabela 1 – Lista de códigos dos itens	72
Tabela 2 – Lista de dispositivos digitais.....	78
Tabela 3 – Lista de dispositivos analógicos	79
Tabela 4 – Funções de ativações	84
Tabela 5 – Quantidade de Neurônios por camada.....	86
Tabela 6 – Instâncias das redes neurais	89
Tabela 7 – Conjunto de entradas da rede Prioridade	91
Tabela 8 – Conjuntos de saídas da rede Prioridade	91
Tabela 9 – Conjunto de entradas da rede Esteira.....	92
Tabela 10 – Conjunto de saídas da rede Esteira	94
Tabela 11 – Conjunto de entradas da rede Expedição	95
Tabela 12 – Conjunto de saídas da rede Expedição	96
Tabela 13 – Conjunto de entradas da rede Elevador	97
Tabela 14 – Conjunto de saídas da rede Elevador.....	98
Tabela 15 – Conjunto de entradas da rede Despacho	99
Tabela 16 – Conjunto de saídas da rede Despacho	100
Tabela 17 – Endereços Modbus Entradas Digitais.....	139
Tabela 18 – Endereços Modbus entradas analógicas	139
Tabela 19 – Endereço Modbus saídas digitais.....	140
Tabela 20 – Endereço Modbus Saídas Analógicas.....	140
Tabela 21 – Erro consolidado das RNAs.....	159
Tabela 22 – Ciclo de varredura.....	169
Tabela 23 – Tempo de escrita modbus por RNA	171
Tabela 24 – Comparação escrita cabeada e wi-fi	172
Tabela 25 – Taxa de atendimento correto por destino e por item	176

LISTA DE ALGORITMOS

Algoritmo 1 – Pseudocódigo de gerenciamento de tempo e itens.....	90
Algoritmo 2 – Pseudocódigo de atualização das variáveis de pendência.....	99
Algoritmo 3 – Pseudocódigo de tratamento das variáveis de estado	105
Algoritmo 4 – Pseudocódigo do processo forward pass.....	107
Algoritmo 5 – Instrução do sistema para requisição à API	122
Algoritmo 6 – Pseudocódigo de treinamento supervisionado de uma rede	131
Algoritmo 7 – Pseudocódigo de verificação de acertos	133
Algoritmo 8 – Instrução do usuário enviado para API.....	179

LISTA DE ABREVIATURAS E SIGLAS

AI	Artificial Intelligence (Inteligência Artificial)
AJAX	Asynchronous JavaScript and XML (JavaScript e XML Assíncronos)
API	Application Programming Interface (Interface de Programação de Aplicações)
CLP	Controlador Lógico Programável
CPS	Cyber-Physical Systems (Sistemas Ciber-Físicos)
CSS	Cascading Style Sheets (Folhas de Estilo em Cascata)
DCS	Distributed Control System (Sistema de Controle Distribuído)
DHCP	Dynamic Host Configuration Protocol (Protocolo de Configuração Dinâmica de Host)
DOM	Document Object Model (Modelo de Objeto de Documento)
GPT	Generative Pre-trained Transformer (Transformador Pré-treinado Generativo)
HTML	HyperText Markup Language (Linguagem de Marcação de Hipertexto)
HTTP	HyperText Transfer Protocol (Protocolo de Transferência de Hipertexto)
HTTPS	HyperText Transfer Protocol Secure (Protocolo de Transferência de Hipertexto Seguro)
IA	Inteligência Artificial
IEC	International Electrotechnical Commission (Comissão Eletrotécnica Internacional)
IEEE	Institute of Electrical and Electronics Engineers (Instituto de Engenheiros Eletricistas e Eletrônicos)
IHM	Interface Homem-Máquina
IIoT	Industrial Internet of Things (Internet das Coisas Industrial)
IoT	Internet of Things (Internet das Coisas)
IP	Internet Protocol (Protocolo de Internet)
JSON	JavaScript Object Notation (Notação de Objetos JavaScript)
MAC	Media Access Control (Controle de Acesso ao Meio)
MBAP	Modbus Application Protocol (Protocolo de Aplicação Modbus)
MLP	Multilayer Perceptron (Perceptron Multicamadas)
Modbus	Modicon Bus (protocolo de comunicação industrial)
MSE	Mean Squared Error (Erro Quadrático Médio)
MySQL	My Structured Query Language (Sistema de Gerenciamento de Banco de Dados)
PDU	Protocol Data Unit (Unidade de Dados de Protocolo)

PHP	PHP: Hypertext Preprocessor (Pré-processador de Hipertexto PHP)
REST	Representational State Transfer (Transferência de Estado Representacional)
RNA	Rede Neural Artificial
SCADA	Supervisory Control and Data Acquisition (Controle Supervisório e Aquisição de Dados)
SGBD	Sistema Gerenciador de Banco de Dados
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
SSID	Service Set Identifier (Identificador do Conjunto de Serviço)
SVG	Scalable Vector Graphics (Gráficos Vetoriais Escaláveis)
TCP	Transmission Control Protocol (Protocolo de Controle de Transmissão)
TCP/IP	Transmission Control Protocol/Internet Protocol (Protocolo de Controle de Transmissão/Protocolo de Internet)
TI	Tecnologia da Informação
TLS	Transport Layer Security (Segurança da Camada de Transporte)
UDP	User Datagram Protocol (Protocolo de Datagrama do Usuário)
Wi-Fi	Wireless Fidelity (termo comercial para redes locais sem fio baseadas no padrão IEEE 802.11)
XML	Extensible Markup Language (Linguagem de Marcação Extensível)
XOR	Exclusive OR (Ou Exclusivo)

SUMÁRIO

1.	INTRODUÇÃO.....	20
1.1.	Aspectos gerais	20
1.2.	Problema.....	21
1.3.	Justificativa	22
1.4.	Objetivos.....	22
1.4.1.	Objetivo geral	23
1.4.2.	Objetivos específicos	23
1.5.	Estrutura do trabalho.....	23
2.	FUNDAMENTAÇÃO TEÓRICA.....	24
2.1.	Automação industrial	24
2.1.1.	Evolução da automação industrial	24
2.1.2.	Sistemas de controle industrial	26
2.1.2.1.	<i>Controlador Lógico Programável (CLP)</i>	27
2.1.2.2.	<i>Arquitetura de Controle Distribuído</i>	28
2.1.3.	Sistemas supervisórios e de monitoramento	30
2.1.3.1.	<i>Interface Homem-Máquina (IHM)</i>.....	30
2.1.3.2.	<i>Controle Supervisório e Aquisição de Dados (SCADA)</i>	30
2.2.	Indústria 4.0	31
2.2.1.	Sistemas Ciber-Físicos.....	32
2.2.2.	Integração de sistemas	33
2.2.2.1.	<i>Integração vertical</i>	33
2.2.2.2.	<i>Integração horizontal</i>	34
2.2.3.	Gêmeo digital.....	35
2.3.	Comunicação industrial	36
2.3.1.	Rede ethernet na indústria	36
2.3.2.	Modbus TCP/IP	38
2.3.2.1.	<i>Arquitetura e unidade de dados</i>.....	39
2.3.2.2.	<i>Registradores e funções</i>	39
2.3.3.	Solicitação HTTP.....	40
2.4.	Computação em nuvem.....	41
2.4.1.	Integração de sistemas com PHP	42
2.4.2.	Banco de dados MySQL	43

2.4.3.	Interface web: HTML, CSS e JavaScript	44
2.5.	Inteligência Artificial.....	45
2.5.1.	Aprendizado de máquina	46
2.5.2.	IA Generativa.....	47
2.6.	Rede Neural Artificial	48
2.6.1.	Perceptron.....	49
2.6.2.	Perceptron multicamadas.....	51
2.6.3.	Propagação direta (forward pass)	52
2.6.4.	Funções de ativação	53
2.6.4.1.	<i>Sigmoide</i>	54
2.6.4.2.	<i>Tangente hiperbólica</i>	55
2.6.4.3.	<i>Softmax</i>.....	56
2.6.5.	Função de Erro.....	57
2.6.5.1.	<i>Entropia cruzada binária</i>.....	57
2.6.5.2.	<i>Entropia cruzada categórica</i>	58
2.6.5.3.	<i>Erro Quadrático Médio</i>	59
2.6.6.	Back propagation	60
2.6.6.1.	<i>Retro propagação (backward pass)</i>	60
2.6.6.2.	<i>Ajuste do bias</i>	62
2.6.6.3.	<i>Gradiente descendente</i>	62
3.	METODOLOGIA.....	64
3.1.	Visão geral	64
3.1.1.	Camadas do sistema.....	65
3.1.2.	Fluxo de comunicação entre camadas	66
3.2.	Planta industrial simulada	67
3.2.1.	Arquitetura da planta	68
3.2.2.	Tratamento de produtos	71
3.2.3.	Direcionamento de produtos.....	72
3.2.4.	Armazenamento de produtos	74
3.2.5.	Expedição	75
3.2.6.	Controle do processo	77
3.2.6.1.	<i>Dispositivos digitais</i>.....	78
3.2.6.2.	<i>Dispositivos analógicos</i>	79
3.2.6.3.	<i>Variáveis de estado</i>	80

3.3.	Implementação da Rede Neural Artificial.....	81
3.3.1.	Funções de ativação das RNAs.....	82
3.3.2.	Definição dos neurônios da RNA	85
3.3.3.	Estrutura matemática da RNA	86
3.3.4.	Composição das Redes Neurais.....	88
3.3.4.1.	<i>Prioridade</i>.....	89
3.3.4.2.	<i>Esteira</i>.....	92
3.3.4.3.	<i>Expedição</i>	94
3.3.4.4.	<i>Elevador</i>	97
3.3.4.5.	<i>Despacho</i>	98
3.4.	Arquitetura de controle.....	100
3.4.1.	Dispositivos de controle	101
3.4.1.1.	<i>Controle local</i>.....	102
3.4.1.2.	<i>Controle por ESP32</i>	103
3.4.2.	Lógica de controle das variáveis de estado.....	104
3.4.3.	Processo forward pass.....	106
3.4.4.	Computação em nuvem	107
3.5.	Integração web	109
3.5.1.	Páginas de navegação	109
3.5.1.1.	<i>Produção</i>	109
3.5.1.2.	<i>Dashboard</i>	112
3.5.1.3.	<i>Treinamento</i>	116
3.5.2.	Integração com API	121
3.5.3.	Banco de dados.....	123
3.5.3.1.	<i>Pedidos</i>	124
3.5.3.2.	<i>Histórico</i>	125
3.5.3.3.	<i>Treinamento</i>	126
3.5.3.4.	<i>Configs</i>	127
3.5.3.5.	<i>API</i>.....	128
3.6.	Treinamento das RNAs	128
3.6.1.	Inicialização dos parâmetros.....	129
3.6.2.	Algoritmo de treinamento.....	130
3.6.3.	Medição do erro.....	131
3.6.4.	Critérios de acurácia	132

3.7.	Comunicação entre camadas	133
3.7.1.	Conexão HTTP	134
3.7.1.1.	<i>Conexão Wifi.....</i>	135
3.7.1.2.	<i>Comunicação com serviço externo.....</i>	136
3.7.2.	Conexão modBus.....	138
4.	RESULTADOS	141
4.1.	Treinamento Inicial das RNAs	142
4.1.1.	Treinamento da rede prioridade.....	142
4.1.2.	Treinamento da rede elevador	146
4.1.3.	Treinamento da rede esteira.....	150
4.1.4.	Treinamento da rede expedição	153
4.1.5.	Treinamento da rede despacho	156
4.1.6.	Erro de treinamento	159
4.2.	Validação da planta industrial	160
4.2.1.	Cenário de teste.....	160
4.2.2.	Acompanhamento do status dos pedidos.....	161
4.2.3.	Comportamento das redes de controle.....	162
4.2.4.	Distribuição dos itens produzidos.....	166
4.3.	Desempenho comunicação modbus TCP/IP.....	168
4.3.1.	Ciclo de varredura.....	169
4.3.2.	Tempo médio de escrita local	170
4.3.3.	Comparação entre comunicação cabeada e wi-fi.....	172
4.4.	Indicadores de desempenho	173
4.4.1.	Tempo médio de atendimento	173
4.4.2.	Distribuição dos itens por destino.....	175
4.4.3.	Taxa de atendimento correto	176
4.5.	Gerenciamento e supervisão Web	177
4.5.1.	Página Produção	177
4.5.1.1.	<i>Indicadores agregados de produção.....</i>	177
4.5.1.2.	<i>Geração automática de relatórios</i>	179
4.5.2.	Página Dashboard	182
4.5.3.	Página Treinamento	183
4.6.	Discussão dos resultados	186
5.	CONCLUSÃO.....	188

5.1.	Verificação dos objetivos.....	188
5.2.	Contribuições do trabalho	191
5.3.	Limitações.....	191
5.4.	Trabalhos futuros	192
5.5.	Considerações finais	193
	REFERÊNCIAS.....	195
	APÊNDICE A – CÓDIGO-FONTE DO SISTEMA.....	200

1. INTRODUÇÃO

A crescente competitividade do setor industrial tem impulsionado a adoção de tecnologias capazes de aumentar a eficiência, reduzir custos operacionais e melhorar a qualidade dos produtos, deste modo, a automação industrial consolidou-se como elemento fundamental para a modernização dos processos produtivos, permitindo maior precisão, repetibilidade e confiabilidade nas operações (GROOVER, 2011). Tradicionalmente, os sistemas de controle industrial são baseados em modelos matemáticos determinísticos com técnicas clássicas de controle, que apresentam bom desempenho em ambientes previsíveis e bem modelados (OGATA, 2010).

Entretanto, com o avanço da digitalização e o surgimento da chamada Indústria 4.0, os sistemas produtivos passaram a incorporar conectividade, integração de dados, além da tomada de decisão descentralizada com integrações web, exigindo soluções mais inteligentes e adaptativas (SCHWAB, 2016). Este tipo de modelo caracteriza-se pela integração de sistemas ciber-físicos, Internet das Coisas, computação em nuvem e Inteligência Artificial (IA), promovendo maior flexibilidade e autonomia aos processos industriais (KAGERMANN; WAHLSTER; HELBIG, 2013).

Neste contexto, a Inteligência Artificial, especificamente por meio das Redes Neurais Artificiais (RNAs), surge como uma solução para a gestão de processos complexos, oferecendo a capacidade de processar grandes volumes de dados sensoriais para o ajuste dinâmico de variáveis. Como destacam Silva, Spatti e Flauzino (2016, p. 32), as redes neurais são "modelos matemáticos que buscam simular a capacidade humana de aprender a partir de experiências", o que as torna ideais para ambientes onde a modelagem matemática exata do processo é difícil ou excessivamente variável, além de permitir uma maior dinâmica com diferentes tipos de sistemas.

1.1. Aspectos gerais

A aplicação de redes neurais artificiais, pode ser uma alternativa aos métodos tradicionais de controle, por possuir a capacidade de aprendizado a partir de dados, adaptação a sistemas não lineares e generalização para diferentes condições operacionais, reduzindo a necessidade de modelagem matemática explícita do processo (HAYKIN, 2009). Cabe ressaltar, contudo, que a adoção de redes neurais não deve ser assumida automaticamente como garantia

de melhoria do processo, devendo sua aplicação ser avaliada e justificada de acordo com as particularidades de cada problema industrial (LENNOX et al., 2001).

Paralelamente, a integração de sistemas industriais com plataformas web e ambientes de computação em nuvem possibilita o monitoramento, a supervisão e o gerenciamento remoto em tempo real, ampliando a capacidade de analisar e atualizar os dados desses modelos inteligentes de forma centralizada (MARINESCU, 2018). Essa convergência entre automação, inteligência artificial e tecnologias de rede, representa uma evolução significativa na arquitetura dos sistemas de controle industriais modernos.

A possibilidade de gerenciar remotamente uma planta, processar dados de Inteligência artificial em servidores externos e receber diagnósticos preditivos através de APIs (Interface de Programação de Aplicações), permite uma escalabilidade sem precedentes. Conforme Veneri e Capasso (2018, p. 24), a Internet das Coisas Industrial (IIoT) viabiliza a "transmissão de dados de dispositivos de borda para plataformas em nuvem, transformando o chão de fábrica em um nó ativo de uma rede de inteligência global".

Observa-se, no entanto, que essas tecnologias são tratadas de forma isolada nas soluções de automação industrial, sem uma arquitetura única capaz de integrá-las em um mesmo sistema de controle adaptativo e supervisionado remotamente. Essa lacuna de integração é discutida na formulação do problema deste trabalho.

1.2. Problema

A crescente digitalização dos processos industriais tem ampliado a demanda por sistemas de controle capazes de operar em cenários com variabilidade de demanda e fluxo de materiais não determinístico, condição em que a modelagem matemática explícita do processo se torna cada vez mais complexa ou pouco viável, sobretudo em sistemas cuja modelagem tradicional apresenta limitações de velocidade computacional e de precisão diante de condições operacionais variáveis (ARAÚJO, 2025; SILVA; SPATTI; FLAUZINO, 2016). Ao mesmo tempo, tecnologias capazes de endereçar essa limitação costumam ser aplicadas de forma isolada, sem compor uma arquitetura única de controle inteligente (KAGERMANN; WAHLSTER; HELBIG, 2013).

Diante desse cenário, como estruturar uma arquitetura integrada, baseada em Redes Neurais Artificiais, capaz de controlar um processo industrial com demanda variável, comunicando-se por meio de protocolo industrial padronizado e sendo supervisionada

remotamente por meio de uma plataforma web integrada à computação em nuvem, sem depender de modelagem matemática explícita do processo?

1.3. Justificativa

A proposta de desenvolvimento de um sistema inteligente para o controle e o gerenciamento remoto de processos industriais, apoiado em RNAs, fundamenta-se na necessidade de integrar diferentes tecnologias em uma arquitetura capaz de atender às demandas de sistemas industriais mais conectados e orientados por dados. Nesse contexto, a solução proposta articula recursos de automação industrial, inteligência artificial, comunicação de dados, supervisão web e computação em nuvem, estabelecendo uma abordagem alinhada às tendências da Indústria 4.0 (KAGERMANN; WAHLSTER; HELBIG, 2013).

Do ponto de vista tecnológico, a utilização do protocolo Modbus TCP/IP possibilita a comunicação entre a planta simulada e os dispositivos de controle, empregando uma tecnologia consolidada e amplamente utilizada na automação industrial. Sobre essa infraestrutura, a aplicação web desenvolvida em PHP possibilita a supervisão e o gerenciamento das informações do processo, enquanto os serviços em nuvem fornecem recursos para o armazenamento dos dados e para a execução das etapas relacionadas ao treinamento dos modelos. Essa organização permite que parte do processamento e do armazenamento seja realizada em uma infraestrutura remota, reduzindo a necessidade de recursos computacionais locais e favorecendo maior flexibilidade e escalabilidade da solução (MARINESCU, 2018).

Além disso, a integração com uma Interface de Programação de Aplicação (API) de Inteligência Artificial Generativa possibilita a análise dos dados operacionais e a geração automatizada de relatórios, agregando uma camada adicional de inteligência ao sistema. Dessa forma, a solução ultrapassa a automação convencional ao integrar controle, supervisão, processamento de dados e análise inteligente em uma única arquitetura, contribuindo para uma abordagem mais conectada, flexível e orientada a dados, características associadas à transformação dos modelos tradicionais de produção e à evolução dos sistemas industriais no contexto da Indústria 4.0 (SCHWAB, 2016).

1.4. Objetivos

Esta seção apresenta os objetivos deste trabalho, de forma a orientar o desenvolvimento, implementação e validação do projeto.

1.4.1. Objetivo geral

Desenvolver e validar um sistema inteligente de controle e gerenciamento remoto de processos industriais baseado em Redes Neurais Artificiais, integrado a tecnologias web, protocolos de comunicação industrial, computação em nuvem e Inteligência Artificial

1.4.2. Objetivos específicos

- Modelar uma planta industrial simulada, com fluxo de produtos e demandas variáveis;
- Implementar um sistema de controle distribuído baseado em redes neurais artificiais;
- Estabelecer um protocolo de comunicação industrial, utilizando Modbus TCP/IP, entre a planta e o sistema de controle;
- Aplicar aprendizado supervisionado para reproduzir decisões de controle definidas a partir das condições operacionais da planta;
- Integrar o sistema de controle a uma plataforma web para supervisão e gerenciamento remoto;
- Utilizar computação em nuvem para treinamento, testes e atualização dos modelos de redes neurais;
- Gerar relatórios com requisição à Interface de Programação de Aplicação (API) de Inteligência Artificial generativa.

1.5. Estrutura do trabalho

O Capítulo 2 aborda a fundamentação teórica, incluindo conceitos de automação industrial, controle de processos, inteligência artificial, redes neurais artificiais, Indústria 4.0 e computação em nuvem, fornecendo a base conceitual necessária para o desenvolvimento do trabalho.

O Capítulo 3 descreve a metodologia adotada no desenvolvimento do sistema proposto, detalhando o tipo de pesquisa, o ambiente de simulação, as ferramentas utilizadas, os procedimentos de desenvolvimento, análise e validação.

O Capítulo 4 expõe os resultados obtidos a partir dos testes realizados, bem como a análise e discussão do desempenho do sistema no cenário operacional proposto, destacando as contribuições do trabalho, suas limitações e as possibilidades de aplicação prática em ambientes industriais reais.

Na sequência, o Capítulo 5 apresenta as conclusões do estudo, destacando os principais resultados obtidos e as contribuições do trabalho, além de propor sugestões para pesquisas e desenvolvimentos futuros.

Por fim, são apresentadas as referências bibliográficas utilizadas como fundamentação teórica e metodológica do estudo, seguido pelos apêndices que reúnem materiais complementares desenvolvidos ao longo do trabalho.

2. FUNDAMENTAÇÃO TEÓRICA

Esta seção reúne o embasamento teórico que orientou a elaboração do projeto, contemplando os principais conceitos relacionados ao tema principal.

2.1. Automação industrial

A automação industrial pode ser compreendida como a aplicação integrada de técnicas, sistemas computacionais, softwares e equipamentos destinados ao controle e à execução de processos produtivos, com o objetivo de aumentar a eficiência operacional, otimizar o uso de recursos, reduzir o consumo de energia e ampliar a segurança dos processos. Segundo Groover (2011, p. 54), a automação evoluiu de mecanismos simples para sistemas cada vez mais complexos, capazes de executar operações com mínima intervenção humana. Essa evolução está diretamente relacionada à integração entre hardware e software, permitindo maior precisão, confiabilidade e capacidade de controle dos processos industriais.

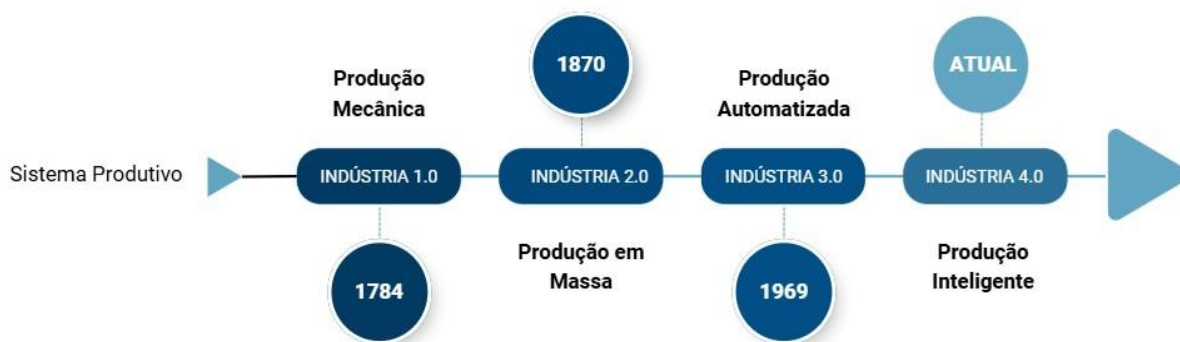
2.1.1. Evolução da automação industrial

A trajetória da automação industrial é comumente dividida em quatro fases distintas, conhecidas como as Revoluções Industriais, cada uma impulsionada por um salto tecnológico específico, sendo:

- a) **Primeira Revolução Industrial:** Caracterizada pela mecanização da produção utilizando energia a vapor e hidráulica no final do século XVIII;
- b) **Segunda Revolução Industrial:** Iniciada no final do século XIX, introduziu a produção em massa e as linhas de montagem alimentadas por energia elétrica;
- c) **Terceira Revolução Industrial:** Ocorreu a partir de 1969 com o advento da eletrônica, da tecnologia da informação e do uso de Controladores Lógicos Programáveis (CLP). Como aponta Bolton (2015, p. 3), o CLP revolucionou o chão de fábrica ao substituir lógicas de relés rígidas por sistemas programáveis e flexíveis;
- d) **Quarta Revolução Industrial (Indústria 4.0):** É o estágio atual, onde o foco reside na conectividade e inteligência de dados.

Cada revolução industrial não substituiu integralmente as tecnologias anteriores, mas sim incorporou novas camadas de complexidade e capacidade aos sistemas produtivos, partindo da mecanização física até a integração digital dos processos em uma progressão contínua rumo à automação, na qual a capacidade de processamento, comunicação e tomada de decisão dos sistemas industriais tem se tornando cada vez mais sofisticada (GROOVER, 2011). Essa trajetória pode ser representada por meio de um sistema produtivo em constante evolução, iniciado em 1784, seguido pela produção em massa a partir da eletricidade, posteriormente caracterizada pela produção automatizada por meio da eletrônica e dos primeiros sistemas computacionais, até o estágio atual, associada à produção inteligente, fruto da integração entre sistemas físicos, digitais e conectados, como ilustrado na Figura 1.

Figura 1 – Evolução da indústria



Fonte: Elaborado pelo autor (2026)

Essa trajetória evidencia que a automação evoluiu de sistemas rígidos, orientados exclusivamente à produção mecânica e em massa, para arquiteturas inteligentes e adaptativas,

capazes de operar com autonomia, conectividade e capacidade de processamento de dados, alinhadas às demandas modernas dos ambientes industriais.

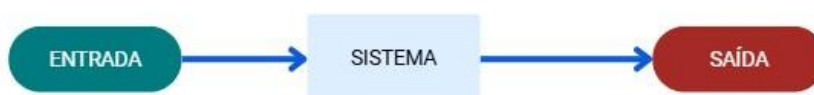
2.1.2. Sistemas de controle industrial

Os sistemas de controle industrial constituem o núcleo funcional da automação, sendo o conjunto de componentes físicos e lógicos responsáveis por gerenciar, comandar e regular o comportamento de dispositivos e processos em um ambiente produtivo. Esses sistemas operam sob uma estrutura hierárquica, na qual sensores coletam dados do campo e atuadores executam as decisões enviadas por um controlador central, podendo atender a diversos critérios de desempenho, segurança e eficiência energética.

De acordo com Ogata (2010, p. 1), um sistema de controle pode ser definido como "um conjunto de componentes que atuam conjuntamente para executar uma função desejada, onde a saída é controlada por meio da manipulação da entrada". No contexto industrial, esses sistemas interagem com processos físicos, sejam eles químicos, térmicos, mecânicos, elétricos ou híbridos, sendo classificados, quanto à sua estrutura de operação, em duas categorias fundamentais: sistemas de controle em malha aberta e sistemas de controle em malha fechada.

Nos sistemas de malha aberta (Figura 2), a ação de controle é determinada exclusivamente a partir de uma referência ou comando predefinido, sem que haja qualquer mecanismo de verificação do resultado obtido no processo. Segundo Ogata (2010), esse tipo de arranjo é adequado apenas quando a relação entre entrada e saída do sistema é bem conhecida e não está sujeita a perturbações significativas, uma vez que erros ou distúrbios não identificados durante a operação não podem ser corrigidos automaticamente pelo próprio sistema.

Figura 2 – Sistema em malha aberta

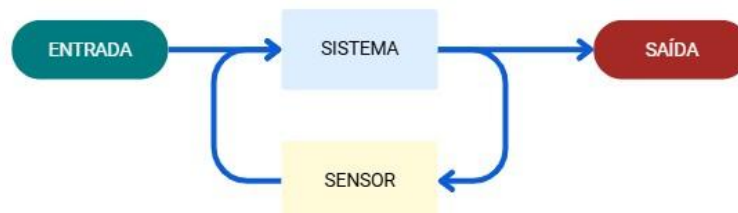


Fonte: Elaborado pelo autor (2026)

Nos sistemas de malha fechada (Figura 3), também denominados sistemas de controle com realimentação (feedback), a saída do processo é continuamente monitorada por sensores e comparada a um valor de referência desejado, sendo a diferença entre ambos, denominada erro, utilizada pelo controlador para ajustar a variável manipulada e conduzir o

sistema à condição desejada (OGATA, 2010). Essa realimentação contínua confere ao sistema maior robustez frente a incertezas do processo e a distúrbios externos, sendo por esse motivo a arquitetura predominante em ambientes industriais modernos, nos quais variações de carga, desgaste de equipamentos e condições ambientais tornam a resposta em malha aberta insuficiente (BOLTON, 2015).

Figura 3 – Sistema em malha fechada



Fonte: Elaborado pelo autor (2026)

Nesse contexto, três conceitos são centrais à compreensão de qualquer tipo de malha de controle industrial:

- a) A variável controlada, que corresponde à grandeza do processo que se deseja manter dentro de uma faixa ou valor desejado;
- b) A variável manipulada, que é o parâmetro ajustado pelo controlador com o objetivo de influenciar a variável controlada;
- c) O distúrbio, que representa qualquer influência externa e não modelada que afeta o processo, exigindo correção contínua por parte do controlador (OGATA, 2010).

A compreensão dessas variáveis é fundamental para o desenvolvimento e a análise de sistemas de controle, uma vez que a interação entre elas determina o comportamento do processo e orienta a atuação do controlador na busca pelas condições desejadas de operação.

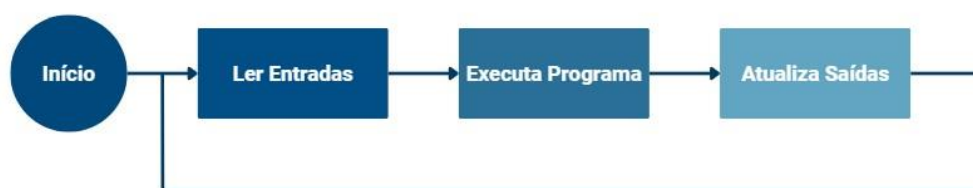
2.1.2.1. Controlador Lógico Programável (CLP)

Os Controladores Lógicos Programáveis (CLPs) constituem uma tecnologia fundamental em sistemas de controle industrial que, segundo Silveira e Santos (2014, p. 45), são equipamentos eletrônicos digitais desenvolvidos para uso em ambientes industriais, capazes de executar funções de lógica, temporização, contagem, sequenciamento e operações aritméticas por meio de programação. Esses dispositivos substituíram sistemas baseados em

relés eletromecânicos, proporcionando maior confiabilidade, flexibilidade e facilidade de manutenção, além de permitirem rápida reconfiguração do processo produtivo por meio de alterações no software, sem necessidade de modificações físicas no painel elétrico.

O CLP se destaca por sua robustez e capacidade de operar em ambientes industriais severos, sujeitos a variações de temperatura, umidade, vibração e interferências eletromagnéticas. Seu funcionamento baseia-se na execução cíclica de lógicas de varredura (scan), processo por meio do qual o controlador lê continuamente o estado das entradas, processa a lógica de controle programada e atualiza as saídas correspondentes, garantindo determinismo e confiabilidade ao sistema (BOLTON, 2015, p. 12). Como visto na Figura 4.

Figura 4 – Fluxo de varredura do CLP



Fonte: Adaptado de BOLTON (2015)

Segundo Capelli (2013), o funcionamento do CLP baseia-se na execução ininterrupta de um ciclo de operação composto pela leitura de todas as entradas físicas do sistema, cujos valores são armazenados na tabela de imagens de entrada, em seguida, o programa do usuário é processado com base nessa tabela, definindo a lógica de controle a ser aplicada, posteriormente, as saídas físicas são atualizadas de acordo com os valores registrados na tabela de imagens de saída, por fim, são realizadas as rotinas de diagnóstico e comunicação, responsáveis por verificar o estado do sistema e garantir a troca de informações com outros dispositivos da rede industrial. É justamente essa execução cíclica e sistemática, repetida continuamente em intervalos de tempo reduzidos, que assegura o determinismo e a confiabilidade do controlador, características destacadas por Bolton (2015).

2.1.2.2. Arquitetura de Controle Distribuído

A Arquitetura de Controle Distribuído (DCS - Distributed Control System) é um modelo de automação no qual as funções de controle são distribuídas entre múltiplos controladores interligados por rede, ao invés de concentradas em um único sistema central, surgindo como evolução dos sistemas centralizados, oferecendo maior confiabilidade,

modularidade e escalabilidade para processos industriais (BELGA, 2011, p. 23), especialmente em aplicações contínuas. A Figura 5 exemplifica essa distribuição.

Figura 5 – Esquema de sistema distribuído



Fonte: Elaborado pelo autor (2026)

Bolton (2015) destaca que a principal característica do controle distribuído é a descentralização da lógica de controle, diferentemente de sistemas centralizados onde uma única falha pode comprometer todo o processo, os DCS oferecem degradação gradual de desempenho e continuidade operacional mesmo com falhas parciais. Conforme destacado por Boyer (2010), a confiabilidade é um dos principais fatores que consolidaram o DCS como solução padrão em plantas industriais de médio e grande porte.

Além da tolerância a falhas, a arquitetura DCS favorece a manutenção e a expansão dos sistemas industriais ao longo do tempo, uma vez que novos controladores podem ser incorporados à rede de comunicação sem a necessidade de projetar novamente toda a lógica de controle existente, bastando que o novo dispositivo seja integrado à infraestrutura de comunicação já estabelecida. Segundo Groover (2011), essa capacidade de expansão modular é particularmente relevante em plantas industriais sujeitas a constantes atualizações tecnológicas, nas quais a substituição ou o acréscimo de equipamentos deve ocorrer com o mínimo de interrupção possível no processo produtivo. Rosário (2010) complementa que, justamente por essa razão, os sistemas DCS tornaram-se a arquitetura predominante em setores como o petroquímico, o de geração de energia e o de processos contínuos de grande porte, nos quais a indisponibilidade do sistema de controle representa custos operacionais elevados e riscos significativos à segurança do processo.

2.1.3. Sistemas supervisórios e de monitoramento

Os sistemas de supervisão e monitoramento são ferramentas fundamentais para a gestão operacional, permitindo que operadores e gestores visualizem, em tempo real, o estado de variáveis críticas do processo produtivo. Segundo Bolton (2015, p. 320), a principal função de um sistema supervisório é fornecer uma visão panorâmica da planta, permitindo a gestão de alarmes, visualização de tendências históricas e a intervenção remota para ajuste do processo, como a IHM e sistemas SCADA. A base de qualquer sistema de monitoramento reside na capacidade de traduzir dados binários de sensores em informações compreensíveis.

2.1.3.1. Interface Homem-Máquina (IHM)

A Interface Homem-Máquina (IHM) é uma camada de controle local, no qual, o ambiente gráfico permite ao operador visualizar variáveis do processo por meio de sinóticos, gráficos de tendência, indicadores e painéis de controle. Ela não é apenas uma tela de visualização, mas um ambiente interativo onde o operador exerce o controle sobre o sistema, tornando-se elemento fundamental para a operação segura e eficiente do processo industrial (ROSÁRIO, 2010).

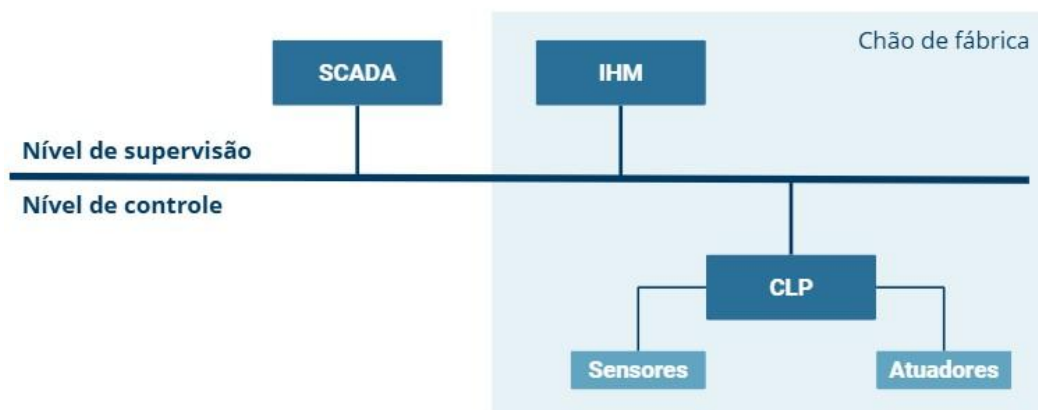
Segundo Silveira e Santos (2014), o projeto adequado de uma IHM deve considerar princípios de ergonomia cognitiva, organizando informações por prioridade e criticidade, de modo que condições anormais do processo sejam rapidamente identificadas em meio ao grande volume de dados exibido na tela, característica especialmente relevante em ambientes industriais complexos, nos quais a sobrecarga de informação pode comprometer o tempo de resposta do operador diante de uma situação de risco.

2.1.3.2. Controle Supervisório e Aquisição de Dados (SCADA)

Historicamente conhecidos como SCADA (Supervisory Control and Data Acquisition – Controle Supervisório e Aquisição de Dados), esses sistemas atuam em uma camada superior ao controle local, coletando dados de diversos controladores e centralizando-os em uma interface gráfica, permitindo que o operador interaja com o processo. O SCADA abrange a planta como um todo, integrando informações provenientes de múltiplos controladores distribuídos ao longo do processo produtivo em uma única visão consolidada (BOYER, 2010).

Segundo Boyer (2010), a arquitetura básica de um sistema SCADA é composta por quatro camadas principais: dispositivos de campo, unidades de controle, sistema de comunicação e estação supervisória. Os dispositivos de campo incluem sensores e atuadores responsáveis pela medição e atuação no processo e as unidades de controle, como CLPs, realizam o processamento local e enviam dados ao sistema supervisório. A Figura 6 demonstra a configuração dessa arquitetura

Figura 6 – Arquitetura sistema SCADA



Fonte: Elaborado pelo autor (2026)

De acordo com Rosário (2010), os sistemas SCADA modernos utilizam arquiteturas cliente-servidor, nas quais um ou mais servidores concentram a base de dados e o processamento principal, enquanto clientes acessam remotamente as informações por meio de interfaces locais ou web.

2.2. Indústria 4.0

Indústria 4.0, também denominada Quarta Revolução Industrial, representa uma nova etapa no processo de evolução da automação e da manufatura, caracterizada pela crescente integração entre tecnologias digitais, sistemas físicos e processos produtivos. Esse paradigma está fundamentado na conectividade entre máquinas, equipamentos, sistemas computacionais e pessoas, possibilitando a comunicação e o compartilhamento de informações ao longo das diferentes etapas da produção. Como resultado, os ambientes industriais passam a incorporar características de maior conectividade, flexibilidade e inteligência, dando origem ao conceito de “fábrica inteligente”. De acordo com Schwab (2016, p. 16), a Indústria 4.0 possui um escopo mais amplo do que a simples conexão e automação de máquinas e sistemas, abrangendo uma

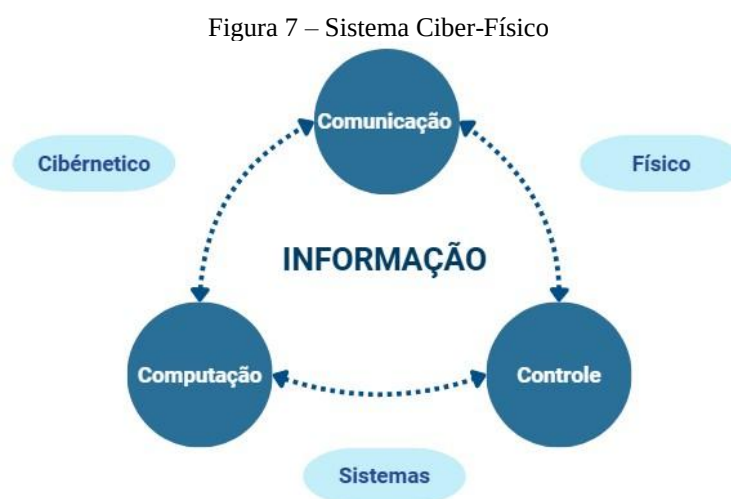
combinação de tecnologias que promove transformações na forma como os processos produtivos são estruturados e gerenciados.

Nesse contexto, os ativos de produção deixam de operar como unidades isoladas e passam a integrar sistemas conectados, capazes de compartilhar informações e interagir com diferentes níveis da estrutura industrial. Segundo Kagermann, Wahlster e Helbig (2013), essa integração permite que os sistemas produtivos respondam de maneira mais dinâmica às alterações nas condições de operação, na demanda e na ocorrência de falhas, utilizando dados provenientes do processo para apoiar a tomada de decisão e a otimização das atividades. Dessa forma, a conectividade e o uso sistemático de dados tornam-se elementos fundamentais para o desenvolvimento de processos industriais mais integrados, flexíveis e orientados à informação.

2.2.1. Sistemas Ciber-Físicos

Os Sistemas Ciber-físicos (Cyber-Physical Systems – CPS) constituem um dos pilares da Indústria 4.0, combinando componentes físicos (máquinas, sensores e atuadores) com sistemas computacionais capazes de monitorar, analisar e controlar o processo em tempo real. Essa integração permite que o sistema físico seja representado digitalmente, possibilitando simulação, otimização e tomada de decisão automatizada (Figura 7).

Segundo Kagermann, Wahlster e Helbig (2013), os CPS promovem descentralização da inteligência, permitindo que diferentes módulos do sistema produtivo operem de forma cooperativa e autônoma. Essa característica está diretamente relacionada ao conceito de controle distribuído e à integração entre hardware e software industrial.



Fonte: Elaborado pelo autor (2026)

CPS caracterizam-se por conectividade extensiva, capacidades de processamento distribuído, autonomia substancial e integração com infraestrutura de rede e computação em nuvem. Rajkumar et al. (2010, p. 731) argumentam que CPS vão além de sistemas embarcados em três aspectos críticos: ênfase na interação física através de sensoriamento e atuação, conectividade em rede permitindo coordenação entre múltiplos sistemas, e capacidades cognitivas incluindo aprendizado e adaptação.

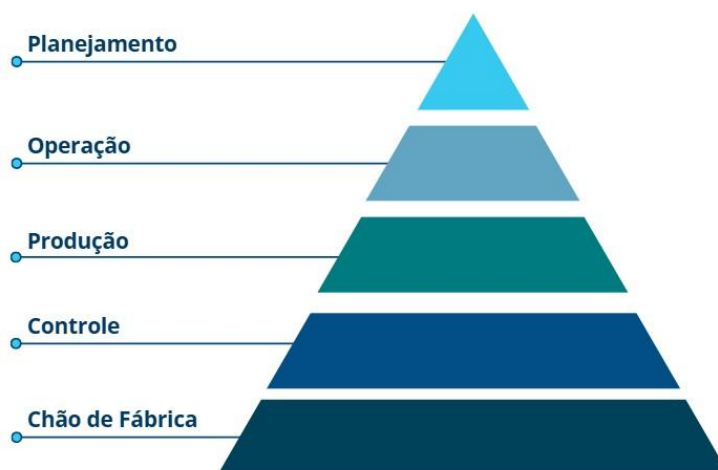
2.2.2. Integração de sistemas

De acordo com Veneri e Capasso (2018, p. 22), a integração pode ser classificada em vertical, correspondente à conexão entre os diferentes níveis da pirâmide de automação, e horizontal, referente à conexão entre diferentes setores da cadeia produtiva, sendo essas duas dimensões complementares uma à outra. Segundo Schwab (2016), independentemente da direção em que ocorre, essa conectividade amplia a transparência dos processos, melhora a rastreabilidade e possibilita decisões baseadas em dados em toda a cadeia de valor, características centrais ao que se convencionou chamar de fábrica inteligente. No ambiente industrial moderno, tanto a integração vertical quanto a horizontal são viabilizadas pela mesma base tecnológica, redes padronizadas, protocolos baseados em Ethernet e sistemas digitais interconectados, o que reforça a convergência entre infraestruturas antes tratadas de forma isolada dentro da planta industrial.

2.2.2.1. Integração vertical

A integração vertical refere-se à interligação dos diferentes níveis, como visto na configuração da Figura 8, desde o nível de campo até o nível corporativo. Isso significa que dados coletados por sensores e atuadores (nível de campo) são processados por controladores (nível de controle), disponibilizados em sistemas supervisórios (nível de supervisão) e posteriormente integrados a sistemas de gestão empresarial. Tradicionalmente, essa hierarquia é representada pelo modelo em camadas conhecido como Pirâmide de Automação, que organiza os sistemas industriais em níveis funcionais crescentes, do chão de fábrica ao planejamento estratégico, cada um operando com escalas de tempo, volumes de dados e capacidade de decisão distintos entre si (BOYER, 2010).

Figura 8 – Pirâmide da integração vertical



Fonte: Elaborado pelo autor (2026)

De acordo com Schwab (2016), essa integração permite que decisões estratégicas sejam fundamentadas em dados operacionais em tempo real, reduzindo o tempo de resposta e aumentando a eficiência produtiva, eliminando barreiras entre os setores técnicos e administrativos ao promover alinhamento entre operação e planejamento. Kagermann, Wahlster e Helbig (2013) reforçam que essa conectividade entre níveis hierárquicos é o que viabiliza, na prática, a descentralização da tomada de decisão característica da Indústria 4.0, uma vez que informações antes restritas ao nível de supervisão passam a estar disponíveis também para camadas superiores de planejamento, e decisões estratégicas podem, por sua vez, refletir-se diretamente em ajustes no nível operacional, movimento que distingue a integração vertical moderna das arquiteturas hierárquicas tradicionais.

2.2.2.2. Integração horizontal

A integração horizontal (Figura 9) refere-se à conexão entre diferentes setores da cadeia de valor, incluindo fornecedores, produção, armazenamento, distribuição e clientes. Ela possibilita que dados circulem entre diferentes unidades produtivas ou organizações, promovendo sincronização logística e maior rastreabilidade.

De acordo com Kagermann et al. (2013), a integração horizontal viabiliza redes de produção inteligentes, nas quais diferentes sistemas compartilham informações para otimizar recursos, reduzir desperdícios e melhorar o atendimento à demanda. Essa conectividade permite maior flexibilidade produtiva e personalização em massa.

Figura 9 – Ilustração integração horizontal



Fonte: Elaborado pelo autor (2026)

Schwab (2016) complementa que esse tipo de integração é o que sustenta, em escala mais ampla, a chamada cadeia de valor digital, na qual a informação passa a fluir de forma tão relevante quanto o próprio material físico, permitindo que decisões de produção em uma unidade sejam ajustadas dinamicamente em função de eventos ocorridos em outras etapas da cadeia sem depender de ciclos de replanejamento manuais e demorados.

2.2.3. Gêmeo digital

O conceito de gêmeo digital (digital twin) foi originalmente proposto por Grieves (2014) como um modelo virtual capaz de representar, de forma dinâmica e atualizada, o estado, o comportamento e o histórico de um sistema físico correspondente, estabelecendo um fluxo bidirecional contínuo de informações entre os dois ambientes. Diferentemente de uma simulação estática, na qual o modelo computacional é construído uma única vez e não reflete necessariamente as condições reais do sistema ao longo do tempo, o gêmeo digital caracteriza-se justamente pela sincronização constante com dados provenientes de sensores e dispositivos de campo, permitindo que decisões tomadas no ambiente virtual sejam validadas antes de sua aplicação física, e que alterações no sistema real sejam refletidas quase instantaneamente em sua contraparte digital.

No contexto da Indústria 4.0, o gêmeo digital assume papel estratégico ao integrar-se aos Sistemas Ciber-Físicos e à computação em nuvem, funcionando como camada intermediária de análise e tomada de decisão entre o chão de fábrica e os sistemas de gestão corporativa. Segundo Kassem e Meier (2016), essa tecnologia possibilita a integração entre o

mundo físico e o digital, favorecendo a tomada de decisão baseada em dados e reduzindo os riscos associados a testes e ajustes realizados diretamente em ambientes produtivos reais. De forma complementar, Tao et al. (2018) destacam que a aplicação de gêmeos digitais em ambientes de manufatura permite não apenas o monitoramento passivo do processo, mas também a simulação prescritiva de cenários futuros, incluindo a previsão de falhas, a otimização de parâmetros operacionais e o teste antecipado de algoritmos de controle antes de sua implementação em campo.

2.3. Comunicação industrial

A comunicação em sistemas industriais evoluiu de arquiteturas centralizadas para modelos distribuídos, nos quais diferentes dispositivos compartilham informações por meio de redes dedicadas, permitindo maior flexibilidade e escalabilidade, operando em ambientes adversos, com alta imunidade a ruídos eletromagnéticos, baixa latência e elevada confiabilidade (BOLTON, 2015).

2.3.1. Rede ethernet na indústria

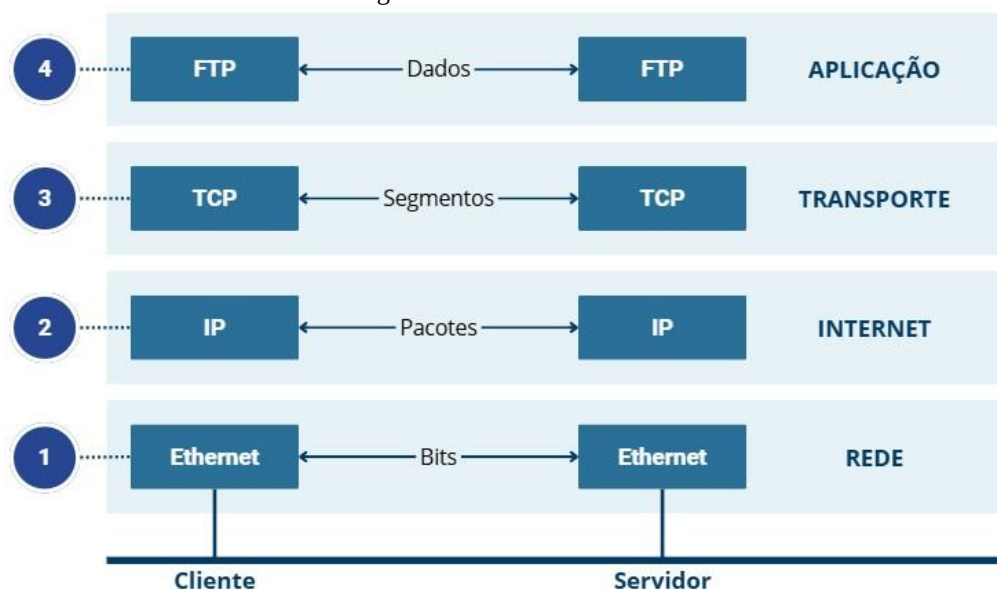
A utilização da rede Ethernet no ambiente industrial representa um dos principais avanços na integração entre sistemas de automação e tecnologias da informação (TI). Inicialmente desenvolvida para aplicações corporativas, a Ethernet passou por adaptações para atender às exigências industriais.

Segundo Veneri e Capasso (2018, p. 52), a Ethernet Industrial difere da comercial por possuir hardware robustecido (cabos blindados e conectores resistentes a vibrações) e protocolos que garantem a entrega de mensagens em tempo real, sendo sua principal vantagem, a capacidade de transportar grandes volumes de dados sob uma infraestrutura padronizada.

O uso da pilha de protocolos TCP/IP (Transmission Control Protocol/Internet Protocol) sobre a Ethernet industrial permite que cada dispositivo na rede possua um endereço IP único, o modelo organiza a comunicação em camadas funcionais, permitindo padronização, interoperabilidade e escalabilidade entre dispositivos distintos (TANENBAUM; WETHERALL, 2011), que facilita a integração direta entre diferentes sistemas, possibilitando a existência de dispositivos inteligentes que podem ser monitorados e configurados remotamente (BOLTON, 2015, p. 308). Segundo Tanenbaum e Wetherall (2011), o modelo

TCP/IP é tradicionalmente dividido em quatro camadas principais: Aplicação, Transporte, Internet e Acesso à Rede.

Figura 10 – Camadas TCP/IP



Fonte: Elaborado pelo autor (2026)

Como visto na Figura 10, cada uma dessas quatro camadas desempenha uma função específica e complementar dentro do processo de comunicação, atuando de forma hierárquica. Os dados originados na camada de Aplicação são progressivamente encapsulados ao passarem pelas camadas de Transporte, Internet e Interface de Rede, até serem efetivamente transmitidos como sinais elétricos pelo meio físico, no sentido inverso, o dispositivo receptor realiza o processo de desencapsulamento, camada por camada, até que os dados originais sejam entregues à aplicação de destino (TANENBAUM; WETHERALL, 2011). As quatro camadas do modelo são descritas a seguir:

- 1) **Camada de Interface de Rede (ou Link de Dados):** É a camada física onde ocorre a transmissão de bits via cabos Ethernet. Segundo Veneri e Capasso (2018, p. 54), nesta camada operam os endereços MAC (Media Access Control), que identificam cada hardware de forma única na rede local da fábrica.
- 2) **Camada de Internet (IP):** Responsável pelo roteamento dos pacotes. O protocolo IP define o endereço lógico de cada dispositivo. De acordo com Erl, Puttini e Mahmood (2013, p. 48), é nesta camada que o tráfego industrial pode ser direcionado para redes externas ou para a nuvem, permitindo que a planta seja acessada via internet.

- 3) **Camada de Transporte (TCP/UDP):** O TCP garante que as mensagens cheguem na ordem correta e sem erros, através de mecanismos de confirmação. Se um pacote for perdido, o TCP solicita a retransmissão automaticamente (TANENBAUM; WETHERALL, 2011).
- 4) **Camada de Aplicação:** Como afirma Rinaldi (2015, p. 47), nesta camada os dados brutos são traduzidos em comandos industriais específicos, como "Ler Registro de Entrada" ou "Escrever em Bobina".

Essa organização em camadas é o que confere ao modelo TCP/IP sua principal vantagem para aplicações industriais: cada nível opera de forma independente e especializada, permitindo que alterações em uma camada não exijam modificações nas demais.

2.3.2. Modbus TCP/IP

O Modbus TCP/IP é uma versão do protocolo Modbus adaptada para operar sobre redes Ethernet utilizando a pilha de protocolos TCP/IP. Originalmente desenvolvido em 1979 pela Modicon para comunicação serial entre controladores industriais, o Modbus tornou-se um dos protocolos mais difundidos na automação industrial devido à sua simplicidade, padronização e ampla compatibilidade entre fabricantes (MODICON, 1996).

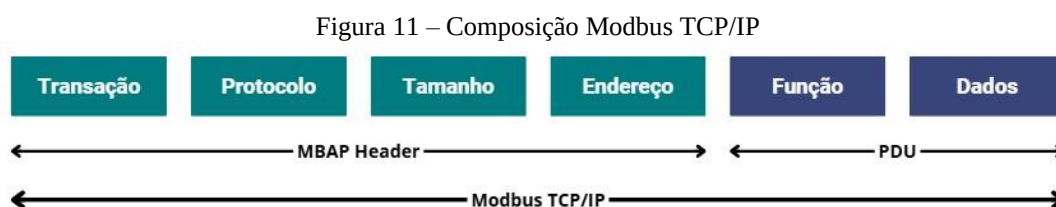
Segundo Boyer (2010), o Modbus é um protocolo do tipo mestre-escravo (ou cliente-servidor, na terminologia moderna), no qual um dispositivo mestre inicia as requisições e os dispositivos escravos respondem com os dados solicitados. Na versão TCP/IP, o modelo passa a utilizar a arquitetura cliente-servidor sobre Ethernet, eliminando as limitações físicas das redes seriais e permitindo maior taxa de transmissão e integração com infraestruturas de rede convencionais. Conforme destaca Rinaldi (2015, p. 12):

"O Modbus tornou-se o padrão de facto na indústria por ser uma linguagem aberta e de fácil implementação, permitindo que dispositivos de diferentes fornecedores troquem dados sem a necessidade de interfaces complexas ou drivers proprietários."

Essa universalidade foi o que permitiu sua transição para o ambiente de redes modernas. Segundo Veneri e Capasso (2018), a adaptação para o TCP/IP foi um passo natural para garantir que o protocolo sobrevivesse à era da internet, encapsulando a estrutura de dados original em pacotes Ethernet.

2.3.2.1. Arquitetura e unidade de dados

A mensagem Modbus TCP/IP é formada pelo MBAP Header (Cabeçalho de Aplicação) seguido pela PDU (Protocol Data Unit). O MBAP Header é responsável por informações de controle e identificação da comunicação, contendo campos como identificador da transação, identificador do protocolo, tamanho da mensagem e identificador da unidade (Unit ID), permitindo que o dispositivo correto seja reconhecido dentro da rede. Conforme destacam Veneri e Capasso (2018, p. 58), o MBAP Header permite a identificação da transação e do dispositivo específico, garantindo que a resposta recebida corresponda à requisição enviada pelo cliente. Já a PDU contém o código da função Modbus (como leitura ou escrita de registradores) e os dados propriamente ditos, representando a parte funcional da mensagem. Essa estrutura organizada (Figura 11), possibilita comunicação confiável e padronizada sobre redes Ethernet, integrando a simplicidade do Modbus.



Fonte: Adaptado de VENERI; CAPASSO (2018)

De acordo com Tanenbaum e Wetherall (2011), protocolos que operam sobre TCP/IP se beneficiam das funcionalidades da camada de transporte, como controle de erros e garantia de entrega dos pacotes, aumentando a confiabilidade.

2.3.2.2. Registradores e funções

Uma das principais características do Modbus é sua estrutura baseada em registradores, uma formatação simples que facilita a implementação, no qual os dados são organizados em quatro tipos principais, conforme especificação do protocolo (MODBUS ORGANIZATION, 2012), sendo eles:

- a) **Coils (bobinas)** – variáveis booleanas de leitura/escrita;
- b) **Discrete Inputs** – entradas digitais somente leitura;
- c) **Holding Registers** – registradores de 16 bits com leitura e escrita;
- d) **Input Registers** – registradores de 16 bits somente leitura.

Essa estrutura de endereçamento permite que o sistema de controle realize o que Bolton (2015) define como controle em tempo real, onde o atraso na rede (latência) é minimizado pela eficiência do encapsulamento TCP.

Os registradores são identificados por códigos de função que determinam o tipo de operação a ser realizada e o tipo de dado acessado, permitindo que o cliente (mestre) solicite leitura ou escrita de variáveis específicas no dispositivo servidor (escravo). Cada requisição especifica o endereço inicial do registrador desejado e a quantidade de posições consecutivas a serem lidas ou escritas, possibilitando que uma única transação atualize diversas variáveis do processo de uma só vez, em vez de exigir uma requisição separada para cada registrador individual. Caso a operação solicitada não possa ser executada, o protocolo prevê códigos de exceção específicos, retornados ao mestre em substituição à resposta normal, permitindo que falhas de comunicação ou de endereçamento sejam identificadas e tratadas pela aplicação de controle (MODBUS ORGANIZATION, 2012). Os principais códigos de função são:

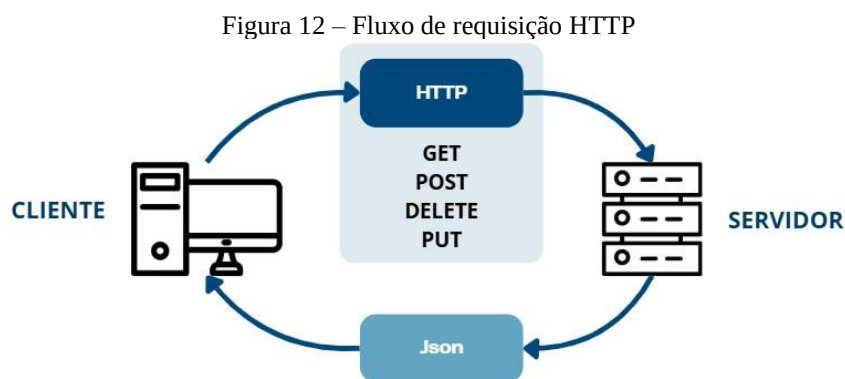
- a) **01 (0x01) – Read Coils:** leitura de bobinas (saídas digitais);
- b) **02 (0x02) – Read Discrete Inputs:** Somente leitura de entradas digitais;
- c) **03 (0x03) – Read Holding Registers:** leitura de registradores de 16 bits com possibilidade de escrita;
- d) **04 (0x04) – Read Input Registers:** leitura de registradores de 16 bits somente leitura;
- e) **05 (0x05) – Write Single Coil:** escrita em uma única bobina;
- f) **06 (0x06) – Write Single Register:** escrita em um único registro;
- g) **15 (0x0F) – Write Multiple Coils:** escrita em múltiplas bobinas;
- h) **16 (0x10) – Write Multiple Registers:** escrita em múltiplos registradores.

Esses códigos definem a ação executada e estruturam a comunicação dentro da PDU. No contexto industrial, os holdings registers (função 03 e 16) são amplamente utilizados para troca de variáveis analógicas, como velocidade, temperatura, contadores e parâmetros de controle, enquanto coils e discrete inputs são empregados para estados lógicos, como acionamento de motores ou leitura de sensores.

2.3.3. Solicitação HTTP

A requisição HTTP (Hypertext Transfer Protocol) é um mecanismo de comunicação baseado no modelo de requisição e resposta (LOCKHART, 2015), amplamente

utilizado para troca de informações na web. Nesse modelo, um cliente, como um navegador ou aplicação web, envia uma solicitação (request) a um servidor, que processa a requisição e retorna uma resposta (response). Estas requisições podem utilizar métodos como GET (para solicitar dados), POST (para enviar dados), PUT (para atualizar informações) e DELETE (para remoção de dados), como na representação da Figura 12.



Fonte: Elaborado pelo autor (2026)

No contexto industrial, as requisições HTTP são empregadas principalmente na camada de supervisão e gestão, permitindo integração entre sistemas de automação e plataformas web. Diferentemente de protocolos industriais de tempo real, como Modbus, o HTTP não é utilizado diretamente para controle determinístico de campo, mas sim para envio de dados a servidores.

A aplicação do HTTP na indústria ocorre predominantemente por meio de APIs REST (Representational State Transfer), que utiliza os métodos HTTP para permitir que sistemas heterogêneos conversem. De acordo com Erl, Puttini e Mahmood (2013), essa arquitetura é ideal para a automação ciber-física, pois é leve e utiliza formatos de dados universais, como o JSON (JavaScript Object Notation).

Segundo Lockhart (2015, p. 94), a facilidade de leitura do JSON por seres humanos e a rapidez de processamento por máquinas, facilita a depuração (debug) do sistema e a integração com bancos de dados modernos.

2.4. Computação em nuvem

A Computação em Nuvem (Cloud Computing) representa um modelo de fornecimento de recursos computacionais sob demanda, no qual processamento, armazenamento, redes e aplicações são disponibilizados por meio da internet. Nesse modelo, a

infraestrutura física é virtualizada, permitindo que múltiplos usuários acessem serviços de forma escalável e flexível sem a necessidade de investimentos elevados em hardware local (MARINESCU, 2018).

No contexto industrial, a computação em nuvem é utilizada para armazenamento massivo de dados de processo, integração de sistemas distribuídos, execução de algoritmos complexos de análise e suporte à tomada de decisão em diferentes níveis da organização.

A sinergia entre o hardware local e o processamento remoto cria o que Buyya, Broberg e Goscinski (2011) definem como um ecossistema de computação distribuída, onde a latência é gerenciada na ponta (edge) e a inteligência de larga escala é processada na nuvem.

2.4.1. Integração de sistemas com PHP

No contexto da computação em nuvem, o PHP (Hypertext Preprocessor) destaca-se como uma linguagem amplamente utilizada para o desenvolvimento de aplicações web executadas no lado do servidor. Por ser interpretado no servidor e integrado de forma simples a bancos de dados, o PHP possibilita a construção de sistemas web escaláveis e acessíveis remotamente, característica essencial em arquiteturas baseadas em nuvem (SILBERSCHATZ; GALVIN; GAGNE, 2018).

Lockhart (2015) afirma que o PHP evoluiu de uma simples ferramenta de criação de páginas web para um ecossistema robusto, capaz de gerenciar tarefas assíncronas e interagir com diversos protocolos de rede, permitindo que o PHP execute três funções fundamentais na arquitetura ciber-física:

- a) **Manipulação de Protocolos de Aplicação:** Através de extensões específicas e bibliotecas de rede, o PHP é capaz de realizar a transposição de dados. Ele atua na camada de aplicação para traduzir requisições de protocolos industriais para o formato HTTP, facilitando a subida de dados para o ambiente de nuvem;
- b) **Consumo e Orquestração de APIs:** O PHP é uma ferramenta eficiente para a interação com Web Services. Conforme Nixon (2018), a linguagem possui funções otimizadas para o consumo de APIs RESTful, permitindo que uma aplicação envie conjuntos de dados complexos para serviços de Inteligência Artificial e processamento de linguagem natural hospedados remotamente, tratando as respostas de forma assíncrona;

- c) **Gestão de Interfaces de Software como Serviço (SaaS):** Por ser uma linguagem fundamentada na Web, o PHP facilita a criação de camadas de visualização que residem na nuvem, permitindo a construção de painéis de controle, oferecendo ao usuário uma interface de gerenciamento centralizada e acessível via navegadores.

De acordo com Veras (2016), a escolha do PHP em arquiteturas de computação em nuvem justifica-se pela sua baixa curva de aprendizado aliada a uma vasta biblioteca de funções para segurança e criptografia, garantindo que o tráfego de informações sensíveis entre o ambiente local e o servidor remoto ocorra de forma íntegra e protegida.

2.4.2. Banco de dados MySQL

O MySQL é um Sistema de Gerenciamento de Banco de Dados (SGBD) relacional amplamente utilizado em aplicações web e sistemas distribuídos. Baseado no modelo relacional proposto por Codd, o MySQL organiza os dados em tabelas estruturadas, compostas por linhas e colunas, permitindo o estabelecimento de relacionamentos entre diferentes entidades por meio de chaves primárias e estrangeiras, sendo a manipulação dos dados realizada por meio da linguagem SQL (Structured Query Language), padrão amplamente difundido para definição, consulta e controle de informações em bancos relacionais (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Segundo Nixon (2018), o MySQL destaca-se pela sua integração eficiente com aplicações web, especialmente quando utilizado em conjunto com linguagens de script do lado do servidor. Sua arquitetura cliente-servidor permite que múltiplos usuários realizem operações simultaneamente, mantendo a integridade e a consistência das informações armazenadas. Além disso, o sistema oferece suporte a mecanismos de controle transacional, índices para otimização de consultas e recursos de segurança baseados em autenticação e privilégios de acesso.

A hospedagem deste Banco de Dados em ambiente de nuvem confere ao sistema escalabilidade vertical e horizontal, permitindo que o volume de dados cresça sem a necessidade de substituição do hardware físico local, mantendo a interoperabilidade necessária para sistemas que operam sob os paradigmas da Indústria 4.0 (VERAS, 2016).

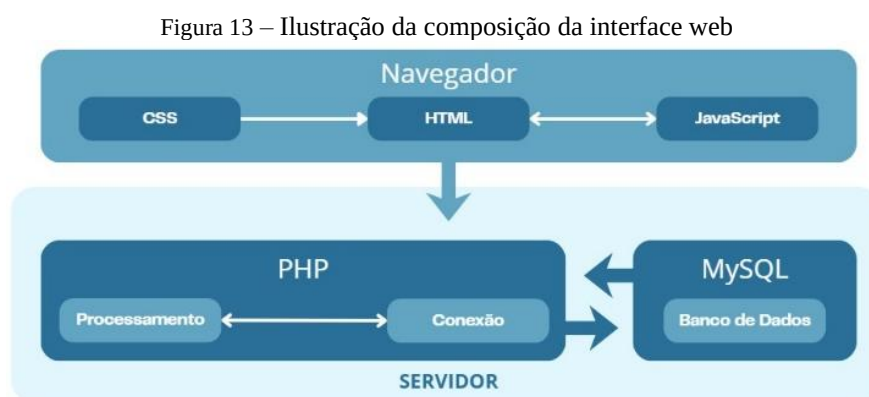
Como ferramenta de gerenciamento do banco de dados MySQL, o phpMyAdmin fornece uma interface web que permite administrar bancos de dados relacionais sem a necessidade de realizar todas as operações diretamente por meio do terminal. A ferramenta

possibilita a criação e a manutenção de bancos de dados, tabelas e usuários, além da execução de comandos SQL, inserção, consulta, alteração e exclusão de registros. De acordo com a documentação oficial do phpMyAdmin, a aplicação foi desenvolvida para oferecer uma interface baseada na web para a administração do MySQL e de outros sistemas compatíveis, facilitando tarefas relacionadas ao gerenciamento e à manutenção das informações armazenadas (THE phpMyAdmin DEVELOPMENT TEAM, 2026).

2.4.3. Interface web: HTML, CSS e JavaScript

O desenvolvimento de interfaces modernas para sistemas de monitoramento baseia-se em padrões web que garantem a acessibilidade e a interatividade da aplicação. Segundo Nixon (2018), essas três tecnologias trabalham de forma integrada, HTML (HyperText Markup Language) define a estrutura e o conteúdo da página, o CSS (Cascading Style Sheets) é responsável pela formatação visual e pelo layout responsivo, e o JavaScript atua como a camada de comportamento e lógica no cliente.

O HTML é responsável pela visualização do conteúdo da página, organizando elementos como títulos, parágrafos, tabelas, formulários e links, definindo a semântica da informação apresentada ao usuário e servindo como base estrutural da aplicação (NIXON, 2018). O CSS por sua vez, é utilizado para definir a apresentação visual dos elementos estruturados em HTML, é possível controlar cores, tipografia, espaçamento, posicionamento e responsividade da interface, permitindo a adaptação da aplicação a diferentes dispositivos e resoluções de tela. O JavaScript é uma linguagem de programação de alto nível que permite a criação de conteúdos dinâmicos que se atualizam sem a necessidade de recarregar a página inteira (FLANAGAN, 2020). A associação dessas linguagens é ilustrada na Figura 13.



Fonte: Elaborado pelo autor (2026)

A execução de scripts em JavaScript no lado do cliente (client-side) permite que a interface de supervisão seja mais do que um documento estático. Conforme destaca Haverbeke (2018), o uso de scripts possibilita:

- a) Manipulação do DOM: Altera dinamicamente elementos visuais do HTML em resposta a novos dados;
- b) Comunicação Assíncrona (AJAX/Fetch): Realiza a troca de dados com servidores e bancos de dados em segundo plano, mantendo a interface ativa;
- c) Tratamento de Dados: Processa fluxos de informações e gera visualizações gráficas diretamente no navegador, otimizando o desempenho do sistema.

Conforme aponta Nixon (2018), essa arquitetura permite que a interface de usuário atue como um terminal inteligente, integrando-se de forma eficiente aos serviços de nuvem e processando dados estruturados de maneira ágil.

2.5. Inteligência Artificial

A Inteligência Artificial (IA) pode ser definida como o campo da ciência da computação dedicado ao desenvolvimento de sistemas capazes de executar tarefas que, tradicionalmente, exigiriam inteligência humana, tais como reconhecimento de padrões, tomada de decisão, aprendizado e resolução de problemas. A área envolve o estudo de métodos computacionais que permitem a construção de agentes capazes de perceber o ambiente, processar informações e agir de maneira racional para alcançar determinados objetivos (RUSSELL; NORVIG, 2021).

No contexto da engenharia e dos sistemas industriais, a Inteligência Artificial tem sido aplicada para otimização de processos, detecção de falhas, previsão de demanda, controle adaptativo e automação inteligente. Diferentemente dos sistemas tradicionais baseados em lógica fixa, os sistemas inteligentes possuem a capacidade de aprender a partir de dados históricos e ajustar seu comportamento diante de novas situações, aumentando a flexibilidade e a eficiência operacional.

Historicamente, a evolução da Inteligência Artificial está associada ao avanço do poder computacional, à disponibilidade de grandes volumes de dados e ao desenvolvimento de métodos matemáticos mais robustos. Ao longo do tempo, a área passou de sistemas baseados predominantemente em regras lógicas e heurísticas para abordagens fundamentadas em

aprendizado a partir de dados, ampliando significativamente sua capacidade de adaptação e generalização (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.5.1. Aprendizado de máquina

O avanço da IA é impulsionado pelo Aprendizado de Máquina (Machine Learning), que permite aos sistemas melhorar seu desempenho por meio da experiência. Segundo Goodfellow, Bengio e Courville (2016), esse paradigma possibilita que o software extraia padrões a partir de dados brutos, permitindo que o sistema tome decisões baseadas em tendências históricas e feedbacks de performance, essa capacidade é essencial para a manutenção da eficiência em ambientes complexos, pois o sistema pode se adaptar a novas condições de trabalho sem a necessidade de intervenção humana constante para reprogramação de parâmetros. De maneira geral, o aprendizado de máquina pode ser classificado em três categorias principais:

- a) **Aprendizado Supervisionado:** No aprendizado supervisionado, o algoritmo é treinado a partir de um conjunto de dados rotulados, isto é, cada entrada possui uma saída desejada associada. O objetivo é aprender uma função que mapeie entradas em saídas, minimizando o erro entre a predição do modelo e o valor esperado. Esse tipo de aprendizado é amplamente utilizado em problemas de classificação e regressão (GOODFELLOW; BENGIO; COURVILLE, 2016, p. 105). O processo de treinamento envolve a definição de uma função de custo e a atualização iterativa dos parâmetros do modelo, buscando otimizar seu desempenho com base nos exemplos fornecidos;
- b) **Aprendizado Não Supervisionado:** O aprendizado não supervisionado é aplicado quando os dados disponíveis não possuem rótulos associados. Nesse caso, o objetivo do algoritmo é identificar estruturas ocultas, padrões ou agrupamentos existentes nos dados. Técnicas desse tipo são empregadas para análise exploratória, segmentação e redução de dimensionalidade (RUSSELL; NORVIG, 2021, p. 720). Diferentemente do aprendizado supervisionado, não há uma saída explícita a ser comparada durante o treinamento; o modelo busca representar a organização interna dos dados com base em critérios estatísticos ou probabilísticos;

c) **Aprendizado por Reforço:** Diferente dos modelos anteriores, este paradigma baseia-se na interação dinâmica entre um agente e um ambiente. Segundo Sutton e Barto (2018), o aprendizado ocorre através de um ciclo de tentativa e erro, onde o agente recebe recompensas ou punições de acordo com as ações tomadas para atingir um objetivo específico. Essa abordagem é ideal para o controle adaptativo, pois permite que o sistema desenvolva estratégias ótimas em ambientes onde as respostas corretas não são conhecidas previamente, mas podem ser avaliadas por um sinal de desempenho.

A integração desses diferentes tipos de aprendizado possibilita que sistemas complexos executem, de forma combinada, desde a previsão antecipada de falhas até a tomada de decisão autônoma em tempo real, adaptando-se dinamicamente às variações e incertezas inerentes aos processos produtivos modernos, caracterizados por elevado grau de conectividade e volume de dados (SUTTON; BARTO, 2018).

2.5.2. IA Generativa

A Inteligência Artificial Generativa representa uma mudança de paradigma no campo da ciência da computação, movendo-se da análise de dados para a criação de novos conteúdos originais por meio de arquiteturas neurais profundas (GOODFELLOW; BENGIO; COURVILLE, 2016). Diferente dos modelos preditivos tradicionais, a IA generativa utiliza arquiteturas neurais profundas para sintetizar informações que imitam a produção humana em diversos formatos, como texto, imagem, áudio e código-fonte.

O avanço dessa área está diretamente relacionado ao desenvolvimento de arquiteturas neurais profundas, especialmente os modelos baseados em mecanismos de atenção e na arquitetura Transformer, que permitem capturar dependências de longo alcance em sequências de dados (VASWANI et al., 2017, p. 3). Esses modelos viabilizaram a criação dos chamados Modelos de Linguagem de Grande Escala (Large Language Models – LLMs), treinados com grandes volumes de textos para compreender e gerar linguagem natural de forma contextualizada.

A aplicação prática dessas inteligências em sistemas de software modernos ocorre predominantemente por meio de Interfaces de Programação de Aplicações (APIs). Uma API atua como um contrato de comunicação que permite a um sistema externo enviar dados para o modelo hospedado na nuvem e receber uma resposta processada. Segundo Buntak et al. (2021,

p. 154), o uso de APIs de IA permite a democratização de recursos computacionais de alto nível, uma vez que o processamento pesado ocorre na infraestrutura do provedor, e não no hardware local.

Atualmente, diferentes provedores disponibilizam LLMs de propósito geral acessíveis via API, cada um baseado em arquiteturas e estratégias de treinamento próprias, ainda que compartilhem o mesmo fundamento teórico do modelo Transformer. O ChatGPT, desenvolvido pela OpenAI é fundamentado na arquitetura GPT (Generative Pre-trained Transformer), baseada em modelos autorregressivos treinados para prever a próxima palavra em uma sequência textual, ajustados posteriormente com técnicas de alinhamento por reforço com feedback humano (RUSSELL; NORVIG, 2021, p. 1023). De forma semelhante, o Gemini, desenvolvido pelo Google DeepMind, adota uma arquitetura nativamente multimodal, projetada desde sua concepção para processar e relacionar diferentes tipos de dados dentro de um único modelo (GEMINI TEAM; GOOGLE, 2023).

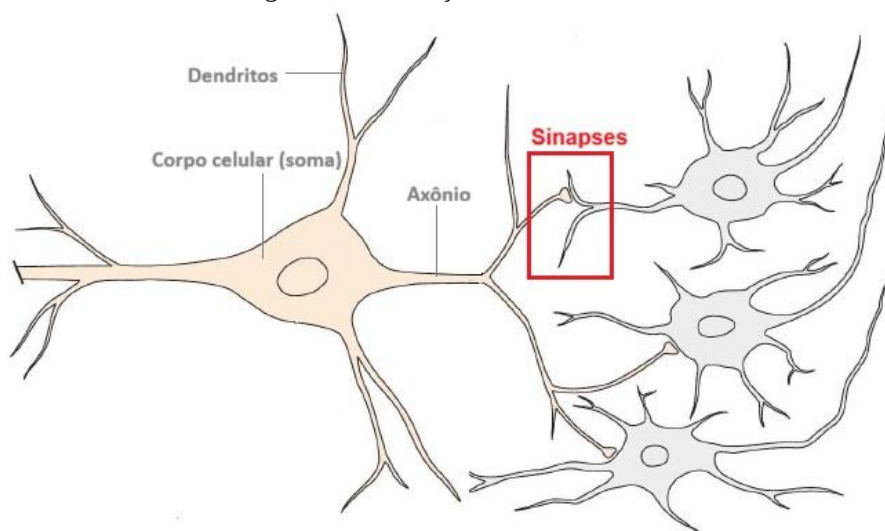
Apesar dos avanços expressivos, a adoção de modelos de IA generativa em sistemas de produção exige atenção a algumas limitações inerentes a essa tecnologia. Modelos generativos estão sujeitos ao fenômeno de alucinação, no qual o sistema produz informações plausíveis, porém factualmente incorretas ou não sustentadas pelos dados de entrada fornecidos, o que reforça a necessidade de validação humana sobre relatórios e diagnósticos gerados automaticamente, especialmente em aplicações industriais nas quais decisões operacionais podem ser influenciadas por essas análises (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.6. Rede Neural Artificial

As Redes Neurais Artificiais (RNAs) são modelos computacionais inspirados na estrutura e no funcionamento do sistema nervoso biológico, fundamenta-se na tentativa de reproduzir, de forma simplificada, o mecanismo de processamento de informações realizado pelos neurônios naturais (HAYKIN, 2001).

O neurônio biológico é composto basicamente por três estruturas principais: dendritos, corpo celular (soma) e axônio, como visto na Figura 14. Os dendritos recebem estímulos elétricos provenientes de outros neurônios, enquanto o corpo celular realiza a integração desses sinais, por fim, o axônio transmite o impulso elétrico resultante para outros neurônios por meio das sinapses. O disparo ocorre quando o potencial elétrico interno ultrapassa determinado limiar, caracterizando um tipo de comportamento não linear (KANDEL et al., 2014).

Figura 14 – Ilustração de um neurônio



Fonte: Adaptado de KANDEL (2014)

De forma análoga, o neurônio artificial é um modelo matemático simplificado composto por entradas ponderadas, um termo de polarização (bias) e uma função de ativação. As entradas representam os sinais recebidos, enquanto os pesos sinápticos modelam a intensidade das conexões, sendo que o somatório ponderado representa a integração dos estímulos, finalizando com a função de ativação que define a saída do neurônio, frequentemente introduzindo não linearidade ao sistema (RUSSELL; NORVIG, 2021).

2.6.1. Perceptron

O Perceptron, introduzido por Frank Rosenblatt em 1958, representa o marco inicial das Redes Neurais Artificiais modernas, sendo matematicamente, um classificador linear que decide se uma entrada pertence a uma classe específica ou não, baseando-se em uma combinação linear de seus componentes, como uma abstração matemática do comportamento observado no neurônio biológico (ROSENBLATT, 1958).

Originalmente concebido como um modelo capaz de simular processos de aprendizado perceptivo, o Perceptron foi um dos primeiros algoritmos a demonstrar que uma máquina poderia ajustar seus próprios parâmetros internos a partir da experiência, sem a necessidade de programação explícita de regras para cada situação, o que o tornou um marco fundador não apenas das redes neurais, mas do próprio campo do aprendizado de máquina (HAYKIN, 2001).

Considere um conjunto de sinais de entrada $x_1, x_2, \dots, x_n$, associados a pesos sinápticos $w_1, w_2, \dots, w_n$, os quais quantificam a relevância de cada informação para o

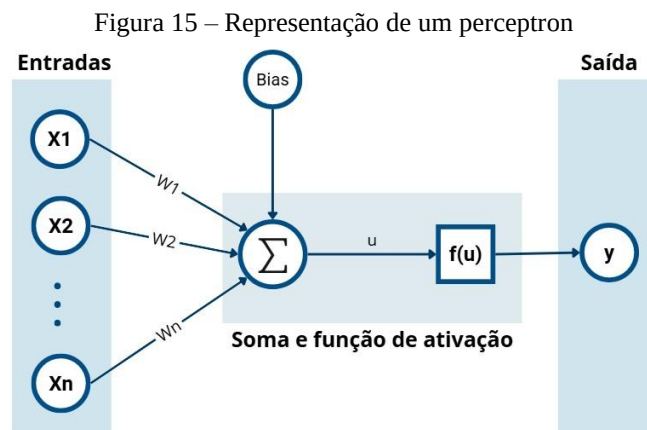
neurônio. A esses sinais é adicionado um termo de polarização b , também denominado viés (bias), responsável por ajustar o limiar de ativação do neurônio, garantindo que este seja capaz de disparar mesmo quando as entradas são nulas (HAYKIN, 2001):

$$u = \sum_{i=1}^n w_i x_i + b \quad (1)$$

Onde:

- u é o potencial de ativação;
- w_i são os pesos sinápticos;
- x_i são as variáveis de entrada;
- b é o bias.

O potencial de ativação u é aplicado a uma função de ativação $f(u)$, produzindo a saída do neurônio (Figura 15). Essa função é o componente responsável por transformar um valor real, resultante da soma ponderada das entradas em uma saída delimitada e interpretável para o restante da rede. Sem esse componente, o neurônio se comportaria como uma simples combinação linear de suas entradas, e uma rede composta exclusivamente por camadas lineares, por mais numerosas que fossem, continuaria sendo equivalente, do ponto de vista matemático, a um único modelo linear, incapaz de representar relações mais complexas presentes nos dados (GOODFELLOW; BENGIO; COURVILLE, 2016).



Fonte: Elaborado pelo autor (2026)

É precisamente a não linearidade introduzida pela função de ativação que confere à rede neural sua capacidade de aproximar funções arbitrariamente complexas, permitindo que o

modelo capture padrões que não seguem uma relação estritamente proporcional entre entradas e saídas. Essa propriedade fundamenta o chamado Teorema da Aproximação Universal, segundo o qual uma rede neural com pelo menos uma camada oculta e funções de ativação não lineares é capaz de aproximar qualquer função contínua com o grau de precisão desejado, característica indispensável para a resolução de problemas de classificação e de controle não linear (HAYKIN, 2001).

2.6.2. Perceptron multicamadas

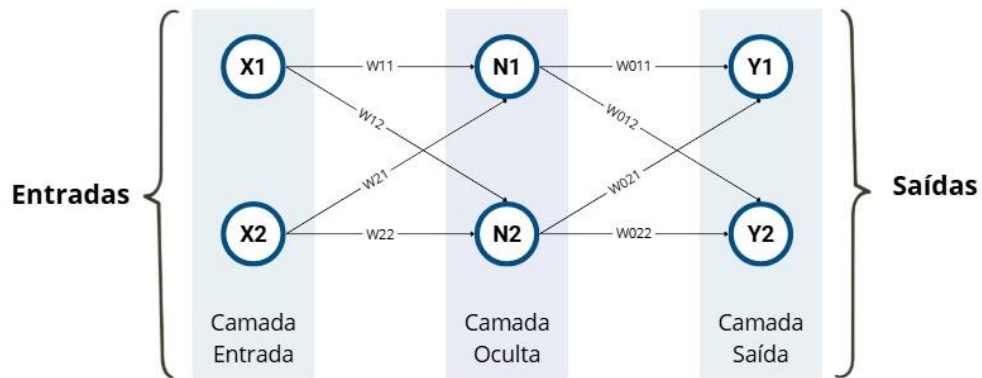
Embora o Perceptron de Rosenblatt tenha sido revolucionário, ele possuía uma limitação teórica severa: a incapacidade de resolver problemas que não fossem linearmente separáveis. O exemplo clássico dessa limitação é o problema da porta lógica XOR (OU-Exclusivo), demonstrado por Minsky e Papert em 1969, o que levou a um período de estagnação nas pesquisas da área (FACELI et al., 2021).

Para superar tais limitações, surgiram as Redes Neurais Multicamadas (Multilayer Perceptron - MLP). Diferente de seu antecessor, a MLP organiza os neurônios em camadas hierárquicas, permitindo que a rede aprenda representações de dados cada vez mais complexas à medida que a informação avança pelas camadas. Cada camada sucessiva constrói suas representações a partir das características já extraídas pela camada anterior, em vez de operar diretamente sobre os dados brutos de entrada (GOODFELLOW; BENGIO; COURVILLE, 2016). Uma rede MLP padrão é composta por:

- a) **Camada de Entrada:** Apenas repassa os dados para a próxima camada;
- b) **Camadas Ocultas (Hidden Layers):** Onde ocorre o processamento intermediário. É aqui que a rede extrai características não lineares;
- c) **Camada de Saída:** Onde o resultado é consolidado.

O processo de soma ponderada das entradas é aplicado de forma independente a cada conexão entre camadas, sendo repetido tantas vezes quanto for o número de camadas da arquitetura. Dessa forma, uma rede MLP pode ser entendida como uma composição sucessiva de transformações do tipo perceptron, cada uma seguida de sua respectiva função de ativação, na qual a saída esperada de uma camada torna-se a entrada da camada seguinte (GOODFELLOW; BENGIO; COURVILLE, 2016).

Figura 16 – Perceptron multicamada



Fonte: Elaborado pelo autor (2026)

As MLPs utilizam funções de ativação contínuas e diferenciáveis, pois, a presença dessas funções, aliada à profundidade das camadas, permite que a rede atue como uma aproximação universal de funções, sendo capaz de modelar relações arbitrariamente complexas entre entradas e saídas (HAYKIN, 2001).

Segundo o Teorema de Cybenko (1989), também conhecido como Teorema da Aproximação Universal, uma rede MLP com pelo menos uma camada oculta e neurônios com funções de ativação não lineares pode aproximar qualquer função contínua definida em um domínio compacto, com o grau de precisão desejado, desde que o número de neurônios da camada oculta seja suficientemente grande (HAYKIN, 2001).

2.6.3. Propagação direta (forward pass)

O forward pass (propagação direta) é a etapa do processamento em redes neurais artificiais na qual os dados de entrada são propagados camada por camada até a obtenção da saída da rede. Nesse processo, cada neurônio realiza o cálculo do seu potencial de ativação por meio de uma combinação linear ponderada das entradas, seguida da aplicação de uma função de ativação não linear. Essa etapa representa a fase de inferência da rede, sendo responsável por produzir a resposta do modelo para determinado conjunto de dados (HAYKIN, 2001).

Matematicamente, para uma camada intermediária, o processamento pode ser descrito como o somatório (z) da multiplicação do vetor de entrada (X) pelos pesos sinápticos (W), acrescida de um termo de bias (B):

$$z^{[n]} = W^{[n]} \cdot X^{[n]} + B^{[n]} \quad (2)$$

Onde:

$$X^{[n]} = \begin{bmatrix} x_1^{[n]} \\ x_2^{[n]} \\ \vdots \\ x_i^{[n]} \end{bmatrix} \quad B^{[n]} = \begin{bmatrix} b_1^{[n]} \\ b_2^{[n]} \\ \vdots \\ b_j^{[n]} \end{bmatrix} \quad W^{[k]} = \begin{bmatrix} w_{11}^{[n]} & w_{12}^{[n]} & \cdots & w_{1i}^{[n]} \\ w_{21}^{[n]} & w_{22}^{[n]} & \cdots & w_{2i}^{[n]} \\ \vdots & \vdots & \ddots & \vdots \\ w_{j1}^{[n]} & w_{j2}^{[n]} & \cdots & w_{ji}^{[n]} \end{bmatrix}$$

Em seguida, é realizada a aplicação da função de ativação sobre o resultado da combinação linear das entradas ponderadas, de modo que o valor obtido (Y) passa a compor o sinal de saída daquele neurônio, tornando-se, por sua vez, a entrada da camada seguinte. Em redes do tipo MLP, esse mecanismo de propagação ocorre de maneira sequencial, unidirecional e sem realimentação, isto é, a informação flui exclusivamente da camada de entrada em direção à camada de saída, sem que haja retorno dos sinais para camadas anteriores (SILVA; SPATTI; FLAUZINO, 2016). De forma analítica, esse processo pode ser representado por:

$$Y^{[n]} = f(z^{[n]}) \quad (3)$$

Onde:

$$Y^{[n]} = \begin{bmatrix} f(z_1^{[n]}) \\ f(z_2^{[n]}) \\ \vdots \\ f(z_k^{[n]}) \end{bmatrix}$$

O forward pass é fundamental tanto na fase de treinamento quanto na fase de operação do sistema. Durante o treinamento supervisionado, ele é utilizado para calcular a saída estimada da rede, que posteriormente será comparada ao valor desejado para determinação do erro. Já na fase de aplicação prática (inferência), o processo de propagação direta é executado isoladamente, sem atualização de pesos, fornecendo as decisões ou previsões do modelo treinado (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.6.4. Funções de ativação

A função de ativação desempenha um papel crítico na arquitetura de uma rede neural, atuando como um mecanismo de decisão que determina se um neurônio deve ser "ativado" ou não. Sua principal função é introduzir não-linearidade no modelo, sem essa característica, a rede se comportaria como uma transformação linear única, tornando-a incapaz de aprender

dados complexos (FACELI et al., 2021). Do ponto de vista matemático, considerando o potencial interno, a saída do neurônio é definida como:

$$y = f(u) \quad (4)$$

A escolha da função de ativação influencia diretamente a capacidade de aprendizado da rede, a estabilidade numérica, a propagação do gradiente durante o treinamento e o tipo de problema. Entre as funções mais utilizadas estão a sigmoide, tangente hiperbólica e softmax, sendo cada uma adequada a contextos específicos de aplicação (HAYKIN, 2001).

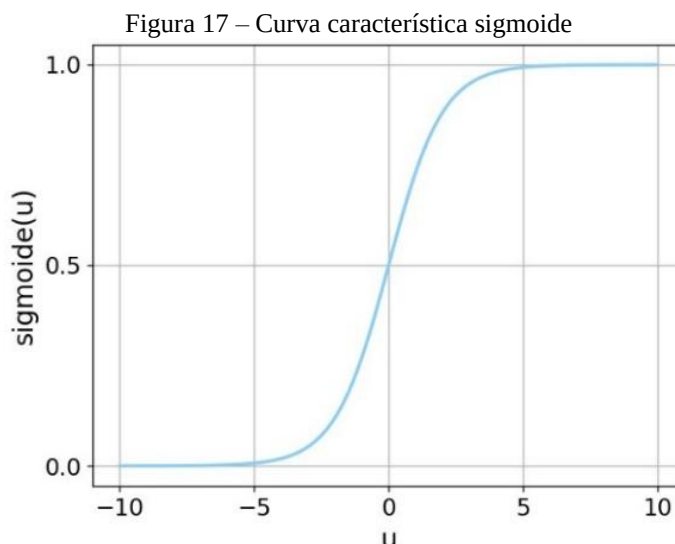
2.6.4.1. Sigmoide

Historicamente, a função Sigmoide (ou logística) foi a mais utilizada em Redes Multicamadas devido à sua semelhança com o comportamento de neurônios biológicos e sua natureza diferenciável. Sua formulação matemática é dada por:

$$\text{sigmoide}(u) = \frac{1}{1 + e^{-u}} \quad (5)$$

A Sigmoide mapeia qualquer valor real para um intervalo entre 0 e 1, o que a torna particularmente útil para modelos que precisam prever probabilidades. Essa ampla adoção também encontra respaldo teórico no Teorema da Aproximação Universal, demonstrado por Cybenko (1989), segundo o qual uma rede Perceptron Multicamadas com apenas uma camada oculta e funções de ativação sigmoidais é capaz de aproximar qualquer função contínua definida em um subconjunto compacto do espaço euclidiano, desde que disponha de neurônios suficientes na camada oculta, resultado que sustentou por muitos anos sua utilização como função padrão nesse tipo de arquitetura.

Entretanto, conforme destacado por Haykin (2001), ela apresenta o problema do Desvanecimento do Gradiente (Vanishing Gradient), ou seja, para valores de entrada muito altos ou muito baixos, a derivada da função torna-se próxima de zero, o que interrompe o aprendizado em redes muito profundas, sendo esse um dos motivos pelos quais funções alternativas, tornaram-se mais populares em arquiteturas mais profundas.



A Figura 17 mostra o formato da curva característica da função sigmoide. Apesar de suas limitações, a função sigmoide permanece relevante em camadas de saída para problemas de classificação binária.

2.6.4.2. Tangente hiperbólica

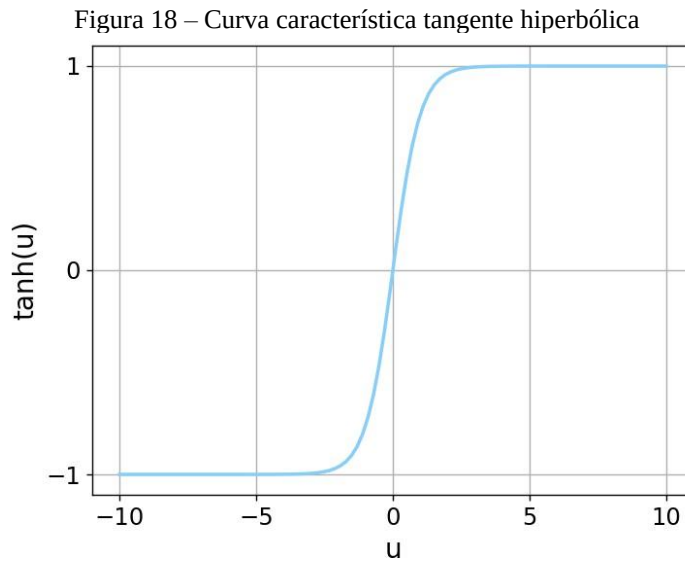
A função Tangente Hiperbólica ($\tanh$) é uma função não linear que apresenta uma forma de "S" semelhante à Sigmoide, porém com uma diferença fundamental em seu intervalo de saída, enquanto a Sigmoide varia entre 0 e 1, a $\tanh$ mapeia as entradas para um intervalo entre -1 e 1. Sua expressão matemática é definida por:

$$\tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} \quad (6)$$

De acordo com Goodfellow, Bengio e Courville (2016), a principal vantagem da $\tanh$ é que ela é centrada no zero, isso significa que a média das saídas dos neurônios em uma camada tende a ser próxima de zero, o que facilita e acelera o processo de convergência durante o treinamento via gradiente descendente.

No caso da Sigmoide, como todas as saídas são positivas, os gradientes dos pesos durante o treinamento tendem a assumir todos o mesmo sinal, o que pode causar um movimento de "zigue-zague" ineficiente na atualização dos parâmetros, enquanto a $\tanh$ resolve esse problema ao permitir valores negativos (HAYKIN, 2001).

Diferentemente da sigmoide, a tangente hiperbólica mapeia os valores de entrada para o intervalo de -1 a 1, sendo centrada em zero, como visto na Figura 18.



Fonte: Elaborado pelo autor (2026)

A $\tanh$ também sofre do problema de saturação, ainda assim é frequentemente utilizada em camadas ocultas de redes neurais tradicionais, especialmente quando se deseja uma ativação simétrica em torno da origem, contribuindo para maior estabilidade no processo de treinamento.

2.6.4.3. Softmax

A função Softmax é uma função de ativação vetorial, aplicada geralmente à última camada de uma rede neural projetada para classificação multiclasse. Sua função primordial é converter um vetor de pontuações brutas (chamados de logits) em uma distribuição de probabilidade discreta (GOODFELLOW; BENGIO; COURVILLE, 2016). Considerando um vetor de ativações $z = (z_1, z_2, \dots, z_n)$, a saída para cada classe i é dada por:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad (7)$$

Essa função transforma o vetor de entradas em uma distribuição de probabilidade, garantindo que a soma de todos os componentes do vetor de saída seja exatamente igual a 1. A softmax pode ser interpretada como um modelo de regressão logística multinomial, no qual

modela a probabilidade condicional de uma classe, dado o vetor de entrada, sendo amplamente utilizada em tarefas de classificação supervisionada (RUSSELL; NORVIG, 2021).

2.6.5. Função de Erro

O cálculo do erro é uma etapa fundamental no processo de treinamento das redes neurais artificiais, com objetivo em ajustar os parâmetros do modelo (pesos e vieses) de modo a minimizar uma função de custo, que quantifica a diferença entre a saída produzida pela rede e o valor desejado.

Considere uma saída prevista $\hat{y}$ e um valor alvo y . A função de erro $E(\hat{y}, y)$ mede essa discrepância, servindo como base para o cálculo do gradiente utilizado no algoritmo de treinamento (HAYKIN, 2001).

Segundo Goodfellow, Bengio e Courville (2016), a escolha da função de perda está intrinsecamente ligada à arquitetura da camada de saída e à natureza do problema. O objetivo do algoritmo de aprendizagem é encontrar um conjunto de pesos e bias que minimize o valor médio dessa perda para todos os exemplos de treino.

A função de erro é uma superfície multidimensional que mapeia o desempenho da rede em relação aos seus pesos. Para que o aprendizado ocorra via métodos baseados em gradiente, essa função deve possuir propriedades matemáticas específicas, sendo a diferenciabilidade a mais crucial delas, assim garante que pequenas mudanças nos pesos resultem em mudanças previsíveis e graduais no erro, permitindo um ajuste fino (FACELI et al., 2021).

2.6.5.1. Entropia cruzada binária

Quando a camada de saída utiliza a função sigmoide, o problema geralmente envolve classificação binária. Nesse cenário, funções de perda mais tradicionais tendem a apresentar dificuldades de convergência devido ao comportamento da derivada da sigmoide, que se torna próxima de zero nas regiões de saturação da função, resultando em atualizações de peso muito pequenas durante as primeiras iterações do treinamento (BISHOP, 2006).

Para se obter maior estabilidade numérica e uma convergência mais eficiente durante o processo de treinamento da rede, especialmente em problemas de classificação, adota-se de forma mais adequada a função de entropia cruzada binária E , a qual penaliza com maior intensidade as previsões que se afastam significativamente do valor real esperado, favorecendo

o ajuste mais preciso dos pesos sinápticos durante o processo de treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016). Essa função é definida como:

$$E = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (8)$$

Do ponto de vista intuitivo, a entropia cruzada aumenta significativamente quando o modelo está confiante e errado, incentivando ajustes mais expressivos nos pesos. Por outro lado, quando a previsão está correta e próxima do valor alvo, o erro diminui suavemente. Essa formulação apresenta vantagens importantes ao calcular a derivada da perda em relação ao potencial u , uma vez que o gradiente resultante é proporcionalmente maior justamente nos casos em que a correção é mais necessária (BISHOP, 2006). A expressão simplificada é definida como:

$$\frac{\partial E}{\partial \hat{y}} = \hat{y} - y \quad (9)$$

Essa simplificação matemática é particularmente relevante para o algoritmo de treinamento, uma vez que reduz significativamente a complexidade computacional envolvida no cálculo dos gradientes durante a retropropagação do erro, além de contribuir para uma maior estabilidade numérica e eficiência do processo de aprendizado da rede (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.6.5.2. Entropia cruzada categórica

Para problemas de classificação multiclasse que utilizam a função Softmax na camada de saída, a função de perda padrão é a Entropia Cruzada Categórica, a qual constitui uma generalização direta da entropia cruzada binária para cenários com mais de duas classes mutuamente exclusivas. Assim como sua contraparte binária, essa função tem como objetivo mensurar a "distância" entre duas distribuições de probabilidade, a distribuição prevista pela rede $\hat{y}_i$ e a distribuição real y_i , esta última normalmente representada por um vetor one-hot, no qual a classe correta recebe probabilidade igual a 1, enquanto todas as demais classes recebem probabilidade igual a 0, no qual, quanto maior a divergência entre a distribuição prevista e a distribuição real, maior será o valor da função de perda, orientando o processo de

ajuste dos pesos sinápticos durante o treinamento no sentido de minimizar esse erro (FACELI et al., 2021). Desta forma, a função de erro (E) é definida como:

$$E = - \sum_{i=1}^k y_i \log \hat{y}_i \quad (10)$$

Como apenas a classe correta contribui de fato para o somatório, uma vez que y_i é igual a zero para todas as demais classes, o valor da função de erro depende, na prática, exclusivamente da probabilidade atribuída pelo modelo à classe verdadeira, sendo tanto menor quanto mais próxima de 1 for essa probabilidade.

A principal vantagem matemática é que, ao derivar essa função de perda em relação aos logits (z) da camada anterior, a exponencial da Softmax e o logaritmo se "cancelam", resultando em um gradiente linear e muito mais estável, assim como ocorre na Entropia Cruzada Binária. Este resultado indica que o ajuste dos pesos é proporcional apenas à diferença direta entre a probabilidade prevista e o valor real (Equação 9), eliminando problemas de saturação do gradiente e acelerando drasticamente a convergência do modelo (BENGIO, 2009).

2.6.5.3. Erro Quadrático Médio

O Erro Quadrático Médio (Mean Squared Error – MSE) é uma métrica estatística amplamente utilizada para quantificar a magnitude do erro entre um conjunto de valores previstos e seus respectivos valores reais, no qual o objetivo é medir o quão distante, em média, as estimativas de um modelo estão dos dados observados (BISHOP, 2006). Sua formulação calcula a média dos quadrados das diferenças entre o valor previsto e o valor desejado para cada exemplo de um conjunto de dados:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (11)$$

Dessa forma, o MSE constitui uma medida importante para a avaliação do desempenho de modelos preditivos, uma vez que permite representar numericamente a diferença entre os valores estimados e os valores reais. Por utilizar o quadrado dos erros, a

métrica atribui maior peso às diferenças de maior magnitude, tornando mais evidente a ocorrência de previsões significativamente distantes dos valores observados. Assim, valores menores de MSE indicam, de modo geral, uma maior proximidade entre as previsões e os dados reais, contribuindo para uma melhor visualização e interpretação da qualidade dos resultados obtidos pelo modelo (JAMES et al., 2021).

2.6.6. Back propagation

O algoritmo de Back propagation, popularizado por Rumelhart, Hinton e Williams em 1986, é o "coração" do treinamento de redes neurais artificiais. Ele permite que o erro calculado na camada de saída seja propagado de volta por todas as camadas ocultas, permitindo que cada peso e viés da rede seja ajustado de forma a minimizar a função de perda global (HAYKIN, 2001).

O funcionamento do algoritmo baseia-se no cálculo sistemático de derivadas parciais, sendo dividido em duas fases principais: o fluxo direto (Forward Pass) e o fluxo reverso (Backward Pass).

No fluxo direto, os dados de entrada percorrem a rede camada por camada e ao final do processo, obtém-se a saída prevista pela rede, no qual é comparada com o valor desejado, e calcula-se a função de erro, que quantifica a discrepância entre o resultado obtido e o esperado, a partir desses dados é aplicado o fluxo reverso.

2.6.6.1. Retro propagação (backward pass)

O verdadeiro poder do algoritmo reside no fluxo reverso em que o erro calculado na saída é propagado da última camada em direção às camadas anteriores, sendo que o princípio fundamental deste algoritmo consiste em determinar quanto cada peso contribuiu para o erro final (RUMELHART; HINTON; WILLIAMS, 1986).

Para ajustar um peso (w) localizado em uma camada profunda, precisamos saber como uma pequena alteração nesse peso afeta o erro final (E). Matematicamente, isso é representado pela derivada parcial, e como a rede é uma composição de múltiplas funções (camadas), utiliza-se a Regra da Cadeia do cálculo diferencial para decompor essa derivada, permitindo calcular a contribuição de cada peso de forma sistemática, camada por camada, sem

a necessidade de recalculá-lo de zero a cada iteração (HAYKIN, 2001). Para um peso w_{ij} que conecta o neurônio i ao neurônio j , o gradiente é calculado como:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial u_j} \cdot \frac{\partial u_j}{\partial w_{ij}} \quad (12)$$

Onde:

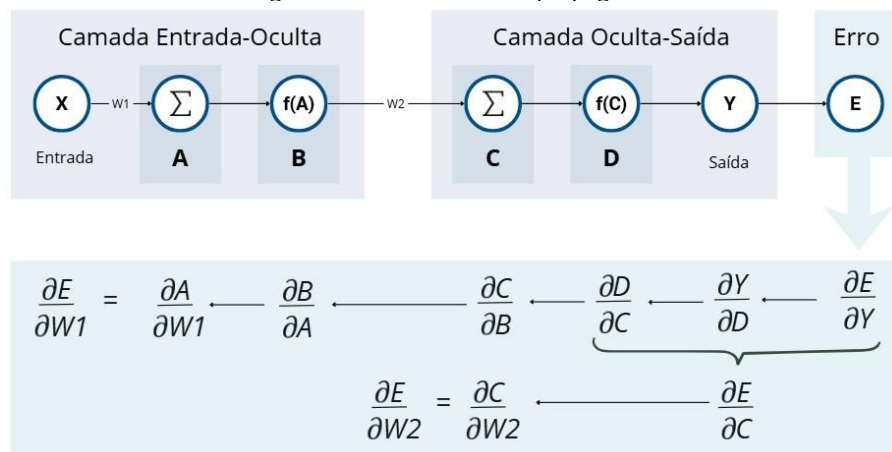
$\frac{\partial E}{\partial \hat{y}}$ é a sensibilidade do erro em relação à saída;

$\frac{\partial \hat{y}}{\partial u_j}$ é a derivada da função de ativação;

$\frac{\partial u_j}{\partial w_{ij}}$ é o valor da entrada (sinal) que passou por aquele peso.

A sensibilidade do erro em relação à saída de uma camada oculta não é calculado diretamente a partir da função de perda, mas sim propagado a partir das sensibilidades já calculadas nas camadas posteriores, multiplicadas pelos respectivos pesos que conectam essas camadas. É justamente essa propagação recursiva do erro, camada por camada, em sentido contrário, que confere ao algoritmo sua eficiência computacional, permitindo calcular o gradiente de todos os pesos da rede em uma única chamada reversa, em vez de recalculá-la de forma independente (GOODFELLOW; BENGIO; COURVILLE, 2016).

Figura 19 – Fluxo do back propagation



Fonte: Elaborado pelo autor (2026)

Como visto na Figura 19, o processo de atualização de cada peso ocorre de forma individualizada, pois cada parâmetro contribui de maneira distinta para o erro final da rede.

Assim, o ajuste deve considerar o caminho específico de propagação do erro através das camadas, respeitando a influência particular de cada conexão no resultado global.

2.6.6.2. Ajuste do bias

No algoritmo de backpropagation, o bias é tratado como um peso adicional (w_b) associado a uma entrada constante de valor unitário. Dessa forma, seu ajuste é realizado pelo mesmo processo empregado para os demais pesos da rede (HAYKIN, 2009). De forma equivalente tem-se:

$$b = w_b \cdot 1 \quad (13)$$

Como a entrada associada ao bias possui valor constante igual a 1, o cálculo depende apenas do erro propagado do neurônio, não sendo influenciado pelos valores das entradas da camada anterior. A atualização do bias contribui para deslocar horizontalmente a função de ativação do neurônio, aumentando a capacidade da rede neural de representar padrões complexos e minimizar o erro durante o treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.6.6.3. Gradiente descendente

O Gradiente Descendente é um dos principais métodos de otimização utilizados no treinamento de redes neurais artificiais. Seu objetivo consiste em minimizar a função de erro do modelo por meio de atualizações iterativas dos pesos sinápticos, buscando encontrar um conjunto de parâmetros que produza o menor erro possível entre as saídas geradas pela rede e os valores esperados. Para isso, o método utiliza o gradiente da função de erro em relação aos parâmetros ajustáveis da rede, determinando a direção na qual o erro apresenta maior crescimento. Os pesos são então modificados na direção oposta, de modo a conduzir o modelo para regiões de menor erro (HAYKIN, 2001).

Em termos geométricos, a função de erro pode ser representada como uma superfície em um espaço de alta dimensionalidade, no qual cada dimensão está associada a um dos pesos da rede neural. Nessa representação, cada ponto da superfície corresponde a uma determinada configuração dos pesos e possui um valor de erro associado. O gradiente indica a

direção de maior crescimento da função de erro, sendo, portanto, necessário deslocar os pesos no sentido contrário ao gradiente para buscar um ponto de menor custo. Esse processo caracteriza o princípio fundamental do Gradiente Descendente. A atualização dos pesos pode ser representada pela equação apresentada a seguir:

$$w_{novo} = w - \eta \frac{\partial E}{\partial w} \quad (14)$$

Onde:

w representa o peso a ser ajustado;

E é a função de erro;

η é a taxa de aprendizado.

A taxa de aprendizado exerce papel fundamental no processo de otimização, pois controla o tamanho do passo realizado durante a atualização dos pesos. Quando esse parâmetro assume valores excessivamente elevados, as atualizações podem ser muito grandes, fazendo com que o algoritmo ultrapasse regiões de mínimo da função de erro e apresente oscilações ou até mesmo divergência. Por outro lado, valores excessivamente baixos produzem alterações muito pequenas nos pesos, tornando o processo de treinamento lento e aumentando consideravelmente o número de iterações necessárias para alcançar uma solução satisfatória. Dessa forma, a escolha adequada da taxa de aprendizado é essencial para obter um equilíbrio entre velocidade de convergência e estabilidade do treinamento (HAYKIN, 2009).

A cada iteração, o modelo calcula suas saídas, determina o erro em relação aos valores desejados e utiliza esse erro para obter as derivadas necessárias à atualização dos pesos. Esse procedimento é repetido ao longo de diversas épocas, permitindo que os parâmetros da rede sejam gradualmente ajustados de maneira a reduzir o erro global do modelo (FACELI et al., 2021).

O Back propagation é responsável pelo cálculo eficiente dos gradientes da função de erro em relação aos pesos da rede, enquanto o Gradiente Descendente utiliza essas informações para determinar a direção e a magnitude das atualizações dos parâmetros. A utilização conjunta desses procedimentos permite otimizar iterativamente os pesos da rede, reduzindo o erro durante o treinamento e possibilitando a construção de modelos com múltiplas camadas e grande quantidade de parâmetros (BISHOP, 2006).

3. METODOLOGIA

De acordo com Vargas (2017), a pesquisa aplicada tem como finalidade a geração de conhecimentos voltados à aplicação prática, direcionados à solução de problemas específicos. Tal definição alinha-se diretamente ao objetivo deste estudo, que consiste no desenvolvimento de uma solução tecnológica voltada ao controle inteligente de processos industriais, integrando conceitos de automação, inteligência artificial e sistemas computacionais.

Classifica-se, ainda, como pesquisa experimental por envolver a implementação e validação prática do sistema em ambiente simulado, permitindo a análise do modelo sob diferentes condições operacionais. Essa abordagem possibilita testar hipóteses, ajustar parâmetros e avaliar o desempenho do modelo antes de uma eventual aplicação em ambiente real, prática comum no desenvolvimento de sistemas automatizados complexos (GROOVER, 2011).

O presente trabalho possui uma abordagem quantitativa que se fundamenta na coleta, mensuração e análise de indicadores objetivos de desempenho, tais como tempo médio de processamento, taxa de atendimento correto, eficiência na alocação de recursos e tempo médio de resposta do sistema proposto. Esses indicadores são amplamente utilizados na avaliação de sistemas de produção e automação industrial, permitindo uma análise comparativa, além da validação de desempenho (PINEDO, 2016).

3.1. Visão geral

O projeto consiste na implementação de uma planta industrial simulada, controlada por meio de um sistema de Redes Neurais Artificiais (RNAs), totalmente integrado a uma interface web destinada a supervisão, monitoramento e gerenciamento remoto de um processo produtivo simulado, permitindo o desenvolvimento de soluções integradas de automação industrial, em consonância com a tendência de sistemas de controle industrial migrarem de uma lógica determinística baseada em regras para uma tomada de decisão adaptativa orientada por dados (SIDDIQUE et al., 2023).

A proposta fundamenta-se na aplicação de técnicas de aprendizado supervisionado em um grupo de RNAs para tomada de decisão em ambientes industriais dinâmicos, nas quais, modelos são treinados a partir de dados de entrada e saída para inferir padrões e otimizar respostas em uma planta industrial simulada, permitindo que os modelos aprendam

comportamentos desejados a partir de exemplos previamente definidos (GOODFELLOW; BENGIO; COURVILLE, 2016), possibilitando o controle autônomo de todo o processo.

A processo produtivo é representado com o auxílio de um software de simulação, no qual produtos de diferentes tipos são inseridos de forma aleatória no sistema, devendo ser processados conforme a demanda de pedidos ativos. A rede neural é responsável por analisar o estado corrente da planta e controlar seus atuadores, considerando variáveis como tipo de produto recebido, fila de pedidos, tempo de atendimento e estado lógico de sensores. A partir dessas informações, o modelo decide a ação mais adequada, podendo ser o direcionamento do produto para uma das duas linhas de produção, caso disponíveis, ou encaminhar o item para um sistema de armazenamento vertical, caso o item recebido não atenda à demanda produtiva imediata.

No âmbito da arquitetura computacional, a interface web desempenha um papel estratégico na integração entre o usuário e o sistema de controle inteligente, atuando como um elemento intermediário entre o ambiente operacional e o nível de supervisão, o que viabiliza o gerenciamento remoto de todo o processo produtivo. Com a implementação do sistema web é possível realizar a inserção de pedidos, o monitoramento da produção, o treinamento de redes industriais inteligentes e o registro de dados históricos para análises posteriores. Todo processo alinha-se aos princípios de digitalização e conectividade da Indústria 4.0 (KAGERMANN; WAHLSTER; HELBIG, 2013), favorecendo a interoperabilidade, a escalabilidade e o acesso distribuído às informações, aspectos essenciais em arquiteturas modernas baseadas em redes de comunicação (TANENBAUM; WETHERALL, 2011).

3.1.1. Camadas do sistema

A arquitetura do sistema proposto foi estruturada em camadas funcionais, de forma a organizar e modularizar as responsabilidades de cada componente, permitindo maior clareza no desenvolvimento, manutenção e escalabilidade da solução (KAGERMANN; WAHLSTER; HELBIG, 2013). Desta forma, o sistema foi desenvolvido da seguinte maneira:

- a) **Camada Física Simulada:** Representa o ambiente industrial virtual, composto por sensores, atuadores, esteiras transportadoras e um sistema de armazenamento vertical;

- b) **Camada de Controle Inteligente:** Implementada por meio de conjuntos de redes neurais do tipo Perceptron Multicamadas, responsável pela tomada de decisão dos processos (HAYKIN, 2001);
- c) **Camada de Integração Web:** Responsável pelo envio de pedidos de produção, treinamento dos modelos, monitoramento em tempo real, registro de dados e análise de desempenho operacional.

Essa organização possibilita maior flexibilidade na evolução do projeto, permitindo que modificações em uma camada específica não impactem diretamente as demais, característica fundamental em arquiteturas distribuídas de sistemas ciberfísicos, que segundo Rajkumar et al. (2010), tais sistemas são projetados para integrar de forma eficiente componentes físicos e computacionais, garantindo modularidade e capacidade de adaptação.

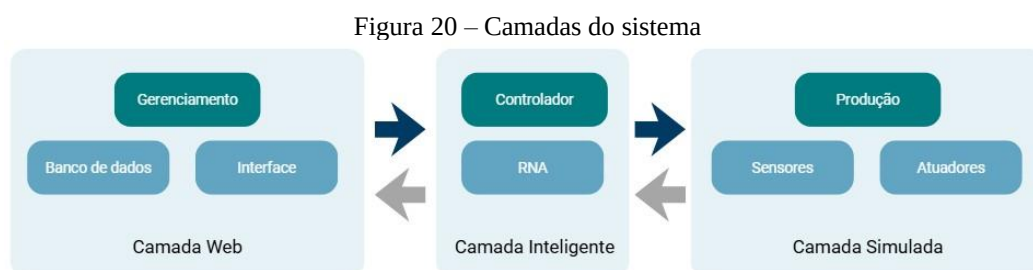
A utilização de ambientes simulados, permite testes sob diferentes cenários operacionais, que segundo Ogata (2010), a modelagem e a simulação constituem etapas fundamentais no desenvolvimento e validação de sistemas de controle, pois possibilitam avaliar o comportamento dinâmico do sistema antes de sua aplicação prática.

3.1.2. Fluxo de comunicação entre camadas

O funcionamento do sistema baseia-se na troca contínua de informações entre as camadas que compõem sua arquitetura, seguindo o princípio da separação de responsabilidades. Inicialmente, a camada física simulada gera dados provenientes dos sensores virtuais que representam o estado do processo produtivo, incluindo informações relacionadas à movimentação de materiais, ocupação das linhas produtivas e condições operacionais dos equipamentos. Esses dados são transmitidos à camada de controle inteligente, responsável pelo processamento das informações e pela tomada de decisão.

Na camada de controle inteligente, a RNA analisa as variáveis de entrada e determina as ações de controle mais adequadas para o sistema, no qual, as decisões geradas pelo sistema são encaminhadas de volta à camada física simulada, onde são executadas por meio dos atuadores virtuais, permitindo a realização das operações previstas. O emprego de técnicas de inteligência artificial em ambientes industriais é uma proposta interessante para problemas de tomada de decisão, otimização de processos e adaptação a condições dinâmicas de operação (RUSSELL; NORVIG, 2021).

Os dados operacionais e os resultados das decisões das redes são disponibilizados à camada de supervisão e interface web, possibilitando o monitoramento do processo em tempo real, além da interação remota por parte do usuário. Segundo Boyer (2010), a função essencial de uma camada supervisória é justamente traduzir dados operacionais dispersos em informação compreensível e acionável, permitindo que o operador avalie o comportamento do sistema sem a necessidade de intervenção direta no ambiente físico.



Fonte: Elaborado pelo autor (2026)

Essa integração entre os níveis operacionais, de controle e supervisão está alinhada aos conceitos industriais mais modernos, que promovem a conectividade entre sistemas, a digitalização dos processos produtivos e a utilização de tecnologias inteligentes para suporte à tomada de decisão (SCHWAB, 2016).

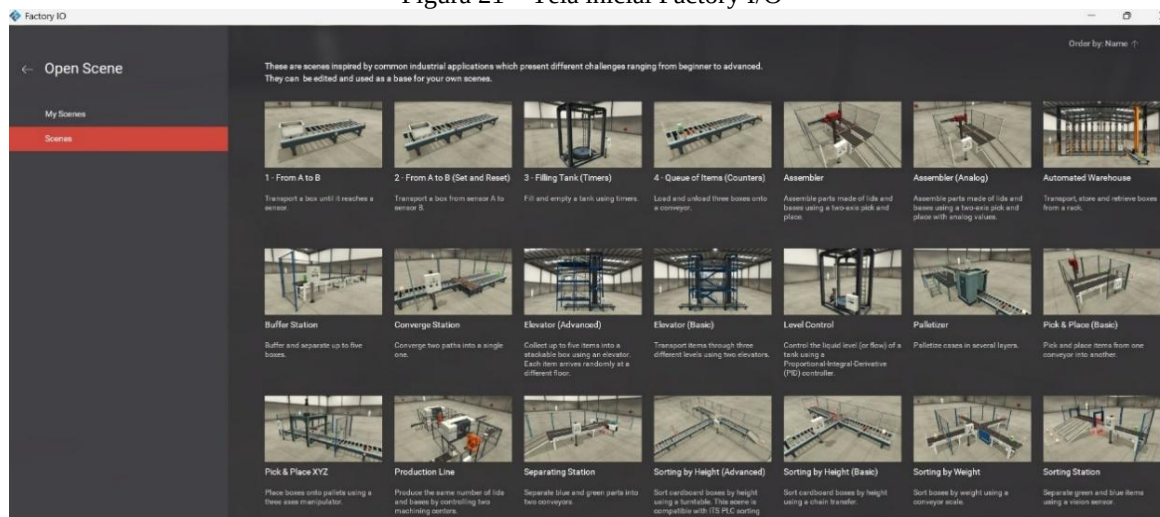
3.2. Planta industrial simulada

A planta industrial utilizada neste trabalho foi desenvolvida no *Factory I/O*, software voltado à modelagem e simulação de processos industriais, amplamente empregado em atividades de ensino, validação de estratégias de controle e testes de sistemas automatizados antes de sua implementação física. A plataforma permite a construção de ambientes tridimensionais interativos compostos por sensores, atuadores, esteiras transportadoras, sistemas articulados, dispositivos de movimentação e estruturas de armazenamento, além de oferecer suporte à integração com CLPs, protocolos industriais e algoritmos externos desenvolvidos em diferentes linguagens de programação. Tal característica possibilitou a construção de um cenário próximo à realidade de uma planta de triagem industrial.

Segundo a documentação técnica da desenvolvedora, o software foi projetado para simular sistemas industriais em tempo real, garantindo sincronização entre eventos, resposta dinâmica dos dispositivos e execução determinística dos cenários implementados (REAL GAMES, 2023). Essa característica é o que viabiliza, no contexto deste trabalho, a comunicação

com um protocolo industrial entre a planta simulada e o controlador externo desenvolvido em diferentes linguagens de programação, reproduzindo o comportamento de uma célula automatizada. A Figura 21 exibi a tela inicial do software Factory I/O.

Figura 21 – Tela inicial Factory I/O



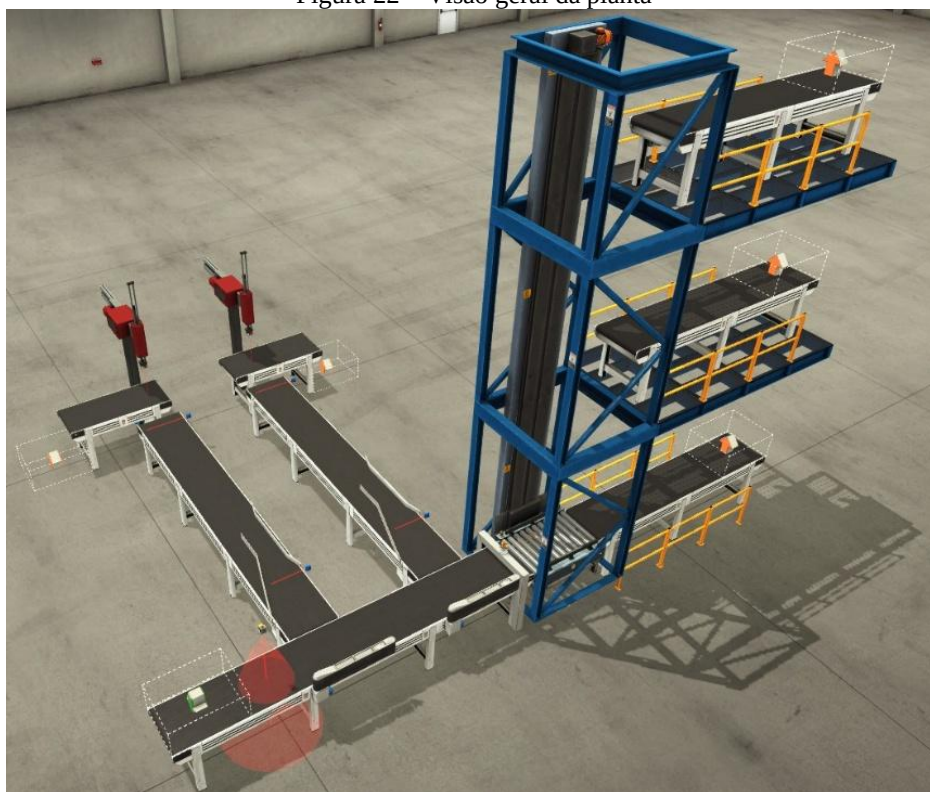
Fonte: Adaptado de REAL GAMES (2023)

A adoção de múltiplos dispositivos com funções distintas, possibilita a representação de cenários industriais dinâmicos, nos quais o fluxo de materiais, a lógica de controle e as condições operacionais variam ao longo do tempo. Segundo Groover (2011), a modelagem desses ambientes é fundamental para compreender a interação entre os elementos que compõem os sistemas de manufatura automatizados e avaliar seu desempenho sob diferentes condições operacionais.

3.2.1. Arquitetura da planta

A arquitetura da planta simulada integra três esteiras transportadoras de velocidade variável, responsáveis pela entrada e produção inicial, um elevador vertical para representar um sistema de armazenamento, este integrado a três esteiras de velocidade fixa, além de dois manipuladores cartesianos pneumáticos voltados à etapa de expedição, auxiliados por duas pequenas esteiras também de velocidade fixa. A velocidade variável das esteiras de entrada atua como parâmetro operacional ajustável pelo sistema de controle inteligente, permitindo modular o fluxo de transporte conforme as demandas dinâmicas do processo. A Figura 22 mostra a implementação da planta simulada.

Figura 22 – Visão geral da planta



Fonte: Elaborado pelo autor (2026)

A primeira delas, denominada esteira principal (Figura 23), é responsável pelo recebimento dos produtos inseridos no sistema de forma aleatória, caracterizando um processo de entrada não determinístico. A partir desse ponto, os itens são distribuídos conforme as decisões do sistema de controle, podendo ser triados para uma das esteiras secundárias existentes, associadas ao atendimento dos pedidos em processamento no sistema, ou encaminhados ao elevador vertical para simulação de um processo de armazenamento.

Figura 23 – Esteira transportadora



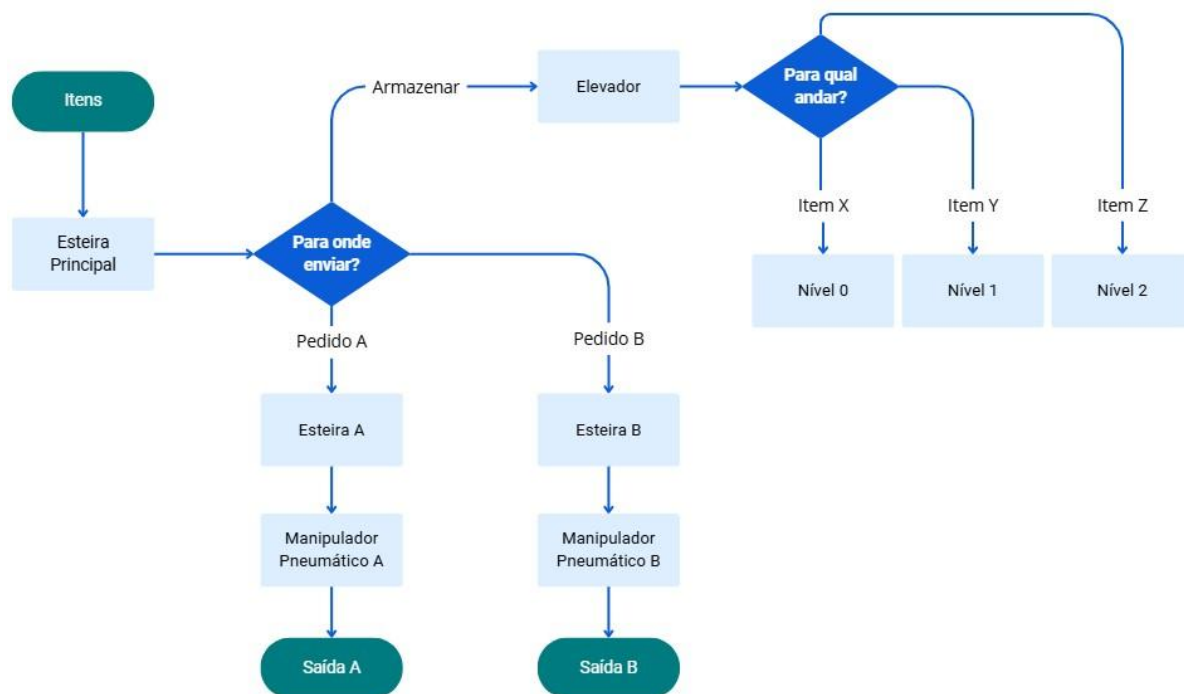
Fonte: Adaptado de REAL GAMES (2023)

Cada pedido gerado pelo sistema é classificado como Pedido A ou Pedido B, sendo processado em sua respectiva esteira secundária, operando com três categorias distintas de itens,

denominadas item X, item Y e item Z. Na fase final, após o item passar pela esteira de pedidos, a expedição é executada pelos manipuladores cartesianos pneumáticos, responsáveis por retirar os produtos das linhas secundárias e transferi-los às esteiras de saída de menor porte, simulando o despacho da produção.

O elevador, por sua vez, realiza a movimentação dos itens entre três níveis de estocagem, cada um dedicado a um tipo específico de produto, no qual o item somente é direcionado ao elevador quando não é requerido para o atendimento imediato de um pedido ativo, caso contrário, é encaminhado diretamente à esteira secundária correspondente. Essa lógica de priorização entre atendimento imediato e lógica de armazenamento reflete um princípio recorrente em sistemas automatizados de manuseio de materiais, nos quais o roteamento dinâmico de itens busca minimizar o tempo de permanência no sistema (GROOVER, 2011).

Figura 24 – Fluxograma do processo industrial



Fonte: Elaborado pelo autor (2026)

A Figura 24 demonstra a sequência de decisões do processo produtivo simulado. Essa configuração reproduz elementos típicos de sistemas automatizados de movimentação e armazenamento industrial, conforme abordado na literatura de automação e controle discreto (SILVEIRA; SANTOS, 2014).

3.2.2. Tratamento de produtos

Os produtos considerados neste trabalho correspondem a modelos pré-definidos disponíveis no ambiente de simulação utilizado. No software Factory I/O, os elementos manipulados são objetos padronizados da biblioteca interna do sistema, sendo neste estudo identificados como item X, item Y e item Z, como demonstrado na Figura 25. Cada item possui uma identificação lógica própria dentro do ambiente de simulação, permitindo sua distinção não por características visuais, mas por meio de sensores e variáveis de controle associadas ao momento de sua criação no sistema.

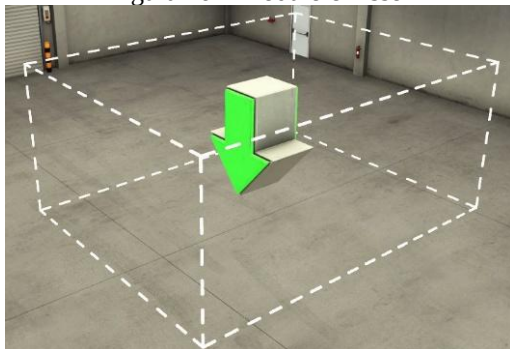
Figura 25 – Itens tratados no sistema



Fonte: Adaptado de REAL GAMES (2023)

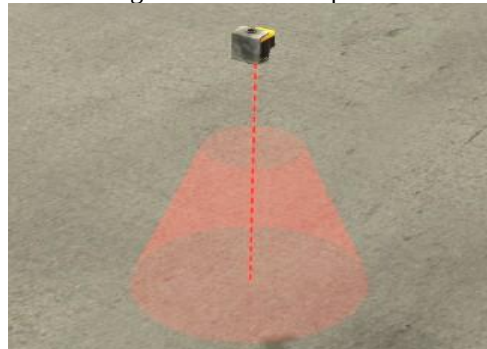
No instante em que um novo produto é inserido aleatoriamente na esteira principal, o módulo emissor (Figura 26) atribui a esse objeto uma identificação numérica padronizada, correspondente ao código fixo associado a cada tipo de item. Essa informação é lida por uma espécie de sensor óptico simulado (Figura 27), posicionado na entrada da esteira principal, durante a passagem do item pelo feixe de leitura, sendo convertida em um sinal analógico proporcional às características associadas ao objeto detectado, sendo uma forma equivalente ao funcionamento de sensores ópticos industriais reais (BOLTON, 2015).

Figura 26 – Módulo emissor



Fonte: Adaptado de REAL GAMES (2023)

Figura 27 – Sensor óptico



Fonte: Adaptado de REAL GAMES (2023)

Após o momento de leitura, o sistema reconhece o tipo de produto e executa a decisão de controle correspondente, direcionando o item a uma das esteiras secundárias ou encaminhando ao sistema de armazenamento vertical, conforme a demanda de pedidos ativa no momento da detecção.

Cada tipo de produto utilizado na simulação está associado a um valor fixo, atribuído pelo módulo emissor no momento de sua criação e posteriormente lido pelo sensor óptico. Esse valor funciona como uma assinatura numérica exclusiva de cada categoria de item, sendo o dado efetivamente utilizado pela camada de controle para diferenciar os produtos ao longo de todo o fluxo produtivo. A Tabela 1 apresenta o valor correspondente a cada item:

Tabela 1 – Lista de códigos dos itens

Item	Código
X	3
Y	6
Z	9

Fonte: Elaborado pelo autor (2026)

Os objetos industriais simulados podem ser parametrizados para representar diferentes tipos de materiais, possibilitando a criação de cenários com múltiplas categorias de produtos dentro do mesmo fluxo produtivo. Essa característica viabiliza a implementação e a validação de algoritmos de tomada de decisão em ambientes que reproduzem diferentes condições operacionais (REAL GAMES, 2023).

3.2.3. Direcionamento de produtos

Após a identificação do tipo de produto pelo sensor óptico da esteira principal, o sistema passa à etapa de direcionamento, responsável por conduzir cada item ao seu destino de processamento de acordo com a demanda de pedidos ativa no sistema. Esse encaminhamento é realizado por dois atuadores de desvio (pivô) integrados ao ambiente de simulação, posicionados ao longo da esteira principal, cada um responsável por desviar o fluxo de produtos para uma das esteiras secundárias correspondentes.

A confirmação da chegada do produto à esteira secundária é realizada por um sensor difuso instalado no início de cada linha, cuja função limita-se a detectar a presença do objeto durante sua passagem pelo feixe de detecção, este sensor cumpre apenas a função de confirmação física de chegada. Existe uma distinção entre sensores de identificação e sensores de confirmação, característica recorrente em sistemas discretos de manuseio automatizado

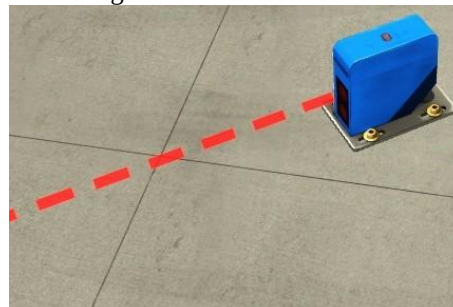
(SILVEIRA; SANTOS, 2014). O pivô de desvio e o sensor difuso são ilustrados nas Figuras 28 e 29, respectivamente.

Figura 28 – Pivô de desvio



Fonte: Adaptado de REAL GAMES (2023)

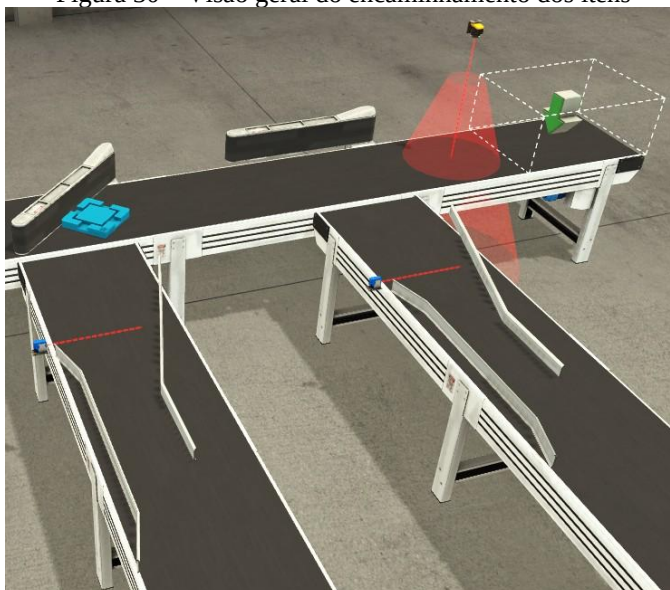
Figura 29 – Sensor difuso



Fonte: Adaptado de REAL GAMES (2023)

A camada de controle recebe como entradas o estado de ocupação de cada esteira secundária, a condição dos pedidos ativos (quantidades e tipos ainda pendentes) e os sinais provenientes dos sensores de identificação e confirmação. Com base na combinação dessas informações, o sistema determina, a cada novo item detectado na esteira principal, se o produto deve ser encaminhado para uma das linhas de atendimento. A Figura 30 representa a composição do ponto de encaminhamento dos itens para as esteiras secundárias.

Figura 30 – Visão geral do encaminhamento dos itens



Fonte: Elaborado pelo autor (2026)

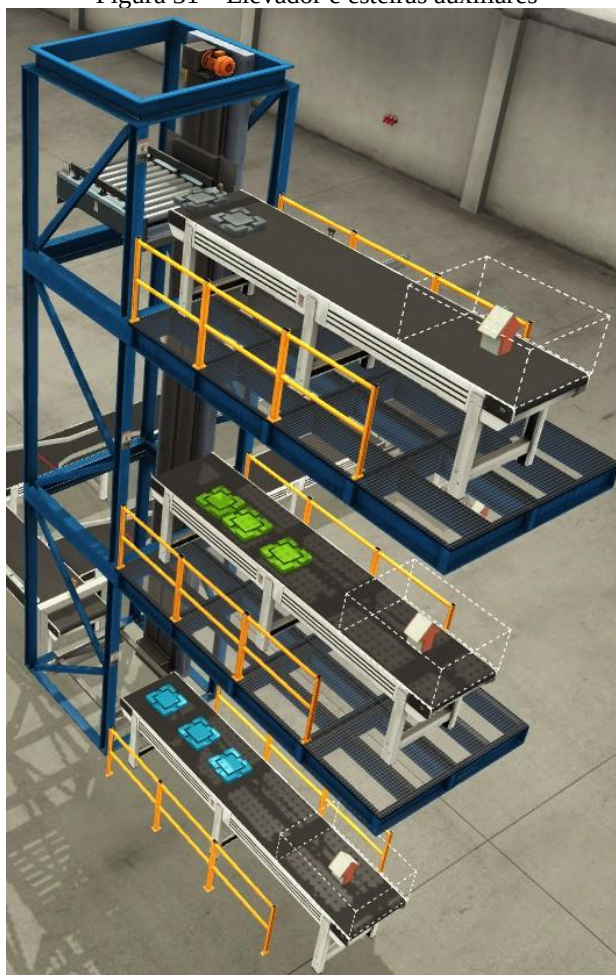
Quando, o item recebido na esteira principal não corresponde a nenhum pedido ativo no momento da decisão, ambos os atuadores de desvio permanecem estáticos, permitindo que o produto prossiga até o final da esteira principal, onde é automaticamente encaminhado

ao sistema de armazenamento vertical. Esse comportamento caracteriza uma lógica de decisão condicional executada a cada novo item detectado, reproduzindo o princípio de roteamento dinâmico presente em sistemas de triagem automatizada, no qual a decisão de destino é tomada em função do estado corrente do sistema (REAL GAMES, 2023).

3.2.4. Armazenamento de produtos

O sistema de armazenamento é composto por um elevador vertical com três níveis operacionais, visto na Figura 31, sendo cada nível destinado exclusivamente a um tipo específico de produto, nível 0 (térreo) para o item X, nível 1 para o item Y e nível 2 para o item Z. Essa organização simplifica a lógica de controle desenvolvida, pois estabelece uma correspondência direta entre o tipo do produto identificado na esteira principal e sua posição final de armazenamento, eliminando a necessidade de lógicas de busca ou realocação dinâmica de espaço, comuns em sistemas de armazenamento mais complexos (GROOVER, 2011).

Figura 31 – Elevador e esteiras auxiliares



Fonte: Elaborado pelo autor (2026)

Cada um dos três níveis do elevador possui uma esteira lateral própria, responsável por receber o produto assim que a plataforma de elevação atinge a posição correspondente. Essas esteiras permanecem inativas durante o deslocamento vertical da plataforma, sendo acionadas apenas no instante em que o elevador se encontra posicionado em uma faixa do nível de destino. Esse tipo de arranjo é característico de estruturas automatizadas de armazenamento vertical (BOLTON, 2015).

O posicionamento do elevador é realizado por meio de um atuador integrado ao ambiente de simulação, enquanto um sensor de posição monitora continuamente o nível em que a plataforma se encontra, fornecendo realimentação em malha fechada ao sistema de controle.

Os itens entram na esteira do elevador no nível 0, a partir desse ponto e com base na identificação previamente realizada na esteira principal, a lógica de controle determina o nível correspondente ao tipo de produto, desloca o elevador até a posição adequada e, somente após a confirmação do posicionamento pelo sensor, aciona a esteira lateral daquele nível para efetivar o recebimento do item. Essa sequência de eventos coordenados entre sensor de posição, atuador de elevação e atuador de transporte lateral reproduz o funcionamento típico de dispositivos automatizados de manuseio, garantindo confiabilidade operacional mesmo em ciclos de alta repetitividade (CAPELLI, 2013).

O armazenamento atua como representação simulada de estoque dentro do processo produtivo, registrando virtualmente os produtos que não são direcionados imediatamente às linhas de atendimento. Os itens encaminhados ao elevador não retornam ao fluxo de produção, sendo o sistema utilizado apenas para simular a organização e representar uma otimização do uso do espaço físico disponível na planta (GROOVER, 2011).

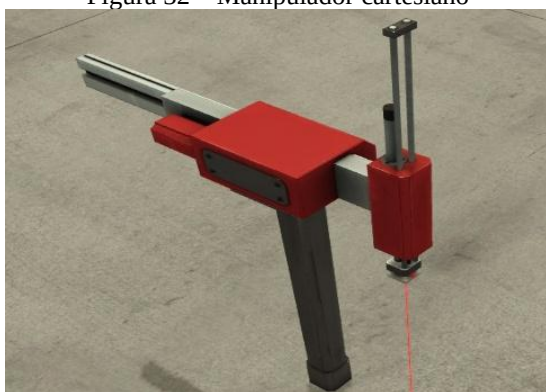
3.2.5. Expedição

A etapa final do processo produtivo corresponde à expedição dos pedidos, executada por dois manipuladores pneumáticos de estrutura cartesiana, um associado a cada esteira secundária (A e B), responsáveis por retirar os produtos das respectivas linhas após a conclusão do atendimento e transferi-los ao ponto de despacho correspondente. Cada manipulador opera em dois eixos de movimento, um eixo horizontal, responsável pelo posicionamento lateral em direção ao produto, e um eixo vertical, responsável pelo deslocamento de descida até a posição de captura. Esses movimentos lineares são executados de forma sequencial, garantindo o correto posicionamento, a captura e a transferência de cada

item para a etapa de remoção definitiva do sistema, simulando sistemas industriais de manuseio de materiais de elevada repetibilidade (BOLTON, 2015).

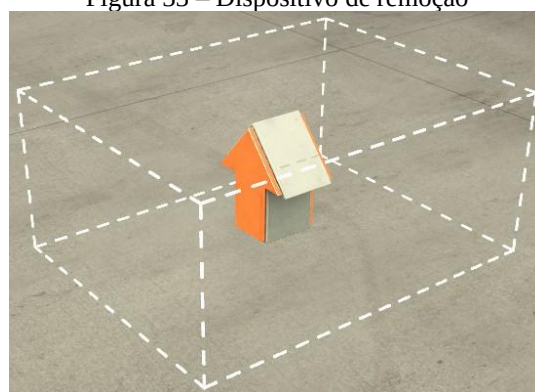
Para realizar a remoção definitiva no sistema, são utilizados dispositivos de remoção, cuja função é efetuar a retirada dos produtos sem a necessidade de controle explícito por parte do sistema. As Figuras 32 e 33 apresentam, respectivamente, o manipulador cartesiano e o dispositivo de remoção.

Figura 32 – Manipulador cartesiano



Fonte: Adaptado de REAL GAMES (2023)

Figura 33 – Dispositivo de remoção

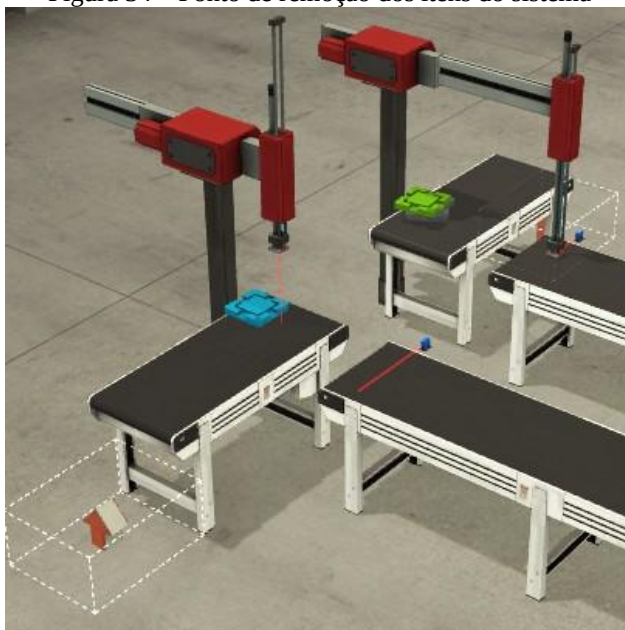


Fonte: Adaptado de REAL GAMES (2023)

O ciclo de operação do manipulador é iniciado quando o produto atinge o final da esteira secundária e é detectado por um sensor difuso, responsável por identificar sua presença na posição de coleta. Após esse acionamento, o manipulador desloca-se até a posição do produto, sendo o correto posicionamento confirmado por meio de sensores de deslocamento. Após essa confirmação, é iniciada a sequência automática de captura do item, em que um sensor localizado na ponta do manipulador detecta a presença do item próximo ao ponto de captura. Concluída a captura, o manipulador executa o movimento de retorno à posição inicial, onde deposita o produto sobre uma esteira auxiliar de menor porte, responsável por transportá-lo até o dispositivo de remoção do sistema.

O ponto de saída do sistema, visto na Figura 34, é constituído por uma esteira de acumulação, responsável por empilhar os produtos recebidos até que a quantidade total correspondente ao pedido seja atingida. Esse mecanismo de acúmulo permite que o sistema aguarde a formação completa do lote antes de finalizar o ciclo, evitando o despacho de pedidos parciais ou incompletos. Após a formação do lote, a esteira de saída é acionada, deslocando o conjunto até o dispositivo de remoção e concluindo o ciclo operacional daquele pedido específico, momento em que o sistema pode voltar a atender uma nova solicitação na linha secundária correspondente.

Figura 34 – Ponto de remoção dos itens do sistema



Fonte: Elaborado pelo autor (2026)

A utilização de atuadores pneumáticos simulados permite representar aplicações industriais de movimentação e manipulação de materiais, como em sistemas integrados a células automatizadas e linhas de produção discretas que compõem arquiteturas típicas de automação industrial voltadas à repetição de tarefas de curto ciclo (PRUDENTE, 2016; CAPELLI, 2013).

3.2.6. Controle do processo

No ambiente de simulação Factory I/O, a interação entre o sistema de controle e o processo simulado é mediada por sensores e atuadores virtuais, que reproduzem o comportamento funcional de seus equivalentes empregados em sistemas industriais reais. Os sensores são responsáveis por informar ao controlador o estado corrente do processo, como presença de itens, posição de dispositivos móveis e condição operacional dos equipamentos, enquanto os atuadores executam as ações determinadas pela lógica de controle. Essa interação contínua entre leitura de estado e execução de comando reproduz o princípio de operação dos sistemas automatizados em malha fechada (GROOVER, 2011; SILVEIRA; SANTOS, 2014).

Os dispositivos empregados na planta simulada podem ser classificados quanto à natureza do sinal que produzem ou recebem, dividindo-se em dispositivos digitais, cujas variáveis assumem apenas dois estados lógicos discretos, e dispositivos analógicos, cujas variáveis representam grandezas contínuas dentro de uma faixa de operação.

3.2.6.1. Dispositivos digitais

No ambiente de simulação desenvolvido, os dispositivos digitais (binários) são aqueles cujas entradas e saídas assumem apenas dois estados lógicos, normalmente representados por 0 e 1 (desligado/ligado ou falso/verdadeiro). Na planta simulada, essa categoria compreende sensores de presença, sensores difusos e atuadores responsáveis por comandos discretos, como o acionamento de esteiras, dispositivos de desvio e alguns comandos dos manipuladores cartesianos. Os sinais digitais são empregados para representar eventos e condições operacionais discretas do processo, possibilitando a implementação de intertravamentos, sequências de operação e lógicas de decisão fundamentais ao funcionamento dos sistemas automatizados (BOLTON, 2015). A Tabela 2 lista todos os dispositivos digitais usados na simulação.

Tabela 2 – Lista de dispositivos digitais

Dispositivo	Função	Quantidade	Tipo
Emissor	Inserir de forma aleatória os produtos no sistema	1	Atuador
Removedor	Remover um item do fim da respectiva esteira	5	Atuador
Sensor Difuso	Reconhecer a presença dos produtos	4	Sensor
Esteiras Digitais	Remover os produtos já processado do sistema	5	Atuador
Atuador Rotativo	Desviar o produto para uma das esteiras secundárias	2	Atuador
Sensor presença	Detectar item associado ao seu respectivo atuador	3	Sensor
Atuador Luminoso	Exibir indicação de prioridade para o elevador	1	Atuador
Pick	Segurar o item na ponta do manipulador	2	Atuador

Fonte: Elaborado pelo autor (2026)

O sistema utiliza sensores distribuídos ao longo das esteiras, com a finalidade de detectar presença de peças e estado operacional dos dispositivos, fornecendo sinais discretos ao sistema de controle, permitindo a atualização das variáveis de estado do processo. Em sistemas de automação industrial, sensores deste tipo constituem elementos fundamentais para a aquisição de dados e tomada de decisão em malhas de controle discreto (SILVEIRA; SANTOS, 2014). Os sinais provenientes dos sensores são utilizados como entradas para o sistema de

controle inteligente, permitindo que cada decisão seja baseada no estado atual da planta simulada.

Foram empregados atuadores discretos com o intuito de reproduzir condições dinâmicas de entrada e saída de materiais, característica comum em processos industriais com variabilidade de demanda e fluxo intermitente, sendo acionados ou desativados de forma pontual em resposta a eventos discretos captados pelos sensores (GROOVER, 2011).

No Factory I/O, esses dispositivos são implementados por meio de entradas e saídas booleanas, utilizadas para estabelecer a comunicação entre o ambiente de simulação e a camada de controle, reproduzindo o comportamento dos dispositivos digitais empregados em aplicações industriais (REAL GAMES, 2023).

3.2.6.2. Dispositivos analógicos

Os dispositivos analógicos presentes na planta são responsáveis pelo processamento e controle de grandezas contínuas, possibilitando maior precisão no acionamento dos equipamentos e maior flexibilidade na operação do sistema. Diferentemente dos dispositivos digitais, cujos sinais assumem apenas dois estados lógicos, os dispositivos analógicos operam com variáveis que podem assumir qualquer valor dentro de uma faixa contínua, representando grandezas físicas como posição ou velocidade (BOLTON, 2015). A Tabela 3 caracteriza os dispositivos analógicos implementados.

Tabela 3 – Lista de dispositivos analógicos

Dispositivo	Função	Quantidade	Tipo
Sensor Óptico	Identificar o tipo de produto que entrou no sistema	1	Sensor
Elevador	Transportar itens excedentes para armazenamento	1	Atuador
Esteiras Analógicas	Transportar os produtos de entrada para o processo	3	Atuador
Sensor de Posição	Medir o deslocamento do elevador e dos eixos dos manipuladores	5	Sensor
Eixos dos Manipuladores	Mover os eixos horizontais e verticais dos manipuladores cartesianos	4	Atuador

Fonte: Elaborado pelo autor (2026)

Na planta desenvolvida, a velocidade é tratada por alguns elementos como variável contínua dentro do modelo, permitindo que o sistema de controle ajuste dinamicamente o fluxo de transporte em função da demanda ativa, do estado das linhas secundárias e das condições de ocupação do sistema. Em aplicações industriais reais, esse tipo de controle é comumente implementado por meio de inversores de frequência acoplados a motores elétricos, possibilitando variação contínua e eficiente da rotação (OGATA, 2010).

O elevador vertical também apresenta comportamento analógico no que se refere ao seu deslocamento, ocorrendo de forma contínua até o alcance da posição desejada, sendo que o controle de posicionamento se baseia em um sensor que fornece sinal proporcional à altura instantânea da plataforma, permitindo alinhamento preciso com cada andar.

O mecanismo de detecção do sensor óptico pode ser representado por leitura analógica, de forma equivalente ao que ocorre em sensores industriais ópticos ou indutivos reais, os quais geram um sinal elétrico proporcional às características físicas do objeto detectado, como refletividade, cor ou distância, e não apenas uma resposta binária de presença/ausência (BOLTON, 2015). No contexto deste trabalho, essa característica é o que permite ao sensor não apenas identificar que um produto está passando pela esteira principal, mas também distinguir qual tipo de item (X, Y ou Z) foi detectado, uma vez que cada categoria de produto está associada a um valor de sinal distinto dentro da faixa contínua lida pelo sensor, sendo esse valor posteriormente tratado pela lógica de controle antes de ser convertido em uma decisão de roteamento.

Segundo Silveira e Santos (2014), a coexistência entre sinais digitais e analógicos é justamente uma das características centrais de plantas industriais automatizadas modernas, nas quais a maior parte dos processos reais não se reduz a estados puramente binários, exigindo controladores capazes de processar simultaneamente ambos os tipos de grandeza de forma coordenada.

3.2.6.3. Variáveis de estado

Além dos sensores e atuadores, o sistema de controle desenvolvido neste trabalho utiliza um conjunto de variáveis de estado, tais variáveis são grandezas internas, calculadas ou derivadas pela própria lógica de controle, que não correspondem diretamente à leitura de um único sensor, mas sim à consolidação de múltiplas informações do processo em um único indicador de estado. Essa abordagem foi adotada porque, na prática, um sensor isolado fornece apenas um dado bruto sobre a planta em um instante específico, enquanto a tomada de decisão

da camada de controle exige que esse dado seja combinado com outras condições operacionais em curso. Segundo Ogata (2010), essa distinção entre variável medida diretamente por um sensor e variável de estado do processo é fundamental em sistemas de controle modernos, uma vez que grande parte das decisões de controle depende da relação dinâmica entre múltiplas grandezas ao longo do tempo.

Do ponto de vista da arquitetura de automação, a existência de variáveis de controle intermediárias também contribui para o tratamento adequado de eventos assíncronos, situações em que múltiplos sensores podem ser acionados em momentos distintos, exigindo que o sistema aguarde a consolidação de todas as condições relevantes antes de executar uma ação. Bolton (2015) destaca que o uso de variáveis de estado é uma prática comum em sistemas de automação industrial baseados em eventos discretos, permitindo que a lógica de controle reaja de forma coerente a sequências de eventos.

No sistema desenvolvido, essas variáveis de estado são atualizadas continuamente a cada ciclo de varredura do controlador, refletindo sempre a condição mais recente do processo antes que uma nova decisão seja tomada. Dessa forma é garantido que mesmo quando um evento já tenha ocorrido, a informação correspondente permaneça disponível na variável de estado até que seja efetivamente considerada pela camada de controle, evitando a perda de eventos transitórios que poderiam ocorrer entre dois ciclos consecutivos de leitura.

3.3. Implementação da Rede Neural Artificial

A implementação do sistema inteligente de controle foi baseada em Redes Neurais Artificiais (RNAs) do tipo Perceptron Multicamadas (MLP), capaz de modelar relações não lineares entre variáveis de entrada e saída, característica essencial em sistemas industriais com múltiplas variáveis e interdependências operacionais (GOODFELLOW; BENGIO; COURVILLE, 2016).

A viabilidade de redes neurais treinadas por aprendizado supervisionado como mecanismo de decisão de controle já foi demonstrada experimentalmente em plantas piloto, nas quais um modelo treinado, utilizado como modelo interno de um controlador preditivo, alcançou rastreamento eficiente de setpoint e rejeição de perturbações (RITA, 1995).

Considerando a complexidade do processo industrial simulado, optou-se por segmentar o sistema de decisão em múltiplas redes neurais independentes, em vez de utilizar uma única rede de grande porte. Essa estratégia foi adotada para:

- a) Reduzir a dimensionalidade do problema;

- b) Diminuir o volume de dados necessários para treinamento;
- c) Facilitar a convergência durante o treinamento;
- d) Permitir modularidade e manutenção de cada subsistema.

Segundo Haykin (2001), a decomposição de problemas complexos em subproblemas menores é uma abordagem recomendada quando o espaço de estados é elevado ou quando diferentes conjuntos de decisões possuem naturezas distintas.

Dessa forma, o sistema foi dividido em cinco redes neurais independentes, cada uma responsável por um conjunto específico de decisões dentro do processo. Cada rede possui estrutura própria de entradas, saídas e funções de ativação, podendo operar com saídas binárias ou saídas analógicas. Essa abordagem modular permite que cada rede seja treinada de forma especializada para sua função específica, reduzindo interferências entre tarefas distintas e tornando o sistema mais robusto e escalável. A composição das RNAs que envolvem a planta simulada é:

- a) **Prioridade:** Responsável por decidir para onde o produto da esteira principal deverá ser enviado;
- b) **Esteira:** Controlar a esteira principal e o momento de inserir novos produtos na produção;
- c) **Expedição:** Tratar os itens recebidos pelas esteiras secundárias e o controle dos manipuladores cartesianos;
- d) **Elevador:** Controlar o elevador e as esteiras de armazenamento para recebimento dos produtos não utilizados;
- e) **Despacho:** Finalização do pedido e remoção do sistema, simulando etapa final de entrega.

Essa característica aproxima a arquitetura proposta do conceito de sistemas multiagentes especializados, nos quais cada agente é responsável por um subconjunto restrito de decisões dentro de um processo maior, comunicando-se por meio de variáveis de estado compartilhadas (HAYKIN, 2001).

3.3.1. Funções de ativação das RNAs

Para todas as redes neurais implementadas, adotou-se uma camada oculta e com a função de ativação tangente hiperbólica ($\tanh$), em virtude de sua característica de saída

centrada em zero, o que favorece a dinâmica do treinamento ao reduzir viés no gradiente durante a retro propagação do erro, contribuindo para uma atualização de pesos mais equilibrada e melhor convergência do algoritmo de gradiente descendente (GOODFELLOW; BENGIO; COURVILLE, 2016).

Na camada de saída, a função de ativação foi definida conforme a natureza do problema tratado por cada rede, uma vez que, diferentemente da camada oculta, na qual a tangente hiperbólica foi adotada de forma padronizada em todas as redes, a camada de saída precisa refletir diretamente o tipo de decisão que a rede deve produzir.

Para decisões binárias, utilizou-se a função sigmoide logística, permitindo a interpretação probabilística da saída no intervalo $[0,1]$. Nesses casos, adotou-se um limiar de decisão de 0.95, de modo que valores de saída (y_d) superiores a esse limiar são convertidos para o estado digital 1 (comando ativado), enquanto qualquer valor abaixo é convertido para o estado digital 0 (comando não ativado):

$$y_d = \begin{cases} 1, & \text{se } y > 0,95 \\ 0, & \text{caso contrário} \end{cases} \quad (15)$$

A adoção de um limiar elevado tem por objetivo aumentar a exigência de confiança do modelo antes do acionamento de comandos críticos, reduzindo a ocorrência de falsos acionamentos decorrentes de saídas próximas à região de incerteza da função sigmoide (BISHOP, 2006).

Para as variáveis de controle contínuo, também se adotou a função sigmoide na camada de saída, porém sem a aplicação do limiar de ativação, nesses casos, a saída da rede, naturalmente limitada ao intervalo $[0,1]$, é tratada como um valor proporcional normalizado, sendo posteriormente reescalada para o intervalo operacional aceito pelos atuadores do ambiente de simulação. No Factory I/O, essa escala de operação para dispositivos analógicos corresponde tipicamente ao intervalo de 0 a 1000, unidade adotada internamente pelo software para representar grandezas contínuas. Dessa forma, a saída bruta da rede neural (y_s) é convertida por meio de um escalonamento linear simples, de modo que o valor 0 corresponda ao limite inferior da faixa de operação e o valor 1 corresponda ao seu limite superior, sendo:

$$y_s = y \cdot 1000 \quad (16)$$

Essa lógica de conversão é conceitualmente equivalente à empregada em sistemas industriais reais, nos quais grandezas analógicas de saída são representadas em registradores digitais com resolução definida em bits, sendo o valor normalizado da rede neural igualmente reescalado para a faixa correspondente à resolução do dispositivo físico (por exemplo, de 0 a 4095 em um conversor de 12 bits), característica típica dos módulos de saída analógica utilizados em Controladores Lógicos Programáveis (BOLTON, 2015). Ainda que a unidade de representação varie entre o ambiente simulado e uma aplicação real, o princípio permanece o mesmo.

Para problemas de decisão, empregou-se a função Softmax, que normaliza as saídas em forma de distribuição de probabilidade, garantindo que a soma de todas as alternativas seja igual a 1 e possibilitando a seleção da ação com maior probabilidade associada (BISHOP, 2006). Diferentemente da sigmoide, que trata cada saída de forma independente, a Softmax calcula a probabilidade de cada classe, de modo que o aumento da probabilidade atribuída a uma alternativa implica necessariamente a redução proporcional das demais. A definição das funções de ativação de cada rede é estabelecida de acordo com a Tabela 4.

Tabela 4 – Funções de ativações

RNA	Camada Oculta	Camada de Saída
Prioridade	Tanh	Softmax
Esteira	Tanh	Sigmoide
Expedição	Tanh	Sigmoide
Elevador	Tanh	Sigmoide
Despacho	Tanh	Sigmoide

Fonte: Elaborado pelo autor (2026)

Vale destacar que, embora a camada de saída de uma mesma rede possa combinar variáveis de natureza distinta, produzindo simultaneamente comandos digitais e variáveis contínuas, todos os neurônios de saída dessas redes mistas empregam a mesma função de ativação sigmoide, diferenciando-se apenas quanto à interpretação aplicada sobre o valor produzido. As saídas contínuas são reescaladas linearmente para a faixa operacional do atuador correspondente, enquanto as saídas destinadas a comandos discretos passam pelo processo adicional de ativação por limiar (Equação 15). A arquitetura da rede permanece uniforme do ponto de vista matemático, todos os neurônios de saída calculam sua ativação por meio da mesma função sigmoide, sendo a distinção entre comportamento binário e contínuo definida exclusivamente na etapa de pós-processamento do sinal, já fora do escopo da rede neural propriamente dita (BISHOP, 2006).

Essa padronização parcial, combinada à flexibilidade da camada de saída, permitiu que uma mesma arquitetura de rede fosse reaproveitada em todos os módulos do sistema de controle, alterando-se apenas a dimensão dos vetores de entrada, vetores de saída e a função de ativação final (HAYKIN, 2009; GOODFELLOW; BENGIO; COURVILLE, 2016).

3.3.2. Definição dos neurônios da RNA

A quantidade de neurônios da camada oculta de cada rede foi definida com base em heurística, na qual o número de neurônios da camada oculta (n_{oculta}) corresponde à média aritmética entre o número de neurônios da camada de entrada e o número de neurônios da camada de saída, arredondada para o inteiro imediatamente superior e acrescido de uma unidade. Essa relação fornece um ponto de partida razoável para a capacidade de representação da camada oculta:

$$n_{oculta} = \left\lceil \frac{n_{entrada} + n_{saída}}{2} \right\rceil + 1 \quad (17)$$

Onde:

$n_{entrada}$ é o número de neurônios da camada de entrada;

$n_{saída}$ é o número de neurônios da camada de saída;

$\lceil \cdot \rceil$ representa o operador de arredondamento para o inteiro superior.

Diferentemente da camada oculta, cujo dimensionamento decorre de uma heurística de cálculo, a quantidade de neurônios da camada de saída não é um parâmetro livre ou ajustável empiricamente, sendo determinado diretamente pela natureza do problema que o sistema deve resolver, correspondendo ao número de variáveis de controle que a rede precisa produzir para comandar os atuadores da planta simulada.

A quantidade de neurônios da camada de entrada, assim como a da camada de saída, também não constitui um parâmetro livre ou ajustável empiricamente, sendo determinada pelo número de variáveis de estado que cada rede precisa monitorar para tomar sua decisão, tais como leituras de sensores e variáveis de estado. Dessa forma, redes responsáveis por decisões que dependem de um conjunto mais amplo de variáveis de estado, apresentam uma camada de entrada proporcionalmente maior do que redes cuja decisão depende de um número mais

restrito de variáveis. Partindo deste princípio básico, a quantidade de neurônio de cada camada para cada uma das redes, é definido conforme a Tabela 5.

Tabela 5 – Quantidade de Neurônios por camada

RNA	Quantidade de Neurônios		
	Entrada	Saída	Oculto
Prioridade	4	3	5
Esteira	8	2	6
Expedição	4	4	5
Elevador	2	5	5
Despacho	2	2	3

Fonte: Elaborado pelo autor (2026)

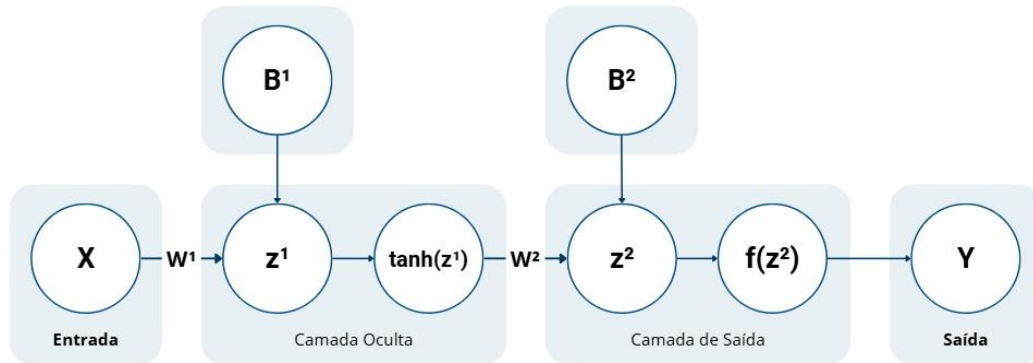
Segundo Bishop (2006), a quantidade de neurônios da camada de saída deve corresponder exatamente à dimensionalidade da informação que se deseja produzir, de modo que adicionar ou remover neurônios dessa camada altera diretamente o significado e a interpretação da resposta produzida pelo modelo.

3.3.3. Estrutura matemática da RNA

Todas as Redes Neurais Artificiais desenvolvidas neste trabalho seguem a mesma estrutura matemática fundamental, independentemente da função específica desempenhada no sistema. Embora tenham sido divididas funcionalmente em diferentes módulos, todas as redes são compostas por camada de entrada, uma camada oculta e camada de saída, com treinamento supervisionado por retro propagação do erro (RUMELHART; HINTON; WILLIAMS, 1986 apud HAYKIN, 2001). Segundo o Teorema da Aproximação Universal, uma rede MLP com apenas uma camada oculta e número suficiente de neurônios é capaz de aproximar qualquer função contínua com o grau de precisão desejado, desde que a função de ativação empregada seja não linear (CYBENKO, 1989), o que justifica a adoção dessa mesma arquitetura básica para tratar problemas de naturezas distintas dentro do sistema.

Uma formulação matricial apropriada representa de forma compacta o processo de propagação direta (forward pass), no qual a camada oculta primeiro extrai uma representação intermediária não linear dos dados de entrada por meio da função tangente hiperbólica, para em seguida essa representação ser combinada linearmente e transformada pela função de ativação de saída específica de cada rede (HAYKIN, 2009; SILVA; SPATTI; FLAUZINO, 2016). A sequência pode ser observada na Figura 35.

Figura 35 – Fluxo para Forward Pass



Fonte: Elaborado pelo autor (2026)

Cada elemento do vetor de entrada é multiplicado pelo peso correspondente da conexão que o liga a cada neurônio da camada seguinte, e os produtos resultantes ($z^{[n]}$) são somados ao respectivo termo de bias, produzindo o potencial de ativação de cada neurônio daquela camada (HAYKIN, 2001). Para o vetor soma da camada oculta $z^{[1]}$, temos:

$$z^{[1]} = W^{[1]} \cdot X + B^{[1]} \quad (18)$$

Onde:

X é o vetor das entradas da rede;

$W^{[1]}$ representa os pesos da camada oculta;

$B^{[1]}$ são os bias da camada oculta.

De forma semelhante, o vetor de potencial de ativação da camada de saída ($z^{[2]}$) é obtido a partir da aplicação dos pesos dessa camada sobre o vetor já processado pela função de ativação da camada oculta, resultando em:

$$z^{[2]} = W^{[2]} \cdot \tanh(z^{[1]}) + B^{[2]} \quad (19)$$

Onde:

$W^{[2]}$ representa os pesos da camada de saída;

$B^{[2]}$ são os bias da camada de saída.

Essa estrutura em duas etapas permite a modelagem de relações não lineares complexas entre variáveis de entrada e saída, sendo a base matemática de praticamente toda a

literatura de aprendizado profundo aplicado a problemas de classificação (GOODFELLOW; BENGIO; COURVILLE, 2016). Do ponto de vista matemático, o comportamento da saída (Y) de cada rede pode ser descrito por uma única equação geral que representa o mapeamento entre vetor de entrada e vetor de saída aplicando a função de saída $f(\cdot)$ correspondente:

$$Y = f(W^{[2]} \cdot \tanh(W^{[1]} \cdot X + B^{[1]}) + B^{[2]}) \quad (20)$$

A capacidade representacional obtida ao adicionar muitas camadas ocultas tende a trazer benefícios decrescentes em problemas de dimensionalidade moderada, favorecendo a escolha por uma estrutura mais enxuta, de treinamento mais rápido e de menor custo computacional para execução em tempo real (HAYKIN, 2009).

3.3.4. Composição das Redes Neurais

O sistema de controle inteligente da planta simulada é composto por cinco redes neurais logicamente independentes, cada uma responsável por um subconjunto específico de decisões dentro do fluxo produtivo. Embora todas compartilhem o mesmo modelo de Perceptron Multicamadas, com camada de entrada, uma camada oculta ativada por tangente hiperbólica e camada de saída ativada conforme a natureza do problema tratado, cada rede foi projetada com um vetor próprio de entradas e saídas, refletindo as variáveis de estado e os atuadores associados à etapa do processo que controla.

Cabe destacar que, embora o sistema seja composto por cinco redes distintas do ponto de vista lógico, a rede de Expedição é instanciada duas vezes durante a operação da planta, uma para cada linha secundária de produção (A e B). Essa duplicação decorre do fato das duas linhas compartilharem exatamente a mesma estrutura de entradas, saídas e lógica de decisão, distinguindo-se apenas quanto ao conjunto específico de sensores e atuadores a que cada instância está associada. Por esse motivo, uma única versão de pesos, obtida a partir de um único processo de treinamento, é compartilhada pelas duas instâncias, sem a necessidade de conjuntos de dados ou ciclos de treinamento independentes para cada linha. A Tabela 6 sintetiza essa relação entre as redes definidas logicamente e as instâncias efetivamente em execução na planta.

Tabela 6 – Instâncias das redes neurais

RNA	Instâncias	Associação
Prioridade	1	Roteamento de destinos dos itens
Esteira	1	Esteira principal e inserir itens no sistema
Expedição	2	Linha secundária A/Linha secundária B
Elevador	1	Elevador e esteiras associadas dos três níveis
Despacho	1	Esteiras de saída A e B

Fonte: Elaborado pelo autor (2026)

Dessa forma, embora o sistema opere com seis instâncias de rede em execução simultânea durante o ciclo produtivo, a arquitetura lógica de controle permanece composta por cinco módulos especializados. Cada rede opera de maneira relativamente independente das demais, comunicando-se apenas por meio das variáveis de estado compartilhadas entre etapas consecutivas do fluxo.

3.3.4.1. Prioridade

A rede de Prioridade é a primeira a ser executada em cada ciclo de varredura, sendo responsável por decidir, a cada novo item identificado na esteira principal, qual dos dois pedidos ativos (A ou B) deve recebê-lo ou se deve encaminhar para o armazenamento vertical, caso não tenha demanda para o item. O vetor de entrada é montado dinamicamente de acordo com qual item foi efetivamente detectado (X, Y ou Z), no instante da leitura.

A seleção do vetor de entrada segue a lógica de faixas de valor já estabelecida na seção 3.2.2 pela leitura do sensor óptico, no qual é realizado uma normalização (X) para limitar esses valores no intervalo (0 a 1), sendo uma forma de ajudar no processo de treinamento, garantindo que os valores estejam na mesma escala da saída esperada das demais camadas, possibilitando uma convergência mais rápida na etapa de treinamento e evitando uma explosão de valores dos pesos relativos. Esse cuidado com o pré-processamento dos dados de entrada é consistente com a literatura, que aponta a etapa de tratamento e normalização dos dados como um dos fatores mais determinantes para o desempenho de aplicações industriais baseadas em redes neurais, podendo consumir a maior parte do esforço de desenvolvimento do projeto (LENNOX et al., 2001). A partir da escala de valores dados na Tabela 1, a normalização da entrada x é definida por através da Equação 21.

$$X = \frac{x}{10} \quad (21)$$

Uma vez com todos os valores de entrada normalizados, é possível atribuir às variáveis de estado a caracterização de cada item detectado por meio de faixas de valor mutuamente exclusivas, sendo de 0,15 a 0,35 para o item X, de 0,35 a 0,65 para o item Y e de 0,65 a 0,95 para o item Z. Essas faixas foram definidas com espaçamento uniforme dentro do intervalo normalizado (0 a 1), reservando ainda uma margem inferior a 0,15 para representar a ausência de item válido no ciclo de leitura, essa margem evita que pequenas variações no processo de normalização ou eventuais ruídos de leitura do sensor levem à classificação incorreta de um item nas proximidades dos limites de cada faixa.

Algoritmo 1 – Pseudocódigo de gerenciamento de tempo e itens

Entrada:

```
prodMain          // valor normalizado do item lido na esteira principal
pedidoA, pedidoB  // estrutura com encomenda, feitos de cada pedido
tempoA ← pedidoA.tempo // tempo de processamento do pedido A
tempoB ← pedidoB.tempo // tempo de processamento do pedido B
```

Procedimento:

```
1. Se prodMain < 0.15 então
    vetor_entrada ← [0, 0, 0, 0] // nenhum item válido no ciclo
2. Senão se prodMain < 0.35 então // item X detectado
    faltamA ← (pedidoA.encomenda[X] - pedidoA.feitos[X]) * 0.1
    faltamB ← (pedidoB.encomenda[X] - pedidoB.feitos[X]) * 0.1
    vetor_entrada ← [faltamA, faltamB, tempoA, tempoB]
3. Senão se prodMain < 0.65 então // item Y detectado
    faltamA ← (pedidoA.encomenda[Y] - pedidoA.feitos[Y]) * 0.1
    faltamB ← (pedidoB.encomenda[Y] - pedidoB.feitos[Y]) * 0.1
    vetor_entrada ← [faltamA, faltamB, tempoA, tempoB]
4. Senão se prodMain < 0.95 então // item Z detectado
    faltamA ← (pedidoA.encomenda[Z] - pedidoA.feitos[Z]) * 0.1
    faltamB ← (pedidoB.encomenda[Z] - pedidoB.feitos[Z]) * 0.1
    vetor_entrada ← [faltamA, faltamB, tempoA, tempoB]
5. Retornar vetor_entrada
```

Fonte: Elaborado pelo autor (2026)

O Algoritmo 1, estabelece como o vetor de entrada da rede é complementado por dois conjuntos de variáveis, calculadas de forma específica para o item detectado. O primeiro conjunto corresponde às quantidades restantes de cada pedido, obtidas pela diferença entre a quantidade encomendada e a quantidade já produzida daquele item específico, escalonada pelo mesmo fator de normalização. Em seguida tem-se as variáveis de tempo relativo, calculadas por uma função complementar responsável por normalizar o tempo decorrido desde o início de cada pedido em relação ao pedido mais antigo em produção, de modo que o pedido de maior

tempo sempre apresente valor próximo de 1, enquanto o tempo do outro representa uma parcela deste valor, garantindo que a rede receba um indicador de urgência pelo tempo de espera (HAYKIN, 2009). Consolidando as variáveis descritas, o vetor de entrada da rede é composto, portanto, por quatro variáveis, sintetizados na Tabela 7.

Tabela 7 – Conjunto de entradas da rede Prioridade

Nome	Fonte	Tipo	Descrição
Item faltante em A	Variável	Analógica	Diferença entre encomendado e produzido em A
Item faltante em B	Variável	Analógica	Diferença entre encomendado e produzido em B
Tempo produção de A	Variável	Analógica	Tempo de processamento de pedidos em A
Tempo produção de B	Variável	Analógica	Tempo de processamento de pedidos em B

Fonte: Elaborado pelo autor (2026)

Após o cálculo do vetor de entrada, a rede executa o forward pass, produzindo três saídas na forma de distribuição de probabilidade via Softmax. Essas probabilidades passam por uma função complementar responsável por identificar a alternativa de maior valor e convertê-la em um vetor binário one-hot, no qual apenas a posição vencedora recebe o valor 1 (ativa) e as demais, 0 (inativa). O resultado é utilizado para acionar simultaneamente o pivô e o atuador de movimento da linha selecionada (A ou B), caso a prioridade escolhida seja o armazenamento, é acionado um atuador luminoso (LED) responsável por indicar que o destino selecionado é o elevador. A Tabela 8 apresenta essa composição

Tabela 8 – Conjuntos de saídas da rede Prioridade

Recurso	Destino	Tipo	Descrição
Pivô A	Atuador/Variável	Digital	Indicador de prioridade para A
Pivô B	Atuador/Variável	Digital	Indicador de prioridade para B
Prioridade Elevador	Atuador/Variável	Digital	Indicador de prioridade para Elevador

Fonte: Elaborado pelo autor (2026)

Essa forma de codificação, quatro entradas dinamicamente reaproveitadas para qualquer um dos três tipos de item, reduz consideravelmente a dimensionalidade do problema em relação a uma abordagem que tratasse cada tipo de item com suas próprias variáveis dedicadas, favorecendo tanto a velocidade de treinamento quanto a generalização da rede entre

os diferentes cenários de produção enfrentados pela planta. Ainda assim, a rede mantém sua capacidade de decisão sensível ao contexto, uma vez que o mesmo vetor de entrada é interpretado de forma distinta a cada ciclo, dependendo exclusivamente do item que está sendo processado naquele instante.

3.3.4.2. Esteira

A rede de Esteira é executada logo após a rede de Prioridade, sendo responsável por duas decisões complementares: controlar a velocidade da esteira principal e determinar o momento adequado para solicitar a inserção de um novo produto pelo módulo emissor. A rede Esteira atua sobre o fluxo de entrada do sistema como um todo, considerando o estado de ocupação de todos os pontos de recebimento antes de autorizar a chegada de um novo item.

O vetor de entrada dessa rede é composto por oito variáveis, sendo quatro delas indicadores de ocupação: o próprio ponto de leitura da esteira principal, da linha secundária A, da linha secundária B e do elevador de armazenamento, além de três correspondentes à decisão já tomada pela rede de Prioridade no mesmo ciclo, permitindo que a rede de Esteira ajuste seu comportamento em função do destino que acabou de ser definido para o item corrente e a oitava variável corresponde ao nível instantâneo do elevador, lido diretamente do registrador analógico correspondente. A Tabela 9 define o conjunto de entradas da rede Esteira.

Tabela 9 – Conjunto de entradas da rede Esteira

Nome	Fonte	Tipo	Descrição
Item no sistema	Variável	Analógica	Determina que a item se movendo pelo sistema
Item na esteira A	Variável	Analógica	Determina que a esteira A está ocupada
Item na esteira B	Variável	Analógica	Determina que a esteira B está ocupada
Item no elevador	Variável	Analógica	Determina que o elevador está ocupado
Pivô A	Atuador	Digital	Indicador de prioridade para pedido A
Pivô B	Atuador	Digital	Indicador de prioridade para pedido B
LED Elevador	Atuador	Digital	Indicador de prioridade para Elevador
Posição do elevador	Sensor	Analógica	Indicador de nível de posição do elevador

Fonte: Elaborado pelo autor (2026)

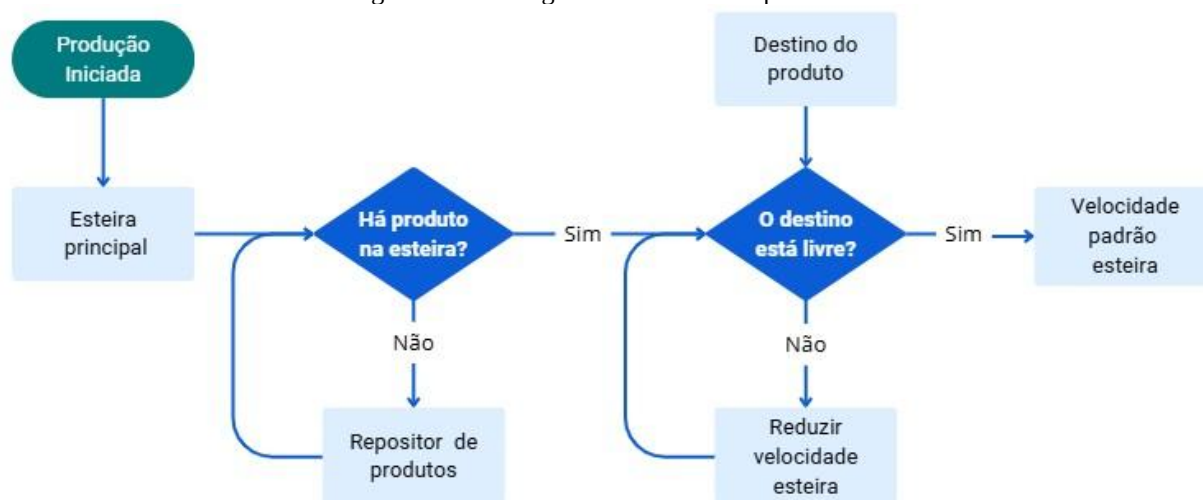
Para os sinais analógicos gerados pela planta, é recomendável realizar uma normalização, a fim de evitar viés da rede para determinadas faixas de valores. Dessa forma, a normalização no intervalo de 0 a 1 é estabelecida conforme demonstrado na Equação 22.

$$X = \frac{x}{1000} \quad (22)$$

Os quatro indicadores de ocupação não são lidos diretamente de um único sensor, mas sim mantidos como variáveis de estado calculadas ao longo do ciclo de controle, seguindo o mesmo princípio de consolidação discutido na seção 3.2.6, no qual, cada uma delas assume o valor do item detectado assim que este entra no respectivo trecho da planta, e retorna a zero apenas quando o sensor de saída correspondente confirma que o item já foi retirado daquele ponto, evitando que a rede autorize uma nova inserção enquanto qualquer um dos quatro destinos ainda estiver ocupado.

A dependência direta da saída da rede de Prioridade como entrada da rede de Esteira caracteriza uma relação sequencial entre os dois módulos, no qual a decisão de destino é tomada primeiro, e somente então a rede de Esteira avalia se as condições operacionais permitem a efetiva liberação de um novo item para aquele destino. Essa ordem de execução, replicada a cada ciclo de varredura, é o que garante que a velocidade da esteira principal seja ajustada de forma coerente com o roteamento já definido, como demonstrado na Figura 36.

Figura 36 – Fluxograma de entrada de produtos



Fonte: Elaborado pelo autor (2026)

As duas saídas produzidas pela rede são tratadas de forma distinta, sendo a primeira, de natureza digital, aciona o módulo emissor sempre que seu valor ultrapassa o limiar (0,95), solicitando a inserção de um novo item na esteira principal e a segunda, de natureza contínua, define a velocidade da esteira principal, reescalada da faixa normalizada (Equação 16). A Tabela 10 exibe esse conjunto de dados.

Tabela 10 – Conjunto de saídas da rede Esteira

Nome	Tipo	Descrição
Repositor	Digital	Acionar inserção de itens no sistema
Esteira principal	Analógica	Controle de velocidade da esteira principal

Fonte: Elaborado pelo autor (2026)

Esse arranjo evidencia uma característica importante da arquitetura modular do sistema, que embora cada rede opere de forma matematicamente independente, a rede Esteira depende funcionalmente da saída da rede de Prioridade, configurando uma cadeia de decisão em que o resultado de um módulo é consumido como entrada por outro, sem que isso comprometa a separação lógica entre ambos e não exige qualquer acoplamento direto entre seus pesos ou parâmetros internos.

3.3.4.3. Expedição

A rede de Expedição é responsável por controlar o manipulador cartesiano e a esteira auxiliar de cada linha secundária, executando a etapa final de retirada dos produtos já processados. Como o sistema possui duas linhas secundárias com lógica operacional idêntica, a mesma arquitetura de rede é instanciada duas vezes de forma independente, uma para a linha A e outra para a linha B, cada arquitetura operando sobre seu próprio conjunto de sensores e atuadores, sem qualquer compartilhamento de estado entre as duas instâncias durante a execução.

O vetor de entrada de cada instância é composto por quatro variáveis, sendo dois sinais digitais, correspondentes ao sensor de fim de esteira e ao sensor de presença de material na extremidade do manipulador, e dois sinais analógicos, correspondentes à posição instantânea dos eixos horizontal e vertical do manipulador cartesiano, lidos diretamente dos registradores de posição.

A Tabela 11 apresenta o conjunto de dados de entrada, definido para o treinamento da rede Expedição.

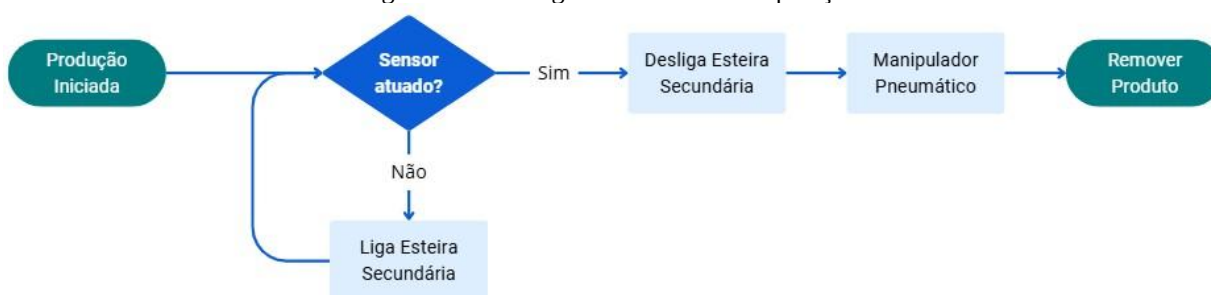
Tabela 11 – Conjunto de entradas da rede Expedição

Nome	Fonte	Tipo	Descrição
Sensor fim da esteira	Sensor	Digital	Sensor posicionado final da esteira de expedição
Sensor na garra	Sensor	Digital	Detectar item na ponta da garra do manipulador
Posição eixo X	Sensor	Analógica	Posição do manipulador no eixo horizontal
Posição eixo Z	Sensor	Analógica	Posição do manipulador no eixo vertical

Fonte: Elaborado pelo autor (2026)

Diferentemente das redes de Prioridade e Esteira, a rede de Expedição não depende de nenhuma variável calculada por outro módulo do sistema, operando exclusivamente sobre leituras diretas de sensores da própria linha controlada. A Figura 37 mostra o fluxo de controle da rede.

Figura 37 – Fluxograma controle de expedição



Fonte: Elaborado pelo autor (2026)

A partir das quatro entradas, a rede produz um vetor de saída com quatro variáveis, todas ativadas por função sigmoide, mas interpretadas de forma distinta conforme sua natureza: a primeira é ajustada pelo limiar de ativação (Equação 15), acionando a garra pneumática apenas quando a confiança da rede é suficientemente alta; as três saídas seguintes permanecem contínuas, sendo reescaladas para a faixa de 0 a 1000 antes de serem escritas nos registradores correspondentes ao deslocamento dos eixos e à velocidade da esteira auxiliar daquela linha. As saídas são representadas de acordo com a Tabela 12.

Tabela 12 – Conjunto de saídas da rede Expedição

Nome	Tipo	Descrição
Agarrar	Digital	Agarra item na ponta do eixo vertical do manipulador
Eixo X	Analógica	Controle de posição eixo horizontal
Eixo Z	Analógica	Controle de posição eixo vertical
Esteira robô	Analógica	Controle Velocidade da esteira de expedição

Fonte: Elaborado pelo autor (2026)

Embora o manipulador pudesse, em tese, ser controlado por meio de saídas puramente digitais, como ocorre com os finais de curso em cilindros pneumáticos convencionais, essa abordagem resultaria em falta de precisão no posicionamento dos eixos, uma vez que o deslocamento ficaria restrito a um número limitado de posições fixas predefinidas, sem a possibilidade de ajuste fino. Por esse motivo, optou-se pelo controle analógico dos eixos, o que confere ao manipulador maior flexibilidade para alcançar posições intermediárias e reduz o risco de falhas na captura decorrentes de pequenas variações no ponto de parada do item na esteira secundária (BOLTON, 2015).

Figura 38 – Eixos do manipulador cartesiano

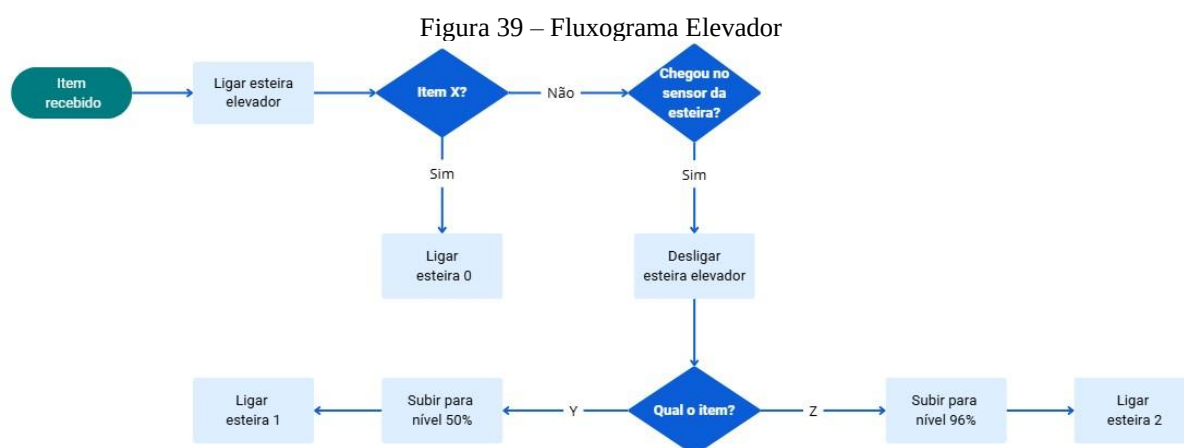


Fonte: Elaborado pelo autor (2026)

A Figura 38 ilustra os dois eixos de deslocamento do manipulador cartesiano, horizontal (X) e vertical (Z), que permite à rede neural ajustar continuamente a posição da garra até a coordenada exigida para a captura do item.

3.3.4.4. Elevador

A rede Elevador é responsável por posicionar a plataforma de elevação no nível correspondente ao tipo de item a ser armazenado e por acionar a esteira interna encarregada de transferi-lo até o ponto de remoção correto, sendo executada sempre que um produto é encaminhado ao sistema de armazenamento vertical. A Figura 39 exibi o fluxograma de funcionamento do Elevador



Fonte: Elaborado pelo autor (2026)

Diferentemente das demais redes do sistema, cujo vetor de entrada reúne múltiplas variáveis de estado consolidadas, a rede de Elevador opera com apenas duas entradas: a posição instantânea da plataforma, lida diretamente do registrador analógico e o valor normalizado do item que foi efetivamente encaminhado ao elevador naquele ciclo, seguindo a mesma convenção de faixas (0,15 a 0,95) empregada na leitura do sensor óptico da esteira principal. Essa economia de entradas é possível porque, diferentemente do roteamento entre linhas de atendimento, que depende do estado de múltiplos pedidos simultâneos, o destino de armazenamento de cada item é determinado exclusivamente pelo seu próprio tipo, não havendo necessidade de considerar variáveis de outros subsistemas da planta. A Tabela 13 resume as duas entradas da rede.

Tabela 13 – Conjunto de entradas da rede Elevador

Nome	Fonte	Tipo	Descrição
Nível do elevador	Sensor	Analógica	Posição da esteira do elevador em relação ao solo
Item no elevador	Variável	Analógica	Item detectado pelo sensor óptico no sistema

Fonte: Elaborado pelo autor (2026)

A partir dessas duas entradas, a rede produz um vetor de saída com cinco variáveis, todas ativadas por função sigmoide: a primeira, de natureza contínua, define o setpoint de posicionamento vertical da plataforma, sendo reescalada para a escala de 0 a 1000 antes de ser escrita no registrador correspondente. As quatro saídas seguintes são ativadas pelo limiar de 0.95, controlando, respectivamente, o acionamento da esteira interna do próprio elevador e das três esteiras laterais associadas a cada um dos níveis de armazenamento (item X, item Y e item Z). A Tabela 14 apresenta as cinco saídas e suas funções no sistema.

Tabela 14 – Conjunto de saídas da rede Elevador

Nome	Tipo	Descrição
Set Point Elevador	Analógica	Nível desejado para posicionar o elevador
Esteira Elevador	Analógica	Deslocar item para movimentação do elevador
Esteira 0	Digital	Esteira de saída do elevador do item X
Esteira 1	Digital	Esteira de saída do elevador do item Y
Esteira 2	Digital	Esteira de saída do elevador do item Z

Fonte: Elaborado pelo autor (2026)

Ao concentrar em uma única rede tanto o posicionamento vertical quanto o acionamento das quatro esteiras associadas, a arquitetura evita a necessidade de uma lógica de sincronização externa entre módulos distintos, uma vez que a própria rede aprende, a partir dos dados de treinamento, a relação entre a posição alcançada pela plataforma e o momento correto de habilitar a esteira do nível correspondente.

3.3.4.5. Despacho

A rede Despacho é responsável pela etapa final do fluxo produtivo, controlando o acionamento das esteiras de saída que efetivamente liberam os produtos já expedidos por cada linha secundária para a área que efetivamente remove o produto do processo.

O vetor de entrada da rede é composto por apenas duas variáveis, uma para cada linha secundária, cada uma representando a quantidade de itens ainda pendentes de produção no pedido correspondente daquela linha. A cada item concluído, o valor correspondente é decrementado em um passo fixo a cada ciclo, aproximando-se progressivamente de zero à medida que a produção avança. Ao atingir o valor zero, sinal de que o pedido foi integralmente

A partir dessas entradas, a rede produz duas saídas, ambas digitais, obtidas por meio da ativação pelo limiar (Equação 15), cada uma responsável por acionar a esteira de saída correspondente a uma das duas linhas de despacho. Diferentemente das demais redes do sistema, cujas saídas combinam comandos digitais e analógicos, a rede de Despacho opera exclusivamente sobre variáveis discretas, refletindo a natureza da decisão que executa a ação de liberar ou não o lote já formado em cada esteira de acumulação para a área de remoção do item do sistema. A Tabela 16 apresenta as duas saídas da rede.

Tabela 16 – Conjunto de saídas da rede Despacho

Nome	Tipo	Descrição
Esteira saída A	Digital	Move esteira de saída do manipulador da expedição A
Esteira saída B	Digital	Move esteira de saída do manipulador da expedição B

Fonte: Elaborado pelo autor (2026)

O fato de essa rede operar com poucas entradas e saídas, a menor dimensionalidade entre todas as redes do sistema, não significa, que sua lógica de decisão seja trivial. A rede Despacho demonstra uma variação particular na forma de estruturar a entrada, utilizando uma única variável contínua para representar, ao mesmo tempo, a quantidade pendente de produção e um mecanismo implícito de temporização.

3.4. Arquitetura de controle

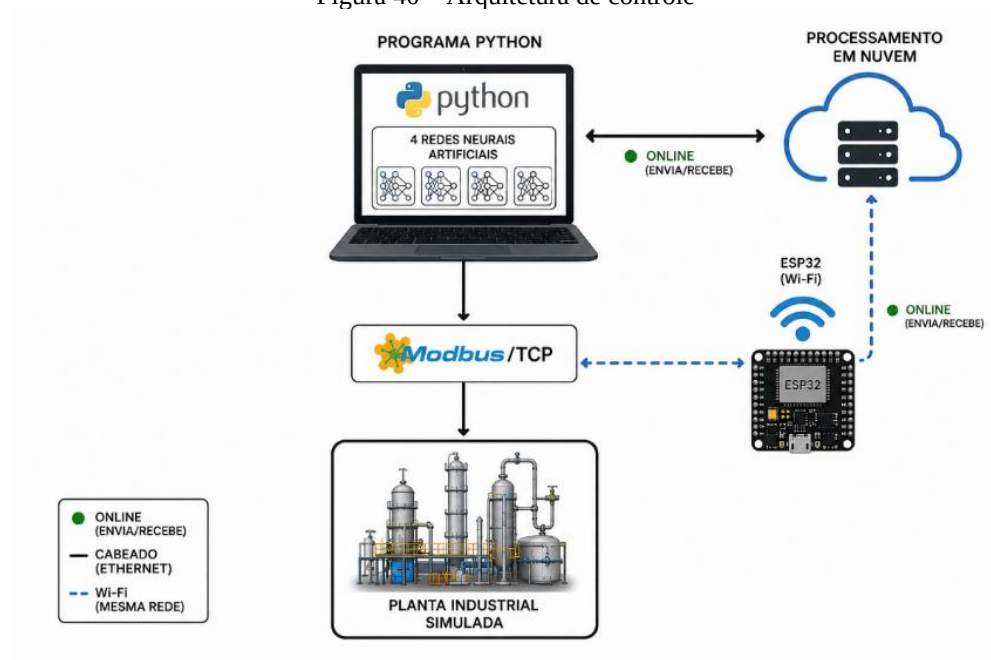
A arquitetura de controle adotada neste trabalho retoma, na prática, o conceito de Controle Distribuído segmentadas entre múltiplos módulos especializados, interligados por uma infraestrutura de comunicação industrial. Nesse sentido, as redes neurais que compõem o sistema de controle assumem o papel equivalente ao de controladores individuais em um DCS industrial tradicional, com a diferença de que, em vez de executarem lógicas fixas predefinidas, essas redes tomam decisões a partir de parâmetros adquiridos durante o processo de treinamento. Rajkumar et al. (2010) destacam que essa combinação entre distribuição funcional do controle e capacidade de processamento adaptativo é uma característica central dos sistemas ciberfísicos, sem que a falha ou a atualização de um módulo comprometa necessariamente o funcionamento dos demais. Esse modelo é amplamente utilizado em ambientes industriais modernos, especialmente em sistemas baseados em Indústria 4.0, devido à sua escalabilidade e facilidade de integração com redes de comunicação (KASSEM; MEIER, 2016).

3.4.1. Dispositivos de controle

O sistema de controle inteligente proposto neste trabalho é composto por dois dispositivos distintos, sendo um servidor local, conectado à rede por meio de uma conexão cabeada, e um dispositivo embarcado ESP32, conectado via Wi-Fi. O servidor local, desenvolvido em Python, é responsável pela execução da maior parte das funções de controle da planta e processando cinco das seis instâncias descritas na Tabela 6. O ESP32, por sua vez, é responsável pelo controle da instância B da rede Expedição, sendo que ambos os dispositivos estabelecem comunicação com a planta por meio do protocolo Modbus TCP/IP.

A divisão da rede Expedição em duas instâncias, A e B, executadas em ambientes distintos, sendo em Python por meio de uma conexão cabeada e por meio de uma conexão Wi-Fi no ESP32, que constitui uma escolha deliberada de projeto. Como se trata da mesma rede neural replicada em diferentes dispositivos, tal configuração possibilita avaliar e comparar o comportamento em diferentes condições de conexão, permitindo analisar eventuais diferenças entre a comunicação cabeada e a comunicação sem fio. A Figura 40 mostra a arquitetura de controle.

Figura 40 – Arquitetura de controle



Fonte: Elaborado pelo autor (2026)

O servidor responsável pela execução da simulação da planta no Factory I/O é identificado pelo endereço IP 192.168.1.17, mas cabe destacar que o endereço utilizado, foi

atribuído dinamicamente pelo protocolo DHCP (Dynamic Host Configuration Protocol) no ambiente de rede utilizado durante os testes. Embora esse tipo de atribuição seja prático por dispensar configuração manual para a simulação, ele exige atenção quanto à estabilidade do endereço ao longo do tempo, uma vez que o servidor DHCP pode, a critério do roteador da rede, atribuir um novo endereço IP ao servidor da planta após reinicializações ou renovações do tempo de concessão (lease time). Nesse cenário, a comunicação entre o servidor e os demais dispositivos de controle permanece viável, desde que ambos sejam devidamente reconfigurados com o novo endereço atribuído ao servidor, o que reforça a importância que para uma aplicação real, adotar-se preferencialmente um endereçamento estático ou uma reserva de IP, prática recomendada para garantir a continuidade da comunicação em redes industriais críticas (TANENBAUM; WETHERALL, 2011).

A proposta do trabalho não está vinculada a nenhuma linguagem de programação ou plataforma computacional específica. Em um ambiente industrial real, a lógica de controle atualmente implementada em Python poderia ser igualmente executada em CLPs ou em outros dispositivos dedicados a sistemas ciberfísicos industriais, projetados para operação contínua, robustez elétrica e confiabilidade em ambientes severos (BOLTON, 2015). A arquitetura híbrida desenvolvida neste projeto, parte em Python e parte embarcada em ESP32, representa uma simulação conceitual de uma arquitetura de controle distribuído aplicada a um contexto industrial, na qual diferentes dispositivos, com diferentes capacidades de processamento, cooperam para executar o controle completo de um processo industrial.

Essa divisão entre processamento externo e processamento embarcado encontra paralelo em estudos que comparam arquiteturas distribuídas, com a rede neural executada em um dispositivo externo ao controlador, e arquiteturas integradas, com a rede executada diretamente no próprio controlador industrial, evidenciando que a escolha entre as duas abordagens envolve um compromisso entre flexibilidade de desenvolvimento e tempo de ciclo de execução (SILVA, 2024).

3.4.1.1. Controle local

A maior parte do processamento das RNAs (Prioridade, Esteira, Expedição A, Despacho e Elevador) foi implementada em Python, executada em um computador conectado à mesma rede local na qual está sendo simulada a planta industrial, comunicando-se com o servidor da planta por meio do protocolo de comunicação industrial no endereço IP atribuído. Nessa infraestrutura, cada ciclo de controle segue a mesma estrutura geral:

- a) Leitura dos sensores digitais e analógicos da planta via Modbus;
- b) Tratamento e normalização das leituras em variáveis de estado;
- c) Inferência da rede neural correspondente a partir do vetor de entrada;
- d) Envio dos comandos de saída aos atuadores da planta.

Essa escolha permitiu maior flexibilidade no desenvolvimento, facilidade de integração com os demais módulos do sistema, além de simplificar a implementação e a depuração de cada uma das cinco redes. Ainda que, na configuração atual, esse computador esteja fisicamente próximo ao servidor da simulação, a comunicação depende exclusivamente do endereçamento IP entre os dispositivos, dessa forma, o mesmo sistema de controle poderia ser executado em um computador distinto da mesma rede ou em um servidor remoto, apenas com os devidos ajustes nos endereçamentos de acesso.

Cabe destacar que as redes de Expedição A e Expedição B não são duas redes distintas, mas sim duas instâncias independentes de uma mesma rede neural, resultado de um único processo de treinamento, conforme já discutido na seção 3.3.4, ou seja, ambas compartilham exatamente a mesma versão de pesos, cada instância responsável por controlar os sensores e atuadores de sua respectiva linha secundária. Enquanto a instância referente à linha A é processada nesta mesma infraestrutura local em Python, descrita anteriormente, a instância referente à linha B é executada em um dispositivo embarcado distinto.

3.4.1.2. Controle por ESP32

A instância da rede de Expedição responsável pela linha B é processada de forma independente por um microcontrolador ESP32 na linguagem C/C++, também conectado à mesma rede local por meio de Wi-Fi, acessando o servidor da planta através do mesmo endereço IP utilizado pelos demais dispositivos. O ESP32 é um sistema em chip de baixo custo, amplamente utilizado em aplicações de Internet das Coisas (IoT), que integra em um único componente, um processador de dois núcleos, memória e módulos de conectividade Wi-Fi e Bluetooth, sendo capaz de executar tarefas de processamento embarcado sem depender de um computador convencional (ESPRESSIF SYSTEMS, 2023). A adoção desse dispositivo para essa instância específica teve como propósito demonstrar a viabilidade de descentralizar fisicamente o processamento de inferência das redes neurais, aproximando a arquitetura desenvolvida do conceito de processamento na borda (edge computing), no qual parte da

inteligência do sistema é executada próxima à origem dos dados, e não exclusivamente em um servidor central (RAJKUMAR et al., 2010).

Por se tratar exatamente da mesma rede neural já processada na infraestrutura local pela instância correspondente à linha A, a execução da rede de Expedição no ESP32 permite uma comparação controlada entre os dois ambientes, isolando o efeito da plataforma de hardware e do meio de comunicação sobre o desempenho do sistema, sem introduzir variáveis adicionais associadas a diferenças de arquitetura ou de lógica de controle entre as instâncias comparadas.

3.4.2. Lógica de controle das variáveis de estado

Conforme mencionado na seção 3.2.6.3, o sistema de controle depende de um conjunto de variáveis internas que consolidam o estado de ocupação de cada ponto relevante da planta, em vez de reagir diretamente e de forma isolada a cada sinal individual de sensor. No dispositivo de controle, essa consolidação é implementada por meio de quatro variáveis de estado, cada uma associada a um dos possíveis destinos do item que trafega pela esteira principal, sendo a esteira principal propriamente dita, a linha secundária A, a linha secundária B e o elevador de armazenamento, além de duas variáveis adicionais específicas da rede de Despacho, já apresentadas na seção 3.3.4.5.

As variáveis de ocupação assumem, a qualquer instante, um de dois estados possíveis, sendo zero, quando o ponto correspondente está livre, ou o valor normalizado do item que ocupa aquela posição, seguindo a mesma convenção de faixas (0.3, 0.6 ou 0.9) estabelecida pela normalização (Equação 21) a partir da leitura do sensor óptico. Diferentemente de uma variável booleana simples, que apenas indicaria ocupado ou livre, essa abordagem permite que a própria variável de estado carregue também a informação de qual tipo de item está sendo processado naquele ponto, sendo essa mesma variável posteriormente utilizada como entrada por as redes do sistema.

A lógica de transferência entre variáveis, na qual o valor de `prodMain` é copiado para a variável de destino, sendo em seguida zerado, garante que um mesmo item nunca seja contabilizado simultaneamente em dois pontos diferentes da planta, mesmo que a detecção de entrada em um novo ponto e a confirmação de saída do ponto anterior não ocorram exatamente no mesmo ciclo de varredura. Essa transferência ocorre apenas mediante confirmação do sensor de entrada correspondente, o que evita que a variável de estado seja atualizada de forma prematura, antes que o item tenha de fato alcançado o novo ponto da planta (BOLTON, 2015).

Já a liberação de cada variável, seu retorno ao valor zero, depende de uma confirmação independente, fornecida pelo sensor de saída daquele mesmo ponto, e não de uma simples temporização. Essa separação entre o evento de ocupação e o evento de liberação é o que permite que outras redes do sistema consultem essas variáveis a qualquer momento e obtenham uma representação do estado da planta, sem risco de considerar um ponto como livre enquanto o item correspondente ainda estiver fisicamente presente. O Algoritmo 3 descreve a lógica de atribuição desses estados a cada ciclo de controle.

Algoritmo 3 – Pseudocódigo de tratamento das variáveis de estado

```

Entrada:
    prodMain, prodA, prodB, prodElev    // variáveis de estado (0 = livre)
    itemLido                            // valor normalizado do item na
esteira principal
    sensorEntradaA, sensorEntradaB      // sensores de entrada das linhas
secundárias
    sensorEntradaElevador               // sensor de entrada do elevador
    sensorSaidaA, sensorSaidaB          // sensores de confirmação de saída
    sensorSaidaElevador                 // sensor de confirmação de saída do elevador

Procedimento:
1. Se itemLido > 0 e prodMain = 0 então
    prodMain ← itemLido                // novo item aguardando roteamento
2. Se sensorEntradaA ativo e prodA = 0 então
    prodA ← prodMain                   // item transferido para a linha A
    prodMain ← 0
3. Se sensorEntradaB ativo e prodB = 0 então
    prodB ← prodMain                   // item transferido para a linha B
    prodMain ← 0
4. Se sensorEntradaElevador inativo e prodElev = 0 então
    prodElev ← prodMain                // item transferido para o elevador
    prodMain ← 0
5. Se sensorSaidaA confirma retirada então
    prodA ← 0                          // linha A liberada
6. Se sensorSaidaB confirma retirada então
    prodB ← 0                          // linha B liberada
7. Se sensorSaidaElevador confirma retirada então
    prodElev ← 0                       // elevador liberado
8. Retornar prodMain, prodA, prodB, prodElev

```

Fonte: Elaborado pelo autor (2026)

As duas variáveis de pendência associadas à rede de Despacho, cuja lógica de atualização foi apresentada no Algoritmo 2 da seção 3.3.4.5, complementam esse mecanismo de controle por variáveis de estado de forma distinta das quatro anteriores. As variáveis prodMain, prodA, prodB e prodElev dependem de confirmação direta de sensores de entrada e saída para serem atualizadas, enquanto as variáveis de pendência da linha A e da linha B são atualizadas continuamente a cada ciclo de varredura, independentemente da ocorrência de

qualquer evento discreto, funcionando como um mecanismo próprio de acompanhamento gradual do estado de produção de cada pedido.

3.4.3. Processo forward pass

O processo de forward pass foi implementado para execução cíclica e determinística, seguindo a mesma lógica em ambos os dispositivos de controle descritos na seção anterior. Após a etapa de treinamento, os pesos sinápticos e os termos de bias foram definidos e armazenados como parâmetros fixos no sistema, permitindo que, a cada ciclo de varredura, o controlador executasse apenas operações de multiplicação matricial, soma acumulada e aplicação das funções de ativação previamente estabelecidas (Equação 20). Dessa forma, o processamento ficou restrito à etapa de inferência, sem necessidade de atualização de pesos durante a operação, o que reduz significativamente a carga computacional exigida de cada dispositivo (HAYKIN, 2009).

A execução ocorre em laço contínuo sincronizado com a simulação da planta, reproduzindo o comportamento típico de varredura (scan cycle) de sistemas de automação industrial, no qual o controlador realiza repetidamente a leitura de entradas, o processamento lógico e a atualização de saídas dentro de um intervalo de tempo determinístico (BOLTON, 2015). Em cada ciclo, o sistema realiza a leitura das variáveis de entrada, a organização do vetor de estado, o cálculo do potencial de ativação nas camadas da rede e a geração das saídas correspondentes.

Embora neste projeto o forward pass tenha sido implementado majoritariamente em Python e C/C++, o algoritmo de inferência da rede neural não depende de nenhuma linguagem específica, sendo que, após o treinamento e a definição dos parâmetros, o mesmo cálculo pode ser igualmente implementado em um CLP, por meio de linguagens como Structured Text, conforme a norma IEC 61131-3, uma vez que a estrutura do algoritmo permanece idêntica, independentemente da plataforma de execução escolhida (GOODFELLOW; BENGIO; COURVILLE, 2016).

O Algoritmo 4 apresenta a implementação do processo de forward pass descrito anteriormente, detalhando as etapas necessárias para a execução do procedimento. Essa implementação pode ser aplicada de forma equivalente a qualquer uma das plataformas de execução consideradas, mantendo a mesma lógica de processamento e garantindo a padronização do processo independentemente da plataforma utilizada.

Algoritmo 4 – Pseudocódigo do processo forward pass

```

Entrada:
    vetor X[n]                // entradas da rede
    matriz W1[n][h]           // pesos camada entrada-oculta
    vetor B1[h]                // bias camada oculta
    matriz W2[h][m]           // pesos camada oculta-saída
    vetor B2[m]                // bias camada de saída

Saída:
    vetor Y[m]                // saída da rede

Procedimento:
    1. Para cada neurônio oculto j de 1 até h faça:
        soma_oculta[j] ← 0
        Para cada entrada i de 1 até n faça:
            soma_oculta[j] ← soma_oculta[j] + X[i] * W1[i][j]
        FimPara
        soma_oculta[j] ← soma_oculta[j] + B1[j]
        H[j] ← f(soma_oculta[j]) // função de ativação
    FimPara
    2. Para cada neurônio de saída k de 1 até m faça:
        soma_saida[k] ← 0
        Para cada neurônio oculto j de 1 até h faça:
            soma_saida[k] ← soma_saida[k] + H[j] * W2[j][k]
        FimPara
        soma_saida[k] ← soma_saida[k] + B2[k]
        Y[k] ← g(soma_saida[k]) // função de ativação da saída
    FimPara
    3. Retornar Y

```

Fonte: Adaptado de HAYKIN (2009)

O elemento central permanece o processo de inferência, que consiste no cálculo das saídas a partir das entradas e dos pesos previamente treinados, caracterizando a etapa operacional da rede neural, processo computacionalmente leve o suficiente para ser executada de forma equivalente tanto em um computador quanto em um dispositivo embarcado de baixo custo, como demonstrado pela implementação da rede Expedição (RAJKUMAR et al., 2010).

3.4.4. Computação em nuvem

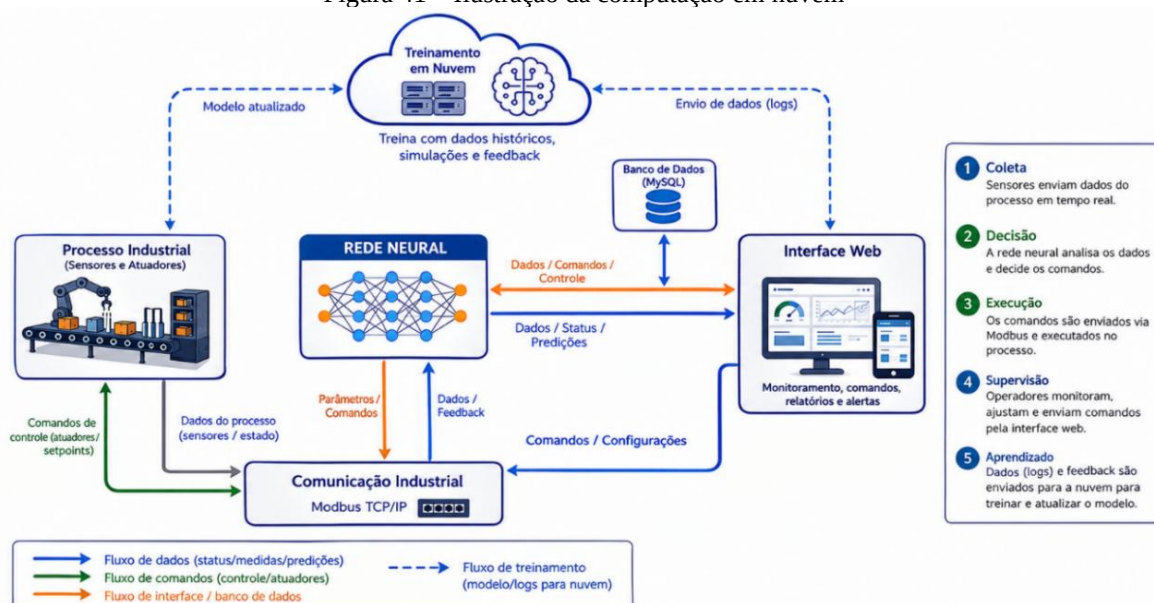
A computação em nuvem foi incorporada à metodologia como elemento estruturante da camada de supervisão e inteligência do sistema, atuando em um nível hierárquico superior de decisão frente à camada de controle. Diferentemente desta última, responsável pela execução em tempo real das redes neurais e pelo comando direto dos atuadores da planta via Modbus, a camada em nuvem concentra funções de gerenciamento, armazenamento dos dados de treinamento e do histórico de execuções, e o próprio processamento de treinamento dos modelos, devolvendo à camada de controle, o conjunto de pesos atualizado a ser utilizado pelas redes neurais. Além dessas funções estratégicas, a camada

em nuvem também é capaz de intervir diretamente sobre a operação da planta por meio do acionamento geral do sistema, permitindo que o usuário inicie ou interrompa remotamente o funcionamento do processo.

Cabe destacar que, mesmo quando o usuário aciona ou interrompe remotamente o funcionamento do sistema por meio da interface web, esse comando não é processado pela nuvem, mas sim recebido pela camada de controle, responsável por traduzi-lo em comandos efetivos sobre os atuadores da planta. Preserva-se, dessa forma, uma separação clara de responsabilidades, no qual a nuvem concentra a inteligência de gerenciamento, armazenamento e treinamento, enquanto toda comunicação com a planta permanece sob responsabilidade exclusiva da camada de controle.

Segundo Marinescu (2018), o processamento em nuvem caracteriza-se pela disponibilização sob demanda de recursos computacionais escaláveis, incluindo processamento, armazenamento e serviços de aplicação, sem que o usuário final precise investir em infraestrutura física própria para cada um desses serviços. De forma complementar, Erl, Puttini e Mahmood (2013) destacam que a arquitetura em nuvem permite separação clara entre infraestrutura, plataforma e aplicação, favorecendo modularidade e elasticidade do sistema, características que na prática, permitem que os módulos de gerenciamento, banco de dados e treinamento das redes sejam desenvolvidos, atualizados e escalados de forma independente entre si, sem impactar diretamente outras camadas do sistema. A Figura 41 mostra o fluxo de troca de informação da rede com o sistema em nuvem.

Figura 41 – Ilustração da computação em nuvem



Fonte: Elaborado pelo autor (2026)

Essa separação entre camada de controle e camada de nuvem também reflete uma distinção importante quanto aos requisitos temporais de cada parte do sistema: enquanto a camada de controle opera sob exigências de resposta em tempo real, executando ciclos de varredura contínuos, a camada em nuvem lida com operações que toleram maior latência, tarefas que não comprometem a operação da planta e que, por esse motivo, podem ser delegadas a uma infraestrutura remota (BUYA; BROBERG; GOSCINSKI, 2011). Conforme discutido por Kagermann, Wahlster e Helbig (2013) e Rajkumar et al. (2010), no contexto moderno, a integração entre dispositivos físicos e serviços em nuvem é elemento fundamental para a construção de sistemas ciberfísicos.

3.5. Integração web

A camada de supervisão do sistema é disponibilizada ao usuário por meio de uma interface web, hospedada na infraestrutura em nuvem, responsável por consolidar tanto o armazenamento dos dados operacionais quanto o envio de comandos que influenciam o comportamento da planta. Essa interface é estruturada em três páginas principais, cada uma responsável por um conjunto específico de funções dentro da arquitetura de gerenciamento remoto proposta, evitando que informações de naturezas distintas sejam concentradas em uma única tela (NIXON, 2018).

3.5.1. Páginas de navegação

Em conjunto, as três interfaces que constituem a camada de supervisão do sistema (Produção, Dashboard e Treinamento) demonstram a atuação da interface web em diferentes níveis da arquitetura proposta, sendo de um lado, como camada de armazenamento e consulta dos dados operacionais gerados pela planta, enquanto de outro, como canal de comando, por meio do qual decisões tomadas remotamente pelo usuário são transmitidas de volta à camada de controle local, fechando o ciclo de comunicação bidirecional entre a planta simulada e a infraestrutura computacional estabelecida em nuvem (TANENBAUM; WETHERALL, 2011).

3.5.1.1. Produção

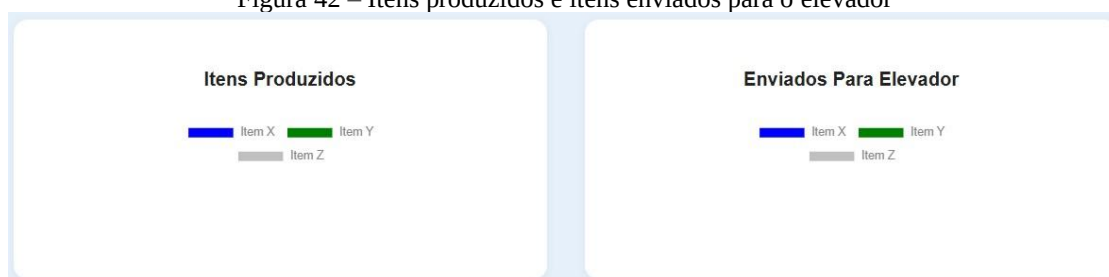
A primeira página da interface web, chamada de PRODUÇÃO, correspondente ao arquivo de índice principal do sistema, apresenta em seu cabeçalho um menu de navegação que

dá acesso às outras páginas que compõem a camada de supervisão do sistema (Dashboard e Treinamento), um princípio recorrente no design de sistemas supervisórios industriais, no qual diferentes contextos de uso são isolados em telas específicas para reduzir a carga cognitiva do operador (SILVEIRA; SANTOS, 2014). O corpo da página é organizado em um painel com seis cartões gráficos (chart cards), cada um contendo um gráfico do tipo pizza renderizado com a biblioteca Chart.js, sendo eles:

- a) Itens Produzidos;
- b) Enviados Para Elevador;
- c) Enviados Para A;
- d) Enviados Para B;
- e) Tempo de Atendimento – Itens;
- f) Tempo de Atendimento – Destinos.

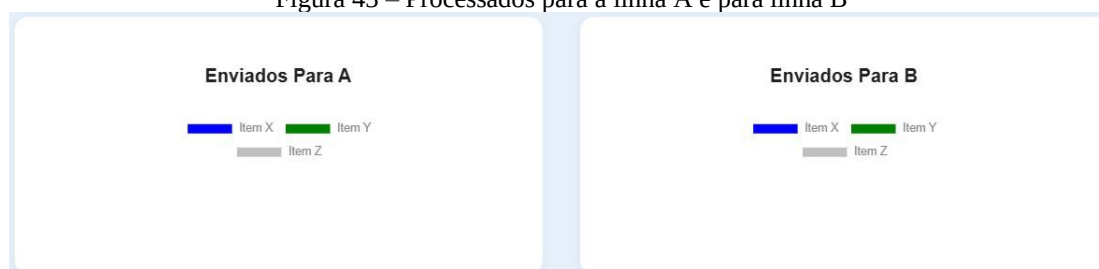
Os quatro primeiros gráficos apresentam a distribuição, por tipo de item (X, Y e Z), da quantidade total de produtos recebidos pelo sistema, da quantidade encaminhada ao elevador de armazenamento e da quantidade expedida em cada uma das duas linhas secundárias de atendimento, permitindo ao operador comparar visualmente o volume de produção associado a cada categoria de item e a cada rota de encaminhamento da planta (BOLTON, 2015).

Figura 42 – Itens produzidos e itens enviados para o elevador



Fonte: Elaborado pelo autor (2026)

Figura 43 – Processados para a linha A e para linha B



Fonte: Elaborado pelo autor (2026)

A contagem exibida em cada um desses gráficos não é calculada de forma independente pela interface, mas deriva diretamente das variáveis de estado já descritas na seção 3.4.2, no qual, para cada transição de uma dessas variáveis do valor zero para o valor normalizado de um item, o backend incrementa o contador correspondente ao tipo de item e ao destino associado àquela variável.

Os dois últimos gráficos apresentam o tempo médio de atendimento sob dois critérios de agrupamento complementares. O primeiro, segmentado por tipo de item (X, Y e Z), mede o intervalo médio entre os horários de entrada de cada item no sistema. O segundo, segmentado por destino (linha A, linha B ou elevador), mede o intervalo médio entre os horários de encaminhamento de cada item ao seu respectivo destino. Ambos possibilitam identificar, de forma imediata, qual tipo de item ou qual destino concentra o maior ou o menor intervalo médio, informação relevante para a análise de gargalos operacionais na planta, prática associada ao conceito de indicadores de desempenho (Key Performance Indicators – KPIs) amplamente utilizados na gestão de sistemas produtivos (PINEDO, 2016). Para ambas as análises, foi adotada a mesma métrica de tempo médio entre itens (T_m), expressa pela seguinte equação:

$$T_m = \frac{T_f - T_i}{N - 1} \quad (23)$$

Onde:

T_f hora do último evento considerado;

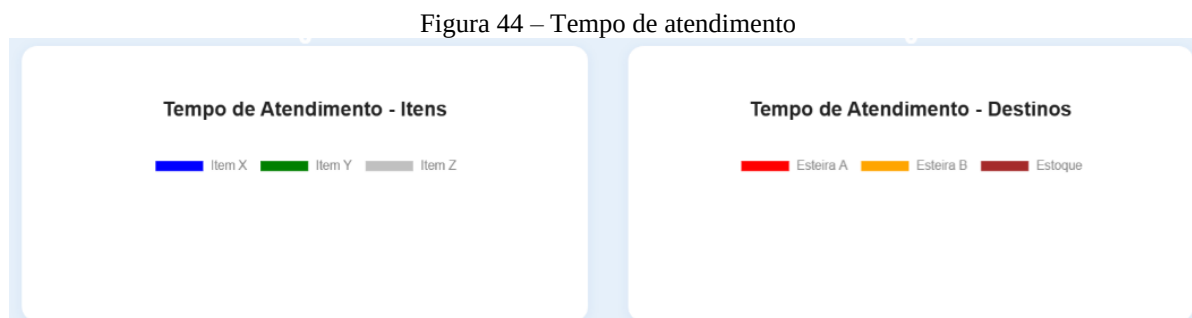
T_i hora do primeiro evento considerado;

N Quantidade de eventos computados no grupo.

Ainda que os dois gráficos apliquem exatamente a mesma equação, o evento de tempo considerado em cada caso é distinto. No gráfico segmentado por tipo de item, os tempos correspondem aos horários de entrada dos itens no sistema, enquanto no gráfico segmentado por destino, correspondem aos horários de encaminhamento dos itens a cada destino. Essa distinção é o que permite que a mesma métrica seja aplicada a dois aspectos diferentes do processo produtivo, medindo tanto a frequência de chegada de itens ao sistema quanto a frequência de encaminhamento a cada destino.

Dessa forma, a análise combinada dos gráficos de distribuição por rota e de intervalo médio entre itens permite não apenas verificar o volume processado por cada destino,

mas também identificar eventuais inconsistências no atendimento dos destinos, oferecendo uma base quantitativa para futuras otimizações na lógica de decisão da rede Prioridade.



Fonte: Elaborado pelo autor (2026)

Ao ser carregada, a página executa automaticamente a inicialização de todos os gráficos e a primeira atualização dos dados de produção por meio de uma requisição assíncrona ao backend. A partir disso, a página passa a repetir essa mesma requisição automaticamente a cada 1 segundo, mantendo os gráficos continuamente atualizados com o estado mais recente da produção, comportamento típico de aplicações que utilizam o padrão AJAX (Asynchronous JavaScript and XML) para manter a interface sempre sincronizada com o backend, sem exigir o recarregamento completo da página (FLANAGAN, 2020).

Além dos gráficos atualizados continuamente, a página conta com um botão de geração de relatório, que utiliza os mesmos dados de produção exibidos na tela no momento do acionamento, para compor por meio de requisição a uma API de Inteligência Artificial Generativa, um relatório analítico do desempenho da planta.

3.5.1.2. Dashboard

A segunda página da interface web, Dashboard, consolida em uma única tela a visualização sinótica do estado físico da planta, o histórico gráfico do comportamento das seis instâncias de redes neurais em operação, o gerenciamento dos pedidos de produção, sendo a função que efetivamente concentra a inserção e o acompanhamento das solicitações do usuário. A página disponibiliza um controle de acionamento geral do sistema, por meio de um botão (Figura 45) que alterna entre os estados "Ligar" e "Parar", refletindo dinamicamente o status operacional da planta enviado pelo backend, funcionalidade que materializa a capacidade da de intervir diretamente sobre a operação da planta.

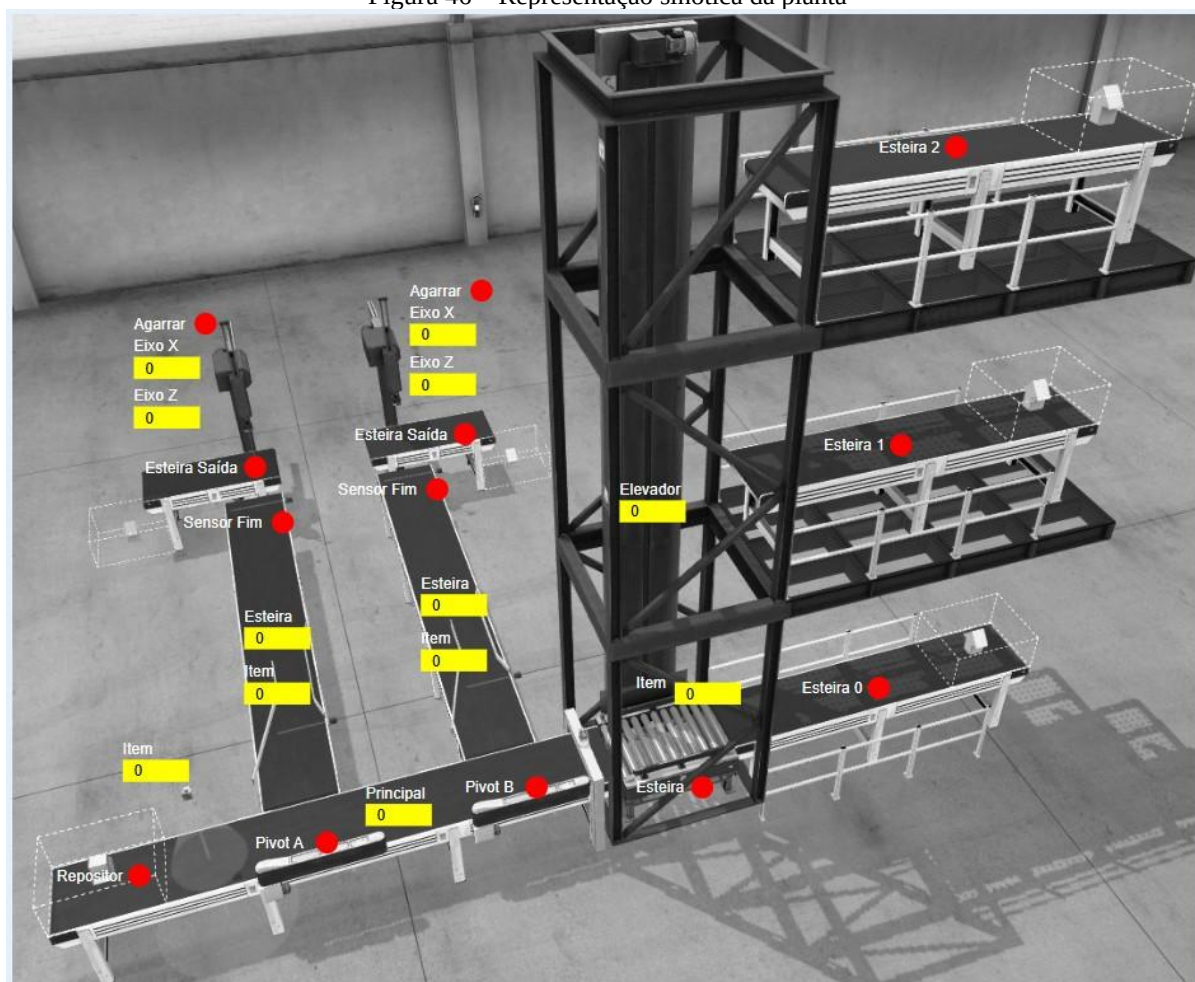
Figura 45 – Comando para ligar o sistema



Fonte: Elaborado pelo autor (2026)

O elemento central do painel é uma representação sinótica da planta, obtida pela sobreposição de uma camada gráfica vetorial (SVG) sobre uma imagem estática do ambiente simulado, como visto na Figura 46.

Figura 46 – Representação sinótica da planta



Fonte: Elaborado pelo autor (2026)

Essa técnica reproduz o conceito de sinótico, no qual o ambiente gráfico de uma IHM permite ao operador visualizar variáveis do processo sobrepostas à representação física da planta (ROSÁRIO, 2010). Cada elemento monitorado da planta é posicionado sobre a imagem por meio de coordenadas fixas dentro do sistema de referência do SVG, sendo representado de

duas formas distintas conforme a natureza do sinal. Variáveis digitais são exibidas como indicadores circulares, cuja cor alterna entre vermelho e verde a partir do mesmo limiar de decisão de 0.95 já estabelecido pela ativação das saídas das redes neurais (seção 3.3.1), enquanto variáveis analógicas são exibidas como valores numéricos, atualizados com três casas decimais dentro de pequenos quadros sobrepostos à imagem. Essa convenção permite que, a partir de uma única imagem, o operador identifique simultaneamente o estado de sensores, atuadores digitais e variáveis contínuas, cobrindo a totalidade dos dispositivos digitais e analógicos apresentados respectivamente nas Tabelas 2 e 3 deste trabalho.

Abaixo do painel sinótico, a página disponibiliza a funcionalidade de gerenciamento de pedidos, implementada por meio de um formulário que permite ao operador definir a quantidade desejada de cada tipo de item (X, Y e Z), respeitando os limites mínimo de dois e máximo de seis itens por pedido. Essa restrição decorre de uma limitação física do manipulador cartesiano na etapa de expedição, valores superiores a esse limite impossibilitariam o empilhamento completo do pedido pelo atuador.

Após o envio, o pedido é registrado na camada em nuvem e passa a ser exibido em uma tabela de acompanhamento, que apresenta, para cada solicitação ativa, o progresso de produção de cada item (quantidade já atendida em relação à quantidade solicitada), o status geral do pedido e o horário de registro, além de permitir sua remoção manual pelo operador, conforme visto na Figura 47.

Figura 47 – Gerenciamento de pedidos

ID	Item X	Item Y	Item Z	Status	Hora	Ação
480	0/1	0/2	0/3	Pendente	2026-09-08 03:53:46	Deletar

Fonte: Elaborado pelo autor (2026)

Na sequência, o painel apresenta seis gráficos de linha, um para cada instância de rede neural em operação: Despacho, Prioridade, Esteira, Elevador, Expedição A e B, respectivamente. A existência de dois gráficos distintos para a expedição confirma, na interface, a decisão metodológica já descrita na seção 3.3.6, na qual a mesma arquitetura de rede é

reaproveitada e instanciada duas vezes para operar de forma independente em cada uma das esteiras secundárias. Cada gráfico exibe a evolução temporal dos valores de saída da respectiva rede ao longo das últimas leituras realizadas, permitindo observar o comportamento dinâmico das decisões de controle. É possível visualizar e comparar o comportamento simultâneo das duas linhas de expedição através da Figura 48.

Figura 48 – Comparação de acionamento das instâncias da rede Expedição



Fonte: Elaborado pelo autor (2026)

Complementarmente, um seletor permite ao operador escolher uma rede específica e consultar, em uma tabela dedicada, o histórico bruto de vetores de entrada e saída associados a cada leitura, funcionando como uma ferramenta de inspeção e depuração do comportamento interno de cada modelo. A Figura 49 ilustra este recurso.

Figura 49 – Lista de histórico de valores das RNAs

Rede		Quantidade	
Elevador		5	
ID	Hora	Entradas	Saídas
5	14:35:38	0.00,0.00	0.01,1.00,0.01,0.00,0.00
4	14:35:37	0.01,0.00	0.01,1.00,0.01,0.00,0.00
3	14:35:36	0.01,0.00	0.01,1.00,0.01,0.00,0.00
2	14:35:35	0.01,0.00	0.01,1.00,0.01,0.00,0.00
1	14:35:34	0.01,0.00	0.01,1.00,0.01,0.00,0.00

Fonte: Elaborado pelo autor (2026)

A atualização de todo o painel ocorre de forma automática por meio de requisições assíncronas repetidas em intervalo de 500 milissegundos, enquanto a tabela de pedidos é

atualizada em um intervalo mais espaçado de 5 segundos, refletindo a diferença de criticidade temporal entre o acompanhamento do estado físico da planta e o acompanhamento administrativo dos pedidos (TANENBAUM; WETHERALL, 2011).

3.5.1.3. Treinamento

A terceira página da interface web, Treinamento, é dedicada exclusivamente ao treinamento das redes neurais, permitindo que o operador gerencie os conjuntos de dados de treinamento, execute o processo de aprendizado e acompanhe sua evolução por meio de indicadores gráficos, para cada uma das cinco redes que compõem o sistema de controle.

O processo se inicia pela seleção da rede desejada, juntamente com a escolha de uma versão de treinamento já existente, a criação de uma nova instância ou a clonagem de uma versão previamente existente. Essa organização por versões permite que múltiplos ciclos de treinamento sejam preservados e comparados entre si para uma mesma rede, sem que um novo treinamento sobrescreva necessariamente os parâmetros já validados anteriormente, prática alinhada ao controle de versionamento de modelos comumente adotado em fluxos de trabalho de aprendizado de máquina, no qual diferentes execuções de treinamento precisam ser rastreáveis e comparáveis (GOODFELLOW; BENGIO; COURVILLE, 2016).

A funcionalidade de clonagem, em particular, permite ao operador criar uma versão a partir dos mesmos dados de treinamento já cadastrados em uma versão existente, sem qualquer necessidade de reinserção manual, viabilizando a condução de testes exploratórios sem comprometer a integridade da versão original, que permanece preservada e disponível para consulta e edição a qualquer momento. Esta implementação é vista na Figura 50.

Figura 50 – Seleção da versão de treinamento da rede

Rede	Treinamento	Ação
Prioridade ▾	v1 ▾	Clonar Deletar

Fonte: Elaborado pelo autor (2026)

Para cada versão selecionada, a página disponibiliza um formulário de inserção de dados de treinamento, no qual o operador fornece manualmente os vetores de entrada e saída correspondentes a um exemplo do conjunto de dados supervisionado, sendo esses valores

validados antes do envio para garantir que estejam em formato numérico adequado. A quantidade de variáveis detectada pelo backend estabelece de forma direta o tamanho da camada de entrada e de saída das redes neurais, de forma equivalente também para camada oculta, já que para esta camada tem a quantidade de neurônios definida pela Equação 17. Os pares de entrada e saída já cadastrados são exibidos em uma tabela de acompanhamento, podendo ser removidos individualmente.

Figura 51 – Inserção de conjunto de dados para treinamento da RNA

ID	Entradas	Saídas	Ação
1	0,0,0,0,0,0,1,0	1,0.3	Deletar

Fonte: Elaborado pelo autor (2026)

O conjunto de entradas, bem como o conjunto de saídas, são vetores inseridos com valores separados por vírgula, cujo tamanho é definido com base na quantidade de dados informados na primeira inserção. Qualquer divergência nesse tamanho nas inserções subsequentes pode levar a um erro de convergência durante o treinamento, o que exige atenção por parte do usuário, uma vez que não há mecanismos de validação para dados inválidos.

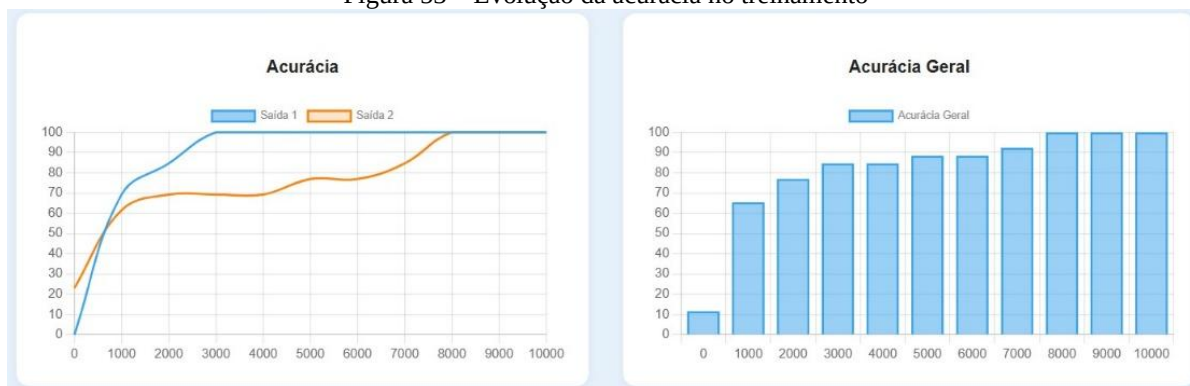
Definido o conjunto de dados, o operador configura os parâmetros do treinamento, número de épocas e taxa de aprendizado, e inicia o processo por meio de um botão dedicado, que exibe uma sobreposição de carregamento durante a execução, informando ao operador que o backend está processando a retro propagação do erro sobre os dados fornecidos. Caso a versão selecionada já possua um treinamento realizado, também é possível redefinir esse conjunto através de um botão de “Reset”, conforme visto na Figura 52.

Figura 52 – Definição da quantidade de épocas e da taxa de aprendizagem

Fonte: Elaborado pelo autor (2026)

Concluído o treinamento, a página é atualizada, permitindo o acompanhamento simultâneo de diferentes aspectos da convergência do modelo, como a acurácia por neurônio de saída, acurácia geral da rede, erro por neurônio de saída e o erro geral da rede.

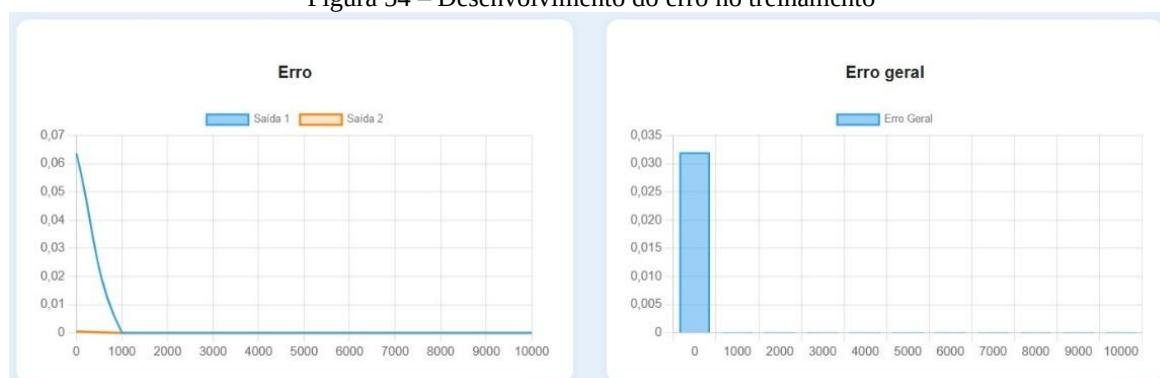
Figura 53 – Evolução da acurácia no treinamento



Fonte: Elaborado pelo autor (2026)

Essa visualização detalhada da trajetória dos parâmetros ao longo do treinamento é o que permite ao operador identificar visualmente se a rede está convergindo de forma estável, ou se apresenta oscilações que indiquem a necessidade de ajuste na taxa de aprendizado ou no número de épocas (HAYKIN, 2009).

Figura 54 – Desenvolvimento do erro no treinamento



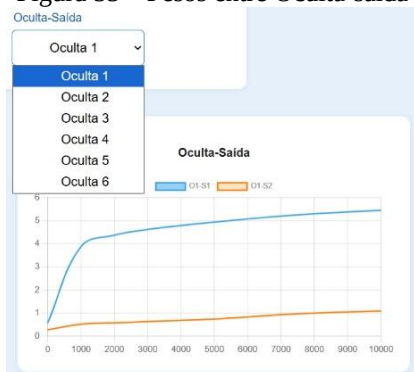
Fonte: Elaborado pelo autor (2026)

Também é possível visualizar a evolução ao longo das épocas, dos pesos sinápticos entre a camada de entrada e a camada oculta, entre a camada oculta e a camada de saída, e dos respectivos termos de bias de cada camada, conforme Figuras 55 e 56.

Como os pesos entre as diferentes camadas formam matrizes bidimensionais, a página permite ao operador selecionar um neurônio de origem específico, conforme Figura 55,

para visualizar isoladamente a evolução dos pesos de suas conexões com a camada seguinte, evitando que os gráficos fiquem sobrecarregados com a totalidade das conexões da rede.

Figura 55 – Pesos entre Oculta-saída



Fonte: Elaborado pelo autor (2026)

Figura 56 – Evolução Bias saída



Fonte: Elaborado pelo autor (2026)

Uma vez validado o comportamento do treinamento, a página oferece duas ações complementares, sendo a possibilidade de testar a rede treinada com um vetor de entrada qualquer, obtendo diretamente a saída correspondente calculada pelo forward pass, e a possibilidade de efetivamente ativar essa versão de pesos como a configuração corrente da rede em operação, substituindo os parâmetros utilizados pela camada de controle local. A funcionalidade de teste não se limita a uma verificação isolada do modelo matemático, uma vez que o vetor de saída obtido é aplicado sobre a própria simulação dos dispositivos da planta, permitindo observar o comportamento resultante nos atuadores e sensores virtuais exatamente como ocorreria em condições reais de operação. Esse comportamento permite que decisões de controle sejam validadas em ambiente simulado antes de sua efetiva aplicação sobre o sistema físico (GRIEVES; VICKERS, 2017). A Figura 57 mostra esta implementação.

Figura 57 – Comando de teste para um conjunto de entradas

Fonte: Elaborado pelo autor (2026)

A ativação de uma versão de treinamento registra o conjunto de pesos e bias correspondente como a configuração ativa para a respectiva rede, substituindo qualquer versão anteriormente em uso. Essa atualização é refletida na camada de controle, pois o dispositivo

requisita periodicamente os parâmetros mais recentes de cada rede junto à camada em nuvem, logo a nova configuração passa a substituir a versão anterior diretamente no ciclo de controle em execução, sem necessidade de reinicializar o sistema ou interromper a operação da planta.

Paralelamente, a interface exibe de forma detalhada todos os valores numéricos finais de pesos e bias da versão ativada, permitindo não apenas a conferência e a documentação completa do modelo aplicado, mas também a implementação direta desses valores nos dispositivos de controle, quando necessário (RUSSELL; NORVIG, 2021). Nessa mesma tela (Figura 58), também são exibidos o erro obtido em cada saída da rede e o erro geral do treinamento, permitindo mensurar tanto o desempenho individual de cada saída quanto a qualidade geral do modelo.

Figura 58 – Exibição dos valores de um conjunto de pesos

```

--- Pesos da entrada para a camada oculta --- [entrada][oculta]
[0][0]: 0.1271677517356831
[0][1]: -0.06036043148439772
[0][2]: 9.175193547723012
[1][0]: -5.503511608988779
[1][1]: -7.333765765740022
[1][2]: 0.044144882663982844

--- Bias da camada oculta --- [oculta]
[0]: 0.30434916627894104
[1]: 0.33476700679896265
[2]: -0.42566184738315926

--- Pesos da camada oculta para a saída --- [oculta][saida]
[0][0]: -1.1644214253300593
[0][1]: 4.93823839828247
[1][0]: 0.7882965543856572
[1][1]: 6.544598617773165
[2][0]: -8.247355364983871
[2][1]: -0.1786574904530483

--- Bias da camada de saída --- [saida]
[0]: 0.31299429268260043
[1]: 0.06026983326832428

--- Erro --- [saida]
[0]: 1.3832069550163664e-8
[1]: 0.0003522245344668664

--- Erro geral ---
0.00017611918326820828

```

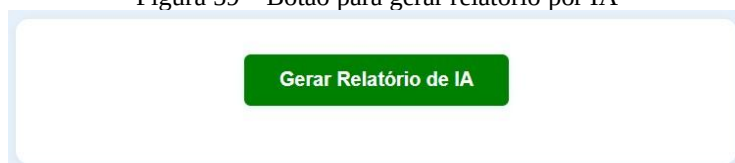
Fonte: Elaborado pelo autor (2026)

Essa etapa de ativação, portanto, não se limita a uma confirmação técnica da operação de substituição, mas constitui o próprio mecanismo pelo qual uma nova versão de rede passa a governar o comportamento da planta. A exibição detalhada dos pesos e bias no momento da ativação, funciona como auditoria da versão do modelo submetida ao crivo humano, imediatamente antes de essa mesma versão assumir o controle efetivo do respectivo subsistema.

3.5.2. Integração com API

O sistema incorpora uma camada adicional voltada à geração automática de relatórios textuais com base nos dados de produção da planta industrial. Essa funcionalidade foi implementada por meio da integração com a API de chat completions da OpenAI, utilizando o modelo GPT-4o-mini, sendo o processamento realizado inteiramente no backend em PHP e acionado pela página de Produção por meio de um botão, conforme Figura 59, dedicado à geração do relatório (OPENAI, 2023).

Figura 59 – Botão para gerar relatório por IA



Fonte: Elaborado pelo autor (2026)

Ao ser solicitado, o backend consulta o registro mais recente do dia armazenado na tabela de histórico do banco de dados, extraindo os vetores de contagem de itens por tipo (X, Y e Z) associados à produção total, ao encaminhamento para o elevador e à expedição em cada uma das duas linhas secundárias, além do intervalo médio entre itens calculado individualmente para cada destino de expedição. Esses dados são então formatados e inseridos em uma requisição estruturada em duas partes: uma instrução de sistema (system prompt), que define o papel do modelo como assistente técnico de automação industrial e estabelece regras específicas para a geração do relatório, e uma instrução de usuário (user prompt), que apresenta os dados numéricos propriamente ditos, já acompanhados da interpretação de cada vetor, solicitando ao modelo uma análise comparativa entre produção, distribuição e tempos de atendimento (OPENAI, 2023).

Algoritmo 5 – Instrução do sistema para requisição à API

```
$systemPrompt = '
  Você é um assistente técnico de automação industrial.
  Sua tarefa é analisar dados resumidos de uma planta industrial e gerar
  um relatório operacional objetivo,
  técnico e claro, em português do Brasil.
  Regras:
  - Considere que os vetores representam contagens de itens X, Y e Z, nesta
  ordem.
  - Leve em consideração que os produtos X, Y e Z entram no sistema de
  forma aleatória.
  - Considere que a ordem de entrada dos produtos pode afetar os tempos
  médios observados.
  - Destaque produção total, distribuição para elevador, esteira A e esteira
  B.
  - Esteiras A e B são responsáveis por atendimento de pedidos solicitados.
  - Elevador é o armazenamento dos produtos que entram no sistema e não
  são necessários nas esteiras A ou B.
  - Analise os tempos médios entre produtos no destino informado.
  - Aponte balanceamento ou desequilíbrio entre os destinos.
  - Aponte possíveis gargalos de forma cautelosa, sem afirmar falhas
  inexistentes.
  - Não invente dados que não existem.
  - Seja técnico, mas com texto fácil de entender.
  - A resposta deve ser em texto corrido, sem tópicos ou listas.';
```

Fonte: Elaborado pelo autor (2026)

Por se tratar de um serviço externo cobrado por uso, o sistema implementa um mecanismo de controle de custo, restringindo o intervalo mínimo entre chamadas consecutivas à API a quinze segundos, caso o operador solicite um novo relatório antes desse intervalo, o backend retorna imediatamente os últimos dados numéricos já disponíveis, acompanhados de uma mensagem informando a necessidade de aguardar, sem realizar uma nova chamada à API. Essa medida é relevante tanto do ponto de vista econômico quanto de responsabilidade no uso de recursos de IA generativa hospedados na nuvem, evitando chamadas redundantes ou excessivas ao serviço externo (BUNTAK et al., 2021).

A comunicação com o serviço externo é realizada exclusivamente no lado servidor, preservando a chave de acesso à API no ambiente backend e impedindo sua exposição em código do lado cliente, uma vez que o navegador do usuário apenas envia o comando de solicitação ao backend em PHP, sem qualquer acesso direto à API da OpenAI. A implementação seguiu organização modular, com separação entre camada de aplicação, acesso a dados e módulo responsável pelas requisições externas, em conformidade com práticas estruturais recomendadas para aplicações web (LOCKHART, 2015; NIXON, 2018).

Todas as interações com a API são registradas em uma tabela específica do banco de dados, garantindo rastreabilidade completa do histórico de relatórios gerados ao longo do

tempo, em conformidade com a abordagem de armazenamento estruturado já adotada nas demais camadas do sistema (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

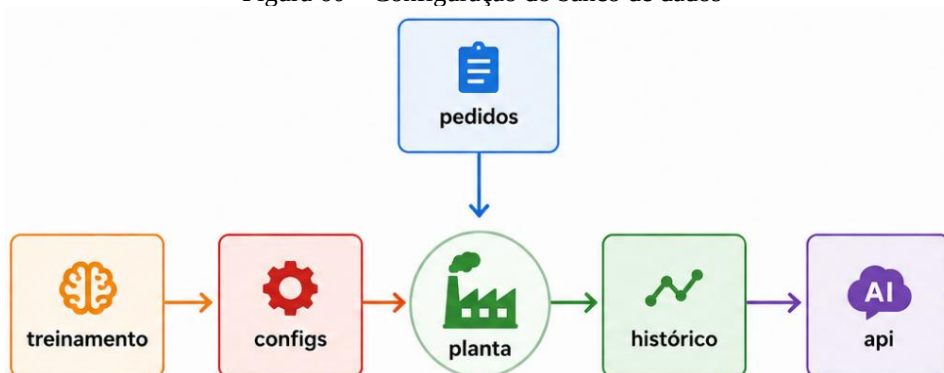
3.5.3. Banco de dados

Todos os dados operacionais do sistema são armazenados em um banco de dados relacional MySQL, hospedado na infraestrutura em nuvem, organizado em cinco tabelas principais, cada uma responsável por um domínio específico da arquitetura:

- a) pedidos;
- b) histórico;
- c) treinamento;
- d) configs;
- e) api.

Essa segmentação reflete diretamente as camadas funcionais já discutidas neste trabalho, gerenciamento de produção, supervisão em tempo real, treinamento das redes neurais, integração com IA generativa e seleção das configurações, cada uma persistida de forma independente, ainda que consultada e atualizada de forma integrada pelos diferentes módulos do backend em PHP (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Figura 60 – Configuração do banco de dados



Fonte: Elaborado pelo autor (2026)

A comunicação entre a aplicação web e o banco de dados foi implementada utilizando prepared statements (consultas parametrizadas) em todas as operações de escrita e leitura, prática que previne vulnerabilidades de injeção de SQL e é considerada essencial em aplicações PHP que interagem diretamente com bancos de dados relacionais expostos à internet

(LOCKHART, 2015; NIXON, 2018). Metodologicamente, o banco de dados integra a arquitetura distribuída como elemento estruturante, conectando a camada de controle, a camada estratégica e a camada de treinamento.

3.5.3.1. Pedidos

A tabela “pedidos” armazena as solicitações de produção inseridas pelo operador na página de Dashboard, contendo o identificador único do pedido, o vetor de quantidades solicitadas por tipo de item, um vetor equivalente com as quantidades já efetivamente produzidas, o status textual da solicitação que transita entre estados como "Pendente" e "Fabricando", sendo automaticamente atualizado para "Finalizado" quando o vetor de itens produzidos se iguala ao vetor solicitado, além do horário da última atualização do registro. Essa verificação de igualdade entre os vetores de pedido e de produção, realizada diretamente na consulta ao banco, dispensa a manutenção de uma variável de controle adicional para sinalizar a conclusão do pedido, sendo o próprio estado dos dados suficiente para essa inferência.

Figura 61 – Estrutura da tabela Pedidos no banco de dados

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	bigint(20)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	pedido	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
3	feitos	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
4	estatus	text	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
5	hora	timestamp			Sim	current_timestamp()			Alterar Eliminar Mais

Fonte: Elaborado pelo autor (2026)

A tabela é composta por cinco atributos que permitem o rastreamento do ciclo de vida das solicitações de produção. O atributo id, do tipo inteiro, é configurado como chave primária com a propriedade AUTO_INCREMENT, mecanismo que gera automaticamente um valor numérico sequencial e crescente a cada nova inserção, sem exigir que esse valor seja informado manualmente, o que garante a identificação única de cada registro. Os atributos “pedido” e “feitos”, do tipo LONGTEXT com colaço binária utf8mb4_bin, armazenam, respectivamente, o vetor de quantidades solicitadas e o vetor de quantidades já produzidas. Como o JSON é, por definição, um formato de dado textual (uma string estruturada segundo uma sintaxe própria), ele é armazenado nesses campos como texto puro, sendo o tipo LONGTEXT escolhido por comportar cadeias de caracteres extensas sem limite prático de

tamanho e a colação `utf8mb4_bin`, por sua vez, realiza comparações byte a byte (binárias), em vez de comparações léxicas sensíveis a regras linguísticas, o que evita que o MySQL considere iguais strings que diferem em maiúsculas/minúsculas, acentuação ou espaçamento, garantindo assim a fidelidade exata na comparação entre os vetores serializados. O atributo `status`, do tipo `TEXT`, representa textualmente o estado da solicitação, sendo atualizado para "Finalizado" quando há igualdade entre os vetores de pedido e feitos, o que dispensa o uso de uma variável de controle adicional, uma vez que o próprio estado dos dados é suficiente para essa inferência. Por fim, o atributo `hora`, do tipo `TIMESTAMP` com padrão `CURRENT_TIMESTAMP()`, registra automaticamente o momento de criação do registro, assegurando a consistência temporal dos dados armazenados (ORACLE CORPORATION, 2024).

3.5.3.2. Histórico

A tabela “histórico” concentra o registro cumulativo do comportamento da planta ao longo de um mesmo dia de operação, armazenando, em uma estrutura de dados serializada em JSON, tanto o histórico recente das últimas leituras de cada uma das seis instâncias de rede neural quanto os indicadores agregados de produção exibidos na página de Produção, como as contagens totais por tipo de item, a distribuição por destino e os tempos médios de atendimento. Diferentemente de um esquema relacional tradicional, no qual cada variável ocuparia uma coluna própria, a opção por armazenar esses dados como um documento estruturado dentro de um único campo reflete a natureza altamente variável e agregada dessas informações, evitando a criação de um número excessivo de colunas ou tabelas para acomodar cada nova métrica derivada (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). A tabela também armazena o comando de acionamento geral da planta (ligar/parar), consultado e alterado pela função de controle da página de Dashboard.

Figura 62 – Estrutura da tabela Historico no banco de dados



#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	bigint(20)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	dados	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
3	comando	tinyint(1)			Sim	0			Alterar Eliminar Mais
4	hora	timestamp			Sim	current_timestamp()			Alterar Eliminar Mais

Fonte: Elaborado pelo autor (2026)

Esta tabela apresenta estrutura de identificação semelhante à da tabela pedidos, mantendo o atributo id como chave primária de tipo inteiro com incremento automático. O atributo “dados”, do tipo longtext com colação utf8mb4_bin, concentra o documento JSON descrito anteriormente, reunindo tanto o histórico das leituras das redes neurais quanto os indicadores agregados de produção em uma única estrutura serializada. Já o atributo comando, do tipo tinyint(1) com valor padrão 0, funciona como uma flag booleana que representa o estado de acionamento geral da planta, sendo alterado entre 0 e 1 conforme a operação de “ligar” ou “parar” solicitada na página de Dashboard. Por fim, o atributo hora, também do tipo timestamp com padrão current_timestamp(), delimita temporalmente cada registro cumulativo, permitindo associar os dados agregados ao respectivo dia e horário operacional da planta.

3.5.3.3. Treinamento

A tabela “treinamento” armazena, para cada rede neural e cada versão de treinamento criada, o conjunto de pares de entrada e saída utilizados como base supervisionada, permitindo que múltiplas versões de dados de treinamento coexistam para uma mesma rede sem sobreposição, conforme o mecanismo de versionamento já descrito na seção sobre a página de Treinamento.

Figura 63 – Estrutura da tabela treinamento no banco de dados

#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	bigint(20)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	rede	text	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
3	versao	bigint(20)			Sim	NULL			Alterar Eliminar Mais
4	treino	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
5	dados	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
6	hora	timestamp			Sim	current_timestamp()			Alterar Eliminar Mais

Fonte: Elaborado pelo autor (2026)

A tabela treinamento mantém o atributo id como chave primária de tipo inteiro com incremento automático, seguindo o mesmo padrão de unicidade adotado nas demais tabelas do sistema. O atributo rede identifica textualmente a rede neural à qual o conjunto de dados de treinamento pertence, do tipo TEXT com colação utf8mb4_unicode_520_ci, essa colação realiza comparações case-insensitive e sensíveis a regras linguísticas, tratando "Rede-A" e "rede-a" como equivalentes em buscas e ordenações, sendo um comportamento adequado para

um campo textual comum, onde a identidade do dado não depende da distinção exata entre maiúsculas e minúsculas (ORACLE CORPORATION, 2024). O atributo “versão”, do tipo inteiro, registra o número da versão correspondente, viabilizando a coexistência de múltiplos conjuntos de treinamento associados a uma mesma rede sem conflito entre eles. Os atributos treino e dados, ambos do tipo JSON, armazenam respectivamente, os pares de entrada e saída que compõem a base supervisionada, estruturados em formato serializado de modo a preservar a integridade dos valores numéricos utilizados no processo de aprendizado. Por fim, o atributo hora, permite o rastreamento cronológico das alterações realizadas na base de treinamento ao longo do tempo.

3.5.3.4. Configs

A tabela “configs” armazena os parâmetros finais dos pesos e bias de cada versão de rede efetivamente ativada pelo operador para uso em produção, sendo essa tabela consultada diretamente pela camada de controle local ao carregar os parâmetros da rede em operação, distinguindo claramente, na modelagem do banco, os dados usados para treinar uma rede dos parâmetros resultantes de um treinamento já validado.

Figura 64 – Estrutura da tabela configs no banco de dados



#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	bigint(20)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	rede	text	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
3	versao	int(11)			Sim	NULL			Alterar Eliminar Mais
4	dados	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
5	hora	timestamp			Sim	current_timestamp()			Alterar Eliminar Mais

Fonte: Elaborado pelo autor (2026)

A tabela configs mantém o atributo id como chave primária de tipo inteiro com incremento automático, o atributo rede, do tipo text com colação utf8mb4_unicode_520_ci, identifica textualmente a rede neural à qual os parâmetros pertencem, enquanto o atributo versao, do tipo inteiro, registra o número da versão de treinamento cujos parâmetros foram efetivamente colocados em produção. O atributo dados, do tipo longtext com colação binária utf8mb4_bin, que é a formatação do tipo JSON, armazena os pesos e bias da rede em formato serializado, preservando a precisão numérica exigida para a correta reconstrução do modelo

pela camada de controle. Por fim, o atributo hora, registra automaticamente o momento em que a configuração foi ativada.

3.5.3.5. API

Por fim, a tabela api registra o histórico completo de interações com o serviço de IA generativa, armazenando a instrução de sistema, a instrução de usuário, a resposta bruta recebida da API e o texto final do relatório gerado, garantindo rastreabilidade completa sobre os relatórios produzidos ao longo do tempo, conforme já discutido na seção 3.5.2.

Figura 65 – Estrutura da tabela api no banco de dados



#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	bigint(20)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	sistema	longtext	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
3	usuario	longtext	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
4	dados	longtext	utf8mb4_bin		Sim	NULL			Alterar Eliminar Mais
5	resposta	longtext	utf8mb4_unicode_520_ci		Sim	NULL			Alterar Eliminar Mais
6	hora	timestamp			Sim	current_timestamp()			Alterar Eliminar Mais

Fonte: Elaborado pelo autor (2026)

A tabela “api” mantém, também nesse caso, o atributo id como chave primária, os atributos sistema e usuario, ambos do tipo longtext com colaço utf8mb4_unicode_520_ci, armazena, respectivamente, a instrução de sistema e a instrução de usuário submetidas ao serviço de IA generativa, utilizando colaço sensível a regras linguísticas por se tratar de conteúdo textual em linguagem natural. O atributo dados, do tipo JSON, registra a resposta bruta recebida da API, preservando integralmente sua formatação original sem qualquer normalização. Já o atributo resposta, do tipo longtext com colaço utf8mb4_unicode_520_ci, contém o texto final do relatório gerado a partir do processamento da resposta bruta. Por fim, o atributo hora, com a mesma funcionalidade das demais tabelas.

3.6. Treinamento das RNAs

O processo de treinamento supervisionado das cinco redes neurais que compõem o sistema de controle foi implementado como um módulo próprio no backend em PHP, responsável tanto pela inicialização dos parâmetros de cada rede quanto pela execução do

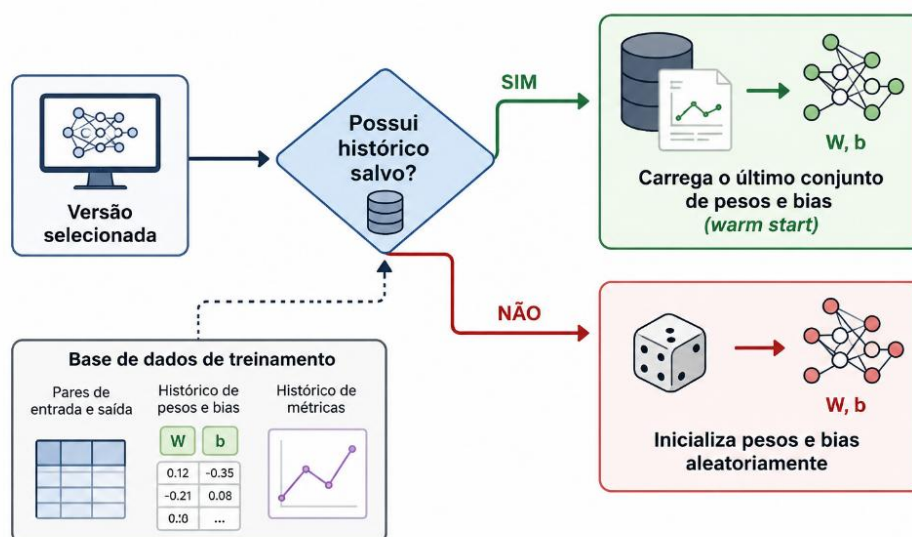
algoritmo de retropropagação do erro ao longo de um número definido de épocas. Diferentemente do módulo de inferência descrito na seção 3.4.3, que apenas executa o forward pass a partir de pesos já treinados, este módulo concentra toda a lógica de aprendizado do sistema, sendo executado exclusivamente na camada em nuvem.

Esta seção detalha os aspectos de implementação desse processo de treinamento, incluindo a forma como os parâmetros da rede são inicializados antes do primeiro ciclo de aprendizado, a estrutura do algoritmo responsável por percorrer o conjunto de dados e ajustar os pesos a cada época, e os critérios utilizados para calcular as métricas de erro e acurácia exibidas ao operador na página de Treinamento. Esse tipo de abordagem, na qual modelos são ajustados a partir de exemplos historicamente observados do comportamento do processo, tende a favorecer a generalização do sistema também para situações operacionais não explicitamente contempladas no conjunto de treinamento (ARAÚJO, 2025).

3.6.1. Inicialização dos parâmetros

Antes de iniciar o treinamento de uma rede, o sistema verifica se já existe um conjunto de parâmetros previamente salvo para aquela combinação específica de rede e versão, consultado diretamente na base de dados de treinamento onde são armazenados os pares de entrada e saída cadastrados e o histórico de pesos, bias e métricas de execuções anteriores (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). A Figura 66 resume bem como funciona essa lógica de decisão:

Figura 66 – Fluxo de inicialização dos parâmetros da rede



Quando já existe histórico, os pesos e bias são recuperados a partir do último conjunto salvo, e não reiniciados do zero, comportamento que reflete a prática usual de treinamento incremental (warm start), na qual a otimização retoma um estado já parcialmente ajustado em vez de partir de uma inicialização aleatória a cada execução (HAYKIN, 2009), permitindo ao operador treinar em lotes de épocas menores e avaliar a evolução do erro antes de decidir se continua ou reinicia o processo. Quando não há histórico, os pesos e bias são gerados aleatoriamente dentro do intervalo contínuo de -1 a 1, arredondados a três casas decimais. Essa faixa de inicialização é prática comum em redes multicamadas, no qual valores próximos de zero, mas não nulos, favorecem a quebra de simetria entre neurônios de uma mesma camada e evitam gradientes iniciais excessivamente pequenos ou grandes (GOODFELLOW; BENGIO; COURVILLE, 2016), já que inicializações de magnitude elevada tendem a saturar as funções de ativação logo nas primeiras épocas (BISHOP, 2006).

Um aspecto particular da implementação é que as dimensões de entrada e saída de cada rede não são fixadas no código, mas inferidas dinamicamente a partir do primeiro par de entrada e saída do próprio conjunto de dados cadastrado, permitindo, em tese, ajustar o número de entradas e saídas de qualquer rede apenas pela composição dos dados, sem alterar o backend (NIXON, 2018). O sistema também permite reiniciar o aprendizado de uma versão sem excluir os dados já cadastrados, apagando apenas o histórico de pesos, bias e métricas, diferente da exclusão completa da versão, útil quando o treinamento anterior converge para um mínimo local insatisfatório da função de erro (FACELI et al., 2021).

3.6.2. Algoritmo de treinamento

O treinamento de cada rede é conduzido por um laço externo de épocas, dentro do qual todo o conjunto de dados cadastrado para aquela versão é percorrido uma vez por época, executando para cada par de entrada e saída, seguido imediatamente da atualização dos pesos por meio do algoritmo de retropropagação, aplicado à saída obtida naquele mesmo exemplo (BISHOP, 2006).

Quando uma mesma versão já possui treinamentos anteriores, a numeração das épocas registradas no histórico não reinicia a partir de zero, mas é somada ao valor da última época já salva, preservando a continuidade da curva de erro exibida ao operador mesmo quando o treinamento de uma versão é realizado em múltiplas sessões separadas, conforme o mecanismo de retomada já discutido na seção 3.6.1. O Algoritmo 6 resume essa estrutura de treinamento.

Algoritmo 6 – Pseudocódigo de treinamento supervisionado de uma rede

```

Entrada:
    entradas[], alvos[]      // conjunto de dados de treinamento
    epocas, taxa             // hiperparâmetros
    pontos                   // número de registros de histórico desejados

Procedimento:
1. Para epoca de 0 até epocas faça:
    acertos ← vetor de zeros (tamanho = saida_tamanho)
    Para cada amostra i no conjunto de dados faça:
        saida ← forward(entradas[i])
        backpropagation(alvos[i], taxa)
        acertos ← verificar_acertos(alvos[i], acertos)
    FimPara
    Se epoca é múltiplo de (epocas / pontos) então
        salvar_dados(tamanho, acertos, erro_medio, epoca)
    FimSe
    FimPara
2. Persistir pesos, bias e histórico na base de dados

```

Fonte: Elaborado pelo autor (2026)

Um aspecto relevante da implementação é que o histórico de pesos, bias e métricas não é registrado a cada época, mas apenas em intervalos regulares, calculados a partir do número de "pontos" desejados para compor os gráficos da página de Treinamento que por padrão são dez pontos ao longo de todo o processo. Essa amostragem esparsa reduz o volume de dados armazenados a cada execução, evitando registrar centenas ou milhares de estados intermediários quando o número de épocas é elevado, sem comprometer a leitura da tendência geral de convergência do modelo (GOODFELLOW; BENGIO; COURVILLE, 2016).

3.6.3. Medição do erro

O erro exibido nos gráficos da página de Treinamento, para todas as cinco redes do sistema, corresponde ao Erro Quadrático Médio calculado por saída (Equação 11), ainda que a atualização dos pesos durante a retropropagação continue sendo realizada por meio das variações da função de entropia cruzada, adequada à natureza probabilística das saídas produzidas pelas funções sigmoide e Softmax. Essa escolha se justifica pela interpretação mais direta do MSE em termos de magnitude absoluta do erro, permitindo dimensionar, em módulo, o quão distante a saída da rede está do valor desejado, funcionando como indicador visual de convergência complementar à métrica efetivamente utilizada na otimização (BISHOP, 2006).

O erro geral exibido no gráfico consolidado corresponde à média dos Erros Quadráticos Médios de todas as saídas da rede naquele ponto do treinamento, oferecendo uma

medida única de convergência do modelo como um todo, complementar à visão individual por saída já fornecida.

Cabe destacar que essa separação entre a métrica utilizada para acompanhamento visual e a função de perda empregada no cálculo do gradiente não compromete a coerência do processo de treinamento, enquanto o MSE cumpre exclusivamente um papel de comunicação e diagnóstico, sendo mais intuitivo por refletir diretamente a distância numérica entre saída e alvo, a entropia cruzada permanece a métrica que de fato orienta o ajuste dos pesos a cada iteração, por apresentar propriedades de gradiente mais estáveis frente à saturação das funções sigmoide e Softmax (GOODFELLOW; BENGIO; COURVILLE, 2016). Essa prática de dissociar a métrica de exibição da função de otimização é comum em ferramentas de acompanhamento de treinamento de redes neurais, nas quais a legibilidade da informação apresentada ao usuário nem sempre corresponde diretamente à formulação matemática interna do algoritmo de aprendizado.

3.6.4. Critérios de acurácia

O critério utilizado para determinar se uma saída da rede é considerada um acerto, exibido nos gráficos de acurácia por saída e acurácia geral da página de Treinamento, varia conforme a função de ativação empregada na camada de saída de cada rede, seguindo a mesma distinção já estabelecida entre saídas sigmoide e saída Softmax discutida na seção 3.3.1.

Para as quatro redes cuja saída utiliza a função sigmoide (Esteira, Expedição, Elevador e Despacho), um acerto é contabilizado sempre que o valor produzido pela rede se encontra dentro de uma margem de tolerância de ± 0.05 em torno do valor alvo correspondente de um determinado conjunto de dados (i):

$$acerto_i = \begin{cases} 1, & \text{se } |\hat{y}_i - y_i| \leq 0,05 \\ 0, & \text{caso contrário} \end{cases} \quad (24)$$

Já para a rede de Prioridade, cuja saída utiliza a função Softmax sobre uma codificação one-hot dos alvos, o critério de acurácia não é aplicado diretamente sobre os valores contínuos de probabilidade produzidos pela rede. Antes da comparação, o vetor de saída é transformado por meio do critério $\arg \max$, no qual a posição de maior probabilidade recebe o valor 1 e todas as demais recebem 0, sendo esse vetor comparado ao alvo original de modo que o acerto só é contabilizado quando a classe de maior probabilidade prevista pela rede

corresponde exatamente à classe indicada nos dados de treinamento, sem qualquer margem de tolerância numérica. O Algoritmo 7 mostra essa lógica condicional:

Algoritmo 7 – Pseudocódigo de verificação de acertos

```

Entrada:
    saida[]          // vetor de saída da rede (após forward pass)
    alvo[]           // vetor de saída desejado
    func_saida       // função de ativação da camada de saída
    acertos[]        // vetor acumulador de acertos por posição

Procedimento:
1. Se func_saida = "softmax" então
    saida ← argmax(saida)           // binariza pela maior probabilidade
    FimSe
2. Para cada posição i do vetor de saída faça:
    Se func_saida = "softmax" então
        Se saida[i] = alvo[i] então
            acertos[i] ← acertos[i] + 1
        FimSe
    Senão
        Se |saida[i] - alvo[i]| ≤ 0,05 então
            acertos[i] ← acertos[i] + 1
        FimSe
    FimSe
    FimPara
3. Retornar acertos

```

Fonte: Elaborado pelo autor (2026)

Essa distinção de critérios reflete a natureza distinta dos dois tipos de problema tratados pelas cinco redes do sistema, enquanto as saídas sigmóides representam comandos digitais ou variáveis de controle contínuo, para as quais uma pequena margem numérica de tolerância é aceitável e até esperada, a saída Softmax representa uma decisão de classificação mutuamente exclusiva, na qual apenas o acerto exato da classe selecionada tem significado prático para a operação da planta (BISHOP, 2006).

3.7. Comunicação entre camadas

A comunicação entre as diferentes camadas da arquitetura proposta foi estruturada como componente fundamental para a operação integrada do sistema. A troca contínua de informações assegura sincronização entre o dispositivo de controle local, o banco de dados hospedado em nuvem, o módulo de treinamento e a interface supervisória web, permitindo atualização permanente dos estados operacionais e dos registros estratégicos. Conforme descrito por Tanenbaum e Wetherall (2011), sistemas distribuídos dependem de protocolos

padronizados para garantir interoperabilidade, integridade e confiabilidade na transmissão de dados entre nós distintos da rede.

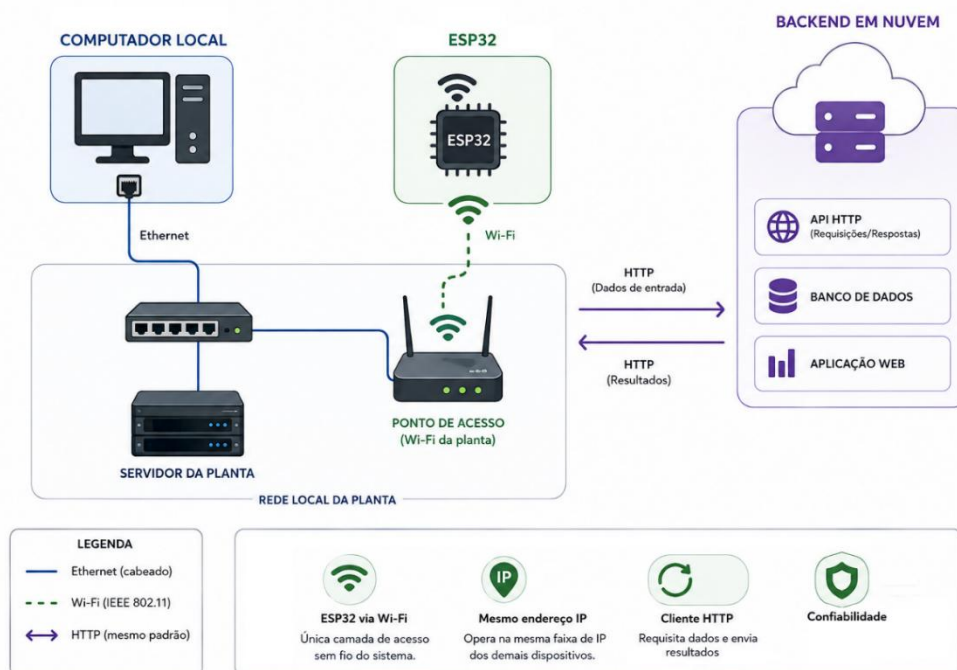
A arquitetura implementada empregou dois protocolos distintos, definidos de acordo com a função desempenhada em cada camada. Para a comunicação entre os dispositivos de campo e o sistema de supervisão local, foi utilizado o protocolo Modbus TCP/IP, amplamente empregado em ambientes industriais para troca de registradores e variáveis de processo (RINALDI, 2015). Já a comunicação entre aplicação web, banco de dados e serviços em nuvem foi realizada por meio do protocolo HTTP, estruturando a troca de dados em modelo cliente-servidor, conforme princípios de redes e aplicações web descritos por Tanenbaum e Wetherall (2011).

3.7.1. Conexão HTTP

A comunicação baseada em HTTP é a camada de comunicação mais amplamente utilizada no sistema desenvolvido, sendo responsável por interligar praticamente todos os módulos que não dependem de resposta imediata, diferentemente do Modbus, empregado exclusivamente para a comunicação de campo entre a planta simulada e os dispositivos de controle. O modelo de comunicação adotado segue a arquitetura cliente-servidor, na qual o backend, implementado em PHP e hospedado em infraestrutura de nuvem, atua como servidor central, recebendo requisições de múltiplos clientes distintos e retornando respostas estruturadas em formato JSON (TANENBAUM; WETHERALL, 2011).

No sistema desenvolvido, essa camada de comunicação é utilizada em três frentes principais. A primeira corresponde à comunicação entre os dispositivos de controle (computador local em Python e microcontrolador ESP32, este conectado via Wi-Fi) e o backend em nuvem, por meio de endpoints responsáveis pela atualização do histórico de produção e pela leitura do estado agregado da planta, nos quais os dispositivos de controle enviam periodicamente os dados coletados de cada rede, ao mesmo tempo em que recebem, na mesma resposta, os parâmetros de rede mais recentes aprovados na página de Treinamento, bem como os pedidos pendentes de atendimento. A segunda frente corresponde à comunicação entre a interface web e o backend, utilizada para o gerenciamento de pedidos, o cadastro e a consulta de dados de treinamento e a execução do treinamento das redes neurais, todas seguindo o mesmo padrão de requisição POST com corpo estruturado em JSON. A terceira frente corresponde à comunicação do backend com serviços externos de terceiros.

Figura 67 – Comunicação HTTP



Fonte: Elaborado pelo autor (2026)

O HTTP, no contexto deste sistema, é empregado para operações que toleram maior latência e que transportam estruturas de dados mais complexas, como o histórico de leituras das seis instâncias de rede neural, os conjuntos de dados de treinamento ou os parâmetros completos de uma rede treinada, características que tornam esse protocolo mais adequado à camada de supervisão e gestão do que ao controle de campo (ERL; PUTTINI; MAHMOOD, 2013). Em todas as requisições, o corpo da mensagem é transmitido em formato JSON, tanto no envio de dados do cliente ao servidor quanto no retorno das respostas, padronização que simplifica a integração entre os diferentes módulos do sistema, independentemente da linguagem ou plataforma de origem da requisição (LOCKHART, 2015).

3.7.1.1. Conexão Wifi

Enquanto o computador local responsável pelo processamento das quatro redes principais se conecta à rede por meio de uma interface Ethernet cabeada, o microcontrolador ESP32, responsável pela rede Expedição B, ingressa na mesma rede local exclusivamente por meio de conectividade Wi-Fi, utilizando o padrão IEEE 802.11, nativamente suportado pelo módulo de rádio integrado ao chip (ESPRESSIF SYSTEMS, 2023). Essa é a única camada de

acesso físico do sistema que depende de um meio de transmissão sem fio, sendo, por esse motivo, tratada separadamente das demais conexões baseadas em Ethernet.

Ao ser inicializado, o firmware do ESP32 estabelece a associação com a rede local por meio das credenciais de acesso (SSID e senha) previamente configuradas em seu código-fonte, obtendo, em seguida, um endereço IP dentro da mesma faixa utilizada pelos demais dispositivos do sistema, conforme descrito na seção 3.4.1. Uma vez conectado, o ESP32 comunica-se com o backend em nuvem exatamente pelo padrão HTTP, requisitando periodicamente os dados de entrada necessários à rede controlada e enviando de volta os resultados do seu processamento local, comportando-se, do ponto de vista da camada de aplicação, de forma equivalente a qualquer outro cliente HTTP do sistema.

A adoção de Wi-Fi para esse dispositivo específico é coerente com o propósito de demonstrar a viabilidade de processamento na borda (edge computing). Por se tratar de um dispositivo embarcado de baixo custo e reduzida capacidade de processamento, o ESP32 não exige a robustez ou a largura de banda de uma conexão cabeada, sendo o Wi-Fi suficiente para sustentar o volume relativamente baixo de dados trafegados pela rede, ao mesmo tempo em que elimina a necessidade de cabeamento físico até o ponto de instalação do dispositivo, vantagem prática recorrente em aplicações industriais de Internet das Coisas (IIoT), nas quais sensores e atuadores de menor porte são frequentemente distribuídos em pontos de difícil acesso ao cabeamento estruturado da planta (VENERI; CAPASSO, 2018). Do ponto de vista da confiabilidade, a conexão sem fio introduz uma fonte adicional de variabilidade de latência e de possível perda momentânea de conectividade.

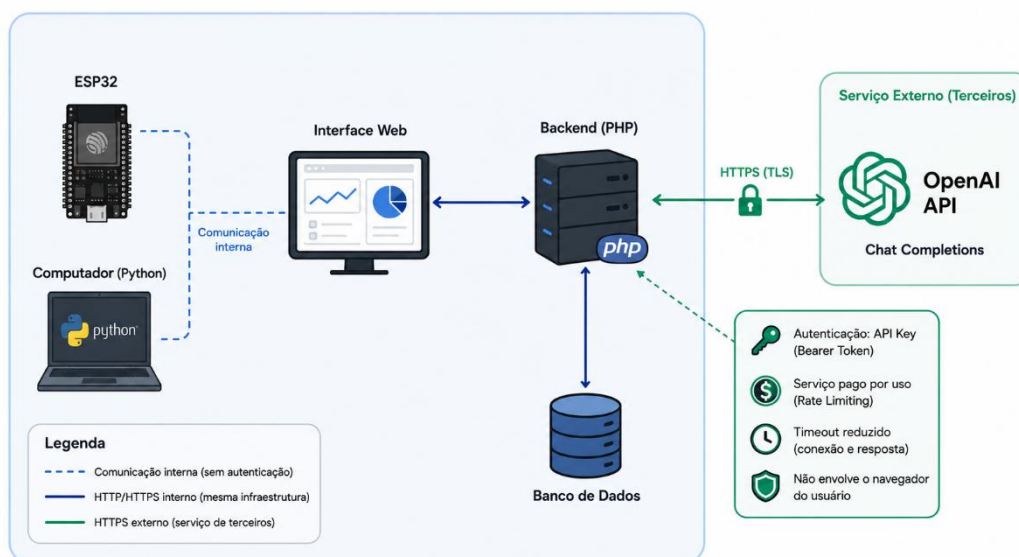
3.7.1.2. Comunicação com serviço externo

Além da comunicação interna entre os dispositivos de controle, a interface web e o backend em nuvem, o sistema também estabelece comunicação com um serviço externo de terceiros: a API da OpenAI, utilizada na geração automática de relatórios analíticos descrita na seção 3.5.2. Ainda que tecnicamente baseada no mesmo protocolo HTTP, mais especificamente, sobre uma conexão HTTPS que adiciona uma camada de criptografia via TLS (Transport Layer Security) à comunicação, essa troca de dados apresenta características distintas das demais comunicações internas do sistema.

A primeira diferença está na direção e na origem da requisição, enquanto as demais comunicações HTTP do sistema ocorrem entre componentes controlados, como dispositivos de controle, interface web e banco de dados, todos hospedados na mesma infraestrutura em nuvem,

a comunicação com a API da OpenAI é realizada exclusivamente pelo backend em direção a um serviço hospedado por um provedor externo, sem qualquer participação direta do navegador do usuário nessa etapa, medida de segurança já discutida anteriormente para evitar a exposição da chave de acesso à API (LOCKHART, 2015). A segunda diferença está na natureza da autenticação exigida, que diferentemente das requisições internas do sistema, que não demandam mecanismo de autenticação por não saírem da infraestrutura controlada, a comunicação com a API externa exige a apresentação de uma chave de acesso (API key) em cada requisição, transmitida por meio do cabeçalho HTTP de autorização, seguindo o padrão Bearer Token, comumente adotado por serviços de API RESTful hospedados na nuvem (ERL; PUTTINI; MAHMOOD, 2013).

Figura 68 – Comunicação com serviço externo



Fonte: Elaborado pelo autor (2026)

A terceira diferença está relacionada ao custo e à criticidade operacional dessa comunicação por se tratar de uma chamada a um serviço pago por uso, o sistema implementa um mecanismo de controle de taxa (rate limiting) na própria aplicação, restringindo o intervalo mínimo entre chamadas consecutivas à API, prática que evidencia que a comunicação com serviços externos exige, além dos cuidados técnicos usuais de qualquer requisição HTTP, uma camada adicional de controle de uso vinculada aos custos operacionais do serviço consumido (BUNTAK et al., 2021).

Por fim, é definido um tempo limite (timeout) reduzido para a conexão e para a resposta da requisição externa, evitando que uma eventual indisponibilidade ou lentidão do serviço de terceiros comprometa indefinidamente a experiência da interface web, isolando, na

medida do possível, a confiabilidade do sistema desenvolvido da confiabilidade do serviço externo do qual ele depende (TANENBAUM; WETHERALL, 2011).

3.7.2. Conexão modBus

A comunicação Modbus TCP/IP do sistema envolve três elementos distintos: o servidor da planta simulada no Factory I/O, atuando como dispositivo escravo (slave), e dois clientes independentes responsáveis por interrogá-lo, o controlador local em Python e o dispositivo embarcado ESP32. O Factory I/O é configurado como servidor Modbus no endereço IP 192.168.1.17, na porta 502, padrão reservada ao protocolo), sob o identificador Slave ID 1 (MODBUS ORGANIZATION, 2012). Esse mesmo servidor é acessado simultaneamente pelos dois clientes de controle, cada um estabelecendo e mantendo sua própria conexão Modbus de forma independente, ambos utilizando a mesma porta e o mesmo Slave ID do servidor.

A cada ciclo, o controlador realiza a leitura simultânea de dois grupos de variáveis da planta, sendo sete entradas digitais (discrete inputs) e seis registradores analógicos de entrada (input registers). Caso a leitura falhe ou a conexão seja interrompida, o sistema registra a falha, encerra a conexão corrente e sinaliza o estado de desconexão, permitindo que o laço principal de controle tente restabelecer a comunicação automaticamente na iteração seguinte, sem interromper a execução do programa (BOYER, 2010).

Figura 69 – Configuração servidor FactoryI/O

Server

☒ Auto start

Host
192.168.1.17

Port
502

Slave ID
1

Network adapter
Qualcomm Atheros QCA9377 Wireless Netwo

Fonte: Adaptado de REAL GAMES (2023)

Figura 70 – Configuração entradas e saídas Factory I/O

I/O Points

	Offset	Count
Digital Inputs	0	7
Digital Outputs	0	14
Register Inputs	0	6
Register Outputs	0	8

Fonte: Adaptado de REAL GAMES (2023)

No controlador local, conectado à rede por meio de conexão cabeada, a comunicação é implementada em Python, utilizando a biblioteca pymodbus para encapsular as operações de leitura e escrita de registradores. O cliente Modbus é instanciado uma única vez no início da execução do controlador e reutilizado ao longo de todo o ciclo de varredura.

Já no dispositivo embarcado ESP32, a comunicação é implementada utilizando a biblioteca ModbusIP_ESP8266.h, responsável por prover, em ambiente de microcontrolador, uma implementação do protocolo compatível com a arquitetura do ESP32, ainda que originalmente concebida para a família ESP8266. Diferentemente do controlador em Python, cuja execução ocorre sobre um sistema operacional completo e dispõe de recursos de processamento mais amplos, a instância executada no ESP32 opera sob as restrições próprias de um sistema embarcado, com memória e capacidade de processamento reduzidas. A Tabela 17 apresenta a associação das entradas digitais e seus respectivos endereços implementados.

Tabela 17 – Endereços Modbus Entradas Digitais

Dispositivo	Associação	Endereço
Sensor início A	Variável ProdA	Input 00
Sensor Fim A	RNA ExpedicaoA/Variável ProdA	Input 01
Sensor com peça manipulador A	RNA ExpedicaoA/Variável ProdA	Input 02
Sensor início B	Variável ProdB	Input 03
Sensor Fim B	RNA ExpedicaoB/Variável ProdB	Input 04
Sensor com peça manipulador B	RNA ExpedicaoB/Variável ProdB	Input 05
Sensor Esteira Elevador	RNA Elevador/Variável Elev	Input 06

Fonte: Elaborado pelo autor (2026)

A Tabela 18 apresenta o endereçamento dos registradores analógicos de entrada, associados aos sensores e variáveis de leitura contínua da planta.

Tabela 18 – Endereços Modbus entradas analógicas

Dispositivo	Associação	Endereço
Sensor entrada	RNA Prioridade/Variável ProdMain	Input Reg 00
Posição manipulador A eixo horizontal	RNA ExpedicaoA	Input Reg 01
Posição manipulador A eixo vertical	RNA ExpediçãoA	Input Reg 02
Posição manipulador B eixo horizontal	RNA ExpediçãoB	Input Reg 03
Posição manipulador B eixo vertical	RNA ExpediçãoB	Input Reg 04
Nível elevador	RNA Elevador/RNA Esteira	Input Reg 05

Fonte: Elaborado pelo autor (2026)

Uma vez processadas pelas seis instâncias das redes neurais responsáveis pelo controle da planta, as decisões resultantes são traduzidas em comandos de escrita sobre dois grupos de saída. Sendo o primeiro deles as bobinas (coils), correspondentes aos atuadores digitais da planta, representados na Tabela 19.

Tabela 19 – Endereço Modbus saídas digitais

Dispositivo	RNA	Saída RNA	Endereço
Repositor	Esteira	1	Coil 00
Pivô A	Prioridade	1	Coil 01
Pivô A Esteira	Prioridade	1	Coil 02
Pivô B	Prioridade	2	Coil 03
Pivô B Esteira	Prioridade	2	Coil 04
LED Elevador	Prioridade	3	Coil 05
Manipulador A agarrar	ExpedicaoA	1	Coil 06
Manipulador B agarrar	ExpedicaoB	1	Coil 07
Elevador esteira	Elevador	2	Coil 08
Esteira nível 0	Elevador	3	Coil 09
Esteira nível 1	Elevador	4	Coil 10
Esteira nível 2	Elevador	5	Coil 11
Esteira saída A	Despacho	1	Coil 12
Esteira saída B	Despacho	2	Coil 13

Fonte: Elaborado pelo autor (2026)

De forma análoga, a Tabela 20 apresenta o endereçamento dos registradores de retenção (holding registers), responsáveis pelas variáveis de controle contínuo da planta.

Tabela 20 – Endereço Modbus Saídas Analógicas

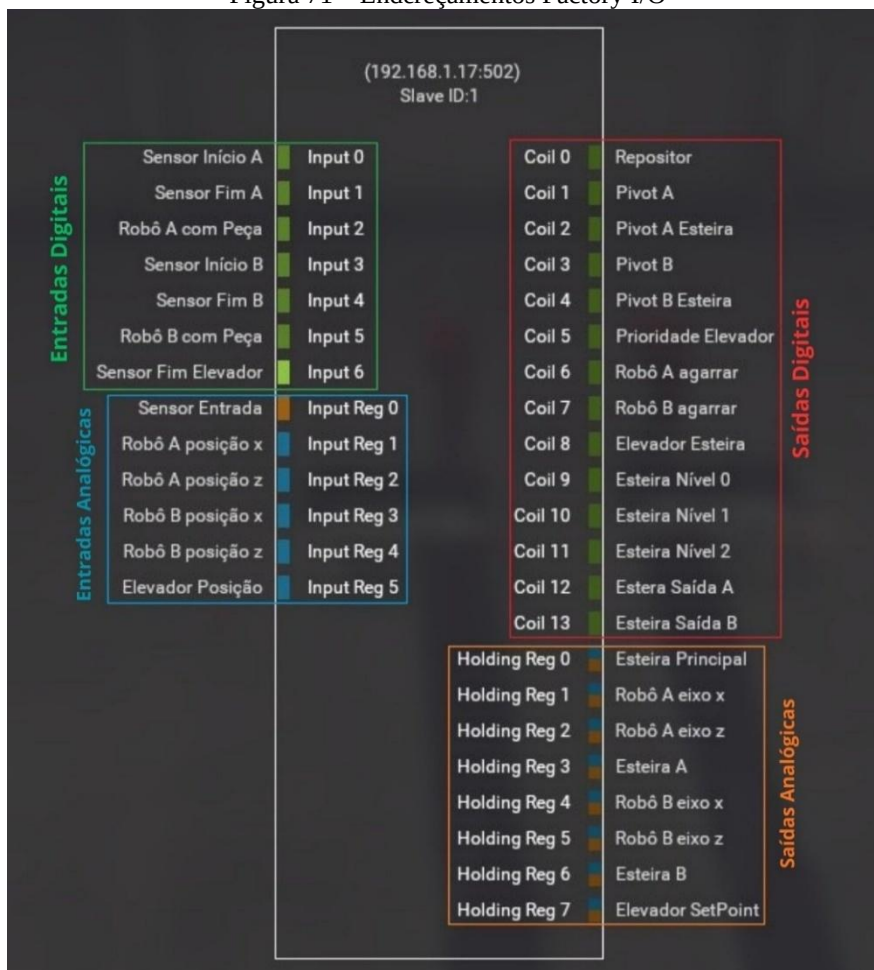
Dispositivo	RNA	Saída RNA	Endereço
Esteira Principal	Esteira	2	Holding Reg 00
Manipulador A eixo horizontal	ExpedicaoA	2	Holding Reg 01
Manipulador A eixo vertical	ExpedicaoA	3	Holding Reg 02
Esteira secundária A	ExpedicaoA	4	Holding Reg 03
Manipulador B eixo horizontal	ExpedicaoB	2	Holding Reg 04
Manipulador B eixo vertical	ExpedicaoB	3	Holding Reg 05
Esteira secundária B	ExpediçãoB	4	Holding Reg 06
Elevador setpoint	Elevador	1	Holding Reg 07

Fonte: Elaborado pelo autor (2026)

As saídas digitais são obtidas diretamente das saídas das redes neurais, aplicando a função de limiar (Equação 15), ao passo que as saídas analógicas são reescaladas antes da escrita no registrador correspondente (Equação 16), uma vez que o protocolo Modbus trabalha

exclusivamente com valores inteiros em registradores de 16 bits, sendo necessário converter o valor contínuo normalizado produzido pela rede neural, no intervalo [0, 1], para um valor inteiro proporcional antes da transmissão (RINALDI, 2015).

Figura 71 – Endereçamentos Factory I/O



Fonte: Elaborado pelo autor (2026)

A adoção do Modbus TCP/IP aproxima a arquitetura desenvolvida de arranjos industriais baseados em redes Ethernet, mantendo compatibilidade com práticas consolidadas de integração entre dispositivos de campo e sistemas de supervisão (BELGA, 201).

4. RESULTADOS

Este capítulo apresenta os resultados obtidos a partir da implementação e do treinamento do sistema inteligente de controle descrito no Capítulo 3, abrangendo desde o processo de treinamento inicial de cada uma das cinco redes neurais que compõem a arquitetura até a análise do comportamento da planta simulada em operação. Os resultados são

apresentados de forma progressiva, iniciando pela etapa de treinamento seguida pela análise do funcionamento integrado da planta, contemplando o fluxo de produção, o comportamento das redes em tempo de operação e o desempenho da arquitetura de comunicação e supervisão proposta.

4.1. Treinamento Inicial das RNAs

O treinamento inicial de cada rede foi conduzido de forma independente, por meio da página de Treinamento descrita na seção 3.5.1.3, seguindo o mesmo procedimento geral para todas as cinco redes do sistema: definição de um conjunto de dados supervisionado representativo do comportamento esperado, execução do algoritmo de retropropagação por um número definido de épocas e taxa de aprendizado, e posterior análise da evolução do erro e da acurácia ao longo do processo, antes da aprovação da versão de pesos para uso.

Para fins de acompanhamento visual do treinamento, o erro exibido nos gráficos de todas as redes é medido por meio do MSE, ainda que a função de perda efetivamente utilizada na atualização dos pesos seja a entropia cruzada. A opção pelo MSE nessa etapa de acompanhamento se justifica por sua interpretação mais direta em termos de magnitude absoluta do erro, permitindo dimensionar o quão distante a saída da rede está do valor desejado, funcionando como indicador complementar de convergência (BISHOP, 2006). Os critérios de erro e acurácia foram definidos nas seções 3.6.3 e 3.6.4, respectivamente.

4.1.1. Treinamento da rede prioridade

O treinamento da rede responsável por decidir o destino de cada item detectado na esteira principal foi conduzido a partir de um conjunto de dados elaborado para representar as diferentes combinações possíveis entre a quantidade restante de itens em cada pedido ativo e o tempo relativo decorrido desde o início de produção de cada um deles, conforme entradas detalhadas na seção 3.3.4.1. Por se tratar da rede que trata o problema de maior variabilidade combinatória entre as cinco que compõem o sistema, uma vez que qualquer um dos três tipos de item pode ocorrer em conjunto com diferentes níveis de demanda pendente em cada pedido, sua composição de dados exigiu maior atenção à cobertura de cenários do que as demais redes do sistema.

A composição dos dados de treinamento buscou representar, de forma equilibrada, três grandes grupos de cenários: situações em que apenas o pedido A ainda demanda o item

detectado, devendo a rede direcioná-lo integralmente para a esteira A, situações simétricas em que apenas o pedido B demanda o item e situações em que ambos os pedidos ainda apresentam demanda pendente para o mesmo tipo de item, cenário em que a decisão de destino passa a depender do indicador de tempo relativo de cada pedido, priorizando aquele que se encontra em produção há mais tempo. Foram também incluídos exemplos em que nenhum dos dois pedidos demanda mais o item detectado, situação em que a rede deve direcioná-lo ao elevador de armazenamento, conforme ilustrado pela Figura 24. A Figura 72 mostra o conjunto de entradas e saídas configurados para treinamento da rede Prioridade.

Figura 72 – Conjunto de dados para treinamento da rede prioridade

ID	Entradas	Saídas
1	0,6,0,6,1,1	0,1,0
2	0,6,0,5,1,1	1,0,0
3	0,5,0,5,0,0	0,1,0
4	0,5,0,4,0,0	1,0,0
5	0,4,0,4,1,0	1,0,0
6	0,3,0,3,0,1	0,1,0
7	0,2,0,1,1,1	1,0,0
8	0,2,0,1,0,1	0,1,0
9	0,1,0,2,1,1	0,1,0
10	0,1,0,2,1,0	1,0,0
11	0,1,0,0,1	1,0,0
12	0,1,0,1,1	1,0,0
13	0,0,1,1,0	0,1,0
14	0,0,1,1,1	0,1,0
15	0,0,0,0	0,0,1
16	0,0,1,1	0,0,1
17	0,0,1,0	0,0,1
18	0,0,0,1	0,0,1
19	0,6,0,1,0,1	0,1,0
20	0,1,0,6,1,0	1,0,0
21	0,6,0,0,1	1,0,0
22	0,0,6,1,0	0,1,0
23	0,1,0,0,1	1,0,0
24	0,0,1,1,0	0,1,0

Fonte: Elaborado pelo autor (2026)

Diferentemente das redes de saída sigmoide já apresentadas, cujo critério de acerto admite uma margem de tolerância de $\pm 0,05$ em torno do valor alvo, o conjunto de dados desta rede foi estruturado considerando o critério de acurácia, exigindo que a classe de maior probabilidade produzida pela função Softmax corresponda precisamente à classe indicada nos dados de treinamento. Essa exigência tornou especialmente importante a inclusão de exemplos que representassem situações de ambiguidade, como demandas simultâneas de mesma magnitude em ambos os pedidos, de modo que a rede aprendesse a utilizar o indicador de tempo relativo como critério de desempate consistente, evitando alternância indesejada entre destinos em cenários muito próximos entre si.

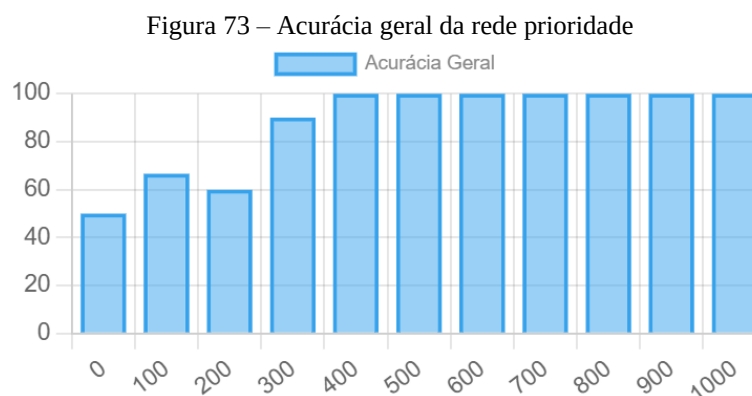
Como ilustrado na Figura 72, a saída é codificada em formato one-hot, indicando o destino correto entre os três possíveis, em que nos casos em que apenas um dos pedidos ainda demanda o item detectado, como nas IDs 11, 12, 21 e 23 (apenas o pedido A demanda) e nas IDs 13, 14, 22 e 24 (apenas o pedido B demanda), o destino é definido de forma trivial, já que o pedido com demanda nula para aquele item não pode ser um candidato válido. De forma semelhante, quando nenhum dos dois pedidos apresenta demanda pendente do item, como nas IDs 15 a 18, o item é encaminhado ao elevador de armazenamento, independentemente do tempo relativo de produção de cada pedido. Nos demais casos, em que ambos os pedidos apresentam demanda pendente do item, o critério de decisão passa a considerar a relação entre demanda e tempo relativo. Quando o tempo relativo é igual entre os dois pedidos, como nas IDs 2, 4, 7 e 9, a rede aprende a priorizar o pedido com maior quantidade pendente do item, evitando que a diferença de progresso entre os dois pedidos se acentue. Quando, por sua vez, a demanda pendente é igual entre os pedidos, como nas IDs 5 e 6, o critério de desempate passa a ser o tempo relativo de produção, priorizando o pedido que está em produção há mais tempo. Nos casos em que tanto a demanda quanto o tempo relativo são idênticos entre os dois pedidos, como nas IDs 1 e 3, a rede prioriza consistentemente o pedido B, funcionando como uma regra de desempate residual para a situação de igualdade total entre os candidatos.

O aspecto mais relevante do conjunto de dados, no entanto, está nas amostras em que demanda e tempo relativo apontam para pedidos diferentes, como nas IDs 8, 10, 19 e 20. Na ID 10, o pedido B possui maior quantidade pendente do item (0,2 contra 0,1 do pedido A), mas é o pedido A quem recebe o item, por apresentar tempo relativo máximo (1,0) frente ao tempo nulo do pedido B, de forma simétrica, na ID 8, o pedido A possui maior demanda pendente, mas o item é direcionado ao pedido B, que aguarda há mais tempo. As IDs 19 e 20 reforçam esse mesmo comportamento diante de diferenças de demanda mais acentuadas, sendo que na ID 19, o pedido A possui demanda pendente muito superior à do pedido B (0,6 contra

0,1), ainda assim o item é direcionado a B, por este apresentar o maior tempo relativo, de forma simétrica, na ID 20, o item é direcionado a A pelo mesmo critério, apesar da demanda consideravelmente maior do pedido B. Esses exemplos evidenciam que o tempo de espera foi tratado como critério prioritário sobre a quantidade de itens ainda pendente, de modo a evitar que um pedido em produção há muito tempo permaneça indefinidamente represado apenas por possuir uma demanda pontualmente menor que a do pedido concorrente.

O treinamento foi conduzido com 1000 épocas e taxa de aprendizado de 0,5 e tendo a acurácia geral de aproximadamente 50% no instante inicial, evoluindo para cerca de 66% na época 100. Entre as épocas 100 e 200 observou-se uma leve queda no percentual de acertos, para aproximadamente 60%, comportamento compatível com o critério de acerto exato adotado por esta rede que torna a métrica mais sensível a pequenos ajustes nos pesos durante as fases iniciais do treinamento, quando a fronteira de decisão entre as três classes ainda está sendo definida.

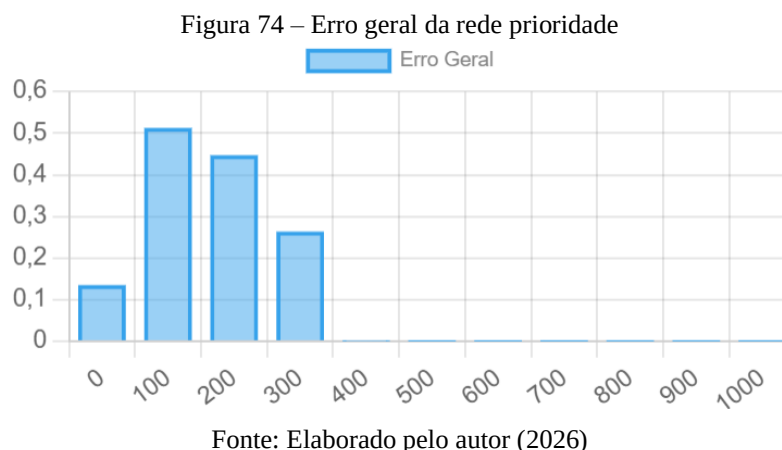
A partir da época 300, a acurácia salta para aproximadamente 90%, atingindo 100% já na época 400 e permanecendo estável nesse patamar até a época 1000, indicando que a rede convergiu para uma solução capaz de classificar corretamente a totalidade dos exemplos do conjunto de treinamento.



Fonte: Elaborado pelo autor (2026)

O erro parte de aproximadamente 0,13 no instante inicial e sobe para seu valor máximo, próximo de 0,51 na época 100, decrescendo em seguida para cerca de 0,44 na época 200 e 0,26 na época 300, até se aproximar de zero a partir da época 400, mantendo-se estável nesse patamar até o final do treinamento. O crescimento inicial do erro entre as épocas 0 e 100, concomitante ao aumento da acurácia no mesmo intervalo, é consistente com a natureza da função de perda de entropia cruzada categórica associada à saída Softmax, discutida na seção 2.6.5.2. Nas primeiras épocas, a rede pode acertar a classe de maior probabilidade em um

número maior de amostras, elevando a acurácia, ainda que as probabilidades atribuídas às demais classes permaneçam pouco definidas, o que eleva o erro quadrático médio calculado sobre o vetor de saída completo.



A partir da época 300, a queda acentuada do erro acompanha o salto observado na acurácia, e ambas as métricas se estabilizam simultaneamente a partir da época 400, evidenciando que o modelo atingiu convergência e foi capaz de associar corretamente as combinações de demanda pendente e tempo relativo de cada pedido à decisão de destino esperada, inclusive nos cenários de maior ambiguidade entre os dois pedidos ativos.

4.1.2. Treinamento da rede elevador

A composição do conjunto de dados de treinamento levou em consideração a natureza determinística do problema tratado por esta rede, diferentemente da rede de Prioridade, que lida com múltiplas combinações possíveis de estado de pedidos, o comportamento esperado do elevador é diretamente determinado pelo tipo de item recebido, havendo pouca variabilidade de cenários a serem cobertos. Por esse motivo, optou-se por um conjunto de dados reduzido, mas representativo de todas as combinações relevantes de entrada. Problemas com número reduzido de situações possíveis e respostas bem definidas permitem que a rede aprenda o mapeamento adequado com poucos exemplos (BISHOP, 2006). Os pares de treinamento foram elaborados de modo a cobrir três grupos de cenários:

- a) Situações de repouso, em que nenhum item foi recebido e o elevador deve permanecer inativo;

- b) Situações de recebimento de cada um dos três tipos de item (X, Y e Z), associando corretamente cada um ao seu nível de armazenamento;
- c) Situações em que a plataforma já se encontra na posição correspondente ao item recebido, de modo que a rede aprenda a manter o acionamento da esteira de destino independentemente da posição de partida do elevador.

Essa cobertura foi considerada suficiente para representar toda a lógica de decisão exigida da rede, sem a necessidade de um conjunto de dados mais extenso. A Figura 75 apresenta os pares de entrada e saída utilizados no treinamento inicial da rede Elevador.

Figura 75 – Conjunto de dados para treinamento da rede elevador

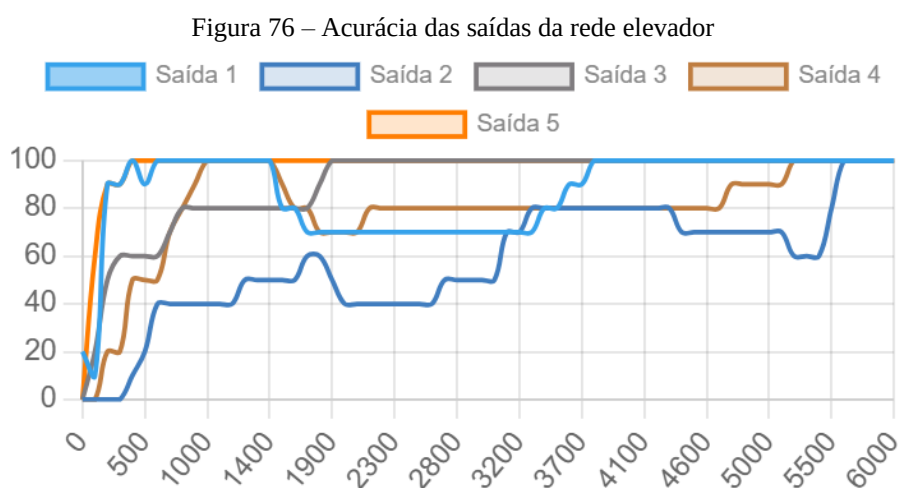
ID	Entradas	Saídas
1	0,0.6	0.51,0,0,0,0
2	0.46,0.6	0.51,0,0,1,0
3	0.51,0.6	0.51,1,0,1,0
4	0,0.9	0.96,0,0,0,0
5	0.91,0.9	0.96,0,0,0,1
6	0.96,0.9	0.96,1,0,0,1
7	0,0.3	0,1,1,0,0
8	0,0	0,1,0,0,0
9	0.96,0	0,0,0,0,0
10	0.1,0.3	0,0,0,0,0

Fonte: Elaborado pelo autor (2026)

Os pares 1, 2 e 3 representam o mesmo item do tipo Y sendo aproximado progressivamente do nível correspondente, sendo que na posição inicial (nível 0), nenhuma esteira é acionada, ainda que o setpoint já indique o nível de destino, à medida que a plataforma se aproxima da posição-alvo, a esteira do elevador e a esteira do nível correspondente passam a ser acionadas. O mesmo padrão se repete nos pares 4, 5 e 6 para o item do tipo Z, confirmando que a rede aprenda a condicionar o acionamento das esteiras à proximidade real entre a posição atual e o setpoint, e não apenas ao tipo de item recebido. Já o par 7 mostra que o item do tipo X corresponde ao próprio nível base do elevador (setpoint igual a zero), sendo por isso, o único caso em que o acionamento ocorre de forma imediata, sem etapa intermediária de aproximação. Os pares 8, 9 e 10 tratam de situações de ausência de item ou de posicionamento intermediário,

quando não há item a posicionar, a rede aprende a redirecionar o setpoint para o nível base, mantendo as esteiras de destino desativadas, garantindo que a plataforma retorne à posição de repouso ao final de cada ciclo de entrega.

O treinamento foi conduzido em lotes de 1000 épocas, com taxa de aprendizado de 0,5 que foram repetidos até que a rede convergisse para 100% de acurácia, resultado atingido após 6 lotes, totalizando 6000 épocas de treinamento. A Figura 76 apresenta a evolução da acurácia de cada uma das cinco saídas da rede ao longo do treinamento.



Fonte: Elaborado pelo autor (2026)

As saídas 1 (setpoint) e 5 (esteira do nível Z) apresentam convergência inicial mais rápida, atingindo aproximadamente 100% de acerto já nas primeiras centenas de épocas. A saída 5 mantém-se estável nesse patamar durante praticamente todo o restante do treinamento, enquanto a saída 1, apesar da convergência inicial igualmente rápida, sofre uma queda para aproximadamente 70% por volta da época 1900, permanecendo nesse patamar até cerca da época 3700, quando volta a convergir para 100% e ali se mantém até o fim do treinamento.

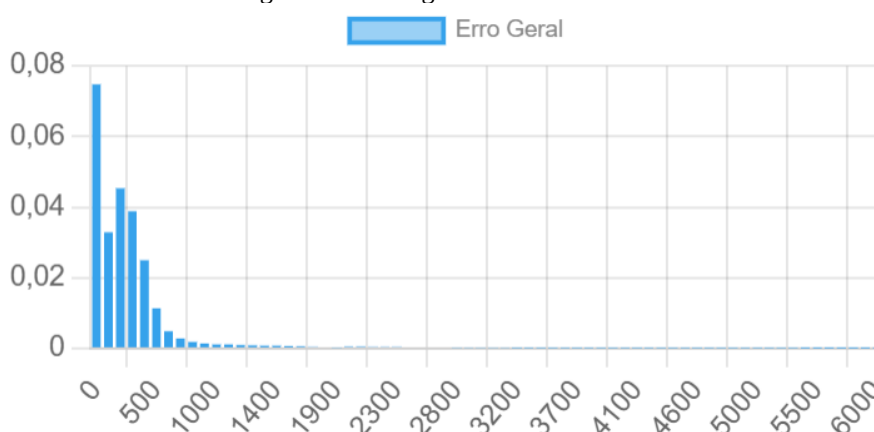
A saída 3 (esteira do nível X) segue trajetória de convergência inicial semelhante, alcançando por volta de 80% já nas primeiras 500 épocas e permanecendo estável nesse patamar até aproximadamente a época 1700, quando sobe rapidamente e se estabiliza próxima de 100% a partir de aproximadamente 1900 épocas. A saída 4 (esteira do nível Y) apresenta convergência mais gradual, permanecendo entre 70% e 90% ao longo de grande parte do treinamento e só atingindo 100% de forma consistente a partir de aproximadamente 4900 épocas. A saída 2 (esteira do elevador) é a que exhibe o comportamento mais irregular, oscilando entre 40% e 75% ao longo de praticamente toda a curva e apresentando inclusive uma queda pontual para cerca

de 60% próximo à época 5300, antes de convergir para 100% apenas ao final do treinamento, por volta da época 5700.

Esse comportamento é coerente com a natureza dessas saídas, cujo acionamento depende de condições de proximidade entre posição atual e setpoint que se repetem em praticamente todas as amostras do conjunto de dados, exigindo mais épocas para que a rede ajuste corretamente a margem de tolerância que caracteriza o acerto, o que também explica por que até mesmo saídas de convergência inicial rápida, como a saída 1, podem sofrer quedas temporárias de acurácia ao longo do treinamento, à medida que os pesos continuam sendo ajustados em função do erro das demais saídas da mesma rede.

O erro geral da rede parte de aproximadamente 0,075 nas primeiras épocas, decrescendo de forma acentuada e praticamente monotônica até se aproximar de zero por volta da época 800, permanecendo estável nesse patamar até o final das 6000 épocas. Esse comportamento indica que, embora a acurácia de algumas saídas, em particular a saída 2, só tenha se estabilizado em 100% nas épocas finais do treinamento, o erro médio agregado sobre o conjunto de saídas já se encontrava consideravelmente baixo desde as primeiras centenas de épocas, uma vez que a maior parte das saídas da rede convergiu rapidamente e que os erros pontuais de classificação da saída mais lenta representam uma parcela pequena do erro total calculado sobre o vetor de saída completo.

Figura 77 – Erro geral da rede elevador



Fonte: Elaborado pelo autor (2026)

A análise conjunta da acurácia e do erro geral permite concluir que a rede foi capaz de aprender corretamente o mapeamento entre a posição do elevador, o tipo de item recebido e o acionamento adequado de cada esteira de destino, sendo a versão final de pesos aprovada para uso em produção conforme o mecanismo de validação descrito na seção 3.5.1.3.

4.1.3. Treinamento da rede esteira

O treinamento da rede responsável pelo controle da esteira principal e o momento de acionamento do repositor de produtos foi conduzido a partir de um conjunto de dados elaborado para cobrir as combinações relevantes entre o estado de ocupação dos quatro possíveis destinos de um item (esteira principal, linha A, linha B e elevador). A Figura 78 contém os vetores de entrada e saída utilizados na condução do treinamento da rede Esteira.

Figura 78 – Conjunto de dados para treinamento da rede esteira

ID	Entradas	Saídas
1	0,0,0,0,0,0,1,0	1,0.3
2	0.3,0,0,0,0,0,1,0	0,0.3
3	0,0,0,0.3,0,0,1,0	1,0.1
4	0.3,0,0,0.9,0,0,1,0.9	0,0.1
5	0.9,0,0,0,0,0,1,0	0,0.3
6	0.9,0,0,0.3,0,0,1,0	0,0.1
7	0.3,0,0,0,0,0,1,0.7	0,0.1
8	0.3,0,0,0,0,0,1,0.5	0,0.15
9	0.9,0,0,0,0,0,1,0.1	0,0.15
10	0,0.9,0,0,1,0,0,0	1,0.1
11	0,0,0,0,1,0,0,0	1,0.2
12	0.3,0.3,0,0,1,0,0,0	0,0.1
13	0.9,0.9,0,0,1,0,0,0	0,0.1
14	0,0,0.9,0,0,1,0,0	1,0.1
15	0,0,0,0,0,1,0,0	1,0.2
16	0.3,0,0.3,0,0,1,0,0	0,0.1
17	0.9,0,0.9,0,0,1,0,0	0,0.1

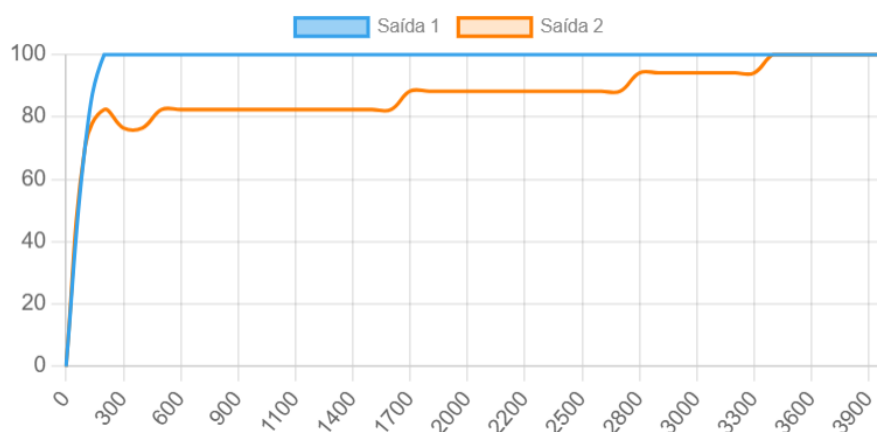
Fonte: Elaborado pelo autor (2026)

A composição dos dados de treinamento buscou representar tanto os cenários em que o destino selecionado está livre, situação em que a esteira deve operar em velocidade nominal e o repositor deve ser autorizado a inserir um novo item, quanto os cenários em que o destino selecionado ainda está ocupado, caso em que a velocidade deve ser reduzida e a inserção

de um novo item deve ser bloqueada, evitando que o produto alcance a zona de transferência antes da liberação do caminho correspondente. Também foram incluídos exemplos em que a esteira principal já possui um item em trânsito, situação em que o repositor deve permanecer bloqueado independentemente do estado dos demais destinos, e exemplos em que o elevador ainda não atingiu o nível de destino, caso em que a velocidade deve ser reduzida até que o posicionamento seja concluído.

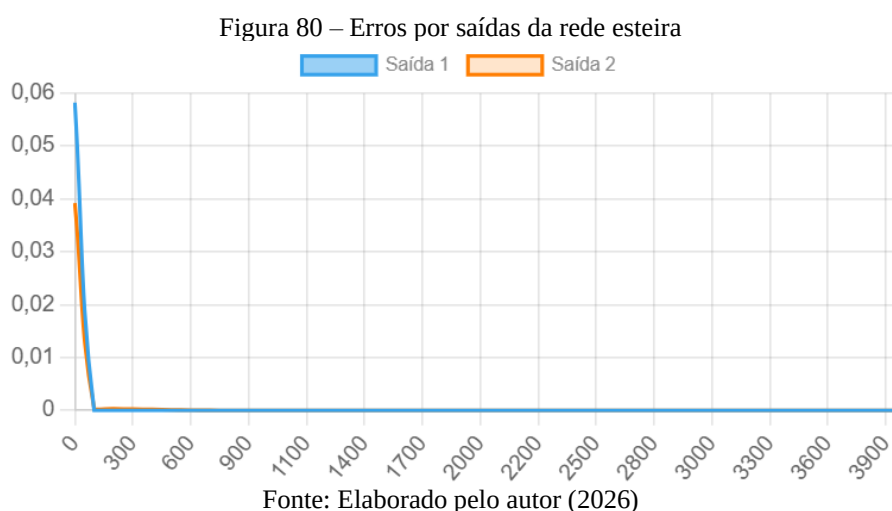
Como ilustrado na Figura 78, cada amostra é composta por oito entradas, sendo elas, o estado de ocupação da esteira principal e dos três destinos (linha A, linha B e elevador), o destino selecionado pela rede de Prioridade, codificado em formato one-hot, e o nível atual do elevador de armazenamento. Observa-se, por exemplo, que nas IDs 1 e 2 o destino selecionado é sempre o elevador, mas o repositor só é liberado (saída igual a 1) quando a esteira principal está livre, sendo bloqueado assim que há um item em trânsito, independentemente do estado dos demais destinos. Já as IDs 3 e 4 evidenciam a influência do nível do elevador sobre a velocidade, com o destino ainda ocupado e o elevador distante do nível de referência, a velocidade é reduzida para 0,1, ao passo que, nas IDs 7, 8 e 9, a velocidade cresce progressivamente à medida que o elevador se aproxima do nível de destino, mesmo com a esteira principal ocupada. Por fim, as IDs 10, 11, 14 e 15 mostram que, quando a esteira principal está livre, o repositor é liberado mesmo que o destino selecionado pela rede de Prioridade esteja momentaneamente ocupado por outro item, casos em que a rede reduz a velocidade de transporte (para 0,1) como forma de compensar a ocupação do destino, mantendo a produção em andamento. Os hiperparâmetros utilizados no treinamento desta rede foram aproximadamente 4000 épocas em uma taxa de aprendizado de 0,5. A Figura 79 apresenta a evolução da acurácia das duas saídas da rede ao longo do treinamento.

Figura 79 – Acurácia das saídas da rede esteira



Fonte: Elaborado pelo autor (2026)

A saída 1, referente à liberação do repositor, converge rapidamente para 100% já nas primeiras 200 épocas, permanecendo estável nesse patamar durante todo o restante do treinamento, comportamento esperado dado o caráter mais determinístico dessa decisão binária. Já a saída 2, referente à velocidade da esteira, apresenta convergência mais gradual, partindo de 0% no início do treinamento, sobe rapidamente para cerca de 85% por volta da época 150, sofre uma leve queda para aproximadamente 75% entre as épocas 300 e 500, e em seguida se estabiliza próxima de 80% ao longo de um intervalo prolongado, entre as épocas 600 e 1600. A partir desse ponto, a acurácia volta a crescer de forma escalonada, atingindo cerca de 88% por volta da época 1700 e permanecendo nesse patamar até aproximadamente a época 2700, quando avança novamente para valores próximos de 95%. Após uma pequena oscilação por volta da época 3300, a saída 2 finalmente converge para 100% ao final do treinamento, próximo da época 3900. Esse padrão de convergência em degraus é coerente com a natureza contínua dessa saída, que precisa aprender a associar diferentes níveis de ocupação e de posicionamento do elevador a diferentes valores de velocidade, exigindo mais épocas de ajuste do que a decisão binária de liberação do repositor.



A Figura 80 apresenta a evolução do Erro de cada uma das duas saídas da rede. Ambas partem de valores relativamente baixos já nas primeiras épocas, aproximadamente 0,059 para a saída 1 e 0,039 para a saída 2, e decrescem de forma acentuada nas primeiras 100 a 200 épocas, atingindo valores próximos de zero e permanecendo estáveis nesse patamar até o final do treinamento. Chama atenção o fato de o erro de ambas as saídas ter convergido rapidamente para valores próximos de zero, ainda que a acurácia da saída 2 tenha continuado a evoluir de forma gradual até praticamente o final do treinamento, esse comportamento é compatível com a margem de tolerância de $\pm 0,05$ adotada como critério de acerto para saídas sigmoide, discutida

na seção 4.1, que permite que pequenos desvios numéricos, suficientes para manter o erro quadrático médio próximo de zero, ainda sejam contabilizados como erro de classificação enquanto a saída não se aproxima o bastante do valor alvo.

A análise conjunta da acurácia e do erro geral indica que a rede foi capaz de aprender corretamente a associação entre o estado de ocupação dos destinos, a decisão de prioridade e o posicionamento do elevador, e a decisão de velocidade e liberação do repositor da esteira principal.

4.1.4. Treinamento da rede expedição

O treinamento da rede responsável pelo controle dos manipuladores cartesianos e das esteiras auxiliares de expedição foi conduzido uma única vez, gerando uma única versão de pesos aplicada simultaneamente às duas instâncias da rede, uma para a linha secundária A e outra para a linha secundária B, uma vez que ambas compartilham exatamente a mesma estrutura de entradas, saídas e lógica de decisão, conforme já justificado na seção 3.3.4.3, diferenciando-se apenas quanto ao conjunto específico de sensores e atuadores a que cada instância está associada na planta simulada. Essa decisão de reaproveitamento reflete-se diretamente na etapa de treinamento, em vez de compor dois conjuntos de dados distintos e conduzir dois processos de treinamento independentes, elaborou-se um único conjunto de dados genérico, válido para qualquer uma das linhas.

A composição dos dados de treinamento buscou representar o ciclo completo de operação do manipulador, contemplando cada uma das etapas sequenciais, como a condição de repouso, em que nenhum item se encontra presente na extremidade da linha secundária e o manipulador permanece parado, a detecção do item pelo sensor de fim de esteira, que deve iniciar o deslocamento do eixo horizontal em direção à posição de captura, o alinhamento vertical do eixo Z até o contato com o produto, seguido da confirmação de presença de material e do correspondente acionamento da garra, por fim, o movimento de retorno dos eixos à posição de descarga, com a consequente liberação da garra.

Por se tratar de uma sequência de movimentos relativamente fixa e previsível, na qual o número de situações possíveis é reduzido e as respostas esperadas são bem definidas, o conjunto de dados necessário para representar adequadamente essa lógica foi comparativamente mais compacto do que o exigido pelas redes de Prioridade e Esteira, sem que isso comprometesse a qualidade do aprendizado, uma vez que a baixa variabilidade do problema favorece a convergência mesmo com poucos exemplos (BISHOP, 2006).

Figura 81 – Conjunto de dados para treinamento da rede expedição

ID	Entradas	Saídas
1	1,0,0,0	0,1,0,0
2	1,0,0.8,0	0,1,0,0
3	1,0,0.9,0	0,1,1,0
4	1,0,1,0.8	0,1,1,0
5	1,0,1,0.9	1,1,1,0
6	1,1,1,1	1,0,0,0
7	0,1,0.2,0.2	1,0,0,0.25
8	0,1,0,0	0,0,0,0.25
9	0,0,0,0	0,0,0,0.25
10	0,1,0.5,0.5	1,0,0,0.25

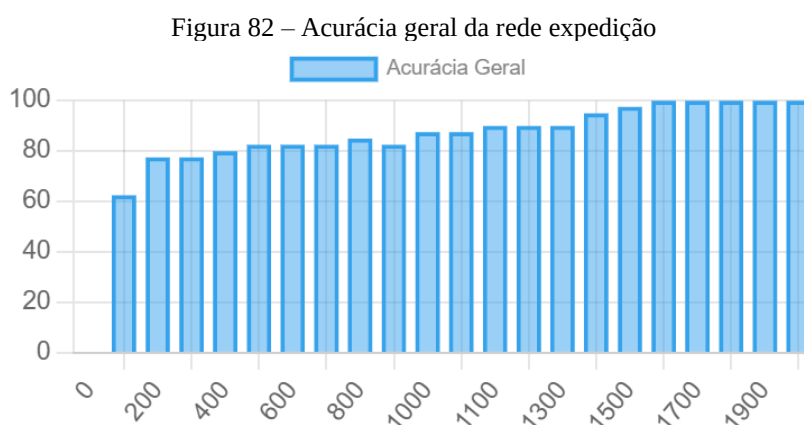
Fonte: Elaborado pelo autor (2026)

Como ilustrado na Figura 81, cada amostra é composta por quatro entradas e por quatro saídas, correspondendo ao acionamento da garra, ao comando dos eixos X e Z e à velocidade da esteira auxiliar. As IDs 1 a 4 representam a fase de aproximação, em que o sensor de fim de esteira já detectou o item, mas o material ainda não foi tocado. Nessa fase, a rede não movimenta os dois eixos simultaneamente, mas de forma sequencial, sendo o comando do eixo X ativado desde a ID 1, aproximando progressivamente o manipulador na horizontal enquanto sua posição reportada avança de 0 até 1 (IDs 1 a 4), somente a partir da ID 3, quando o eixo X já está próximo do valor-alvo, o comando do eixo Z também é ativado, iniciando o deslocamento vertical, cuja posição reportada avança de 0 até 0,9 nas IDs 4 e 5. Essa sequência é coerente com uma estratégia de aproximação horizontal seguida de descida vertical, evitando colisões prematuras do manipulador com os itens ao longo do percurso.

A ID 5 representa o instante em que a garra é finalmente acionada, com os dois eixos praticamente na posição-alvo, enquanto a ID 6 mostra o momento seguinte, já com o sensor de material confirmando o contato, a garra permanece fechada, mas os comandos dos eixos são zerados, indicando que o manipulador deve manter-se parado com o item preso antes de iniciar o retorno. As IDs 7, 8 e 10 representam a fase de retorno e descarga, caracterizada pelo sensor de fim de esteira desligado (o item já foi retirado da esteira principal) e pelo acionamento constante da esteira auxiliar em velocidade 0,25 durante todo o deslocamento de

retorno, independentemente da posição intermediária dos eixos, nesse trecho, os comandos de X e Z permanecem zerados, já que o deslocamento de retorno não depende de uma nova instrução de posicionamento da rede, sendo a garra liberada apenas ao final do percurso, quando os eixos retornam à posição de origem, conforme observado na ID 8. Já a ID 9, que representa a condição de repouso sem qualquer item presente, apresenta a esteira auxiliar mantida na mesma velocidade de 0,25 mesmo sem material em trânsito, o que sugere que esse valor pode representar antes um patamar mínimo de operação da esteira auxiliar do que uma resposta a um item específico. Os parâmetros utilizados no treinamento desta rede foram 2000 épocas e taxa de aprendizado de 0,2.

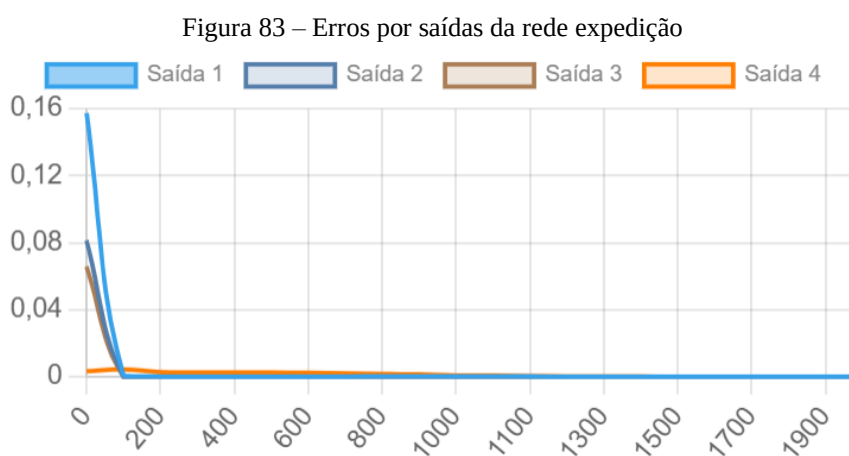
A Figura 82 apresenta a evolução da acurácia geral da rede, calculada de forma agregada sobre as quatro saídas do modelo ao longo das 2000 épocas de treinamento. A acurácia parte de aproximadamente 62% nas primeiras épocas e cresce de forma consistente, ainda que com pequenas oscilações pontuais, passando por cerca de 77% até a época 400, estabilizando-se por volta de 82% entre as épocas 500 e 900, e avançando em seguida para valores próximos de 90% entre as épocas 1000 e 1300. A partir da época 1400 observa-se um novo salto, com a acurácia atingindo cerca de 95%, a partir da época 1600, estabilizando-se em 100% até o final do treinamento, indicando convergência do modelo para a sequência completa de movimentos exigida pelo processo de captura e liberação dos itens.



Fonte: Elaborado pelo autor (2026)

Diferentemente da acurácia, apresentada de forma agregada, a Figura 83 permite observar o comportamento individual do Erro de cada uma das quatro saídas da rede. As saídas 1, 2 e 3, acionamento da garra e comandos dos eixos X e Z, respetivamente partem de valores relativamente elevados nas primeiras épocas, entre aproximadamente 0,065 e 0,16, e decrescem de forma acentuada já nas primeiras 100 a 150 épocas, aproximando-se de zero e permanecendo

estáveis nesse patamar durante todo o restante do treinamento. Já a saída 4, referente à velocidade da esteira auxiliar, parte de um valor consideravelmente menor, próximo de 0,01 a 0,02, e apresenta decaimento mais gradual, levando cerca de 600 a 800 épocas para se estabilizar próxima de zero. Essa diferença de comportamento é coerente com a natureza das saídas envolvidas, no qual as três primeiras saídas alternam entre valores próximos dos extremos 0 e 1 ao longo do conjunto de dados, gerando erros iniciais mais altos, mas de correção mais rápida por parte do algoritmo, enquanto a saída de velocidade da esteira assume valores intermediários e constantes em grande parte das amostras, exigindo um ajuste mais fino e, por isso, um número maior de épocas para convergir.



Fonte: Elaborado pelo autor (2026)

A convergência de todas as quatro saídas para valores próximos de zero antes do final do treinamento, associada à estabilização da acurácia geral em torno de 100%, indica que o modelo foi capaz de reproduzir corretamente a sequência de movimentos esperada para a captura e a liberação dos itens, sendo essa mesma versão de pesos igualmente eficaz quando aplicada tanto à instância da linha A quanto à instância da linha B.

4.1.5. Treinamento da rede despacho

Para a rede despacho, a composição dos dados de treinamento buscou cobrir os cenários em que apenas uma das linhas apresenta produção pendente, devendo apenas a esteira correspondente ser acionada, os cenários em que ambas as linhas estão simultaneamente ativas, exigindo o acionamento concomitante das duas esteiras de saída e os cenários em que nenhuma das duas linhas possui produção em andamento, situação em que ambas as esteiras devem permanecer desativadas. Foram também incluídos exemplos contemplando diferentes estágios

do indicador de tempo de permanência, desde valores baixos, associados ao início recente da produção em uma linha, até valores negativos mais acentuados, associados a um período prolongado sem novas conclusões de produção, de modo que a rede aprendesse a associar corretamente o avanço desse contador ao momento adequado de acionamento de cada esteira, e não apenas à simples presença de produção ativa na linha correspondente.

Por se tratar da rede com a menor dimensionalidade de entrada e saída entre as cinco que compõem o sistema, apenas duas entradas e duas saídas, ambas digitais, conforme já discutido na seção 3.3.4.5, o conjunto de dados necessário para representar essa lógica de decisão foi correspondentemente reduzido, exigindo poucas combinações para cobrir integralmente o espaço de decisão do problema.

Figura 84 – Conjunto de dados de treinamento da rede despacho

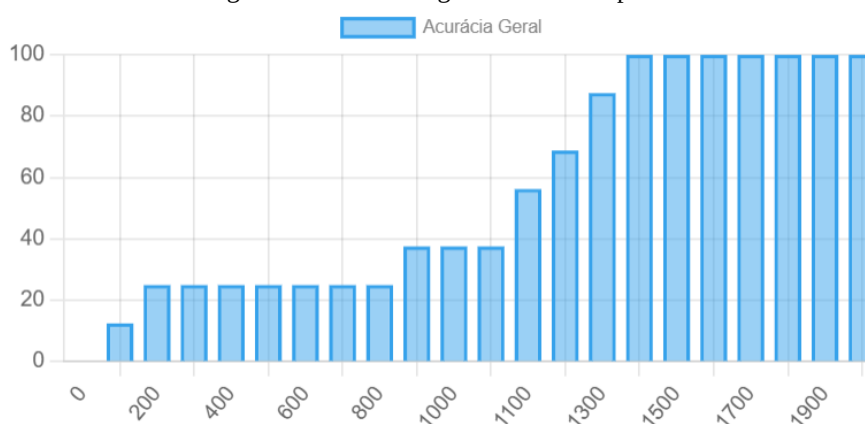
ID	Entradas	Saídas
1	0,0,1	1,0
2	0,1,0	0,1
3	0,1,0,1	0,0
4	0,0	1,1
5	-0,9,0	1,1
6	0,-0,9	1,1
7	0,1,-0,9	0,1
8	-0,9,0,1	1,0

Fonte: Elaborado pelo autor (2026)

Observa-se que o critério de decisão adotado no conjunto de dados é baseado em um limiar simples, que valores de entrada iguais ou inferiores a zero indicam que não há mais itens pendentes na linha, resultando no acionamento da respectiva esteira, como nas IDs 1, 2, 4, 5, 6 e 8, ao passo que valores positivos, indicando itens ainda faltantes na linha, mantêm a esteira correspondente desativada, como observado na linha B em ID 1, na linha A da ID 2, e em ambas as linhas da ID 3 e da ID 7. As IDs 5, 6 e 8, em particular, evidenciam que o critério responde de forma independente para cada linha, ativando isoladamente a esteira de A ou de B mesmo quando a outra linha ainda apresenta itens pendentes, o que confirma que a rede aprende a tratar as duas decisões de despacho de forma desacoplada, sem exigir que ambas as linhas estejam prontas simultaneamente para liberar qualquer uma delas.

Os hiperparâmetros utilizados no treinamento desta rede foram aproximadamente 2000 épocas e taxa de aprendizado de 0,2. Observa-se uma convergência consideravelmente mais lenta do que a verificada nas demais redes do sistema, a acurácia parte de apenas 15% nas primeiras épocas, sobe para cerca de 25% entre as épocas 200 e 800, permanecendo estagnada nesse patamar por um intervalo prolongado, e avança para aproximadamente 37% entre as épocas 900 e 1000. A partir da época 1100 a acurácia volta a crescer de forma mais acentuada, atingindo cerca de 55% na época 1100, 70% na época 1200 e 87% na época 1300, até finalmente convergir para 100% a partir da época 1400, patamar em que se mantém estável até o final do treinamento. Esse longo período de estagnação entre as épocas 200 e 900 é um comportamento atípico frente às demais redes do sistema e pode estar relacionado à baixa dimensionalidade do problema combinada a uma superfície de erro com regiões de gradiente pequeno perto do limiar de decisão em zero, exigindo um número maior de épocas para que os pesos escapem dessa região antes de convergir para a solução final.

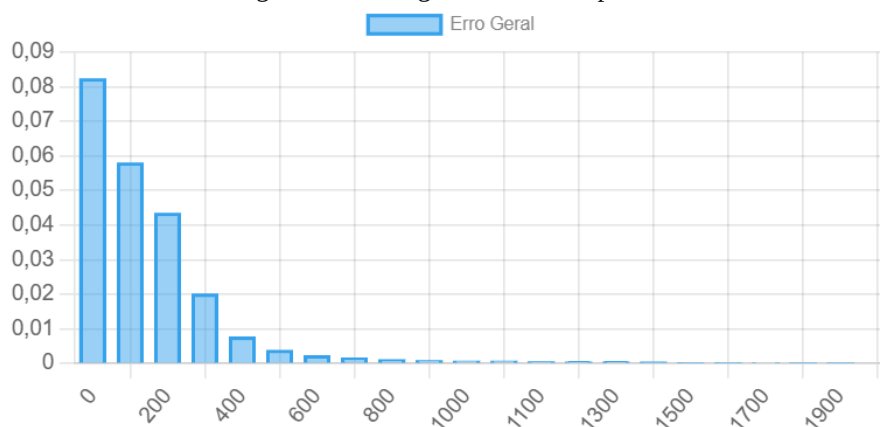
Figura 85 – Acurácia geral da rede despacho



Fonte: Elaborado pelo autor (2026)

O erro parte de aproximadamente 0,082 no instante inicial e decresce de forma consistente ao longo das primeiras 800 épocas, passando por 0,058 na época 100, oscilando para 0,043 na época 200 e caindo abaixo de 0,01 já na época 400, aproximando-se de zero por volta da época 900 e permanecendo estável nesse patamar até o final do treinamento. Nota-se que o erro geral (Figura 86) já se encontrava próximo de zero bem antes da estabilização da acurácia em 100%, o que é coerente com o longo platô observado entre as épocas 200 e 900 na Figura 85.

Figura 86 – Erro geral da rede despacho



Fonte: Elaborado pelo autor (2026)

Mesmo com a maior parte dos exemplos já sendo corretamente aproximada em magnitude pela rede, uma parcela das amostras ainda permanecia do lado incorreto do limiar de normalização, contribuindo pouco para o erro, mas o suficiente para impedir a contagem de acerto exigida pelo critério de acurácia.

4.1.6. Erro de treinamento

A fim de sintetizar o desempenho de cada uma das cinco redes neurais que compõem o sistema, a Tabela 21 apresenta o número de épocas e a taxa de aprendizado efetivamente utilizados no treinamento de cada rede, juntamente com o Erro Quadrático Médio (MSE) geral obtido ao final desse processo.

Tabela 21 – Erro consolidado das RNAs

RNA	Épocas	LR	MSE Geral
Prioridade	1000	0,5	0.00023314885
Esteira	4000	0,5	0.00000625416
Expedição	2000	0,2	0.00001116877
Elevador	6000	0,5	0.00005611177
Despacho	2000	0,2	0.00026126058

Fonte: Elaborado pelo autor (2026)

Observa-se que a rede Esteira apresentou o menor MSE geral entre as cinco redes, seguida pela rede Expedição e pela rede Elevador. Prioridade e Despacho, por sua vez, apresentaram os maiores valores de erro geral, com Despacho registrando o MSE mais elevado do conjunto. Entre as três redes treinadas com a mesma taxa de aprendizado de 0,5 (Prioridade, Esteira e Elevador) não se observa uma relação direta entre o número de épocas e o erro obtido.

A rede Elevador, treinada pelo maior número de épocas entre todas (6.000), apresentou erro superior ao da rede Esteira, treinada por apenas 4.000 épocas, o que indica que a redução do erro está associada não apenas ao número de épocas de treinamento, mas também à complexidade de decisão específica de cada rede.

Os valores de MSE obtidos são compatíveis com resultados relatados na literatura para sistemas físicos reais controlados por redes neurais treinadas por retropropagação, nos quais os desvios entre a saída do modelo e a saída do sistema mostraram-se pequenos o suficiente para serem desconsiderados frente aos erros inerentes a ruídos de medição (RITA, 1995).

4.2. Validação da planta industrial

Após a validação individual de cada uma das cinco redes neurais na etapa de treinamento, descrita na seção 4.1, esta seção apresenta o funcionamento do sistema em operação integrada, demonstrando a coordenação entre os módulos de controle, comunicação e supervisão ao longo do ciclo completo de atendimento de pedidos, desde sua inserção até a efetiva liberação dos lotes na área de despacho. O objetivo desta análise é evidenciar que as redes treinadas de forma isolada, cada uma sobre seu próprio conjunto de dados supervisionado, são capazes de cooperar de forma coerente quando executadas simultaneamente sobre a planta simulada, sem que a especialização de cada módulo comprometa a consistência do comportamento do sistema como um todo.

4.2.1. Cenário de teste

Para essa análise, foram inseridos simultaneamente dois pedidos de produção idênticos, de identificadores 432 e 433, por meio da página de Dashboard, cada um solicitando uma unidade de cada um dos três tipos de item (X, Y e Z), conforme o mecanismo descrito na seção 3.5.1.2. A escolha por dois pedidos com composição idêntica teve como objetivo específico observar o comportamento da rede de Prioridade diante de uma situação de concorrência direta entre as duas linhas secundárias, avaliando se o critério de tempo relativo entre encaminhamentos seria capaz de distribuir os itens de forma equilibrada entre os pedidos A e B, evitando que um deles avançasse desproporcionalmente às custas do outro.

O acompanhamento do ciclo de vida dos dois pedidos foi realizado por meio da tabela de pedidos disponível na interface web, que exhibe, para cada pedido ativo, a quantidade

produzida de cada item em relação à quantidade solicitada, além do status corrente e do horário da última atualização.

Imediatamente após a inserção, os pedidos de ID 432 e 433 foram registrados com status "Pendente" às 09:53:00, ambos com contadores de produção zerados (0/1) para os três tipos de item, conforme a Figura 87. Esse estado inicial confirma que o sistema aguardava o início do ciclo de varredura da planta antes de iniciar o direcionamento dos itens, sem qualquer processamento prematuro dos pedidos recém-inseridos.

Figura 87 – Inserção de pedidos Pendentes

ID	Item X	Item Y	Item Z	Status	Hora	Ação
432	0/1	0/1	0/1	Pendente	2026-09-03 09:53:00	Deletar
433	0/1	0/1	0/1	Pendente	2026-09-03 09:53:00	Deletar

Fonte: Elaborado pelo autor (2026)

O ciclo completo de atendimento dos dois pedidos, desde a ativação do botão "Ligar" na página Dashboard até a finalização de ambas as linhas, teve duração total de aproximadamente 1 minuto e 56 segundos, cronometrado durante a execução do processo.

4.2.2. Acompanhamento do status dos pedidos

Aproximadamente 57 segundos após a inserção dos pedidos, os seus respectivos status foram atualizados para refletir a linha secundária em que cada um passou a ser fabricado. Como mostra a Figura 88, o pedido 432 passou a ser fabricado na linha A e o pedido 433 na linha B, evidenciando que a camada web distribuiu os dois pedidos concorrentes para linhas distintas, evitando a concentração de produção em uma única linha secundária.

Figura 88 – Pedidos em fabricação

ID	Item X	Item Y	Item Z	Status	Hora	Ação
432	0/1	0/1	0/1	Fabricando em A	2026-09-03 09:53:57	Deletar
433	0/1	0/1	0/1	Fabricando em B	2026-09-03 09:53:57	Deletar

Fonte: Elaborado pelo autor (2026)

Conforme ilustrado na Figura 89, ao término do atendimento de cada pedido, com a igualdade entre os vetores de quantidade solicitada e quantidade produzida, o status foi automaticamente atualizado para "Finalizado", seguindo a lógica descrita na seção 3.5.3.1.

Figura 89 – Pedidos finalizados

ID	Item X	Item Y	Item Z	Status	Hora	Ação
432	1/1	1/1	1/1	Finalizado em A	2026-09-03 09:56:02	Deletar
433	1/1	1/1	1/1	Finalizado em B	2026-09-03 09:55:46	Deletar

Fonte: Elaborado pelo autor (2026)

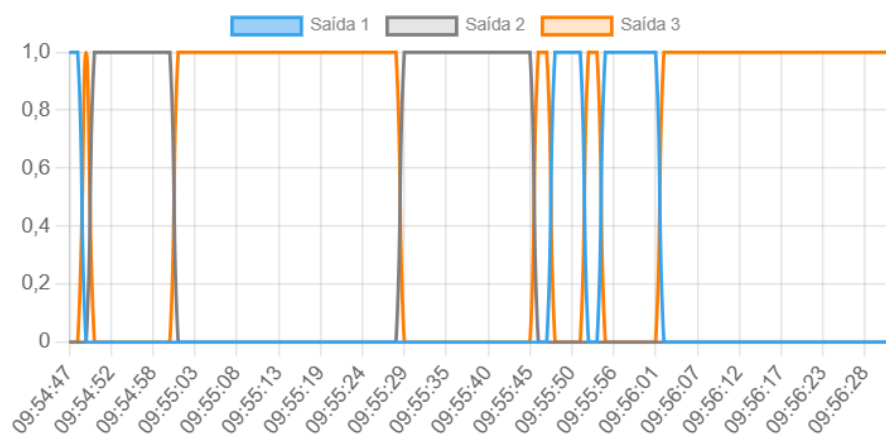
A Figura 89 confirma a conclusão de ambos os pedidos, com os contadores de produção atingindo 1/1 para os três tipos de item em cada um deles. Observa-se que o pedido 433 (linha B) foi finalizado às 09:55:46, aproximadamente dezesseis segundos antes da finalização do pedido 432 (linha A), às 09:56:02, diferença de tempo compatível com a variação natural no processo de distribuição e atendimento da demanda entre as duas linhas secundárias.

4.2.3. Comportamento das redes de controle

Além do acompanhamento pela tabela de pedidos, a interface web disponibiliza, para cada uma das cinco redes do sistema, um gráfico de linha em tempo real com os valores de saída de cada instância durante a execução, permitindo correlacionar as decisões tomadas por cada rede com os eventos observados na tabela de pedidos.

A Figura 90 apresenta as três saídas da rede de Prioridade (destino A, destino B e destino Elevador) ao longo de aproximadamente cem segundos de execução. O gráfico evidencia o padrão de alternância exclusiva entre as três saídas, cada uma assumindo o valor 1 de forma isolada a cada novo item detectado na esteira principal, confirmando o comportamento one-hot esperado da camada Softmax discutido na seção 2.6.4.3. Observa-se uma concentração de comutações rápidas entre aproximadamente 09:55:45 e 09:56:01, intervalo em que a demanda pendente de cada linha secundária se aproxima do esgotamento item a item, com o destino variando conforme qual tipo de item ainda resta em cada pedido. A partir de aproximadamente 09:56:01, a saída correspondente ao Elevador estabiliza-se em 1 de forma contínua até o final do intervalo registrado, sem qualquer nova alternância, comportamento coerente com o fato de que, a partir desse instante, a demanda de ambos os pedidos já havia sido integralmente atendida (conforme os horários de finalização registrados na Figura 89), de modo que todo item subsequente passa a ser corretamente direcionado ao armazenamento, por não haver mais linha secundária com demanda pendente.

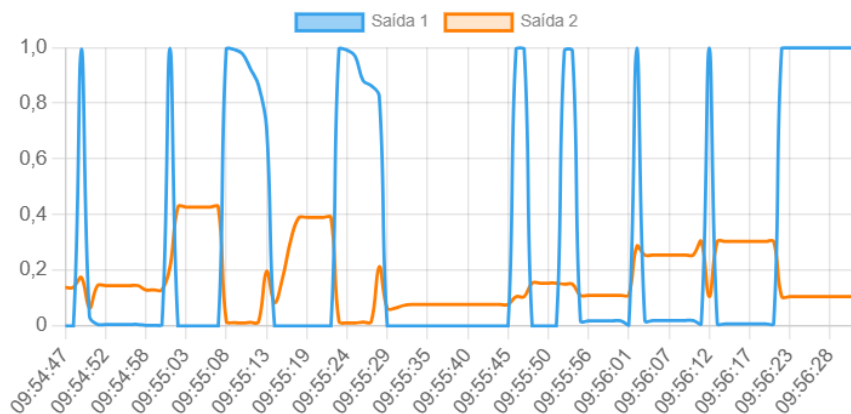
Figura 90 – Saídas da rede prioridade



Fonte: Elaborado pelo autor (2026)

Na Figura 91, a saída 1 (liberação do repositor) apresenta uma sucessão de pulsos que sobem rapidamente a 1 e retornam a 0, cada um correspondendo à liberação de um novo item na esteira principal, enquanto a saída 2 (velocidade) oscila predominantemente entre aproximadamente 0,05 e 0,4 nos intervalos entre pulsos, refletindo a redução de velocidade sempre que o destino selecionado pela rede de Prioridade ainda se encontra ocupado, conforme o comportamento já discutido na seção 4.1.3. Destaca-se, entretanto, um intervalo entre aproximadamente 09:55:03 e 09:55:29 em que a saída 2 se eleva a valores próximos de 1,0 antes de decair gradualmente, padrão distinto da oscilação mais contida observada no restante do gráfico e que sugere uma condição momentânea de menor restrição de destino nesse trecho da execução.

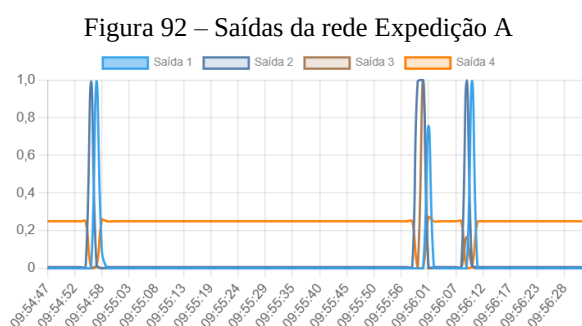
Figura 91 – Saídas da rede esteira



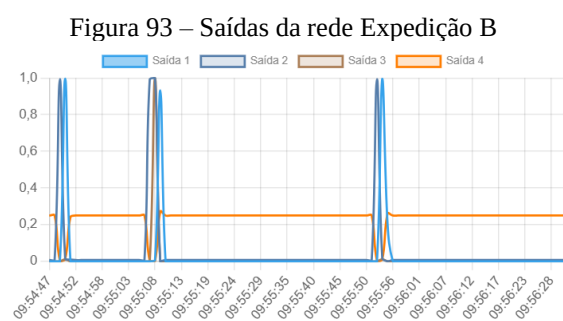
Fonte: Elaborado pelo autor (2026)

As Figuras 92 e 93 apresentam, respectivamente, o comportamento das instâncias de Expedição A e Expedição B durante o mesmo intervalo de execução. Em ambos os gráficos,

a saída 4 (velocidade da esteira auxiliar) permanece constante em aproximadamente 0,25 durante praticamente todo o período, sendo interrompida apenas nos curtos intervalos em que as saídas 1, 2 e 3 disparam simultaneamente para valores próximos de 1, representando o acionamento da garra e o deslocamento dos eixos durante a captura de um item, comportamento consistente com a sequência de movimentos descrita na seção 4.1.4. A linha B registra três eventos de captura ao longo da execução, por volta de 09:54:50, 09:55:08 e 09:55:53, enquanto a linha A registra igualmente três eventos, por volta de 09:54:55, 09:56:03 e 09:56:12. Observa-se que o último evento de captura de cada linha coincide com os respectivos horários de finalização dos pedidos registrados na Figura 89, o da linha B próximo à finalização do pedido 433 às 09:55:46, e o da linha A próximo à finalização do pedido 432 às 09:56:02, confirmando que a última captura de cada linha corresponde exatamente ao item que completa o respectivo pedido.



Fonte: Elaborado pelo autor (2026)



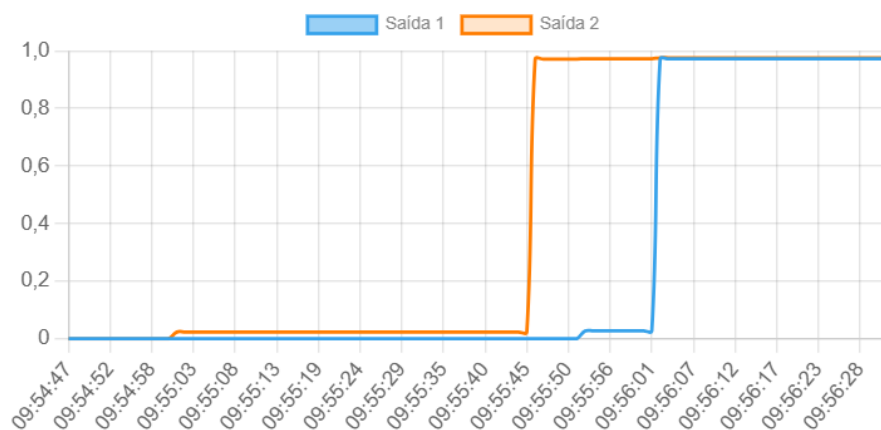
Fonte: Elaborado pelo autor (2026)

Complementarmente, observa-se que, apesar de as instâncias A e B serem executadas em dispositivos distintos, conforme descrito na seção 3.4.1, ambas apresentaram comportamento equivalente ao longo da execução, com o mesmo padrão de pulsos nas saídas 1, 2 e 3 e a mesma constância na saída 4 entre os eventos de captura. Essa equivalência reforça que a divisão da rede Expedição entre os dois ambientes de execução não compromete a consistência do comportamento da rede, validando a viabilidade de reproduzir a mesma lógica de controle em plataformas computacionais com capacidades de processamento distintas.

A Figura 94 confirma a correlação entre o comportamento da rede de Despacho e os horários de finalização registrados na tabela de pedidos. A saída 2, correspondente à esteira de saída da linha B, sobe para aproximadamente 0,98 por volta de 09:55:45, coincidindo com a finalização do pedido 433, enquanto a saída 1, correspondente à linha A, permanece próxima de 0 até por volta de 09:56:01, quando sobe para o mesmo patamar, coincidindo com a finalização do pedido 432 às 09:56:02. Essa correspondência praticamente exata entre os dois

horários confirma que a rede de Despacho respondeu corretamente à conclusão de cada pedido de forma independente, sem aguardar a finalização simultânea de ambas as linhas para liberar a primeira delas.

Figura 94 – Saídas da rede despacho

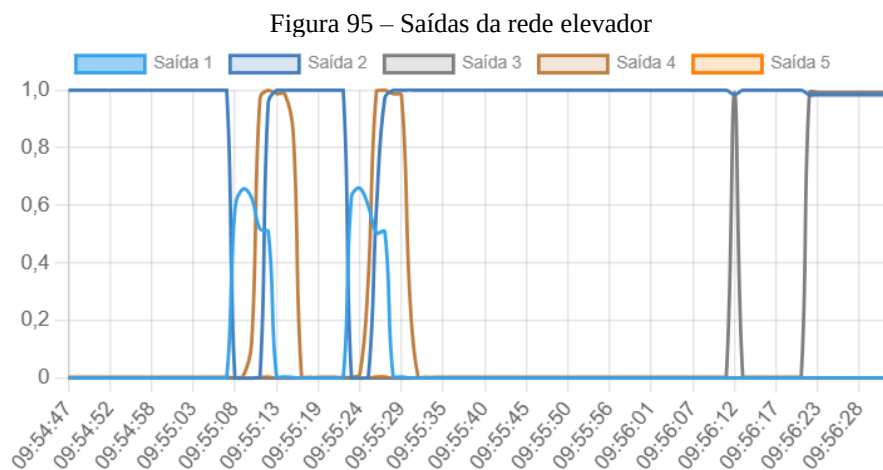


Fonte: Elaborado pelo autor (2026)

A Figura 95 apresenta o comportamento das cinco saídas da rede Elevador durante o mesmo intervalo de execução, no qual foram processados dois itens do tipo X e dois itens do tipo Y. A saída 2 (esteira do elevador) permanece ativa durante praticamente todo o período monitorado, auxiliando a remoção dos itens armazenados, sendo interrompida apenas nos breves instantes em que o elevador está subindo entre níveis. A saída 1 (setpoint) nunca ultrapassa aproximadamente 0,6 ao longo de todo o intervalo, uma vez que, não havendo item do tipo Z a ser armazenado ou removido nesse trecho, o elevador não precisa se deslocar até o nível mais alto ($\sim 0,96$), permanecendo sua movimentação restrita entre o nível de entrada (0) e o nível intermediário ($\sim 0,5$) correspondente ao item Y.

Os dois primeiros eventos de atividade, entre aproximadamente 09:55:08 e 09:55:19, e entre 09:55:24 e 09:55:31, correspondem à remoção dos dois itens do tipo Y: nesses instantes, a saída 1 eleva-se até aproximadamente 0,5, posicionando o elevador no nível correspondente, enquanto a saída 4 (esteira de remoção do item Y) é acionada, auxiliada pela saída 2 (esteira do elevador), que se mantém ativa durante a remoção. Já os dois últimos eventos, por volta de 09:56:12 e 09:56:23, correspondem à remoção dos dois itens do tipo X, como esse item é armazenado no mesmo nível em que entra no elevador (nível 0), nenhuma elevação do setpoint é necessária, e a saída 3 (esteira de remoção do item X) é acionada diretamente, coincidindo com a breve interrupção da saída 2, que se desliga apenas nesse instante para permitir a remoção sem interferência da esteira de entrada.

Ao longo de todo o intervalo, a saída 5 (esteira de remoção do item Z) permanece em 0, uma vez que nenhum item desse tipo foi processado nesse trecho da execução, confirmando que essa saída só é acionada quando há de fato um item Z armazenado a ser removido no nível mais alto do elevador.



Fonte: Elaborado pelo autor (2026)

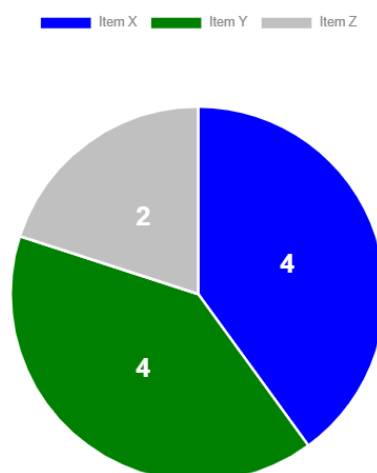
Dessa forma, a ordem cronológica de armazenamento observada no intervalo analisado evidência que o elevador de armazenamento atende, de forma intercalada, tanto os itens excedentes processados pelas linhas de expedição quanto os itens diretamente destinados ao armazenamento pela decisão inicial da rede de Prioridade.

4.2.4. Distribuição dos itens produzidos

Complementando o acompanhamento do sistema, a interface web disponibiliza um conjunto de indicadores agregados ao final da execução, permitindo avaliar a distribuição total dos itens processados entre os diferentes destinos da planta durante o período analisado.

Conforme a Figura 96, foram produzidos ao todo dez itens durante a execução do teste, sendo quatro do tipo X, quatro do tipo Y e dois do tipo Z. Como os dois pedidos juntos demandavam apenas duas unidades de cada tipo de item (uma para o pedido A e uma para o pedido B), a diferença entre o total produzido e o total efetivamente consumido pelos pedidos evidencia que itens adicionais, não requisitados no momento de sua detecção, continuaram a ser gerados pela planta simulada e precisaram ser corretamente direcionados pela rede Prioridade.

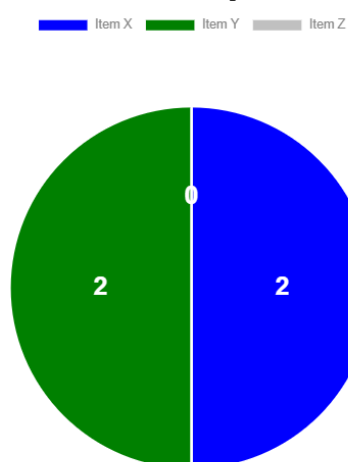
Figura 96 – Distribuição dos itens produzidos



Fonte: Elaborado pelo autor (2026)

A Figura 97 mostra que exatamente quatro itens, sendo dois do tipo X e dois do tipo Y, foram direcionados ao elevador de armazenamento por não serem, no instante de sua detecção, demandados por nenhum dos dois pedidos ativos, validando na prática o comportamento de armazenamento de excedentes já discutido na seção 3.2.3.

Figura 97 – Itens enviados para armazenamento

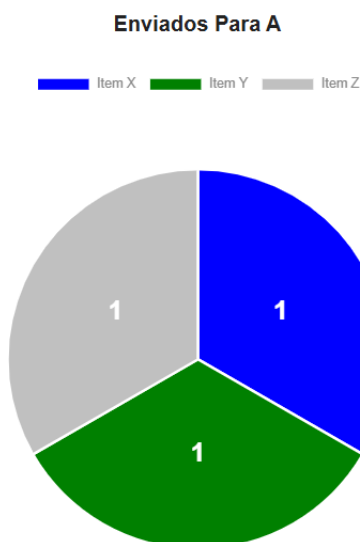


Fonte: Elaborado pelo autor (2026)

Nota-se que nenhum item do tipo Z foi enviado ao elevador, o que é consistente com o fato desse tipo ter sido produzido em menor quantidade ao longo do teste, com a totalidade de suas duas unidades geradas sendo consumida diretamente pelos pedidos A e B.

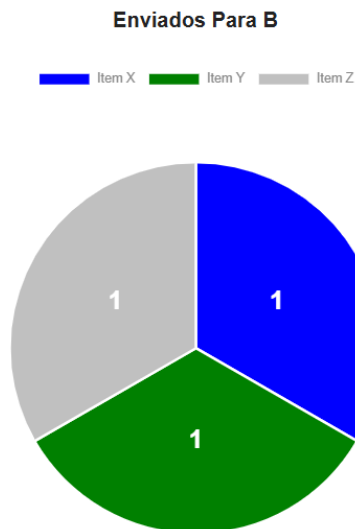
Por fim, as Figuras 98 e 99 confirmam que cada um dos dois pedidos recebeu exatamente uma unidade de cada tipo de item, uma unidade de X, uma de Y e uma de Z, em perfeita conformidade com a quantidade originalmente solicitada por ambos.

Figura 98 – Itens enviados para esteira A



Fonte: Elaborado pelo autor (2026)

Figura 99 – Itens enviados para esteira B



Fonte: Elaborado pelo autor (2026)

Esse resultado demonstra que, mesmo diante da concorrência direta entre dois pedidos idênticos processados simultaneamente, a rede de Prioridade foi capaz de distribuir corretamente cada item entre as duas linhas secundárias, sem excesso, sem falta e sem favorecer sistematicamente uma linha em detrimento da outra, validando de forma consolidada a operação integrada das seis instâncias que compõem o sistema de controle da planta simulada.

4.3. Desempenho comunicação modbus TCP/IP

Esta seção apresenta a análise do desempenho da camada de comunicação de campo, responsável por interligar o dispositivo de controle à planta industrial simulada por meio do protocolo Modbus TCP/IP. O objetivo é verificar se essa camada de comunicação atende a continuidade operacional mínima de um sistema de controle industrial, avaliando o tempo de execução do ciclo de varredura, o tempo de escrita dos comandos Modbus por rede e o comportamento comparado entre a comunicação cabeada e a comunicação sem fio, executada de forma embarcada no microcontrolador ESP32.

4.3.1. Ciclo de varredura

O ciclo de leitura e escrita de registradores foi cronometrado ao longo de sucessivas iterações do laço principal de controle, permitindo estimar o tempo necessário para a execução completa de um ciclo de varredura que compreende a leitura das entradas digitais e analógicas, o processamento das redes neurais executadas em cada dispositivo e a escrita dos comandos correspondentes nos registradores de saída. Na amostragem de dados, foram coletadas 200 amostras para cada um dos dois meios de comunicação, intercaladas por um atraso (delay) configurado entre cada varredura, a fim de evitar sobrecarga de processamento entre requisições sucessivas. Esse atraso foi deliberadamente configurado de forma distinta para cada dispositivo, no qual essa diferença tem como objetivo compensar, ainda que parcialmente, a maior demanda de tempo já inerente à comunicação Wi-Fi, sujeita a etapas adicionais de negociação do protocolo IEEE 802.11, além da menor capacidade de processamento do microcontrolador, concedendo ao dispositivo embarcado um intervalo artificial de espera menor entre ciclos, de modo a não penalizar ainda mais sua capacidade de resposta com um atraso adicional desnecessariamente alto, o que resultaria em um ciclo de varredura ainda mais lento do que o já imposto pelas limitações físicas do próprio meio sem fio. Desta forma, os dados coletados são exibidos na tabela a seguir:

Tabela 22 – Ciclo de varredura

Fonte	Tempo (ms)		
	Maior	Média	Delay
Cabeada	1516.306	120.185	20
Wi-fi	1476.000	775.395	10

Fonte: Elaborado pelo autor (2026)

Como mostra a Tabela 22, mesmo com um atraso configurado propositalmente menor (10 ms contra 20 ms da comunicação cabeada), o tempo médio de ciclo da comunicação Wi-Fi (775,395 ms) permaneceu consideravelmente mais alto do que o da comunicação cabeada (120,185 ms), aproximadamente 6,45 vezes superior, confirmando que a diferença de desempenho entre os dois meios decorre da própria natureza da comunicação sem fio e da limitação de processamento do microcontrolador, e não do atraso artificial configurado entre ciclos, uma vez que reduzir esse atraso não foi suficiente para aproximar os tempos médio.

Esse comportamento é qualitativamente consistente com estudos que comparam arquiteturas de processamento de redes neurais em automação industrial, nos quais

configurações com processamento embarcado diretamente no controlador apresentaram tempos de ciclo sensivelmente superiores aos de configurações com processamento externo ao controlador, o que reforça que o meio e o local de processamento, e não apenas a complexidade do modelo, são fatores determinantes no tempo de resposta do sistema (SILVA, 2024).

É importante notar, ainda, que o laço executado na infraestrutura cabeada processa cinco redes neurais em cada ciclo (Prioridade, Esteira, Expedição A, Despacho e Elevador), enquanto o laço executado no ESP32 processa apenas a rede de Expedição B, ainda assim, ao concentrar uma única rede por ciclo, o dispositivo embarcado apresentou tempo médio de ciclo muito superior ao do dispositivo cabeado, que processa cinco redes no mesmo intervalo.

Por outro lado, ao comparar os valores máximos registrados em cada meio, observa-se que o maior tempo de ciclo da comunicação cabeada (1.516,306 ms) foi ligeiramente superior ao maior tempo registrado na comunicação Wi-Fi (1.476 ms), uma diferença pequena em termos absolutos, mas que revela um padrão de comportamento distinto entre os dois meios. Enquanto a comunicação cabeada opera, em média, em um patamar de tempo bastante baixo (120,185 ms), estando sujeita a picos ocasionais de latência que elevam seu valor máximo a mais de dez vezes sua própria média (cerca de 12,6 vezes), prováveis oscilações da rede local ou do sistema operacional, a comunicação Wi-Fi já opera, em média, em um patamar consideravelmente mais alto, e seu valor máximo, embora quase o dobro de sua própria média (cerca de 1,9 vez), representa uma variação relativa bem menor do que a observada na comunicação cabeada. Essa distinção reforça que a variabilidade da comunicação cabeada está associada a eventos esporádicos e pontuais de latência, enquanto a lentidão da comunicação Wi-Fi é um comportamento sistemático e recorrente, inerente ao próprio meio de comunicação e à capacidade de processamento do dispositivo embarcado.

4.3.2. Tempo médio de escrita local

Complementarmente ao tempo de ciclo completo, foi medido o tempo de escrita de cada comando Modbus de forma individual da rede local (cabeada), agregado por rede, permitindo identificar se alguma das cinco redes neurais concentra tempos de resposta desproporcionalmente mais altos que as demais.

O tempo médio de escrita acumulado por rede varia proporcionalmente ao número de saídas que cada uma delas possui, uma vez que esse valor corresponde à soma do tempo médio de escrita de todas as saídas de uma mesma rede em um único ciclo. A rede Esteira, com apenas duas saídas, apresenta o menor tempo médio acumulado (6,808 ms), ao passo que as

redes de Prioridade e Elevador, com cinco saídas cada, apresentam os maiores tempos médios acumulados (19,620 ms e 17,970 ms, respectivamente).

Tabela 23 – Tempo de escrita modbus por RNA

Rede	Saídas	Tempo (ms)	
		Maior	Média
Prioridade	5	93.144	19.620
Esteira	2	46.150	6.808
Expedição	4	52.765	14.812
Despacho	2	96.824	8.332
Elevador	5	62.299	17.970

Fonte: Elaborado pelo autor (2026)

Quando normalizado pelo número de saídas, contudo, o tempo médio de escrita por saída individual mostra-se bastante homogêneo entre as cinco redes, girando em torno de 3,4 a 4,2 ms por saída, o que indica que a diferença observada na Tabela 23 decorre exclusivamente da quantidade de comandos escritos por ciclo, e não de uma diferença real de desempenho entre as redes.

Observam-se, ainda, valores máximos pontuais consideravelmente mais altos que a média em todas as redes, característicos de picos ocasionais de latência da rede local, e não de um comportamento sistemático de qualquer rede específica. A rede Despacho, em particular, destaca-se das demais nesse aspecto, seu valor máximo (96,824 ms) chega a ser aproximadamente 11,6 vezes superior à sua própria média (8,332 ms), razão consideravelmente mais alta do que a observada nas demais redes, cujos valores máximos variam entre 3,5 e 6,8 vezes suas respectivas médias, o que sugere que, apesar de seu baixo tempo médio de escrita, a rede esteve sujeita a um pico de latência pontual particularmente acentuado durante a amostragem.

Adicionalmente, foi medido o tempo total decorrido entre requisições sucessivas ao servidor Modbus, valor que, por construção, deveria corresponder à soma entre o tempo médio de ciclo completo (120,185 ms, já apresentado na Tabela 22) e o atraso de 20 ms configurado deliberadamente entre ciclos, totalizando um valor esperado de 140,185 ms. O tempo médio efetivamente medido entre requisições sucessivas foi de 142,112 ms, apresentando uma diferença residual de aproximadamente 1,9 ms em relação ao valor esperado, atribuível ao overhead inerente à própria medição, como o tempo de chamada da função de atraso e pequenas variações do sistema operacional. Esse resultado é, portanto, consistente com o tempo agregado de leitura, processamento das cinco redes e escrita de todos os comandos de saída em uma única

iteração do laço principal, somado ao intervalo de espera artificial entre ciclos, confirmando que o tempo entre requisições sucessivas reflete corretamente a soma dos dois componentes que o compõem.

4.3.3. Comparação entre comunicação cabeada e wi-fi

É comparado o tempo de escrita Modbus das redes executadas na infraestrutura local, conectadas por rede cabeada, com o tempo de escrita observado na rede de Expedição B, executada de forma embarcada no microcontrolador ESP32 e conectada à planta simulada por meio de comunicação Wi-Fi. Os valores de referência para a comunicação cabeada correspondem à própria instância local da rede de Expedição (Expedição A), já apresentada na Tabela 23, permitindo uma comparação direta entre duas instâncias de uma mesma rede neural executadas em ambientes distintos.

Tabela 24 – Comparação escrita cabeada e wi-fi

Fonte	Tempo (ms)	
	Maior	Média
Cabeada	52.765	14.812
Wi-fi	116.000	17.665

Fonte: Elaborado pelo autor (2026)

Como mostra a Tabela 24, o tempo médio de escrita da rede de Expedição via Wi-Fi (17,665 ms) é apenas cerca de 19% superior ao tempo médio da mesma rede executada de forma cabeada (14,812 ms), diferença consideravelmente menor do que a esperada para uma comunicação sem fio processada por um microcontrolador de capacidade limitada, o que sugere que, para o tempo médio de escrita, o custo adicional do meio Wi-Fi é discreto quando ambas as instâncias operam em condições normais.

A diferença torna-se muito mais expressiva, contudo, ao observar os valores máximos, sendo o maior tempo de escrita registrado via Wi-Fi (116 ms) mais do que o dobro do maior tempo registrado na comunicação cabeada (52,765 ms), resultado coerente com as etapas adicionais de processamento do protocolo IEEE 802.11 e com a maior variabilidade de latência inerente ao meio sem fio em comparação com uma conexão cabeada. Mais relevante ainda, esse valor máximo (116 ms) corresponde a aproximadamente 96,5% do tempo médio de ciclo completo da planta (120,185 ms), ou seja, em seu cenário de pico observado, uma única escrita Modbus da instância embarcada pode consumir isoladamente quase a totalidade do tempo médio disponível para um ciclo de varredura completo, o que representa um ponto de

atenção relevante para a robustez da arquitetura distribuída em cenários de degradação momentânea do enlace Wi-Fi, ainda que tal pico não tenha comprometido a continuidade da operação durante os testes realizados.

De modo geral, os resultados confirmam que é possível estabelecer com sucesso a comunicação Modbus TCP/IP dentro de um modelo de arquitetura distribuída, como o proposto neste trabalho, tanto na infraestrutura local quanto na comunicação sem fio com um dispositivo embarcado, ainda que os picos ocasionais de latência do enlace Wi-Fi mereçam atenção em eventuais aplicações de maior criticidade.

4.4. Indicadores de desempenho

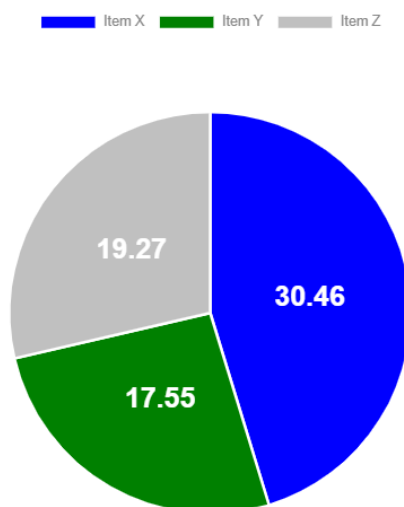
Esta seção apresenta a análise quantitativa do desempenho operacional da planta simulada, com base nos indicadores agregados disponibilizados pela página de Produção da interface web, descrita na seção 3.5.1.1. São analisados dois grupos de indicadores complementares, o primeiro relativo ao tempo médio de atendimento segmentado tanto por tipo de item e o segundo relativo à distribuição dos itens entre os diferentes destinos da planta (elevador de armazenamento, linha A e linha B). Essa abordagem é consistente com o conceito de indicadores de desempenho (Key Performance Indicators – KPIs), amplamente utilizado na gestão e no monitoramento de sistemas produtivos (PINEDO, 2016). Esta seção concentra-se na mensuração numérica agregada do desempenho da planta ao longo da sessão de testes analisada, permitindo avaliar objetivamente a distribuição da produção e a regularidade do atendimento aos pedidos processados pelo sistema.

4.4.1. Tempo médio de atendimento

O tempo médio entre itens de cada tipo corresponde ao intervalo médio decorrido entre a entrada de itens do mesmo tipo no sistema, calculado ao longo de toda a sessão de teste. Como a métrica exige ao menos duas unidades do mesmo tipo para ser calculada, uma vez que corresponde à divisão pelo número de intervalos entre entradas ($N-1$), tipos de item produzidos em menor quantidade resultam em uma média baseada em um número reduzido de intervalos observados, tornando o indicador mais suscetível à variação pontual entre poucas amostras. Além disso, como a geração de itens pela planta ocorre de forma aleatória, o instante exato em que o primeiro e o último item de cada tipo foram inseridos não corresponde necessariamente ao início ou ao fim da sessão de execução, de modo que o intervalo medido reflete a distribuição

observada dentro da janela de amostragem, e não uma taxa de geração fixa e previamente conhecida.

Figura 100 – Tempo médio entre itens no sistema



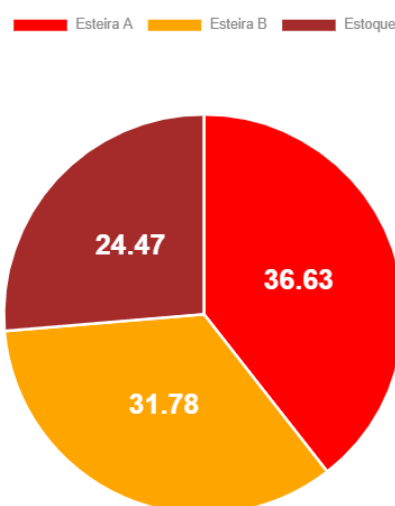
Fonte: Elaborado pelo autor (2026)

Nesse contexto, o item do tipo Y apresentou o menor intervalo médio entre entradas consecutivas, de aproximadamente 17,55 segundos, seguido de perto pelo item do tipo Z, com 19,27 segundos, e pelo item do tipo X, com o maior intervalo médio, de 30,46 segundos. Cabe destacar, que os valores de X e Y foram calculados a partir de quatro unidades produzidas ao longo da sessão, resultando em uma média sobre três intervalos consecutivos observados para cada tipo, ao passo que o valor de Z foi calculado a partir de apenas duas unidades produzidas, correspondendo a um único intervalo observado entre as duas entradas, sem qualquer efeito de média. Por esse motivo, o valor apresentado para o item Z deve ser interpretado com cautela, como uma estimativa pontual sujeita à variação aleatória do instante de geração de cada unidade, e não como uma média estatisticamente consolidada da forma como ocorre para X e Y. Ainda assim, o resultado indica que o item do tipo Y foi gerado com maior frequência ao longo da sessão, enquanto o item do tipo X apresentou os intervalos mais espaçados entre uma entrada e a seguinte no sistema. Como a taxa de geração de cada tipo de item pela planta simulada é determinada externamente ao sistema de controle, essa diferença de intervalos reflete diretamente a própria frequência de ocorrência de cada tipo observada nesta sessão, e não uma diferença de eficiência da rede de Prioridade no roteamento de um tipo específico, uma vez que a métrica aqui apresentada independe de qualquer decisão tomada pelo sistema de controle após a entrada do item.

4.4.2. Distribuição dos itens por destino

Esta seção apresenta o tempo médio de atendimento segmentado por destino, que diferentemente do indicador segmentado por tipo de item, que considera o horário de entrada de cada item no sistema, este é calculado a partir do horário de encaminhamento de cada item ao seu respectivo destino, refletindo a frequência com que a rede de Prioridade direciona itens para cada uma das três rotas possíveis da planta.

Figura 101 – Tempo médio de encaminhamento



Fonte: Elaborado pelo autor (2026)

O destino Estoque apresentou o menor intervalo médio entre encaminhamentos, de aproximadamente 24,47 segundos, seguido pela esteira B, com 31,78 segundos, e pela esteira A, com o maior intervalo médio, de 36,63 segundos. Nesta sessão de testes, os três destinos foram calculados a partir de quantidades distintas de itens encaminhados, quatro itens ao Estoque, resultando em três intervalos observados, e três itens a cada uma das esteiras A e B, resultando em apenas dois intervalos observados para cada uma. Assim como no indicador segmentado por tipo de item, os valores calculados sobre um número reduzido de intervalos, neste caso, os das esteiras A e B, devem ser interpretados como estimativas pontuais, mais suscetíveis à variação aleatória do instante de geração de cada item do que como médias estatisticamente robustas. Ainda assim, o resultado indica que o elevador de armazenamento recebeu itens com maior regularidade ao longo da sessão do que as duas linhas secundárias, comportamento coerente com o fato de que o encaminhamento ao estoque ocorre sempre que nenhuma das duas linhas apresenta demanda pendente no instante de detecção do item,

condição que tende a se tornar mais frequente à medida que os pedidos se aproximam da conclusão.

4.4.3. Taxa de atendimento correto

A taxa de atendimento correto foi mensurada por meio da verificação da correspondência entre o tipo de item efetivamente entregue em cada destino e o tipo de item esperado para aquele destino no momento de sua detecção, permitindo quantificar a confiabilidade da decisão tomada pela rede de Prioridade em condições reais de operação, para além dos indicadores de acurácia já observados na etapa de treinamento apresentada na seção 4.1. Essa distinção é relevante porque a acurácia de treinamento reflete o desempenho da rede sobre um conjunto de dados supervisionado conhecido, ao passo que a taxa de atendimento correto avalia o comportamento da rede diante da variabilidade real de chegada de itens e de estados de pedidos simultâneos, imposta pela própria dinâmica da planta simulada.

Tabela 25 – Taxa de atendimento correto por destino e por item

Destino	Item demandado			Item atendido			Taxa de Acerto
	X	Y	Z	X	Y	Z	
Esteira A	1	1	1	1	1	1	100%
Esteira B	1	1	1	1	1	1	100%
Estoque	2	2	0	2	2	0	100%
Total	4	4	2	4	4	2	100%

Fonte: Elaborado pelo autor (2026)

Conforme a Tabela 25, das dez unidades produzidas ao longo da sessão de teste, as seis unidades demandadas pelos dois pedidos ativos foram corretamente entregues às suas respectivas linhas, uma unidade de cada tipo de item (X, Y e Z) para a esteira A e uma de cada tipo para a esteira B, cada uma correspondendo exatamente ao tipo de item esperado por aquele pedido no momento da detecção. As quatro unidades remanescentes, duas do tipo X e duas do tipo Y, não demandadas por nenhum pedido ativo no instante de sua detecção, foram corretamente identificadas como excedentes e direcionadas ao elevador de armazenamento, em vez de erroneamente entregues a uma das linhas secundárias. Nenhuma unidade do tipo Z foi encaminhada ao estoque, uma vez que a totalidade das duas unidades desse tipo produzidas na sessão foi diretamente consumida pelos dois pedidos. Dessa forma, a taxa de atendimento correto observada nesta sessão de teste foi de 100% em todos os destinos e tipos de item

avaliados, sem nenhum caso de item mal direcionado, entregue em duplicidade ou retido indevidamente em uma linha sem demanda correspondente.

4.5. Gerenciamento e supervisão Web

Esta seção apresenta a análise do funcionamento da camada de supervisão do sistema, estruturada nas três páginas web descritas na seção 3.5.1, avaliando sua capacidade de atender aos requisitos de monitoramento remoto, gerenciamento de pedidos e verificação de consistência entre os modelos de controle treinados e seu comportamento em operação real.

4.5.1. Página Produção

A página de Produção foi avaliada quanto à sua capacidade de consolidar os indicadores agregados de desempenho da planta, permitindo ao operador acompanhar visualmente a distribuição de itens processados por tipo, por rota de encaminhamento e por tempo médio de atendimento, sem a necessidade de interpretar diretamente os dados brutos armazenados na infraestrutura em nuvem. Diferentemente do Dashboard, cuja atualização ocorre a cada 500 milissegundos por concentrar dados de controle em tempo real, a página de Produção opera sob um intervalo de atualização mais espaçado, compatível com a natureza agregada de seus indicadores.

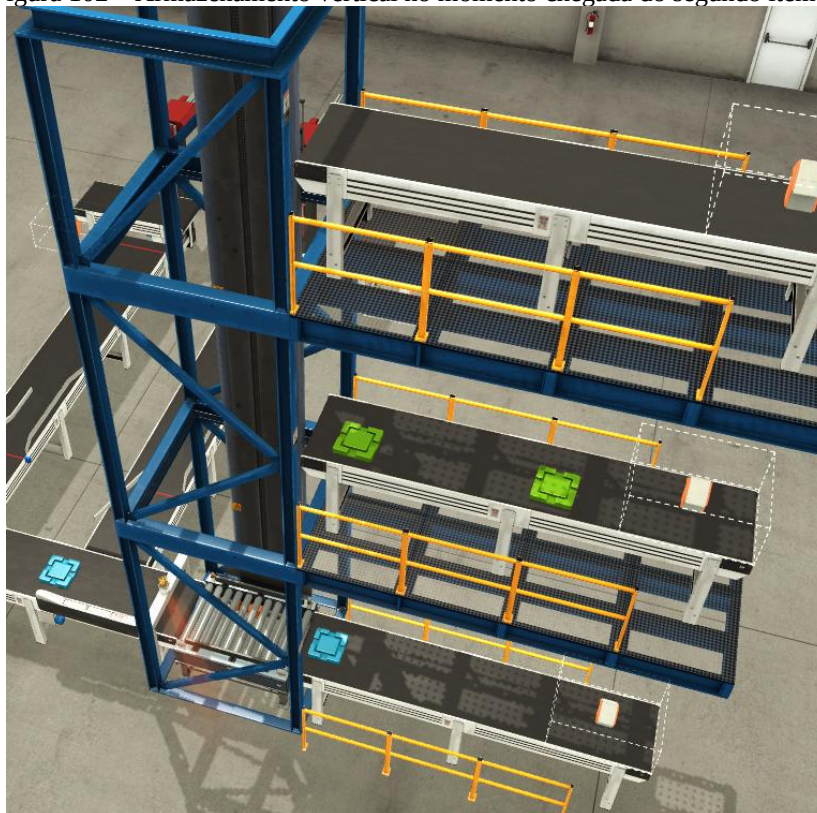
4.5.1.1. Indicadores agregados de produção

Além de exibir os totais consolidados de produção discutidos na seção 4.2, um resultado relevante da página de Produção é a consistência entre o indicador numérico agregado exibido na interface e o estado físico real da planta simulada em um instante específico, validando que a contagem exibida ao operador não é apenas um número calculado de forma independente, mas reflete fielmente a condição real do sistema de armazenamento.

A Figura 102 apresenta o estado físico do sistema de armazenamento vertical durante a sessão de validação dos pedidos 432 e 433, capturado no instante em que o segundo e último item do tipo X (azul) está sendo recebido pelo elevador, evento que corresponde ao último registro de atividade observado na rede Elevador, por volta de 09:56:23, já discutido na Figura 95 da seção 4.2.3. Nesse mesmo instante, observam-se já armazenadas as duas unidades do tipo Y (verde) no nível intermediário de armazenamento (~50%) e uma unidade do tipo X

(azul) já posicionada no nível inferior (nível 0), enquanto o segundo X, capturado em trânsito na imagem, completa o total de quatro itens excedentes gerados na sessão. Nenhuma unidade do tipo Z está presente no nível superior (~96%), consistente com a ausência de itens desse tipo entre os excedentes gerados.

Figura 102 – Armazenamento vertical no momento chegada do segundo item X



Fonte: Elaborado pelo autor (2026)

Comparando esse instante físico com o indicador agregado "Enviados Para Elevador" exibido na página de Produção ao final da mesma sessão, já apresentado na Figura 97 da seção 4.2.4, confirma-se a correspondência exata entre os dois. O indicador registra duas unidades do tipo X e duas do tipo Y, correspondendo precisamente aos quatro itens observados na Figura 102, sendo o segundo X justamente aquele capturado em trânsito, além de nenhuma unidade do tipo Z, resultado consistente com o fato de que a totalidade das duas unidades de Z produzidas na sessão foi consumida diretamente pelos pedidos A e B, sem excedente, conforme já discutido na seção 4.2.4. Esses mesmos valores reaparecem, de forma consolidada, na linha "Estoque" da Tabela 25, na qual as duas unidades de X e as duas de Y encaminhadas ao elevador correspondem exatamente às unidades classificadas como excedentes, com taxa de atendimento de 100% também nesse destino.

4.5.1.2. Geração automática de relatórios

Esta seção apresenta a análise da funcionalidade de geração automática de relatórios analíticos, implementada por meio da integração com a API da OpenAI descrita na seção 3.5.2, avaliando a coerência entre os dados numéricos de produção e o texto produzido pelo modelo de linguagem natural, bem como sua utilidade prática como ferramenta de apoio à interpretação dos indicadores operacionais da planta.

Para essa análise, foi solicitada a geração de um relatório por meio do botão dedicado disponível na página de Produção, a partir dos mesmos indicadores agregados apresentados na seção 4.4, referentes à sessão de dez itens produzidos durante o atendimento dos pedidos 432 e 433. Essa escolha permite comparar diretamente o conteúdo textual produzido pela API com os valores numéricos que o originaram, a partir da instrução do usuário. O algoritmo 8 demonstra a formatação da instrução (Prompt).

Algoritmo 8 – Instrução do usuário enviado para API

```
$userPrompt = "
  Analise os seguintes dados resumidos da planta industrial.
  Interpretação dos vetores:
  - total = quantidade total produzida de itens [X, Y, Z]
  - elev = quantidade de itens [X, Y, Z] enviados para o elevador
  - expA = quantidade de itens [X, Y, Z] enviados para a esteira A
  - expB = quantidade de itens [X, Y, Z] enviados para a esteira B

  Interpretação do vetor tempo:
  tempo = [medio_x, medio_y, medio_z, medio_total, medio_A, medio_B,
  medio_elevador]
  As médias de tempo refere-se ao intervalo médio entre a entrada dos itens:
  (ultimo-primeiro)/(total-1).

  Dados:
  total = [" . implode(',', $total) . "]
  elev = [" . implode(',', $elev) . "]
  expA = [" . implode(',', $expA) . "]
  expB = [" . implode(',', $expB) . "]
  tempo = [" . implode(',', $tempo) . "]

  Gere um relatório operacional resumido.
  Compare produção, distribuição e tempos.
  Faça análise de aproveitamento percentual de cada destinação.
  Aponte qual item aparenta maior participação no processo,
  qual destino aparenta maior uso e se os tempos apresentam bom atendimento
  para A e B.
  Analise se muitos itens estão sendo direcionados para o elevador em vez
  das esteiras.
  Analise se os itens inseridos estão sendo direcionados corretamente.
  Retorne somente texto corrido, sem tópicos ou listas.";
```

Fonte: Elaborado pelo autor (2026)

A instrução de sistema enviada ao modelo (Algoritmo 5, seção 3.5.2), combinada com a instrução de usuário, esta última configurada diretamente no sistema como exemplo padrão, podendo ser livremente ajustada conforme a necessidade de análise, define explicitamente o significado de cada vetor numérico fornecido, além de orientar o modelo a comparar produção, distribuição e tempos, apontar o item de maior participação, avaliar qual destino apresenta melhor aproveitamento e identificar se há um volume excessivo de itens sendo direcionado ao elevador em vez das esteiras.

A instrução fornece ao modelo unicamente os vetores numéricos agregados, sem qualquer informação contextual adicional sobre a lógica de negócio por trás da alocação de itens excedentes ao elevador, aspecto que se mostrará relevante na análise qualitativa do texto gerado, apresentada a seguir. A Figura 103 é resultado da resposta da API para o conjunto de dados da simulação.

Figura 103 – Relatório gerado por IA generativa

Gerar Relatório de IA

A análise dos dados resumidos da planta industrial revela informações importantes sobre a produção e a distribuição dos itens X, Y e Z, além dos tempos médios observados entre as entradas. A produção total dos itens foi de 10 unidades, sendo 4 de X, 4 de Y e 2 de Z. A distribuição para os destinos mostra que 2 unidades de cada um dos itens X e Y foram enviadas para o elevador, enquanto as esteiras A e B receberam 1 unidade de cada item, totalizando 3 unidades para cada esteira. Em termos de aproveitamento percentual, a esteira A e a esteira B apresentaram um uso equivalente e balanceado, com 30% da produção total direcionada a cada uma delas. O elevador, por sua vez, recebeu 40% da produção total, o que sugere uma quantidade significativa de itens armazenados em vez de serem utilizados nas esteiras. Essa distribuição indica um possível desequilíbrio, uma vez que os itens X e Y foram mais direcionados ao elevador, enquanto Z não foi direcionado para o elevador, mas também não foi solicitado pelas esteiras. Analisando os tempos médios, observamos que a média para os itens que foram para a esteira A é de 36,63 segundos e para a esteira B é de 31,78 segundos, enquanto o tempo médio geral é de 12,91 segundos. Esses tempos são relativamente altos, principalmente quando comparados ao tempo médio do elevador, que é de 24,47 segundos. Isso pode indicar que as esteiras A e B estão apresentando uma eficiência inferior no atendimento das solicitações, o que pode ser um indicativo de gargalos ou necessidade de ajustes operacionais. Considerando a participação dos itens, tanto X quanto Y têm uma presença significativa no processo, com 40% da produção total e uma distribuição equilibrada entre os destinos. No entanto, a quantidade elevada de itens direcionados ao elevador sugere que pode haver uma necessidade de reavaliação dos critérios de alocação dos produtos, para garantir que os itens que realmente são necessários para atender aos pedidos sejam priorizados nas esteiras. Assim, a planta deve considerar estratégias para melhorar a distribuição e o fluxo de trabalho, garantindo um melhor aproveitamento das esteiras e reduzindo a quantidade de itens armazenados no elevador.

A análise do relatório gerado revela que o modelo reproduziu com exatidão a totalidade dos dados numéricos fornecidos, a produção total de dez unidades, a distribuição de quatro itens ao elevador, três à esteira A e três à esteira B, os percentuais de aproveitamento de cada destino (30% para cada esteira, 40% para o elevador) e os sete valores do vetor de tempos médios. Entre esses, o relatório cita um tempo médio geral de 12,91 segundos, calculada pela mesma equação (23) já definida anteriormente, mas aplicada ao conjunto de todas as entradas de itens no sistema, sem distinção por tipo, refletindo o intervalo médio entre a chegada de qualquer item consecutivo à planta.

A análise qualitativa do texto, contudo, expõe a limitação já antecipada, que ao interpretar o alto percentual de itens direcionados ao elevador (40%) como um "possível desequilíbrio" e recomendar a "reavaliação dos critérios de alocação dos produtos" para "reduzir a quantidade de itens armazenados no elevador", o modelo trata como uma falha operacional exatamente o comportamento que a rede de Prioridade foi projetada para executar. Conforme demonstrado na seção 4.4.3, a totalidade dos itens excedentes, não demandados por nenhum pedido ativo no momento de sua detecção, foi corretamente direcionada ao armazenamento, com taxa de atendimento de 100% em todos os destinos avaliados. O modelo, ao dispor apenas dos vetores numéricos agregados, sem qualquer informação sobre a lógica de decisão da rede de Prioridade ou sobre a distinção entre itens demandados e itens excedentes, não possui como diferenciar um alto volume de itens armazenados por ausência de demanda de um alto volume de itens armazenados por falha de roteamento.

Um segundo ponto de imprecisão é identificado na afirmação de que o item Z "não foi solicitado pelas esteiras", quando, na realidade, uma unidade desse tipo foi corretamente entregue a cada uma das duas linhas secundárias, conforme a Tabela 25. O modelo parece ter inferido essa conclusão exclusivamente a partir da ausência de itens Z no vetor de itens enviados ao elevador, extrapolando incorretamente essa ausência para uma suposta falta de demanda das esteiras, sem que o prompt fornecesse os vetores individuais de composição de cada esteira necessários para sustentar essa afirmação com precisão.

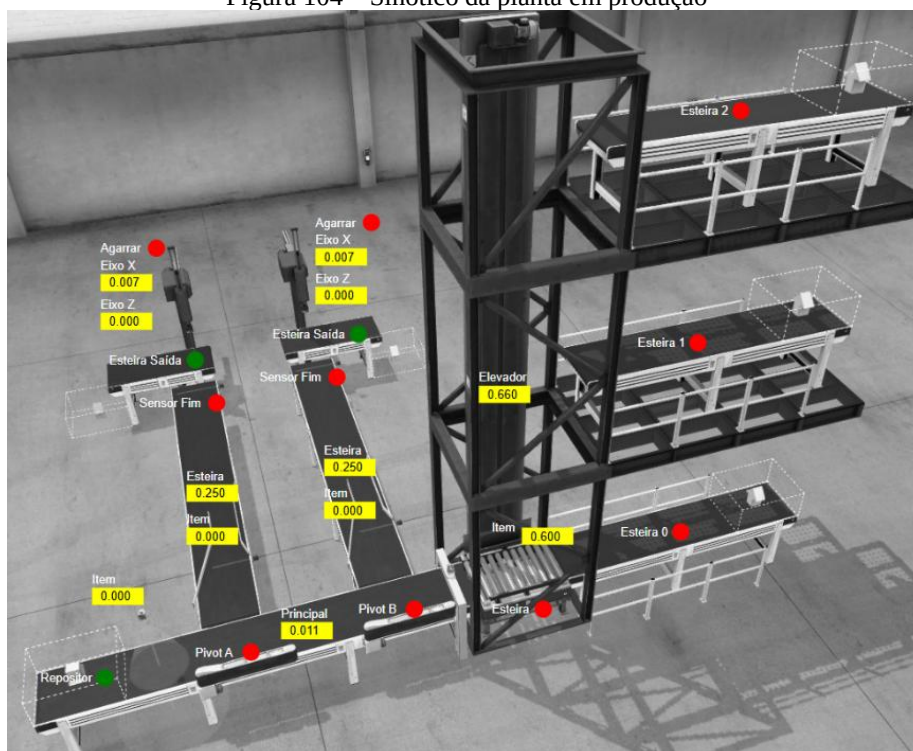
Esses resultados indicam que, embora a funcionalidade de geração automática de relatórios seja eficaz na consolidação e na descrição fiel dos dados numéricos agregados, sua camada de interpretação analítica permanece limitada pelo escopo de informação fornecido ao modelo, podendo produzir conclusões tecnicamente incorretas ou operacionalmente equivocadas quando a lógica de negócio subjacente aos dados não é explicitada no prompt. Essa constatação reforça a importância de tratar o relatório gerado como uma ferramenta de apoio à leitura dos indicadores, e não como uma fonte autônoma e definitiva de diagnóstico

operacional, devendo suas conclusões ser sempre validadas por um operador com conhecimento do contexto operacional do sistema.

4.5.2. Página Dashboard

A interface web disponibiliza uma página de Dashboard voltada ao acompanhamento visual do estado físico da planta em tempo real. Seu elemento central é um painel sinótico, sobreposto à representação tridimensional da planta simulada no Factory I/O, que exibe, junto a cada sensor e atuador mapeado nas Tabelas 17 a 20, o valor corrente da respectiva variável Modbus, atualizado continuamente ao longo da operação.

Figura 104 – Sinótico da planta em produção



Fonte: Elaborado pelo autor (2026)

A Figura 104 apresenta o painel sinótico capturado durante a sessão de atendimento dos pedidos 432 e 433, exibindo simultaneamente o estado de todos os subsistemas da planta em um mesmo instante de execução. O indicador Repositor sinaliza a liberação de um novo item na esteira principal, enquanto o valor Principal (0,011) reflete a velocidade corrente da esteira principal, reduzida nesse instante em razão da ocupação do elevador pelo item Y em trânsito, comportamento coerente com a lógica de controle de velocidade da rede Esteira já discutida na seção 4.2.3, na qual a esteira reduz sua velocidade sempre que o destino

selecionado pela rede de Prioridade ainda se encontra ocupado. Os atuadores Pivô A e Pivô B são responsáveis por direcionar o item à linha secundária correspondente, conforme essa mesma decisão. Em cada uma das duas linhas secundárias, o indicador Esteira (0,250) reflete a velocidade corrente da respectiva esteira de expedição, enquanto o indicador Item, codificado por tipo (Equação 21 aplicada à Tabela 1), identifica qual tipo de item está presente naquele ponto da esteira, no instante capturado, ambas as linhas secundárias apresentam o valor 0,000, indicando a ausência de item. Os valores de Agarrar, Eixo X (0,007) e Eixo Z (0,000), exibidos para cada um dos dois manipuladores, aproximam-se de zero e devem ser lidos como a posição inicial de repouso dos eixos, uma vez que pequenas variações residuais nesses valores decorrem de ruído das saídas da rede, sem representar um deslocamento efetivo nesse instante.

Na porção central do painel, o valor Elevador (0,660) corresponde ao setpoint corrente definido para o elevador de armazenamento, indicando que a rede está direcionando o elevador para o nível de referência do item Y ($\sim 0,5$), entre os três níveis já discutidos na seção 4.2.3 (nível 0, para o item X; $\sim 0,5$, para o item Y; e $\sim 0,96$, para o item Z). O indicador Item, posicionado na base da torre, apresenta o valor 0,600, identificando o item Y como o tipo atualmente presente na entrada do elevador, coerente com o setpoint direcionado a esse mesmo nível e com a redução de velocidade observada na esteira principal. Por fim, os indicadores Esteira 0, Esteira 1 e Esteira 2, posicionados nas três prateleiras do sistema de armazenamento, sinalizam a atividade de remoção de itens em cada um dos três níveis, permitindo ao operador acompanhar visualmente qual tipo de item está sendo armazenado em cada instante.

4.5.3. Página Treinamento

Além de sua função original de cadastro de dados e execução de treinamentos, já discutida na seção 3.5.1.3, a página de Treinamento também disponibiliza uma funcionalidade de teste manual, na qual o operador pode fornecer um vetor de entrada arbitrário e obter, de forma imediata, a saída calculada pela versão de pesos atualmente em uso por determinada rede. Essa funcionalidade permite constatar que os valores de saída produzidos pelos dispositivos de controle em campo durante a operação real da planta correspondem exatamente aos valores já conhecidos a partir do próprio processo de treinamento, para uma mesma entrada. Ou seja, ao submeter à página de Treinamento a mesma entrada observada em campo, é possível confirmar, de forma independente e desacoplada da planta física, que o dispositivo em operação está reproduzindo fielmente o comportamento aprendido pela rede, caracterizando um comportamento de gêmeo digital (digital twin).

Essa validação ganha relevância adicional por envolver três linguagens de programação distintas ao longo da arquitetura, Python no controlador local, C++ no dispositivo embarcado ESP32 e PHP na página de Treinamento, cada uma com sua própria representação interna de números de ponto flutuante. Confirmar que as três produzem resultados iguais ou virtualmente iguais para uma mesma entrada é o que permite tratar os valores exibidos pela página web como uma referência confiável do comportamento esperado de qualquer dispositivo do sistema, sem depender de acesso direto a cada um deles para saber, de antemão, qual saída uma determinada entrada deve produzir.

Como exemplo, é possível verificar que a saída registrada pelo microcontrolador ESP32, responsável pela execução embarcada da rede Expedição B, para a entrada [1, 0, 0.994, 0] coincide até a décima casa decimal com a saída obtida para a mesma entrada na página de Treinamento, confirmando que o dispositivo embarcado está reproduzindo fielmente, durante a operação real da planta, o comportamento aprendido pela rede ao longo do treinamento.

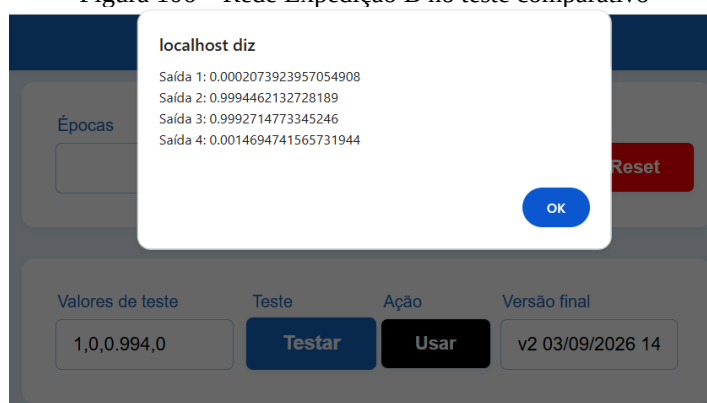
Figura 105 – Print do log ESP32

```
ExpedicaoB: [1.000000,0.000000,0.994000,0.000000] -> [0.0002073923,0.9994462133,0.9992714773,0.0014694741]
65 -> Tempo de escrita Modbus (ms): 15
```

Fonte: Elaborado pelo autor (2026)

A Figura 106 apresenta a captura de tela correspondente a esse mesmo teste, realizado diretamente na página de Treinamento com a entrada [1, 0, 0.994, 0]. A correspondência entre os quatro valores exibidos e os quatro valores registrados no log do ESP32, apresentados na Figura 105, confirma que o comportamento observado em campo já era previamente conhecido a partir do treinamento, sem exigir qualquer inspeção adicional do dispositivo embarcado para validar sua saída.

Figura 106 – Rede Expedição B no teste comparativo



Fonte: Elaborado pelo autor (2026)

Um segundo teste de mesma natureza foi conduzido para a rede Elevador, dessa vez comparando a saída registrada pelo controlador local, executado em Python e responsável pela execução dessa rede durante a operação da planta, com o resultado obtido na página de Treinamento para a mesma entrada. A Figura 107 apresenta o log de execução do controlador local em Python para a entrada [0, 0], submetida à rede Elevador durante a operação real da planta, registrando os cinco valores de saída correspondentes.

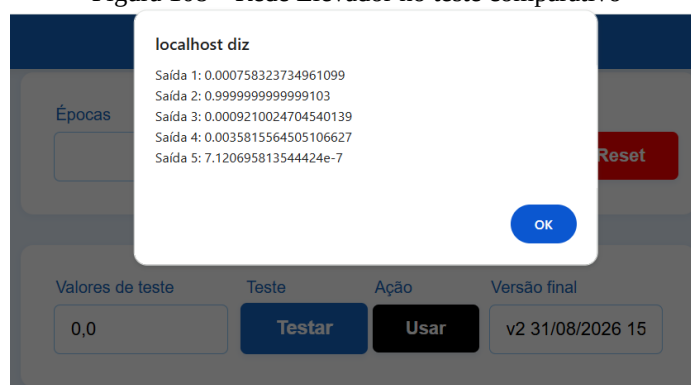
Figura 107 – Print do log do controlador Python

```
despacho: [0, 0] -> [0.9717084118023415, 0.9759391981166644]
elevador: [0.0, 0] -> [0.0007583237349610984, 0.9999999999999103, 0.0009210024704540132, 0.0035815564505106627, 7.120695813544424e-07]
prioridade: [0.0, 0.0, 1.0, 1.0] -> [9.392646000902772e-08, 5.472298241707244e-07, 0.9999993588437158]
esteira: [0.9, 0, 0, 0, 0, 0, 1, 0.0] -> [8.734159196661224e-06, 0.2871970021898856]
expedicaoA: [False, False, 0.0, 0.0] -> [0.0001469394831281643, 0.006955544711818307, 3.3810209463929024e-07, 0.24988572673248943]
```

Fonte: Elaborado pelo autor (2026)

A Figura 108, por sua vez, apresenta a captura de tela do mesmo teste realizado na página de Treinamento, para a mesma entrada [0, 0] da rede elevador. Os valores obtidos nos dois ambientes são idênticos até a precisão de ponto flutuante da própria linguagem, sem qualquer diferença perceptível entre as cinco saídas, apesar de o controlador local ser executado em Python e a página de Treinamento em PHP, duas linguagens com implementações distintas de aritmética de ponto flutuante.

Figura 108 – Rede Elevador no teste comparativo



Fonte: Elaborado pelo autor (2026)

Considerados em conjunto, os dois testes comparativos, Expedição B (ESP32, em C++) e elevador (controlador local, em Python), demonstram que a página de Treinamento, implementada em uma terceira linguagem (PHP), reproduz com fidelidade o comportamento de inferência de qualquer rede do sistema, independentemente do dispositivo ou da linguagem em que essa rede esteja efetivamente em execução. Esse resultado confirma, na prática, que é

possível consultar remotamente, por meio da página web, o valor de saída esperado para qualquer entrada de qualquer uma das cinco redes, com a garantia de que esse valor corresponde ao que os dispositivos de controle em campo efetivamente produziram diante da mesma entrada.

4.6. Discussão dos resultados

Os resultados apresentados ao longo deste capítulo, considerados em conjunto, permitem avaliar de forma integrada o desempenho do sistema inteligente de controle e gerenciamento remoto proposto neste trabalho, confrontando o que foi efetivamente observado com os objetivos específicos estabelecidos na seção 1.3.2.

O treinamento inicial das cinco redes neurais, apresentado na seção 4.1, demonstrou que a segmentação do problema de controle em módulos especializados, motivada por razões de dimensionalidade e simplicidade computacional discutidas na fundamentação metodológica, não comprometeu a qualidade do aprendizado de nenhuma das redes, mesmo naquelas treinadas com conjuntos de dados reduzidos, como as redes de Expedição e Despacho.

A operação integrada da planta, analisada na seção 4.2, evidenciou que redes treinadas de forma isolada foram capazes de cooperar de maneira coerente durante a execução simultânea, confirmando que a comunicação por meio de variáveis de estado consolidadas, em vez de acoplamento direto entre os parâmetros das redes, é suficiente para coordenar um fluxo produtivo com múltiplas etapas sequenciais e dependências entre módulos. A distribuição correta de dois pedidos concorrentes entre as linhas, o direcionamento sem erros dos itens excedentes ao armazenamento e a taxa de atendimento correto de 100% observada sustentam diretamente o objetivo específico de implementar um sistema de controle baseado em redes neurais artificiais capaz de operar de forma coordenada sobre uma planta industrial simulada.

Do ponto de vista da infraestrutura de comunicação, os resultados da seção 4.3 indicam que o protocolo Modbus TCP/IP se mostrou capaz de controlar a planta simulada tanto na infraestrutura local em Python quanto no microcontrolador ESP32. O tempo médio de ciclo completo observado no ESP32 via Wi-Fi mostrou-se cerca de 6,5 vezes superior ao da infraestrutura cabeada, resultado coerente com a combinação esperada entre a menor capacidade de processamento de um microcontrolador de baixo custo e a maior variabilidade de latência inerente à comunicação sem fio. Já o tempo médio de escrita Modbus específico da rede replicada em ambos os ambientes permaneceu bem mais próximo entre si, cerca de 19% superior no Wi-Fi. Picos ocasionais de latência, contudo, levaram o valor máximo de escrita

via Wi-Fi a se aproximar consideravelmente do próprio tempo médio de ciclo completo da planta, evidenciando que, embora o comportamento médio do sistema permaneça responsivo, cenários de pico no enlace sem fio merecem atenção em aplicações de maior criticidade temporal. Esses resultados atendem ao objetivo específico de estabelecer um protocolo de comunicação industrial entre a planta e a rede de controle, ao mesmo tempo em que evidenciam a viabilidade prática de descentralizar parte do processamento de inferência para dispositivos embarcados de baixo custo.

Os indicadores quantitativos apresentados na seção 4.4 confirmaram, com dados concretos, o que as seções anteriores já haviam demonstrado qualitativamente. A rede de Prioridade distribuiu os itens de forma equilibrada entre as três rotas da planta, com taxa de atendimento correto de 100% tanto no roteamento aos pedidos quanto no direcionamento de excedentes ao armazenamento, e o intervalo médio entre itens, tanto por tipo quanto por destino, embora variável entre poucas amostras, não revelou nenhum padrão de acúmulo ou represamento em nenhuma das duas linhas secundárias. A correspondência exata entre os indicadores agregados exibidos na interface e o estado físico da planta em um instante intermediário da execução reforçou, por fim, a confiabilidade desses números como base de decisão para o operador.

A camada de supervisão, analisada na seção 4.5, cumpriu o papel de integrar o sistema de controle a uma plataforma web acessível remotamente, sendo a estrutura em três páginas dedicadas, determinante para separar claramente os diferentes níveis de interação do operador com o sistema, indicadores agregados de produção, representação sinótica do estado físico da planta e verificação da fidelidade computacional dos modelos em operação. Em particular, a comparação entre os valores calculados em produção, tanto pelo controlador local em Python quanto pelo firmware do ESP32, e os valores obtidos para as mesmas entradas na página de Treinamento confirmou, na prática, o comportamento de gêmeo digital proposto para essa interface, com correspondência numérica praticamente exata entre os ambientes. Já a integração com a API de Inteligência Artificial Generativa, discutida na seção 4.5.1, demonstrou ser capaz de transformar dados operacionais brutos em diagnósticos textuais compreensíveis, com alta fidelidade na reprodução de valores agregados fornecidos pelo backend, cumprindo o objetivo específico de analisar o funcionamento do processo com auxílio de IA generativa sem comprometer o caráter determinístico do controle industrial subjacente. Cabe ressaltar que a integração de camadas de supervisão remota, computação em nuvem e Inteligência Artificial Generativa introduz também novas superfícies de exposição em termos de segurança da informação, aspecto que não foi objeto de investigação neste trabalho, mas que

a literatura aponta como um desafio central para a adoção de soluções de automação industrial baseadas em IA em ambientes de produção reais (SIDDIQUE et al., 2023).

A análise qualitativa dos relatórios gerados revelou, no entanto, um erro pontual e uma interpretação que desconsiderou a lógica de controle da arquitetura ao tratar o armazenamento correto de itens excedentes como um desequilíbrio operacional, reforçando que essa funcionalidade deve ser tratada como camada complementar de apoio, e não como fonte primária de verificação dos indicadores.

Os resultados discutidos ao longo deste capítulo permitem observar que o comportamento do sistema esteve alinhado às premissas adotadas para sua concepção, tanto no que se refere à atuação coordenada das redes neurais quanto à comunicação entre os diferentes componentes e à representação das informações operacionais na camada de supervisão. As diferenças observadas entre os ambientes de comunicação, particularmente em relação à latência do Wi-Fi, bem como as limitações identificadas na interpretação dos dados pela IA generativa, também contribuem para uma compreensão mais abrangente do comportamento da arquitetura, ao evidenciar que seu desempenho não depende exclusivamente da capacidade de inferência dos modelos, mas da interação entre controle, comunicação, processamento e supervisão. Nesse sentido, os resultados apresentados fornecem elementos para discutir as potencialidades e as limitações da abordagem adotada, estabelecendo uma base experimental para a análise final do trabalho.

5. CONCLUSÃO

Este capítulo encerra o trabalho, retomando, de forma objetiva, o percurso realizado desde a proposta inicial até os resultados obtidos com a implementação do sistema inteligente de controle e supervisão da planta industrial simulada. A partir da síntese apresentada na discussão dos resultados, ao final do Capítulo 4, são revisitados os objetivos definidos na seção 1.3, avaliando-se em que medida cada um deles foi efetivamente atendido pelo sistema desenvolvido.

5.1. Verificação dos objetivos

Os resultados obtidos ao longo dos experimentos demonstram a viabilidade da arquitetura proposta para integração de diferentes tecnologias aplicadas à automação industrial. A operação conjunta da planta simulada, das redes neurais artificiais, da comunicação via

Modbus TCP/IP, da supervisão web e da camada de Inteligência Artificial Generativa evidenciou que esses recursos podem atuar de forma coordenada em uma mesma estrutura. Dessa forma, a arquitetura desenvolvida não apenas reuniu tecnologias distintas, mas estabeleceu uma relação funcional entre elas, na qual cada camada desempenha uma função específica dentro do processo de automação.

A planta implementada no Factory I/O apresentou comportamento consistente diante de diferentes condições de demanda, no qual a divisão da lógica de controle em cinco redes neurais, distribuídas em seis instâncias independentes, permitiu que as diferentes etapas do processo fossem executadas de maneira coordenada. A utilização de variáveis de estado compartilhadas como mecanismo de comunicação entre as instâncias mostrou-se particularmente relevante, pois possibilitou a coordenação das decisões sem a necessidade de estabelecer dependências diretas entre os parâmetros das redes. Com isso, o sistema foi capaz de direcionar os itens de acordo com os pedidos ativos e encaminhar os excedentes ao armazenamento, mantendo a sequência operacional prevista para a planta. Esse comportamento indica que a abordagem baseada em redes neurais pode representar uma alternativa para a implementação de lógicas de decisão, corroborando estudos que validaram experimentalmente a viabilidade de redes neurais treinadas por aprendizado supervisionado como mecanismo eficaz de decisão de controle (RITA, 1995).

O desempenho obtido no treinamento foi satisfatório devido ao fato das cinco redes alcançarem 100% de acurácia nas saídas, embora tenham apresentado diferentes velocidades de convergência. Esse resultado demonstra que, para as condições utilizadas no desenvolvimento e na validação, as redes foram capazes de reproduzir adequadamente as decisões esperadas a partir dos estados apresentados pela planta. Um aspecto importante desse resultado é que a lógica de decisão não precisou ser implementada por meio de regras condicionais explícitas para cada situação operacional. Entretanto, uma vez que esse desempenho não permite assegurar o mesmo comportamento diante de situações não contempladas durante a etapa de aprendizagem.

A comunicação entre o controlador e a planta também apresentou resultados compatíveis com a proposta. No ambiente cabeado, executado em Python, foi obtido tempo médio de ciclo de varredura de 120,185 ms, enquanto a comunicação realizada pelo ESP32 por meio de Wi-Fi apresentou média de 775,395 ms, conforme os resultados apresentados nas Seções 4.1 e 4.3. O funcionamento observado no ambiente sem fio demonstra a possibilidade de utilizar uma plataforma embarcada como alternativa de comunicação e execução, ampliando a flexibilidade da arquitetura. Por outro lado, os resultados também indicam que a escolha entre

comunicação cabeada e sem fio deve considerar os requisitos temporais da aplicação, especialmente em processos industriais nos quais atrasos de comunicação possam comprometer a dinâmica do controle. Esse mesmo padrão, no qual configurações com processamento ou comunicação mais próximos do controlador apresentam tempos de resposta inferiores aos de configurações embarcadas, também foi observado em estudos comparativos de arquiteturas de integração de redes neurais à automação industrial (SILVA, 2024).

A camada de supervisão acrescentou uma dimensão de acessibilidade e gerenciamento ao sistema, possibilitando acompanhar o processo, inserir pedidos, consultar indicadores e realizar o treinamento das redes sem a necessidade de acesso direto ao ambiente de execução do controlador. A integração entre o backend desenvolvido em PHP e o banco de dados MySQL permitiu ainda centralizar informações operacionais, parâmetros de treinamento, configurações e versões dos pesos das redes. Esse mecanismo mostrou-se relevante não apenas para o armazenamento dos dados, mas também para a manutenção do vínculo entre treinamento e operação, uma vez que os pesos obtidos em uma nova etapa de treinamento puderam ser registrados e posteriormente utilizados como configuração corrente do sistema. A possibilidade de realizar o treinamento por meio da própria interface web também contribui para reduzir a dependência do ambiente de desenvolvimento utilizado na implementação das redes.

A integração da Inteligência Artificial Generativa acrescentou uma camada distinta às demais, voltada à interpretação das informações produzidas pelo sistema. A partir dos dados agregados disponibilizados pelo backend, a API foi capaz de produzir diagnósticos em linguagem natural e reproduzir com elevada fidelidade os valores numéricos utilizados como entrada, demonstrando o potencial dessa tecnologia como mecanismo de apoio à interpretação dos indicadores, principalmente quando se considera a necessidade de apresentar informações técnicas de maneira mais acessível ao operador. Entretanto, os testes também evidenciaram uma limitação importante dessa camada, sendo que em uma análise, o elevado percentual de itens encaminhados ao armazenamento em razão da ausência de demanda foi interpretado como possível desequilíbrio operacional. Nesse caso, a resposta gerada estava numericamente coerente com os dados fornecidos, mas a interpretação do significado operacional desses dados não foi adequada. Assim, embora a Inteligência Artificial Generativa possa contribuir para a interpretação e contextualização dos indicadores, suas respostas não devem substituir os mecanismos determinísticos de validação. Essa constatação está alinhada à literatura, que destaca a necessidade de uso responsável e de medidas adequadas de verificação ao integrar Inteligência Artificial a sistemas de controle industrial, de modo a preservar a confiança e a segurança na adoção dessas tecnologias (SIDDIQUE et al., 2023).

5.2. Contribuições do trabalho

De maneira geral, os experimentos indicam que a principal contribuição da arquitetura está na integração das diferentes camadas em um fluxo único, no qual controle, comunicação, supervisão, armazenamento de dados e análise são tratados de forma complementar. Os resultados obtidos pelas redes neurais demonstram a capacidade de automatizar as decisões previstas para o processo, enquanto os testes de comunicação comprovam a possibilidade de utilizar diferentes meios de conexão entre controlador e planta. Paralelamente, a interface web amplia a capacidade de supervisão e gerenciamento, e a camada generativa oferece uma forma adicional de interpretação das informações operacionais.

5.3. Limitações

Apesar dos resultados positivos obtidos, algumas limitações devem ser consideradas. A principal delas está relacionada às condições utilizadas para o treinamento e a validação das redes neurais. Os modelos foram desenvolvidos e avaliados a partir de condições operacionais previamente conhecidas e definidas para o funcionamento da planta simulada, não abrangendo a complexidade e a variabilidade que podem estar presentes em sistemas de maior escala e com maior quantidade de estados operacionais. Essa característica está diretamente relacionada à utilização do aprendizado supervisionado, uma vez que o treinamento depende da disponibilidade de dados que representem adequadamente as situações às quais a rede poderá ser submetida e das respectivas decisões esperadas. Essa dependência é amplamente reconhecida na literatura, que aponta a disponibilidade de dados de qualidade, livres de ruído e capazes de cobrir todo o espaço operacional relevante, como um dos fatores mais determinantes para o sucesso de aplicações industriais baseadas em redes neurais (LENNOX et al., 2001). Quanto maior a complexidade do processo, maior tende a ser a necessidade de conhecer e caracterizar previamente seu funcionamento para construir um conjunto de dados de treinamento capaz de representar suas diferentes condições.

Outra limitação está relacionada à própria planta utilizada na validação, que foi desenvolvida em ambiente de simulação. Embora o Factory I/O tenha possibilitado reproduzir o fluxo produtivo e avaliar a interação entre os diferentes componentes da arquitetura, determinadas características e incertezas encontradas em processos industriais não foram contempladas nos experimentos. Entre elas, podem ser citadas variações não previstas no processo, falhas de sensores ou atuadores e alterações nas condições de comunicação.

Também deve ser considerada a diferença observada entre os tempos médios de comunicação nos ambientes cabeado e sem fio. O maior tempo de ciclo identificado na comunicação por Wi-Fi demonstra que o meio de transmissão pode influenciar o comportamento temporal do sistema, sendo necessário considerar esse aspecto de acordo com a criticidade de cada aplicação. Uma análise estatística mais formal dessa diferença, como um teste de hipótese para comparação de médias entre as duas amostras de tempos de ciclo, poderia fornecer um nível de confiança mais rigoroso para essa conclusão, sendo essa uma abordagem recomendada pela literatura de estatística aplicada à engenharia para a validação de diferenças observadas entre grupos de medições (MONTGOMERY; RUNGER, 2016).

A estrutura do prompt da IA priorizou determinados indicadores e informações operacionais, podendo direcionar a interpretação do modelo para determinadas perspectivas, produzindo uma análise tendenciosa em relação ao comportamento observado. Essa limitação ficou evidenciada quando um comportamento esperado da planta foi interpretado como possível desequilíbrio operacional, em razão da ausência de informações contextuais que permitissem à IA compreender que aquela condição estava associada à lógica normal de funcionamento do sistema. Dessa forma, os resultados indicam que a qualidade da análise generativa está diretamente relacionada não apenas à precisão dos dados fornecidos, mas também à abrangência e à contextualização das informações presentes na instrução do sistema.

5.4. Trabalhos futuros

Como continuidade deste trabalho, propõe-se inicialmente o estudo de um sistema adaptativo baseado em aprendizado por reforço, buscando superar a característica predominantemente estática do treinamento supervisionado empregado na arquitetura desenvolvida. Enquanto as redes atuais aprendem a reproduzir decisões previamente definidas a partir dos estados apresentados durante o treinamento, uma abordagem baseada em aprendizado por reforço permitiria investigar a capacidade de adaptação das redes diante de diferentes condições operacionais, utilizando recompensas associadas ao desempenho do processo. Essa direção converge com sugestões semelhantes presentes na literatura de controladores baseados em inteligência artificial, que apontam a comparação entre diferentes abordagens, incluindo o aprendizado por reforço, como uma via promissora para identificar a técnica mais adequada a diferentes tipos de sistemas industriais (ARAÚJO, 2025).

Outro direcionamento consiste na implementação das redes neurais e da lógica de controle em um Controlador Lógico Programável (CLP), considerando que esse dispositivo é

desenvolvido especificamente para aplicações de automação e controle industrial, apresentando características de robustez, confiabilidade e determinismo. Essa etapa permitiria avaliar as adaptações necessárias para executar modelos de inteligência computacional em uma plataforma tradicionalmente empregada na indústria, bem como investigar as limitações relacionadas aos recursos computacionais.

Outra possibilidade consiste no desenvolvimento de mecanismos de detecção de anomalias associados à camada de supervisão. Os dados históricos armazenados pelo sistema podem ser utilizados para estabelecer padrões de comportamento da planta e identificar situações que se afastem das condições esperadas. Essa funcionalidade poderia atuar de forma complementar às redes responsáveis pelo controle, permitindo diferenciar uma alteração legítima na demanda de um comportamento potencialmente anômalo do processo ou da própria comunicação entre os componentes da arquitetura.

Especificamente em relação à camada de Inteligência Artificial Generativa, sugere-se avaliar o desempenho de modelos de linguagem alternativos na geração dos relatórios, comparando a qualidade interpretativa entre diferentes provedores comerciais de IA generativa, como os modelos da OpenAI (OPENAI, 2023), do Google (GEMINI TEAM; GOOGLE, 2023) e da Anthropic (ANTHROPIC, 2024), de modo a verificar se a análise tendenciosa identificada neste trabalho é uma limitação específica do modelo utilizado ou um comportamento recorrente entre diferentes arquiteturas de linguagem.

Por fim, a arquitetura pode evoluir para uma estrutura mais modular, permitindo substituir ou adicionar métodos de inteligência computacional sem modificar integralmente as demais camadas do sistema. Essa característica seria particularmente relevante para comparar diferentes abordagens de controle, como redes neurais supervisionadas, aprendizado por reforço e outros métodos de aprendizado de máquina, utilizando a mesma infraestrutura de comunicação, supervisão e armazenamento. Com isso, seria possível estabelecer uma plataforma experimental para avaliação sistemática de diferentes técnicas de inteligência aplicada à automação industrial.

5.5. Considerações finais

Diante dos resultados obtidos, conclui-se que a arquitetura desenvolvida demonstrou ser tecnicamente viável para a integração de redes neurais artificiais, comunicação industrial, supervisão web e Inteligência Artificial Generativa em uma estrutura única de automação. Os experimentos evidenciaram que os diferentes componentes puderam atuar de

forma coordenada, permitindo que as decisões produzidas pelas redes neurais fossem aplicadas ao processo, que as informações operacionais fossem disponibilizadas para supervisão e que os dados resultantes fossem posteriormente utilizados em uma camada de análise.

Os resultados alcançados pelas redes neurais, associados ao funcionamento da comunicação Modbus TCP/IP nos ambientes avaliados e à integração entre controle, banco de dados e interface web, demonstram que a arquitetura é capaz de estabelecer um fluxo contínuo entre processamento, operação, armazenamento e supervisão. Ao mesmo tempo, as limitações identificadas evidenciam que o desempenho obtido está relacionado às condições conhecidas utilizadas no treinamento e à quantidade de contexto disponibilizada para a análise generativa. Dessa forma, os resultados não representam apenas a validação dos componentes individualmente, mas demonstram as possibilidades e os limites da integração dessas tecnologias dentro da estrutura proposta.

A solução desenvolvida estabelece uma base para o estudo da aplicação de técnicas de inteligência computacional em sistemas de automação, ao mesmo tempo em que evidencia a importância da qualidade dos dados, do conhecimento sobre o processo e da adequada contextualização das informações para o funcionamento dessas tecnologias. Os resultados obtidos sustentam a viabilidade da abordagem proposta e fornecem uma base experimental para o aprofundamento de arquiteturas de automação capazes de incorporar mecanismos de aprendizado, adaptação e análise inteligente.

REFERÊNCIAS

- ANTHROPIC. **The Claude 3 Model Family: Opus, Sonnet, Haiku.** Anthropic, 2024. Disponível em: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf. Acesso em: 04 ago. 2026.
- ARAÚJO, Rafael Domingos Nobre de. **Integração do Controle de Processos com Inteligência Artificial: Abordagem do Sistema e do Controlador.** 2025. Dissertação (Mestrado em Ciências) – Escola Politécnica, Universidade de São Paulo, São Paulo, 2025.
- BELGA, A. G. **Automação Industrial e Sistemas de Controle.** São Paulo: Érica, 2011.
- BENGIO, Yoshua. **Learning Deep Architectures for AI.** Hanover: Now Publishers Inc, 2009.
- BISHOP, C. M. **Pattern Recognition and Machine Learning.** New York: Springer, 2006.
- BOLTON, W. **Programmable Logic Controllers.** 6. ed. Oxford: Newnes, 2015.
- BOYER, Stuart A. **SCADA: Supervisory Control and Data Acquisition.** 4. ed. Research Triangle Park: ISA, 2010.
- BUNTAK, N. et al. **Application of Artificial Intelligence in Contemporary Business Management.** International Journal for Quality Research, v. 15, n. 1, p. 145-162, 2021.
- BUYYA, R.; BROBERG, J.; GOSCINSKI, A. M. **Cloud Computing: Principles and Paradigms.** Hoboken: Wiley, 2011.
- CAPELLI, Alexandre. **Automação Industrial: Controle do Movimento e Processos Contínuos.** 2. ed. São Paulo: Érica, 2013.
- CYBENKO, George. **Approximation by superpositions of a sigmoidal function.** Mathematics of Control, Signals and Systems, v. 2, n. 4, p. 303-314, 1989.
- ERL, T.; PUTTINI, R.; MAHMOOD, Z. **Cloud Computing: Concepts, Technology & Architecture.** Upper Saddle River: Prentice Hall, 2013.
- ESPRESSIF SYSTEMS. **ESP32 Series Datasheet.** Version 5.3. Shanghai: Espressif Systems, 2023. Disponível em: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Acesso em: 04 ago. 2026.
- FACELI, Katti et al. **Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina.** 2. ed. Rio de Janeiro: LTC, 2021.
- FLANAGAN, D. **JavaScript: O Guia Definitivo.** 7. ed. Porto Alegre: Bookman, 2020.

GEMINI TEAM; GOOGLE. **Gemini: A Family of Highly Capable Multimodal Models**. arXiv:2312.11805, 2023. Disponível em: <https://arxiv.org/abs/2312.11805>. Acesso em: 10 abr. 2026.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.

GRIEVES, M. **Digital Twin: Manufacturing Excellence through Virtual Factory Replication**. White paper, 2014. Disponível em: https://www.researchgate.net/publication/275211047_Digital_Twin_Manufacturing_Excenlence_through_Virtual_Factory_Replication. Acesso em: 04 ago. 2026.

GRIEVES, M.; VICKERS, J. **Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems**. In: KAHLEN, F.-J.; FLUMERFELT, S.; ALVES, A. (Eds.). *Transdisciplinary Perspectives on Complex Systems*. Cham: Springer, 2017. p. 85–113.

GROOVER, M. P. **Automation, Production Systems, and Computer-Integrated Manufacturing**. 3. ed. Boston: Pearson, 2011.

HAYKIN, Simon. **Redes Neurais: Princípios e Prática**. 2. ed. Porto Alegre: Bookman, 2001.

HAYKIN, S. **Neural Networks and Learning Machines**. 3. ed. New York: Pearson, 2009.

HAVERBEKE, Marijn. **Eloquent JavaScript: a modern introduction to programming**. 3. ed. San Francisco: No Starch Press, 2018.

JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. **An Introduction to Statistical Learning: with Applications in R**. 2. ed. New York: Springer, 2021

KAGERMANN, H.; WAHLSTER, W.; HELBIG, J. **Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0**. Frankfurt: acatech, 2013.

KANDEL, Eric R. et al. **Princípios de neurociência**. 5. ed. Porto Alegre: AMGH, 2014.

KASSEM, A.; MEIER, F. **Industrie 4.0: The Industrial Internet of Things**. Stuttgart: VDMA Verlag, 2016.

LENNOX, Barry; MONTAGUE, Gary A.; FRITH, Andy M.; GENT, Chris; BEVAN, Vic. **Industrial application of neural networks — an investigation**. *Journal of Process Control*, v. 11, n. 5, p. 497-507, 2001.

LOCKHART, J. **PHP Moderno: Novas Características e Boas Práticas**. 1. ed. São Paulo: Novatec, 2015.

MARINESCU, D. C. **Cloud Computing: Theory and Practice**. 2. ed. San Francisco: Morgan Kaufmann, 2018.

MODBUS ORGANIZATION. **Modbus Application Protocol Specification V1.1b3**. Modbus Organization, 2012. Disponível em: https://modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf. Acesso em: 04 ago. 2026.

MODICON, Inc. **Modbus Protocol Reference Guide**. PI-MBUS-300 Rev. J. North Andover: Modicon, Inc., 1996.

MONTGOMERY, D. C.; RUNGER, G. C. **Estatística Aplicada e Probabilidade para Engenheiros**. 6. ed. Rio de Janeiro: LTC, 2016.

NIXON, R. **Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5**. 5. ed. Sebastopol: O'Reilly Media, 2018.

OGATA, K. **Modern Control Engineering**. 5. ed. Upper Saddle River: Prentice Hall, 2010.

OLIVEIRA, Thiago Lima Alves de. **Sistema Inteligente de Controle e Gerenciamento Remoto de Processos Industriais Baseado em Redes Neurais Artificiais: código-fonte**. GitHub, 2026. Disponível em: <https://github.com/thiago-tlado/planta-industrial>. Acesso em: 31 ago. 2026.

OPENAI. **GPT-4 Technical Report**. arXiv:2303.08774, 2023. Disponível em: <https://arxiv.org/abs/2303.08774>. Acesso em: 04 ago. 2026.

ORACLE CORPORATION. **MySQL 8.0 Reference Manual**. Redwood City, CA: Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 04 abr. 2026.

PINEDO, M. L. **Scheduling: Theory, Algorithms, and Systems**. 5. ed. New York: Springer, 2016.

PRUDENTE, Francesco. **Automação industrial pneumática: teoria e aplicações**. Rio de Janeiro: LTC, 2013. E-book.

RAJKUMAR, Ragunathan; LEE, Insup; SHA, Lui; STANKOVIC, John. **Cyber-Physical Systems: The Next Computing Revolution**. In: DESIGN AUTOMATION CONFERENCE, 47., 2010, Anaheim. Proceedings [...]. New York: ACM, 2010. p. 731-736.

REAL GAMES. **Factory I/O: user manual and technical documentation**. Uxbridge: Real Games, 2023. Disponível em: <https://docs.factoryio.com/>. Acesso em: 04 ago. 2026.

RINALDI, J. S. **Modbus: The Everyman's Guide to Modbus**. [S.l.]: Real Time Automation, 2015.

RITA, Dalva Janine. **Controle de Processos Usando Redes Neurais Artificiais: Uma Aplicação Experimental**. 1995. Dissertação (Mestrado em Engenharia Química) – Universidade Federal de Santa Catarina, Florianópolis, 1995.

ROSÁRIO, João Maurício. **Automação Industrial**. São Paulo: Érica, 2010.

ROSENBLATT, Frank. **The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain**. Psychological Review, v. 65, n. 6, p. 386-408, 1958.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. **Learning representations by back-propagating errors**. Nature, v. 323, p. 533-536, 1986.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Harlow: Pearson, 2021.

SCHWAB, Klaus. **A Quarta Revolução Industrial**. São Paulo: Edipro, 2016.

SIDDIQUE, Sunzida; HAQUE, Mohd Ariful; RIFAT, Rakib Hossain; DAS, Laxmi Rani; TALUKDER, Sajedul; ALAM, Syed Bahauddin; GUPTA, Kishor Datta. **Challenges and Opportunities of Computational Intelligence in Industrial Control System (ICS)**. In: IEEE SYMPOSIUM SERIES ON COMPUTATIONAL INTELLIGENCE (SSCI), 2023, Mexico City. Proceedings [...]. [S.l.]: IEEE, 2023. p. 1158-1163. DOI: 10.1109/SSCI52147.2023.10371954.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Operating System Concepts**. 10. ed. Hoboken: Wiley, 2018.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

SILVA, Gerson Fabio da. **Case Study of MLP and Kolmogorov-Arnold Networks Applied to Industrial Automation**. 2024. Dissertação (Mestrado em Ciências) – Escola Politécnica, Universidade de São Paulo, São Paulo, 2024.

SILVA, I. N.; SPATTI, D. H.; FLAUZINO, R. A. **Redes Neurais Artificiais para Engenharia e Ciências Aplicadas: Fundamentos Teóricos e Aspectos Práticos**. 2. ed. São Paulo: Artliber, 2016.

SILVEIRA, Paulo Rogério da; SANTOS, Winderson E. **Automação e Controle Discreto**. 9. ed. São Paulo: Érica, 2014.

SUTTON, R. S.; BARTO, A. G. **Reinforcement Learning: An Introduction**. 2. ed. Cambridge: MIT Press, 2018.

TANENBAUM, Andrew S.; WETHERALL, David J. **Computer Networks**. 5. ed. Boston: Pearson, 2011.

TAO, F. et al. **Digital twin-driven product design, manufacturing and service with big data**. The International Journal of Advanced Manufacturing Technology, v. 94, p. 3563-3576, 2018.

THE PHPMYADMIN DEVELOPMENT TEAM. **phpMyAdmin documentation**. [S. l.]: phpMyAdmin, 2026. Disponível em: <https://docs.phpmyadmin.net/en/latest/>. Acesso em: 8 set. 2026.

VARGAS, Milton. **Metodologia da Pesquisa Tecnológica**. Rio de Janeiro: Globo, 2017.

VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N.; KAISER, Lukasz; POLOSUKHIN, Illia. **Attention Is All You Need. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 30.**, 2017, Long Beach. Proceedings [...]. p. 5998-6008.

VENERI, G.; CAPASSO, A. **Hands-On Industrial Internet of Things: Create a powerful Industrial IoT infrastructure using real-world projects.** Birmingham: Packt Publishing, 2018.

VERAS, M. **Cloud Computing: Produtividade na Nuvem.** Rio de Janeiro: Brasport, 2016.

APÊNDICE A – CÓDIGO-FONTE DO SISTEMA

Em razão da extensão do código-fonte desenvolvido para este trabalho, optou-se por não reproduzi-lo integralmente neste documento. O código-fonte completo encontra-se disponível em repositório público, de autoria do próprio autor deste trabalho, hospedado na plataforma GitHub, podendo ser acessado em: <https://github.com/thiago-tlado/planta-industrial> (OLIVEIRA, 2026). O repositório foi disponibilizado com o objetivo de complementar as informações apresentadas neste trabalho e possibilitar a consulta à implementação dos componentes que constituem o sistema desenvolvido.

O repositório está organizado em diretórios correspondentes às diferentes camadas e componentes da arquitetura apresentada, contemplando o dispositivo de controle local, o firmware do dispositivo embarcado, os módulos do backend e os arquivos referentes à interface web. Também é disponibilizado o backup da estrutura do banco de dados MySQL, permitindo a consulta individual dos componentes, bem como uma análise mais detalhada da implementação entre as diferentes partes do sistema proposto (OLIVEIRA, 2026).